

Hướng dẫn sử dụng bảng điều khiển 3X-UI

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский · Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

Phiên bản 3X-UI: 3.5.0. Hướng dẫn được biên soạn theo phiên bản này và có hiệu lực cho phiên bản đó. Tóm tắt các thay đổi của 3.5.0 so với 3.4.2 – trong mục «[Có gì mới trong 3.5.0](#)».

Hướng dẫn chi tiết bằng tiếng Việt về bảng điều khiển web **3X-UI** (quản lý Xray-core): các tính năng, cấu hình và vận hành, với giải thích từng trường và nút chuyển trong giao diện.

Tên và nhãn tương ứng với giao diện bảng điều khiển. Các từ *inbound* / *outbound* không được dịch.

Mục lục

- Có gì mới trong 3.5.0
- 1. Giới thiệu, yêu cầu và cài đặt
 - 1.1. 3X-UI là gì
 - 1.2. Hệ điều hành và kiến trúc được hỗ trợ
 - 1.3. Các phương thức cài đặt
 - 1.4. Lần khởi chạy đầu tiên và thông tin xác thực mặc định
 - 1.5. Vị trí các file
 - 1.6. Lệnh quản lý `x-ui` (menu script)
 - 1.7. Lệnh con `x-ui` (không dùng menu tương tác)
 - 1.8. Chuyển đổi SQLite sang PostgreSQL
- 2. Đăng nhập vào bảng điều khiển và bảo mật truy cập
 - 2.1. Biểu mẫu đăng nhập
 - 2.2. Xác thực hai yếu tố (2FA / TOTP)
 - 2.3. Giới hạn số lần đăng nhập (login limiter / bảo vệ chống dò mật khẩu)
 - 2.4. Thay đổi tên đăng nhập và mật khẩu quản trị viên
 - 2.5. Đường dẫn bí mật (URI path / webBasePath) và cổng bảng điều khiển
 - 2.6. Thời gian tồn tại phiên (timeout)
 - 2.7. LDAP (đồng bộ hóa và xác thực)
- 3. Tổng quan / Bảng điều khiển
 - 3.1. Nguyên tắc chung thu thập dữ liệu
 - 3.2. CPU
 - 3.3. Bộ nhớ (RAM)
 - 3.4. Bộ nhớ đệm trao đổi (Swap)
 - 3.5. Ổ đĩa (Storage)
 - 3.6. Thời gian hoạt động của hệ thống (Uptime)
 - 3.7. Tải hệ thống (Load average)
 - 3.8. Mạng: tốc độ và tổng lưu lượng
 - 3.9. Địa chỉ IP của máy chủ
 - 3.10. Kết nối TCP/UDP
 - 3.11. Trạng thái Xray và quản lý tiến trình
 - 3.12. Cập nhật panel (3X-UI)
 - 3.13. Cập nhật tệp địa lý (GeoIP / GeoSite)
 - 3.14. Sao lưu và khôi phục cơ sở dữ liệu
 - 3.15. Các phần tử giao diện bổ sung
- 4. Inbounds: tạo mới và các tham số chung
 - 4.1. Các trường chung của form
 - 4.2. Sniffing (Nghe lén)
 - 4.3. Allocate (chiến lược phân bổ cổng)
 - 4.4. External Proxy (Proxy bên ngoài)
 - 4.5. Fallbacks (Các fallback)
 - 4.6. Đặt lại lưu lượng định kỳ
 - 4.7. JSON входящего (nâng cao)
 - 4.8. Các thao tác với inbound: QR / Edit / Reset / Delete và thống kê

- 5. Giao thức
 - 5.1. Danh sách giao thức được hỗ trợ
 - 5.2. Giao thức nào hỗ trợ TLS / REALITY / transport
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (bộ chuyển tiếp trong suốt)
 - 5.8. SOCKS / HTTP (giao thức `mixed`)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (mặc định v2)
 - 5.11. MTPROTO (proxy cho Telegram)
 - 5.12. Bảng tóm tắt nhanh về chọn giao thức
- 6. Truyền tải (Stream Settings)
 - 6.1. Chọn mạng truyền tải
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Truyền tải Hysteria (`hysteriaSettings`)
 - 6.9. Các thông số đi kèm
- 7. Bảo mật kết nối: TLS, XTLS và REALITY
 - 7.1. Sự khác biệt: TLS vs XTLS vs REALITY
 - 7.2. Chế độ «Không» (`none`)
 - 7.3. Chế độ TLS
 - 7.4. Chế độ REALITY
 - 7.5. Khuyến nghị thực tế về cấu hình
- 8. Clients
 - 8.1. Các trường của client
 - 8.2. Liên kết với inbound
 - 8.3. Các thao tác trên client
 - 8.4. Thao tác hàng loạt
 - 8.5. Tìm kiếm, lọc và sắp xếp
 - 8.6. Biểu tượng và trạng thái
- 9. Nhóm khách hàng
 - 9.1. Nhóm khách hàng là gì và dùng để làm gì
 - 9.2. Mối liên hệ của nhóm với khách hàng, inbound, node và giao thức
 - 9.3. Danh mục nhóm và các nhóm "rỗng"
 - 9.4. Các trường và cột của nhóm
 - 9.5. Tạo nhóm
 - 9.6. Đổi tên nhóm
 - 9.7. Thêm khách hàng vào nhóm
 - 9.8. Xóa khách hàng khỏi nhóm (không xóa bản thân khách hàng)
 - 9.9. Đặt lại lưu lượng của nhóm

- 9.10. Xóa nhóm và xóa khách hàng của nhóm
- 9.11. Liên kết với trang «Khách hàng»
- 9.12. Tổng hợp các endpoint API
- 9.13. Lưu lượng theo nhóm
- 10. Đăng ký (Subscription)
 - 10.1. subId là gì và cách tạo liên kết
 - 10.2. Cài đặt server đăng ký
 - 10.3. Định dạng đầu ra
 - 10.4. Trang thông tin đăng ký và mã QR
 - 10.5. Mẫu tùy chỉnh trang đăng ký
- 11. Xray: định tuyến, outbounds, DNS và phần mở rộng
 - 11.1. Cấu trúc trình chỉnh sửa: tab/chế độ
 - 11.2. Cài đặt chính (General)
 - 11.3. Các quy tắc định tuyến (routing)
 - 11.4. Outbounds (kết nối đi)
 - 11.5. Bộ cân bằng tải (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Reverse-proxy và TUN
 - 11.10. Nhật ký và thống kê (Stats, metrics)
 - 11.11. Lưu, khởi động lại và các chuyển đổi tự động
 - 11.12. Outbound từ đăng ký (với tự động cập nhật)
 - 11.13. Xoay vòng IP trong WARP
- 12. Nút (bảng điều khiển, master/slave)
 - 12.1. Tóm tắt ở đầu danh sách
 - 12.2. Thêm và chỉnh sửa nút
 - 12.3. Kiểm tra TLS (cho nút https)
 - 12.4. Thông tin hiển thị cho từng nút
 - 12.5. Các thao tác trên nút
 - 12.6. Lịch sử số liệu
 - 12.7. Cách đồng bộ inbound và client
 - 12.8. Chuỗi nút (nút con / nút chuyển tiếp)
 - 12.9. Các bản sửa lỗi 3.5.0 (độ ổn định của nút)
 - 12.10. Nút: điểm mới trong 3.3.0
- 13. Cài đặt bảng điều khiển
 - 13.1. Lưu và khởi động lại bảng điều khiển
 - 13.2. Cài đặt chung (tab «Bảng điều khiển» / *General*)
 - 13.3. Quyền truy cập bảng điều khiển: IP, cổng, đường dẫn, tên miền, chứng chỉ
 - 13.4. Phiên, proxy bảng điều khiển và proxy tin cậy (tab «Proxy và máy chủ» / *Proxy and Server*)
 - 13.5. Telegram bot (tab «Telegram bot» / *Telegram Bot*)
 - 13.6. Ngày và giờ (tab «Ngày và giờ» / *Date and Time*)
 - 13.7. Lưu lượng bên ngoài và hành vi Xray (tab «Lưu lượng bên ngoài» / *External Traffic*)
 - 13.8. Khác: template cấu hình Xray và URL kiểm tra
 - 13.9. Tài khoản quản trị viên và API token

- 13.10. Thay đổi API trong 3.3.0 (quan trọng cho các tích hợp)
- 14. Telegram Bot
 - 14.1. Bật và cấu hình bot
 - 14.2. Menu chính và các nút
 - 14.3. Lệnh bot
 - 14.4. Quản lý client (chỉ quản trị viên)
 - 14.5. Thông báo và báo cáo
 - 14.6. Sao lưu và nhật ký
 - 14.7. Đặc điểm hoạt động
- 15. Cơ sở dữ liệu địa lý (geopip / geosite và tùy chỉnh)
 - 15.1. geopip.dat và geosite.dat là gì
 - 15.2. Tập địa lý tiêu chuẩn và cách cập nhật
 - 15.3. Tự động cập nhật dữ liệu địa lý bằng Xray (Geodata Auto-Update)
 - 15.4. Xác thực và giới hạn
 - 15.5. Kiểm tra tự động khi khởi động panel
 - 15.6. Sử dụng cơ sở dữ liệu địa lý trong quy tắc định tuyến
- 16. Vận hành: sao lưu, nhật ký, cập nhật, CLI
 - 16.1. Sao lưu và khôi phục cơ sở dữ liệu
 - 16.2. Xem nhật ký
 - 16.3. Cập nhật và cấu hình ghi nhật ký Xray
 - 16.4. Quản lý Xray: dừng và khởi động lại
 - 16.5. Khởi động lại và cập nhật bảng điều khiển
 - 16.6. Các tác vụ định kỳ (cron)
 - 16.7. Menu console và CLI (`x-ui`)
 - 16.8. Xóa bảng điều khiển
 - 16.9. Lệnh `x-ui migrateDB`

Có gì mới trong 3.5.0

Phiên bản 3.5.0 là một bản phát hành lớn: MTProto chuyển sang mô hình đa khách hàng (engine mtg-multi, secret cá nhân, hạn mức và ad-tag), Hosts được quản lý trở thành host nhóm (nhiều inbound và nhiều địa chỉ trong một bản ghi), khôi phục trên panel PostgreSQL chấp nhận bản sao lưu SQLite, outbound có thêm «Target Strategy», chế độ kiểm tra «Real delay» và các cột Egress/Country, bộ cân bằng tải có thể dùng một bộ cân bằng tải khác làm fallback. Đi kèm là lỗi Xray 26.7.11. Dưới đây là các thay đổi so với 3.4.2, được nhóm theo các mục của hướng dẫn.

Thay đổi ở phần 1 – Giới thiệu, yêu cầu và cài đặt

- Lỗi Xray được cập nhật lên **26.7.11**. Các di chuyển tự động đi kèm: các phương thức mã hóa Shadowsocks `none / plain` và VMess `none / zero` đã bị xóa khỏi lõi (các cấu hình đã lưu được ghi lại tự động), còn VLESS/Trojan-outbound không mã hóa đến địa chỉ công khai bị từ chối khi lưu.
- Lệnh mới `x-ui pgclient [phiên bản]` và mục **10. Install/Upgrade client tools (pg_dump/pg_restore)** trong menu PostgreSQL – cài đặt/cập nhật các công cụ client PostgreSQL.
- Sửa lỗi script: cài đặt PostgreSQL và fail2ban trên họ RHEL (EPEL), Arch không còn chạy `pacman -Syu` đầy đủ, tên nhị phân Xray chính xác trên ARM 32-bit (`xray-linux-arm32`), xác nhận IPv4 được tự phát hiện trước khi phát hành chứng chỉ IP, đã sửa thông báo sai «Your input is invalid» khi chọn cổng ACME mặc định.

Thay đổi ở phần 2 – Đăng nhập bảng điều khiển và bảo mật truy cập

- Giới hạn IP: kết nối «chết» nay chỉ bị chặn **một lần**, chứ không phải ở mỗi lần quét 10 giây – bộ đếm fail2ban không còn bị tăng ảo, không cần nâng cao `maxretry` nữa.

Thay đổi ở phần 4 – Inbounds: tạo mới và các tham số chung

- Danh sách inbound có thêm **tìm kiếm** (theo ghi chú, cổng và giao thức), còn các danh sách thả xuống chọn node («Развернуть на», bộ lọc «Node») nay hỗ trợ tìm kiếm khi nhập.

Thay đổi ở phần 5 – Giao thức

- MTProto chuyển sang mô hình đa khách hàng** (engine mtg-multi): người dùng MTProto nay là các client thông thường với secret, hạn mức, thời hạn, ad-tag riêng và liên kết `tg://proxy` cá nhân. Trường «Secret» ở cấp inbound đã bị bỏ (các inbound hiện có được chuyển đổi tự động), «FakeTLS domain» trở thành domain mặc định cho các secret mới. Các trường inbound mới: **Max connections** (giới hạn kết nối), **Public IPv4/IPv6** (cho ad-tag middle proxy). Các thay đổi của client được áp dụng «ngay lập tức», không làm rơi các phiên Telegram của người khác.
- WireGuard: menu inbound nhận được bộ thao tác client đầy đủ (Export All URLs, liên kết/hủy liên kết, nhóm), phần xuất được chia thành các tab **Config** và **Links**, trường «**IP được phép WireGuard**» nay có thể **chỉnh sửa** (nhiều mục nhập cách nhau bằng dấu phẩy), còn trong cấu hình client của inbound trên node, `Endpoint` nay trở đến địa chỉ của node.

Thay đổi ở phần 7 – Bảo mật kết nối: TLS,XTLS và REALITY

- Tổ hợp **Finalmask + REALITY** bị **từ chối** khi lưu (nó từng làm Xray-core sập ở kết nối đầu tiên); placeholder `minClientVer` được cập nhật lên 26.3.27.
- Loại mặt nạ TCP Finalmask mới – **XMC (Minecraft)**: nguy trạng luồng thành lưu lượng Minecraft (Hostname, Usernames, Password bắt buộc với tự động tạo).

Thay đổi ở phần 8 – Clients

- Cột mới **«Tốc độ»** – tốc độ trực tiếp của từng client (↑/↓, trung bình trượt ~5 giây).
- Tìm kiếm client lại tìm được theo **Telegram ID**; trong biểu mẫu client, các inbound đã tắt bị ẩn khỏi danh sách liên kết; đã sửa việc tích lũy bản ghi trùng lặp trong cửa sổ «Hủy liên kết».
- Client MTProto có các trường riêng: **«MTProto secret»** (với nút tạo lại) và **«Ad-tag (kênh tài trợ)»** (32 ký tự hex); hạn mức và thời hạn nay thực sự được áp dụng cho MTProto.

Thay đổi ở phần 9 – Nhóm khách hàng

- Cửa sổ thông tin client nay hiển thị **nhóm** của client.

Thay đổi ở phần 10 – Đăng ký (Subscription)

- **Hosts được quản lý trở thành host nhóm**: một bản ghi bao phủ **nhiều inbound** (chọn nhiều) và **nhiều địa chỉ** (nhập dạng thẻ, mỗi mục có thể có : cổng riêng; tự động gợi ý địa chỉ; để trống – kế thừa địa chỉ của inbound). Các cột của danh sách hiển thị chip địa chỉ và inbound (với «+N»), các thao tác và API làm việc với nhóm (`groupId`), xuất hiện endpoint hàng loạt `POST /panel/api/hosts/bulk/add` . Việc sắp xếp host nay là toàn cục (theo thứ tự, sau đó theo remark).
- Nội dung **thông báo** (`subAnnounce`) nay được hiển thị dưới dạng banner trên trang thông tin đăng ký; các mẫu tùy chỉnh có sẵn biến `announce` .
- Trang thông tin trong trình duyệt nay cũng mở được qua **liên kết JSON/Clash** (không chỉ qua liên kết chính).
- Các cài đặt host **Final Mask** và **Allow insecure** nay cũng có hiệu lực tương ứng trong các liên kết raw (`fm=`) và cho **Hysteria2** (`insecure=1 / skip-cert-verify: true`).
- Phạm vi «Khoảng thời gian cập nhật đăng ký» (`subUpdates`) được sửa thành **0–525600** (giới hạn UI cũ 168 từng chặn việc lưu cài đặt sau khi nâng cấp từ 2.x).
- Các client **WireGuard gốc** nay được đưa vào **đăng ký Clash và JSON** (trước đây – chỉ vào raw).

Thay đổi ở phần 11 – Xray: định tuyến, outbounds, DNS và phần mở rộng

- Trình chỉnh sửa outbound: trường mới **«Target Strategy»** (11 giá trị từ `AsIs` đến `ForceIPv4`), chế độ kiểm tra **«Real delay»** (toàn bộ thời gian bao gồm cả thiết lập tunnel; chế độ HTTP nay được đo trên kết nối «âm»), các cột **Egress** (IP đầu ra ẩn sau biểu tượng «con mắt») và **Country** (cờ + quốc gia, nhãn WARP) sau bài kiểm tra HTTP/Real.
- **Fallback của bộ cân bằng tải có thể trở đến một bộ cân bằng tải khác**: bảng điều khiển tự xây dựng đối tượng loopback ẩn (`_bl_...`), bảo vệ khỏi vòng lặp và khỏi việc xóa bộ cân bằng tải đang được sử dụng; tiền tố `_bl_` được dành riêng.

- Tab «**Định tuyến cơ bản**» nhận được bộ chọn **«Default Outbound»** – outbound nào xử lý lưu lượng không khớp với bất kỳ quy tắc nào (outbound được chọn sẽ được chuyển lên vị trí đầu tiên).
- Các máy chủ DNS trên IP riêng tư không còn bị quy tắc `geoip:private` chặn – bảng điều khiển tự duy trì một quy tắc allow được quản lý.
- Happy Eyeballs trong cài đặt dial (sockopt) nay thực sự được bật; «Try delay» mặc định 250 ms, giá trị 0 đặt rõ ràng được giữ nguyên.
- Nhập đăng ký outbound: các liên kết `ss://` có `?plugin=/dấu /` ở cuối nay được phân tích cổng chính xác.

Thay đổi ở phần 12 – Nút (đa bảng điều khiển, master/slave)

- Gói các bản sửa lỗi: lưu client không thay đổi không còn phá vỡ lưu lượng đang chạy của các inbound trên nút; các ghi đề Host của nút được nhận vào master ở lần chấp nhận đầu tiên; tự động gia hạn mở cửa sổ hạn mức mới; xóa client trên master xóa hoàn toàn client đó trên các nút; các inbound của nút không bị quét sạch trước lần chấp nhận đầu tiên; một inbound không hợp lệ không dừng việc đồng bộ lưu lượng của nút; kiểm tra xung đột cổng được giới hạn trong nút của chính nó.

Thay đổi ở phần 14 – Telegram Bot

- Menu lệnh của bot được bổ sung `/usage`, `/inbound`, `/restart` và lệnh quản trị mới `/clearall` (đặt lại lưu lượng của tất cả client, có xác nhận).
- Danh sách client trực tuyến được ghi nhãn dạng `email – remark inbound`; các tin nhắn sao lưu và nhật ký chặn chứa tên host; tìm kiếm theo Telegram ID hoạt động bất kể định dạng của cài đặt.

Thay đổi ở phần 16 – Vận hành: sao lưu, nhật ký, cập nhật, CLI

- **Khôi phục trên panel PostgreSQL chấp nhận tệp SQLite:** bản sao lưu `.db` thông thường hoặc `.dump` di chuyển được nhập thẳng vào PostgreSQL (trong một giao dịch duy nhất, với các kiểm tra trước khi dừng Xray). Hộp thoại chọn tệp chấp nhận `.dump`, `.db` trên cả hai DBMS; «Tải xuống tệp di chuyển» chỉ còn trên các panel PostgreSQL.
- Trước khi khôi phục tệp lưu trữ `pg_dump`, bảng điều khiển kiểm tra khả năng đọc của dump và khi phiên bản không khớp sẽ gợi ý lệnh chính xác `x-ui pgclient <phiên bản>`.
- Tự động sửa chữa khi khởi động: các bộ đếm lưu lượng bị tràn được kẹp lại và khôi phục; gỡ bỏ ràng buộc UNIQUE lỗi thời trên cổng inbound (từng cản trở multi-node).
- Nhật ký Xray: tác vụ mới mỗi 10 phút cắt bớt access-log và error-log khi vượt quá **64 MiB**; việc dọn dẹp hàng ngày nay làm sạch cả hai.
- Docker: tự động gia hạn chứng chỉ đã được sửa (crond được khởi chạy, trạng thái acme.sh được lưu trong volume).

1. Giới thiệu, yêu cầu và cài đặt

1.1. 3X-UI là gì

3X-UI là bảng điều khiển web mã nguồn mở dành cho máy chủ **Xray-core**. Bảng điều khiển cung cấp giao diện web đa ngôn ngữ thống nhất để triển khai, cấu hình và giám sát nhiều giao thức proxy và VPN: từ VPS đơn lẻ đến các cấu hình phân tán gồm nhiều nút (node).

3X-UI là một nhánh nâng cao của dự án X-UI gốc. So với dự án gốc, 3X-UI bổ sung hỗ trợ nhiều giao thức hơn, tăng cường độ ổn định, thống kê lưu lượng theo từng khách hàng và nhiều tính năng tiện ích khác.

Tính năng chính:

- **Inbound với nhiều giao thức** – VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN và **MTPROTO** (proxy Telegram, thêm vào từ phiên bản 3.3.0).
- **Transport và mã hóa hiện đại** – TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade và XHTTP, được bảo vệ bởi TLS, XTLS và REALITY.
- **Fallback** – phục vụ nhiều giao thức trên cùng một cổng (ví dụ: VLESS và Trojan trên cổng 443) thông qua cơ chế fallback trong Xray.
- **Quản lý theo từng khách hàng** – hạn mức lưu lượng, ngày hết hạn, giới hạn IP, hiển thị trạng thái "online", liên kết mời một chạm, mã QR và đăng ký (subscription).
- **Thống kê lưu lượng** – theo từng inbound, khách hàng và outbound, với khả năng đặt lại.
- **Hỗ trợ nhiều nút** – quản lý và mở rộng quy mô trên nhiều máy chủ từ một bảng điều khiển duy nhất.
- **Outbound và định tuyến** – WARP, NordVPN, quy tắc định tuyến tùy chỉnh, bộ cân bằng tải, chuỗi proxy.
- **Máy chủ đăng ký tích hợp** với nhiều định dạng đầu ra.
- **Bot Telegram** để giám sát và quản lý từ xa.
- **REST API** với tài liệu Swagger tích hợp.
- **Lưu trữ linh hoạt** – SQLite (mặc định) hoặc PostgreSQL.
- **Giao diện 13 ngôn ngữ**, chủ đề tối và sáng.
- **Tích hợp Fail2ban** để áp dụng giới hạn IP theo từng khách hàng.

Lưu ý quan trọng: dự án chỉ dành cho mục đích cá nhân. Không khuyến khích sử dụng cho các mục đích bất hợp pháp hoặc trong môi trường sản xuất.

1.2. Hệ điều hành và kiến trúc được hỗ trợ

Hệ điều hành

Script cài đặt xác định bản phân phối theo trường `ID` trong `/etc/os-release` (hoặc `/usr/lib/os-release`). Các hệ điều hành được hỗ trợ chính thức:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine và Windows.

Trên các hệ thống thuộc họ Alpine, dịch vụ được quản lý bằng OpenRC (`rc-service` / `rc-update`), trên các hệ thống còn lại – bằng systemd. Với CentOS 7, gói được cài đặt qua `yum`, với các phiên bản mới hơn – qua `dnf`. Nếu bản phân phối không được nhận dạng, script sẽ mặc định thử sử dụng trình quản lý gói `apt-get`.

Kiến trúc bộ xử lý

Kiến trúc được xác định theo kết quả của `uname -m` và được ánh xạ sang một trong các giá trị được hỗ trợ:

Giá trị <code>uname -m</code>	Kiến trúc 3X-UI
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

Nếu kiến trúc không thuộc danh sách này, script sẽ hiển thị thông báo «Unsupported CPU architecture!» và dừng cài đặt.

Phụ thuộc cơ bản

Trước khi cài đặt bảng điều khiển, script sẽ tự động cài đặt một bộ gói cơ bản (tên gói khác nhau tùy bản phân phối): `cron` / `cronie` / `dcron`, `curl`, `tar`, `tzdata` / `timezone`, `socat`, `ca-certificates`, `openssl`.

1.3. Các phương thức cài đặt

Phương thức 1. Script cài đặt (khuyến nghị)

Cài đặt được thực hiện bằng một lệnh với quyền root:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

Script bắt buộc yêu cầu quyền root: nếu chạy không phải với quyền root, sẽ hiển thị «Please run this script with root privilege» và kết thúc với lỗi.

Các bước mà trình cài đặt thực hiện:

1. Xác định hệ điều hành và kiến trúc.
2. Cài đặt các phụ thuộc cơ bản.
3. Tải xuống gói phát hành `x-ui-linux-<arch>.tar.gz` và giải nén vào thư mục `/usr/local/x-ui`.
4. Tải xuống script quản lý `x-ui.sh` và cài đặt nó như lệnh `/usr/bin/x-ui`.
5. Tạo thư mục log `/var/log/x-ui`.

6. Chạy cấu hình ban đầu: chọn cơ sở dữ liệu, tạo thông tin xác thực, chọn cổng, cấu hình SSL tùy chọn.
7. Cài đặt và khởi động dịch vụ tự khởi động (unit systemd `x-ui.service` hoặc script init OpenRC cho Alpine).

Chọn cơ sở dữ liệu khi cài đặt. Trình cài đặt đề xuất:

- 1) SQLite (mặc định, khuyến nghị khi số lượng khách hàng < 500) – một file `/etc/x-ui/x-ui.db`, không cần cấu hình.
- 2) PostgreSQL (khuyến nghị khi số lượng khách hàng lớn hoặc có nhiều node). PostgreSQL có thể được cài đặt cục bộ (tạo người dùng và cơ sở dữ liệu riêng với tên `xui`) hoặc chỉ định DSN đến máy chủ hiện có. Các tham số kết nối được ghi vào file môi trường của dịch vụ (`/etc/default/x-ui`, `/etc/conf.d/x-ui` hoặc `/etc/sysconfig/x-ui` tùy bản phân phối) dưới dạng biến `XUI_DB_TYPE` và `XUI_DB_DSN`.

Ví dụ: ghi tham số PostgreSQL vào file môi trường dịch vụ. Sau khi chọn PostgreSQL và chỉ định DSN, trình cài đặt sẽ thêm các dòng tương tự như sau vào file môi trường:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Ở đây `xui` là tên người dùng và cơ sở dữ liệu, `127.0.0.1:5432` – địa chỉ và cổng máy chủ, `sslmode=disable` phù hợp cho kết nối cục bộ (đối với máy chủ từ xa thường dùng `require`).

Cài đặt phiên bản cụ thể (cũ hơn). Có thể chỉ định tag phiên bản rõ ràng – trình cài đặt sẽ tải phát hành tương ứng:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

Phiên bản tối thiểu cho kiểu cài đặt này là `v2.3.5`; nếu chỉ định phiên bản cũ hơn sẽ hiển thị «Please use a newer version (at least v2.3.5)».

Cài đặt bản dev. Ngoài tag phiên bản, trình cài đặt cũng nhận đối số `dev-latest` (bí danh `dev`) – điều này cài đặt bản dev rolling theo commit mới nhất của nhánh `main`:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

Bản dev là bản phát hành trước theo từng commit (tag `dev-latest`), không phải phiên bản ổn định, vì vậy không có kiểm tra phiên bản tối thiểu cho nó. Khi chạy sẽ hiển thị cảnh báo «Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version.». Không có đối số, trình cài đặt sẽ cài phiên bản ổn định mới nhất. Chỉ nên dùng bản dev để kiểm tra các bản sửa lỗi chưa được phát hành; trong sử dụng thông thường hãy cài đặt các phiên bản ổn định.

Phương thức 2. Docker

Khởi chạy với SQLite mặc định:

```
docker compose up -d
```

Để khởi chạy với dịch vụ PostgreSQL tích hợp, cần bỏ chú thích các dòng `XUI_DB_*` trong `docker-compose.yml` và khởi chạy với profile:

```
docker compose --profile postgres up -d
```

Image bao gồm Fail2ban (mặc định được kích hoạt) để áp dụng giới hạn IP theo khách hàng. Fail2ban chặn vi phạm qua `iptables`, yêu cầu khả năng `NET_ADMIN`. Trong `docker-compose.yml` khả năng này đã được cấp qua `cap_add`. Từ 3.5.0, trong container đã sửa xong **tự động gia hạn chứng chỉ** được phát hành qua menu SSL: endpoint khởi chạy `crond` và đăng ký lại tác vụ cron của `acme.sh`, còn `docker-compose.yml` có thêm volume riêng cho trạng thái `acme.sh` – việc gia hạn vẫn hoạt động sau khi tạo lại container (trước đây chứng chỉ âm thầm hết hạn sau ~90 ngày). Khi khởi chạy thủ công qua `docker run`, cần tự thêm các khả năng này, nếu không các lệnh chặn chỉ được ghi vào log mà không được áp dụng:

Ví dụ: lệnh `docker run` đầy đủ. Phiên bản tối thiểu với chuyển tiếp cổng bảng điều khiển, khả năng mạng và volume lâu dài cho cơ sở dữ liệu:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

Volume `/etc/x-ui` lưu file `x-ui.db` giữa các lần khởi động lại container, nếu không cài đặt và tài khoản sẽ bị mất.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

Trong Docker, bảng điều khiển là tiến trình chính của container: tự khởi động được kiểm soát bởi chính sách khởi động lại container (ví dụ: `restart: unless-stopped`), không phải bởi dịch vụ bên trong container.

1.4. Lần khởi chạy đầu tiên và thông tin xác thực mặc định

Khi cài đặt lần đầu (khi vẫn đang dùng thông tin xác thực mặc định), trình cài đặt **tạo ngẫu nhiên** tên người dùng, mật khẩu và đường dẫn web, cũng như cổng:

Tham số	Cách tạo khi cài đặt	Ghi chú
Tên người dùng (Username)	chuỗi ngẫu nhiên 10 ký tự	được tạo tự động
Mật khẩu (Password)	chuỗi ngẫu nhiên 10 ký tự	được tạo tự động
Đường dẫn web bảng điều khiển (WebBasePath)	chuỗi ngẫu nhiên 18 ký tự	bảo vệ bảng điều khiển khỏi bị phát hiện qua URL gốc

Tham số	Cách tạo khi cài đặt	Ghi chú
Cổng bảng điều khiển (Port)	mặc định là cổng ngẫu nhiên trong khoảng 1024–62000; có thể đặt thủ công nếu muốn	giá trị "xuất xưởng" của webPort là 2053, nhưng trình cài đặt sẽ ghi đè

Khi cài đặt hoàn tất, script hiển thị tóm tắt kết quả: tên người dùng, mật khẩu, cổng, đường dẫn web, token API và liên kết đăng nhập sẵn sàng (Access URL) có dạng:

```
https://<tên-miền-hoặc-IP>:<cổng>/<đường-dẫn-web>
```

Nếu chứng chỉ SSL chưa được cấu hình, liên kết sẽ dùng http://, và script sẽ hiển thị cảnh báo về việc cần cấu hình SSL (mục menu 19).

Thay đổi thông tin xác thực bắt buộc. Vì thông tin đăng nhập và mật khẩu được tạo ngẫu nhiên, cần **lưu chúng ngay sau khi cài đặt**. Có thể thay đổi chúng bất cứ lúc nào qua mục menu «Reset Username & Password» (xem bên dưới) hoặc từ giao diện web trong cài đặt bảng điều khiển. Sau khi đặt lại, script nhắc nhở: «Please use the new login username and password to access the X-UI panel. Also remember them!».

Sau khi cài đặt, lệnh `x-ui` được dùng để mở menu quản lý (xem mục 1.6).

1.5. Vị trí các file

Đường dẫn	Mục đích
<code>/usr/local/x-ui/</code>	thư mục cài đặt bảng điều khiển (binary <code>x-ui</code> , script <code>x-ui.sh</code>)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	binary Xray-core (trên armv5/armv6/armv7 được đổi tên thành <code>xray-linux-arm</code>)
<code>/usr/bin/x-ui</code>	script quản lý (lệnh <code>x-ui</code>)
<code>/etc/x-ui/x-ui.db</code>	file cơ sở dữ liệu SQLite (mặc định)
<code>/var/log/x-ui/</code>	thư mục log của bảng điều khiển
<code>/etc/systemd/system/x-ui.service</code>	unit systemd của dịch vụ (không dùng cho Alpine)
<code>/etc/init.d/x-ui</code>	script init OpenRC (chỉ dùng cho Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	file biến môi trường của dịch vụ (đường dẫn tùy thuộc bản phân phối); các biến <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code> được ghi vào đây

Thư mục cơ sở dữ liệu có thể được thay đổi bằng biến môi trường `XUI_DB_FOLDER` (mặc định `/etc/x-ui`), và thư mục binary Xray – bằng biến `XUI_BIN_FOLDER` (mặc định `bin` tương đối so với thư mục bảng điều khiển). Tên file cơ sở dữ liệu là `x-ui.db`.

Ví dụ: chuyển cơ sở dữ liệu sang ổ đĩa riêng. Để lưu `x-ui.db` không phải trong `/etc/x-ui` mà ở ổ đĩa được mount, ví dụ `/data`, hãy đặt biến trong file môi trường dịch vụ và khởi động lại bảng điều khiển:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

Đường dẫn đầy đủ đến cơ sở dữ liệu sẽ là `/data/x-ui/x-ui.db`.

Các biến môi trường chính

Biến	Mục đích	Mặc định
<code>XUI_DB_TYPE</code>	backend cơ sở dữ liệu: <code>sqlite</code> hoặc <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	chuỗi kết nối PostgreSQL (khi <code>XUI_DB_TYPE=postgres</code>)	–
<code>XUI_DB_FOLDER</code>	thư mục file cơ sở dữ liệu SQLite	<code>/etc/x-ui</code>
<code>XUI_INIT_WEB_BASE_PATH</code>	đường dẫn URI ban đầu của bảng điều khiển web (chỉ khi khởi tạo lần đầu)	<code>/</code>
<code>XUI_DB_MAX_OPEN_CONNS</code>	số kết nối mở tối đa (pool PostgreSQL)	–
<code>XUI_DB_MAX_IDLE_CONNS</code>	số kết nối nhàn rỗi tối đa (pool PostgreSQL)	–
<code>XUI_ENABLE_FAIL2BAN</code>	bật áp dụng giới hạn IP qua Fail2ban	<code>true</code>
<code>XUI_LOG_LEVEL</code>	mức độ ghi log (<code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code>)	<code>info</code>
<code>XUI_DEBUG</code>	chế độ gỡ lỗi	<code>false</code>

Ví dụ: tạm thời bật ghi log chi tiết. Để chẩn đoán sự cố, hãy nâng mức log lên `debug` và khởi động lại dịch vụ:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log    # xem log gỡ lỗi
```

Sau khi chẩn đoán, hãy trả về giá trị `info` để log không phiền to.

Đường dẫn ban đầu của bảng điều khiển web qua môi trường. Biến `XUI_INIT_WEB_BASE_PATH` đặt đường dẫn URI của bảng điều khiển web (`webBasePath`) khi khởi tạo cài đặt lần đầu. Điều này tiện lợi khi triển khai trong Docker hoặc qua systemd để ngay lập tức cố định đường dẫn đăng nhập vào bảng điều khiển. Giá trị được chuẩn hóa tự động – dấu gạch chéo đầu và cuối được thêm khi cần, giá trị trống hoặc chỉ gồm khoảng trắng sẽ bị bỏ qua (khi đó áp dụng đường dẫn mặc định `/`). Biến chỉ ảnh hưởng **đến khởi tạo lần đầu**: nếu cài đặt đã được tạo, đường dẫn được thay đổi trong giao diện web hoặc qua mục menu «Reset Web Base Path».

1.6. Lệnh quản lý `x-ui` (menu script)

Sau khi cài đặt, lệnh `x-ui` (chạy với quyền root) mở menu tương tác «3X-UI Panel Management Script». Chọn mục bằng cách nhập số tương ứng (phạm vi 0–27). Nhiều mục cũng có thể dùng dưới dạng lệnh con cho script (xem mục 1.7).

Menu được chia thành các khối theo chủ đề.

Cài đặt và cập nhật

- **1. Install** – cài đặt bảng điều khiển (chạy `install.sh`). Trước khi cài đặt, kiểm tra xem bảng điều khiển đã được cài đặt chưa.
- **2. Update** – cập nhật tất cả các thành phần x-ui lên phiên bản mới nhất. Dữ liệu không bị mất; sau khi cập nhật bảng điều khiển sẽ tự động khởi động lại. Yêu cầu xác nhận.
- **3. Update Menu** – chỉ cập nhật script quản lý (`x-ui.sh` / lệnh `x-ui`) lên phiên bản mới nhất mà không cần cài lại bảng điều khiển.
- **4. Legacy Version** – cài đặt phiên bản cụ thể (cũ hơn) của bảng điều khiển. Script yêu cầu nhập số phiên bản (ví dụ: `2.4.0`) và tải phát hành tương ứng.
- **5. Uninstall** – gỡ cài đặt hoàn toàn bảng điều khiển **cùng với Xray**. Dịch vụ được dừng và vô hiệu hóa, các thư mục `/etc/x-ui/` và `/usr/local/x-ui/` , file môi trường dịch vụ và script quản lý bị xóa. Yêu cầu xác nhận (mặc định «không»).

Thông tin xác thực và cài đặt

- **6. Reset Username & Password** – đặt lại tên người dùng và mật khẩu bảng điều khiển. Có thể nhập giá trị của riêng mình hoặc để trống để tạo ngẫu nhiên (tên ngẫu nhiên – 10 ký tự, mật khẩu ngẫu nhiên – 18 ký tự). Ngoài ra đề nghị tắt xác thực hai yếu tố (2FA) nếu đã được cấu hình. Sau khi đặt lại bảng điều khiển sẽ khởi động lại.
- **7. Reset Web Base Path** – đặt lại đường dẫn web của bảng điều khiển: tạo đường dẫn ngẫu nhiên mới (18 ký tự), bảng điều khiển khởi động lại. Dừng khi đường dẫn cũ bị lộ hoặc bị quên.
- **8. Reset Settings** – đặt lại tất cả cài đặt bảng điều khiển về giá trị mặc định. **Thông tin xác thực (tên người dùng và mật khẩu) và dữ liệu tài khoản không bị mất.** Yêu cầu xác nhận; sau khi đặt lại bảng điều khiển sẽ khởi động lại.
- **9. Change Port** – thay đổi cổng bảng điều khiển web. Nhập số cổng (1–65535); sau khi đặt cần khởi động lại để cổng có hiệu lực.
- **10. View Current Settings** – xem cài đặt hiện tại (`x-ui setting -show`). Hiển thị bao gồm backend cơ sở dữ liệu đang dùng (SQLite hoặc PostgreSQL với mật khẩu được che trong DSN) và liên kết truy cập sẵn sàng (Access URL). Nếu SSL chưa được cấu hình, đề nghị phát hành chứng chỉ Let's Encrypt cho địa chỉ IP.

Quản lý dịch vụ

- **11. Start** – khởi động dịch vụ bảng điều khiển. Nếu bảng điều khiển đã chạy, hiển thị thông báo rằng không cần khởi động lại.
- **12. Stop** – dừng dịch vụ bảng điều khiển.
- **13. Restart** – khởi động lại dịch vụ bảng điều khiển.
- **14. Restart Xray** – khởi động lại chỉ nhân Xray-core mà không khởi động lại bảng điều khiển (qua `systemctl reload x-ui` , trong Docker – bằng tín hiệu `USR1` đến tiến trình bảng điều khiển).
- **15. Check Status** – kiểm tra trạng thái dịch vụ (`systemctl status x-ui` hoặc `rc-service x-ui status`).

- **16. Logs Management** – quản lý log: xem log gỡ lỗi (Debug Log, qua `journalctl`) và, ngoài Alpine, xóa tất cả log (Clear All logs).

Tự khởi động

- **17. Enable Autostart** – bật tự khởi động bảng điều khiển khi hệ điều hành khởi động (`systemctl enable x-ui` hoặc `rc-update add`).
- **18. Disable Autostart** – tắt tự khởi động khi hệ điều hành khởi động.

Trong Docker, tự khởi động được kiểm soát bởi chính sách khởi động lại container, vì vậy các mục này chỉ hiển thị gợi ý tương ứng.

Bảo mật và mạng

- **19. SSL Certificate Management** – quản lý chứng chỉ SSL qua acme.sh: phát hành chứng chỉ cho tên miền, thu hồi, gia hạn bắt buộc, xem các tên miền hiện có, chỉ định đường dẫn chứng chỉ cho bảng điều khiển, cũng như phát hành chứng chỉ ngắn hạn (~6 ngày, với tự động gia hạn) cho địa chỉ IP.
- **20. Cloudflare SSL Certificate** – phát hành chứng chỉ SSL qua xác thực DNS Cloudflare.
- **21. IP Limit Management** – quản lý giới hạn số IP theo khách hàng (dựa trên Fail2ban): xem và gỡ bỏ các lệnh chặn, v.v.
- **22. Firewall Management** – quản lý tường lửa (mở/đóng cổng và xem quy tắc).
- **23. SSH Port Forwarding Management** – cấu hình chuyển tiếp cổng SSH để mở bảng điều khiển từ máy cục bộ qua tunnel SSH.

Hiệu suất và bảo trì

- **24. Enable BBR** – bật/tắt thuật toán kiểm soát tắc nghẽn TCP BBR (menu con với các mục Enable BBR / Disable BBR).
- **25. Update Geo Files** – cập nhật cơ sở dữ liệu địa lý (file `.dat`) với chọn nguồn: Loyalsoldier (`geoip.dat`, `geosite.dat`), chocolate4u (`geoip_IR.dat`, `geosite_IR.dat`), runetfreedom (`geoip_RU.dat`, `geosite_RU.dat`) hoặc All (tất cả). Sau khi cập nhật bảng điều khiển sẽ khởi động lại.
- **26. Speedtest by Ookla** – chạy kiểm tra tốc độ mạng qua Speedtest by Ookla.
- **27. PostgreSQL Management** – quản lý phiên bản PostgreSQL tích hợp/được kết nối (bật và các thao tác liên quan).
- **0. Exit Script** – thoát khỏi menu.

1.7. Lệnh con `x-ui` (không dùng menu tương tác)

Để sử dụng trong script, lệnh `x-ui` hỗ trợ các lệnh con trực tiếp (chạy `x-ui` không có đối số sẽ mở menu):

Lệnh	Hành động
<code>x-ui</code>	mở menu quản lý
<code>x-ui start</code>	khởi động bảng điều khiển
<code>x-ui stop</code>	dừng bảng điều khiển

Lệnh	Hành động
<code>x-ui restart</code>	khởi động lại bảng điều khiển
<code>x-ui restart-xray</code>	khởi động lại Xray
<code>x-ui status</code>	trạng thái hiện tại của dịch vụ
<code>x-ui settings</code>	cài đặt hiện tại
<code>x-ui enable</code>	bật tự khởi động khi hệ điều hành khởi động
<code>x-ui disable</code>	tắt tự khởi động
<code>x-ui log</code>	xem log
<code>x-ui banlog</code>	xem log chặn Fail2ban
<code>x-ui update</code>	cập nhật bảng điều khiển
<code>x-ui update-all-geofiles</code>	cập nhật tất cả file địa lý
<code>x-ui migrateDB [file]</code>	chuyển đổi <code>.db</code> <code>?</code> <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	cài đặt phiên bản cũ
<code>x-ui install</code>	cài đặt bảng điều khiển
<code>x-ui uninstall</code>	gỡ cài đặt bảng điều khiển

1.8. Chuyển đổi SQLite sang PostgreSQL

Có thể chuyển cài đặt hiện có từ SQLite sang PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# sau đó đặt XUI_DB_TYPE và XUI_DB_DSN trong /etc/default/x-ui và khởi động lại:
systemctl restart x-ui
```

File SQLite nguồn vẫn được giữ nguyên – chỉ xóa thủ công sau khi kiểm tra backend mới hoạt động tốt.

Ví dụ: kiểm tra việc chuyển sang PostgreSQL. Sau khi chuyển đổi, hãy xác nhận bảng điều khiển thực sự đang chạy trên backend mới bằng lệnh xem cài đặt – trong kết quả đầu ra phải có PostgreSQL (mật khẩu trong DSN được che):

```
x-ui settings | grep -i -E 'db|dsn'
```

Nếu bảng điều khiển mở được và các tài khoản đầy đủ, có thể xóa `x-ui.db` nguồn.

2. Đăng nhập vào bảng điều khiển và bảo mật truy cập

Phần này mô tả tất cả những gì liên quan đến xác thực quản trị viên của bảng điều khiển 3X-UI: biểu mẫu đăng nhập, xác thực hai yếu tố (TOTP), bảo vệ chống dò mật khẩu, thay đổi thông tin đăng nhập, thay đổi đường dẫn bí mật và cổng bảng điều khiển, thời gian tồn tại phiên, cũng như đồng bộ hóa/xác thực qua LDAP.

2.1. Biểu mẫu đăng nhập

Trang đăng nhập được phục vụ tại gốc của đường dẫn bí mật (`webBasePath`). Nếu người dùng đã đăng nhập, họ sẽ tự động được chuyển hướng đến `.../panel/` . Trang có công tắc chủ đề, lựa chọn ngôn ngữ giao diện và chính biểu mẫu đăng nhập.

Các trường biểu mẫu:

Trường	Gợi ý/tiêu đề	Bắt buộc	Mô tả
Tên đăng nhập	«Имя пользователя»	Có	Tên đăng nhập của quản trị viên. Giá trị trống bị từ chối ngay ở phía máy khách, còn trên máy chủ – bằng thông báo «Введите имя пользователя».
Mật khẩu	«Пароль»	Có	Mật khẩu quản trị viên. Giá trị trống bị từ chối bằng thông báo «Введите пароль».
Mã 2FA	«Код 2FA»	Chỉ khi bật 2FA	Trường này xuất hiện chỉ khi bảng điều khiển đã bật xác thực hai yếu tố. Mã 6 chữ số từ ứng dụng xác thực.

Nút «Войти» gửi biểu mẫu đến `POST /login` .

Hành vi và thông báo:

- Khi đăng nhập thành công, hiển thị «Вход выполнен успешно» và chuyển hướng đến `.../panel/` .
- Với bất kỳ lỗi thông tin đăng nhập nào hoặc mã 2FA sai, máy chủ trả về thông báo **duy nhất**: «Неверные данные учетной записи.» (tiếng Anh: *Invalid username or password or two-factor code.*). Điều này được thực hiện có chủ ý – bảng điều khiển không gợi ý cụ thể điều gì sai (tên đăng nhập, mật khẩu hay mã), để không tạo điều kiện cho việc dò mật khẩu.
- Trường «Код 2FA» được bảng điều khiển hiện hoặc ẩn dựa trên yêu cầu `POST /getTwoFactorEnable` , trả về trạng thái 2FA hiện tại ngay cả trước khi xác thực.
- Nếu phiên máy chủ hết hạn, ở yêu cầu tiếp theo sẽ hiển thị «Сессия истекла. Войдите в систему снова» và người dùng được chuyển hướng đến trang đăng nhập.

Ghi chú về CSRF: trước khi gửi biểu mẫu, máy khách lấy CSRF token (`GET /csrf-token`); các yêu cầu `/login` và `/logout` được bảo vệ bằng kiểm tra CSRF.

Ví dụ: đăng nhập qua API. Khi 2FA tắt, chỉ cần tên đăng nhập và mật khẩu; khi 2FA bật, thêm trường `twoFactorCode` :

```
# Без 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# С включённой 2FA — добавляется 6-значный код
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

Khi thành công, máy chủ sẽ trả về `Set-Cookie` với cookie phiên — đây là cookie cần truyền trong các yêu cầu tiếp theo đến `/panel/api/...`.

2.2. Xác thực hai yếu tố (2FA / TOTP)

2FA trong 3X-UI được triển khai theo chuẩn **TOTP** và tương thích với mọi ứng dụng xác thực (Google Authenticator, Aegis, FreeOTP, v.v.). Các tham số được cố định: thuật toán **SHA1**, 6 chữ số, chu kỳ **30** giây, issuer `3x-ui`, nhân `Administrator`.

Ví dụ: otpauth URI được mã hóa trong mã QR. Nếu ứng dụng xác thực không quét được camera, có thể thêm token thủ công qua liên kết sau (thay `JBSWY3DPEHPK3XP` bằng Base32 secret của bạn):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3XP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

Các tham số `algorithm=SHA1`, `digits=6`, `period=30` tương ứng với các giá trị cố định của bảng điều khiển — không cần thay đổi chúng.

Cài đặt nằm trong phần **Настройки** → **Учетная запись**, tab «**Двухфакторная аутентификация**».

Phần tử	Văn bản	Mô tả
Công tắc	«Включить 2FA»	Bật/tắt xác thực hai yếu tố.
Mô tả	«Добавляет дополнительный уровень аутентификации для повышения безопасности.»	Gợi ý bên dưới công tắc.

Cách bật 2FA

Khi bật công tắc, bảng điều khiển **tạo cục bộ một secret mới** — chuỗi ngẫu nhiên mã hóa Base32 (bảng chữ cái `A–Z` và `2–7`). Một cửa sổ «Включить двухфакторную аутентификацию» mở ra với hướng dẫn từng bước:

1. «Отсканируйте этот QR-код в приложении для аутентификации или скопируйте токен рядом с QR-кодом и вставьте его в приложение». Bên dưới mã QR hiển thị bản thân secret dạng văn bản — khi nhập vào mã QR, secret được sao chép vào clipboard (hiển thị «Скопировано»).

2. **«Введите код из приложения»** — cần nhập mã 6 chữ số do ứng dụng tạo ra. Mã được kiểm tra **phía trình duyệt**: bảng điều khiển tự tính TOTP hiện tại theo secret vừa tạo và so sánh với mã đã nhập. Nếu mã sai — «Неверный код»; trường chỉ chấp nhận đúng 6 chữ số.

Chỉ sau khi xác nhận thành công, secret và cờ bật mới được lưu lại. Khi lưu, hiển thị «Двухфакторная аутентификация была успешно установлена».

Lưu ý: các thay đổi trong phần cài đặt được áp dụng bằng nút chung **«Сохранить»**, sau đó thường cần khởi động lại bảng điều khiển («Сохраните изменения и перезапустите панель для их применения»). Khi lần đầu bật 2FA, máy chủ còn **vô hiệu hóa tất cả các phiên đang hoạt động** (tăng «login epoch»), vì vậy sau khi áp dụng cài đặt cần đăng nhập lại — lần này đã có mã 2FA.

Cách tắt 2FA

Nhập lại công tắc sẽ mở cửa sổ «Отключить двухфакторную аутентификацию» với gợi ý «Введите код из приложения, чтобы отключить двухфакторную аутентификацию.». Sau khi nhập mã đúng (từ 3.4.2 mã được kiểm tra **ở phía máy chủ**), cờ và secret được xóa, hiển thị «Двухфакторная аутентификация была успешно удалена».

Kiểm tra mã khi đăng nhập

Khi đăng nhập, máy chủ lấy secret đã lưu và so sánh TOTP hiện tại với mã 2FA được gửi. Không khớp được coi là đăng nhập thất bại, nhưng người dùng vẫn thấy thông báo chung «Неверные данные учетной записи.».

Khôi phục quyền truy cập (recovery)

3X-UI **không có** cơ chế «mã khôi phục» riêng biệt. Nếu mất quyền truy cập vào ứng dụng xác thực, không thể khôi phục đăng nhập qua giao diện bảng điều khiển. Cách duy nhất là tắt 2FA trực tiếp trong cơ sở dữ liệu trên máy chủ: đặt lại khóa `twoFactorEnable` thành `false` (và nếu cần xóa `twoFactorToken`) trong bảng cài đặt, sau đó khởi động lại bảng điều khiển. Vì vậy, khi bật 2FA nên lưu secret (Base32 token) ở nơi an toàn.

Ví dụ: tắt khẩn cấp 2FA trên máy chủ. Sau khi truy cập máy chủ qua SSH, dùng bảng điều khiển, đặt lại các khóa trong bảng cài đặt và khởi động lại bảng điều khiển:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

Sau đó, đăng nhập chỉ bằng tên đăng nhập và mật khẩu, còn 2FA có thể được thiết lập lại nếu muốn.

Liên quan đến thay đổi thông tin đăng nhập: khi thay đổi tên đăng nhập/mật khẩu (xem 2.4), 2FA **tự động tắt** trên máy chủ để secret cũ không chặn quyền truy cập với thông tin đăng nhập mới.

2.3. Giới hạn số lần đăng nhập (login limiter / bảo vệ chống dò mật khẩu)

Bảng điều khiển có bộ giới hạn đăng nhập thất bại tích hợp sẵn (tương tự fail2ban ở cấp ứng dụng). Các tham số được cố định trong mã nguồn và **không thể cấu hình** qua giao diện:

Tham số	Giá trị	Mục đích
Số lần thất bại tối đa	5	Số lần thử thất bại cho phép trong cửa sổ thời gian.
Cửa sổ tính toán	5 phút	Cửa sổ trượt để tích lũy các lần thất bại (các lần cũ hơn bị loại bỏ).
Khóa (cooldown)	15 phút	Thời gian khóa khóa sau khi vượt ngưỡng.

Cách hoạt động:

- Khóa chặn được xây dựng từ **cặp «IP + tên đăng nhập»** (tên đăng nhập được chuyển thành chữ thường, cắt khoảng trắng). Tức là khóa chặn áp dụng cho cặp cụ thể «địa chỉ + tên người dùng», không phải toàn bộ bảng điều khiển.
- Mỗi lần thử thất bại (tên đăng nhập/mật khẩu sai hoặc mã 2FA sai), bộ đếm tăng lên. Sau **5 lần thất bại trong 5 phút**, khóa bị chặn trong **15 phút**. Trong thời gian bị chặn, mọi thử nghiệm của cặp đó ngay lập tức bị từ chối bằng thông báo «Неверные данные учетной записи.», dù thông tin có đúng.
- **Đăng nhập thành công ngay lập tức đặt lại** bộ đếm và gỡ chặn cặp đó.
- Địa chỉ IP của máy khách được xác định có tính đến proxy tin cậy (xem `trustedProxyCIDRs`): header `X-Real-IP` và `X-Forwarded-For` chỉ được chấp nhận nếu yêu cầu đến từ địa chỉ tin cậy. Nếu không, sử dụng địa chỉ kết nối thực tế, còn nếu không trích xuất được – chuỗi `unknown`.

Tất cả các lần thử đều được ghi nhật ký. Với các lần thất bại, cảnh báo được ghi vào nhật ký máy chủ kèm tên người dùng, IP, lý do và, khi bị chặn, thời gian `blocked_until`. Nếu bật thông báo đăng nhập qua bot Telegram (`tgNotifyLogin` – «Уведомление о входе»), quản trị viên còn nhận được tên người dùng, IP và thời gian của cả các lần thử thành công, thất bại và bị chặn.

Ví dụ: thông báo đăng nhập trong Telegram. Khi `tgNotifyLogin` được bật, sau mỗi lần thử, quản trị viên nhận được tin nhắn dạng như sau:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Với cặp «IP + tên đăng nhập» bị chặn, trạng thái sẽ chỉ ra rằng lần thử bị từ chối bởi bộ giới hạn.

2.4. Thay đổi tên đăng nhập và mật khẩu quản trị viên

Phần **Настройки** → **Учетная запись**, tab «Учетные данные администратора». Các trường:

Trường	Văn bản	Mô tả
Tên đăng nhập hiện tại	«Текущий логин»	Tên người dùng hiện tại. Phải khớp với tên đăng nhập hiện tại, nếu không thay đổi bị từ chối.

Trường	Văn bản	Mô tả
Mật khẩu hiện tại	«Текущий пароль»	Mật khẩu hiện tại để xác nhận danh tính.
Tên đăng nhập mới	«Новый логин»	Tên người dùng mới. Không được để trống.
Mật khẩu mới	«Новый пароль»	Mật khẩu mới. Không được để trống.

Thay đổi được áp dụng bằng nút **«Подтвердить»** và gửi đến `POST /panel/setting/updateUser`.

Logic và thông báo từ máy chủ:

- Nếu «Текущий логин» không khớp với thực tế hoặc «Текущий пароль» sai – «Произошла ошибка при изменении учетных данных администратора.» với giải thích «Неверное имя пользователя или пароль».
- Nếu tên đăng nhập mới hoặc mật khẩu mới trống – giải thích «Новое имя пользователя и новый пароль должны быть заполнены».
- Khi thành công – «Вы успешно изменили учетные данные администратора.». Mật khẩu được lưu dưới dạng bcrypt hash.

Xác nhận bằng mã 2FA (từ 3.4.2). Nếu bảng điều khiển đã bật xác thực hai yếu tố, việc thay đổi tên đăng nhập/mật khẩu còn yêu cầu thêm việc **nhập mã hiện tại** từ ứng dụng xác thực. Cửa sổ **«Thay đổi thông tin đăng nhập»** mở ra với gợi ý «Введите код из приложения, чтобы изменить учётные данные администратора.»; mã (`twoFactorCode`) được kiểm tra ở phía máy chủ, khi mã sai hoặc trống thì thay đổi không được áp dụng. Cùng một xác nhận đó nay cũng được yêu cầu khi tắt 2FA (xem 2.2).

Ví dụ: thay đổi thông tin đăng nhập qua API. Yêu cầu cần có cookie phiên hợp lệ (lấy khi đăng nhập) và xác nhận tên đăng nhập/mật khẩu hiện tại:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=БАША_СЕССИОННАЯ_КОOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

Sau khi thành công, phiên hiện tại bị hủy – cần đăng nhập lại với thông tin đăng nhập mới.

Các hiệu ứng quan trọng khi thay đổi thông tin đăng nhập:

- **Tất cả các phiên hiện có đều bị hủy** (tăng bộ đếm `login_epoch` của người dùng), do đó sau khi thay đổi, bảng điều khiển tự động đăng xuất và chuyển hướng đến trang đăng nhập – cần đăng nhập lại.
- Nếu tại thời điểm thay đổi **2FA đang được bật, nó tự động tắt** (cờ và secret được đặt lại). Xác thực hai yếu tố sau khi thay đổi tên đăng nhập/mật khẩu cần được thiết lập lại.

Nếu 2FA đang bật, trước khi gửi biểu mẫu sẽ mở cửa sổ «Изменить учетные данные» với gợi ý «Введите код из приложения, чтобы изменить учетные данные администратора.» – chỉ có thể thay đổi thông tin đăng nhập sau khi xác nhận mã 2FA hiện tại.

2.5. Đường dẫn bí mật (URI path / webBasePath) và cổng bảng điều khiển

Các tham số này nằm trong phần **Настройки** → **Панель** và ảnh hưởng trực tiếp đến khả năng «ẩn» và tính khả dụng của bảng điều khiển. Được áp dụng sau khi lưu và **khởi động lại bảng điều khiển**.

Trường	Văn bản	Giá trị mặc định	Mô tả
Cổng bảng điều khiển	«Порт панели» (<code>panelPort</code>), gợi ý «Порт, на котором работает панель»	2053	Cổng TCP của giao diện web.
URI path	«URI-путь» (<code>panelUrlPath</code>), gợi ý «Должен начинаться с '/' и заканчиваться '/'»	/	Đường dẫn cơ sở bí mật (<code>webBasePath</code>). Bảng điều khiển chỉ có thể truy cập qua đường dẫn này (ví dụ: <code>/мой-секрет/</code>).
Địa chỉ IP để quản lý bảng điều khiển	«IP-адрес для управления панелью» (<code>panelListeningIP</code>), gợi ý «Оставьте пустым для подключения с любого IP»	trống	Địa chỉ mà bảng điều khiển lắng nghe. Trống = tất cả giao diện.
Tên miền bảng điều khiển	«Домен панели» (<code>panelListeningDomain</code>), gợi ý «Оставьте пустым для подключения с любых доменов и IP.»	trống	Giới hạn truy cập theo tên miền (Host).
Đường dẫn đến khóa công khai chứng chỉ bảng điều khiển	<code>publicKeyPath</code> , gợi ý «Введите полный путь, начинающийся с '/'»	trống	Chứng chỉ TLS để truy cập HTTPS vào bảng điều khiển.
Đường dẫn đến khóa riêng tư chứng chỉ bảng điều khiển	<code>privateKeyPath</code> , gợi ý tương tự	trống	Khóa riêng tư TLS.

Hành vi của đường dẫn cơ sở (`webBasePath`):

- Giá trị được chuẩn hóa tự động: nếu không bắt đầu bằng `/` , ký tự được thêm vào đầu; nếu không kết thúc bằng `/` , được thêm vào cuối. Tức là đường dẫn thực tế luôn có dạng `/.../` .
- Đường dẫn cơ sở áp dụng cho bản thân bảng điều khiển, cho các asset và cho cookie phiên (cookie chỉ được cấp cho đường dẫn này).

Khuyến nghị bảo mật (phần «Предупреждения безопасности»): bảng điều khiển tự hiển thị cảnh báo nếu cấu hình «quá công khai»:

- «Панель работает по обычному HTTP – настройте TLS для продакшна.»
- «Стандартный порт 2053 широко известен – измените его на случайный.»
- «Базовый путь по умолчанию "/" широко известен – измените его на случайный.»

Nói cách khác, đối với máy chủ thực tế, cần đặt **cổng không chuẩn**, **URI path không tầm thường** và **chứng chỉ TLS**.

Ví dụ: cấu hình bảng điều khiển «[ẩn](#)» cho môi trường sản xuất. Trong phần **Настройки** → **Панель**, đặt các giá trị xấp xỉ như sau:

Trường	Giá trị
Cổng bảng điều khiển	34571 (ngẫu nhiên, thay vì 2053)
URI path	/aXf9Qm2/ (không tasm thường, bắt đầu và kết thúc bằng /)
Đường dẫn đến khóa công khai chứng chỉ bảng điều khiển	/etc/letsencrypt/live/panel.example.com/fullchain.pem
Đường dẫn đến khóa riêng tư chứng chỉ bảng điều khiển	/etc/letsencrypt/live/panel.example.com/privkey.pem

Sau khi lưu và khởi động lại, bảng điều khiển chỉ có thể truy cập qua `https://panel.example.com:34571/aXf9Qm2/`, và các cảnh báo bảo mật sẽ biến mất.

2.6. Thời gian tồn tại phiên (timeout)

Trường **«Продолжительность сессии»** (`sessionMaxAge`) nằm trong phần cài đặt bảng điều khiển/khoảng thời gian.

Trường	Văn bản	Giá trị mặc định	Đơn vị	Mô tả
Thời gian tồn tại phiên	«Продолжительность сессии», gợi ý «Продолжительность сессии в системе (значение: минута)»	360	phút	Thời gian tồn tại của cookie phiên quản trị viên.

Hành vi:

- Giá trị được đặt bằng **phút** (mặc định 360 phút = 6 giờ) và khi cấu hình cookie được chuyển đổi sang giây.
- Nếu giá trị **lớn hơn 0**, cookie phiên được đặt `MaxAge` tương ứng. Sau khi hết thời hạn này, cookie ngừng hoạt động và ở yêu cầu tiếp theo, người dùng nhận được «Сессия истекла. Войдите в систему снова».
- Phiên cũng trở nên không hợp lệ sớm hơn khi thay đổi thông tin đăng nhập hoặc lần đầu bật 2FA (thông qua cơ chế `login_epoch`, xem 2.4 và 2.2) và khi đăng xuất tường minh (`POST /logout`).
- Cookie phiên được đánh dấu `HttpOnly`, với chính sách `SameSite=Lax`; cờ `Secure` được đặt khi truy cập HTTPS trực tiếp vào bảng điều khiển.

Ngoài chính timeout còn có thông báo liên quan: **«Задержка уведомления об истечении сессии»** (`expireTimeDiff`, gợi ý «Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)», mặc định `0`) – cho phép nhận cảnh báo trước.

2.7. LDAP (đồng bộ hóa và xác thực)

Phần LDAP cung cấp hai khả năng: (1) xác thực đăng nhập quản trị viên qua LDAP nếu mật khẩu cục bộ không khớp, và (2) định kỳ đồng bộ hóa trạng thái của các client (bật/tắt cờ VLESS) từ thư mục.

Cách sử dụng khi đăng nhập: máy chủ trước tiên kiểm tra bcrypt hash mật khẩu cục bộ. Nếu **không khớp** và LDAP được bật, bảng điều khiển cố gắng xác thực người dùng trong thư mục: với **Bind DN** được đặt, thực hiện bind dịch vụ, sau đó theo bộ lọc và thuộc tính tìm kiếm bản ghi người dùng, và thử bind dưới DN tìm thấy với mật khẩu đã nhập. Thành công có nghĩa là đăng nhập. (Sau khi xác thực LDAP thành công, nếu 2FA được bật, mã TOTP vẫn được kiểm tra.)

Các trường trong phần:

Trường	Văn bản	Giá trị mặc định	Mô tả
Bật đồng bộ hóa LDAP	«Включить LDAP-синхронизацию» (enable)	false	Công tắc chính của tích hợp LDAP.
LDAP host	«LDAP-хост» (host)	trống	Địa chỉ máy chủ LDAP.
Cổng LDAP	«Порт LDAP» (port)	389	Cổng. Đối với LDAPS thường là 636.
Sử dụng TLS (LDAPS)	«Использовать TLS (LDAPS)» (useTls)	false	Khi bật, sử dụng scheme ldaps:// (mặc định – với kiểm tra chứng chỉ máy chủ).
Bỏ qua kiểm tra chứng chỉ TLS	«Бỏ qua kiểm tra chứng chỉ TLS» (ldapInsecureSkipVerify)	false	Thêm vào 3.4.2. Tắt việc kiểm tra chứng chỉ máy chủ khi dùng LDAPS – dành cho CA nội bộ/tự ký. Gợi ý: «Không an toàn – tắt việc kiểm tra chứng chỉ máy chủ. Chỉ dùng với các CA nội bộ/ không tin cậy». Công tắc chỉ khả dụng khi đã bật «Sử dụng TLS (LDAPS)».
Bind DN	«Bind DN» (bindDn)	trống	DN của tài khoản dịch vụ để bind/tìm kiếm ban đầu. Nếu trống – không thực hiện bind (tìm kiếm ẩn danh).
Mật khẩu bind	gợi ý: «Настроено; оставьте пустым, чтобы сохранить текущий пароль.» / «Не настроено.» / «Настроено – введите новое значение для замены»	trống	Mật khẩu cho Bind DN . Được lưu riêng; để giữ nguyên mật khẩu cũ, để trống trường này.
Base DN	«Base DN» (baseDn)	trống	Gốc của cây con nơi thực hiện tìm kiếm (tìm kiếm đệ quy, toàn bộ cây con).
Bộ lọc người dùng	«Фильтр пользователя» (userFilter)	(objectClass=person)	Bộ lọc LDAP để chọn tài khoản. Khi xác thực, tên đăng nhập được thay vào bộ lọc có escaping.
Thuộc tính người dùng (username/email)	«Атрибут пользователя (username/email)» (userAttr)	mail	Thuộc tính được so khớp với tên đăng nhập/định danh client (ví dụ: mail hoặc uid).

Trường	Văn bản	Giá trị mặc định	Mô tả
Thuộc tính cờ VLESS	«Атрибут VLESS-флага» (<code>vlessField</code>)	<code>vless_enabled</code>	Thuộc tính xác định xem quyền truy cập VLESS của client có nên được bật hay không.
Thuộc tính cờ chung (tùy chọn)	«Общий атрибут флага (опц.)» (<code>flagField</code>), gợi ý «Если задано, переопределяет флаг VLESS — напр. <code>shadowInactive</code> .»	trống	Nếu được đặt, dùng thay cho <code>vless_enabled</code> .
Giá trị truthy	«Truthy-значения» (<code>truthyValues</code>), gợi ý «Через запятую; по умолчанию: <code>true,1,yes,on</code> »	<code>true,1,yes,on</code>	Danh sách giá trị của thuộc tính cờ được coi là «bật».
Đảo ngược cờ	«Инвертировать флаг» (<code>invertFlag</code>), gợi ý «Включите, когда атрибут означает «отключено» (напр. <code>shadowInactive</code>).»	<code>false</code>	Đảo ngược ý nghĩa của cờ.
Lịch đồng bộ hóa	«Расписание синхронизации» (<code>syncSchedule</code>), gợi ý «Строка типа cron, напр. <code>@every 1m</code> »	<code>@every 1m</code>	Tần suất đồng bộ hóa theo định dạng cron.
Tag các inbound	«Теги входящих» (<code>inboundTags</code>), gợi ý «Входящие, на которых LDAP-синхронизация может авто-создавать или авто-удалять клиентов.»	trống	Giới hạn các inbound nào được phép thực hiện thao tác tự động. Nếu không có inbound: «Входящие не найдены. Сначала создайте входящий.»
Tự động tạo client	«Авто-создание клиентов» (<code>autoCreate</code>)	<code>false</code>	Tạo client trong các inbound đã chỉ định nếu client xuất hiện trong thư mục.
Tự động xóa client	«Авто-удаление клиентов» (<code>autoDelete</code>)	<code>false</code>	Xóa client nếu client biến mất khỏi thư mục.
Dung lượng mặc định (GB)	«Объём по умолчанию (ГБ)» (<code>defaultTotalGb</code>)	<code>0</code>	Giới hạn lưu lượng cho các client được tạo tự động (0 = không giới hạn).
Thời hạn mặc định (ngày)	«Срок по умолчанию (дни)» (<code>defaultExpiryDays</code>)	<code>0</code>	Thời hạn cho các client được tạo tự động (0 = vĩnh viễn).
Giới hạn IP mặc định	«Лимит IP по умолчанию» (<code>defaultIpLimit</code>)	<code>0</code>	Giới hạn số lượng IP đồng thời (0 = không giới hạn).

Đặc điểm logic của cờ đồng bộ hóa: khi đọc thuộc tính cờ (`flagField` , mặc định `vless_enabled`), giá trị được coi là «bật» nếu nó nằm trong danh sách giá trị truthy; khi bật đảo ngược, kết quả bị đảo ngược. Thuộc tính người dùng (`userAttr`) được dùng làm khóa so khớp (email/tên) – các bản ghi không có giá trị của thuộc tính này bị bỏ qua.

Bảo mật: khuyến nghị bật **TLS (LDAPS)** để mật khẩu bind và mật khẩu được kiểm tra không bị truyền dưới dạng văn bản rõ, còn với Bind DN nên sử dụng tài khoản có quyền đọc tối thiểu cần thiết.

Ví dụ: cấu hình LDAP điển hình để đồng bộ hóa (Active Directory). Điền các trường trong phần cho một thư mục nơi trạng thái truy cập được lưu trong thuộc tính tương tự cờ `userAccountControl`, và so khớp theo email:

Trường	Giá trị
LDAP host	<code>ldap.example.com</code>
Cổng LDAP	<code>636</code>
Sử dụng TLS (LDAPS)	bật
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
Bộ lọc người dùng	<code>(objectClass=person)</code>
Thuộc tính người dùng (username/email)	<code>mail</code>
Thuộc tính cờ VLESS	<code>vless_enabled</code>
Giá trị truthy	<code>true,1,yes,on</code>
Lịch đồng bộ hóa	<code>@every 5m</code>

Với cấu hình này, cứ mỗi 5 phút, bảng điều khiển sẽ duyệt qua cây con `OU=Users`, so khớp client theo `mail` và bật/tắt quyền truy cập VLESS theo giá trị `vless_enabled`.

3. Tổng quan / Bảng điều khiển

Bảng điều khiển (*Overview*) là trang khởi động của panel. Nó hiển thị trạng thái máy chủ và tiến trình Xray theo thời gian thực. Tất cả các chỉ số đều đến từ phía máy chủ. Bộ lập lịch nền tái tạo ảnh chụp **mỗi 2 giây** và phát đến tất cả các tab đang mở qua WebSocket; mỗi phút một lần, các hàng chỉ số tích lũy được ghi xuống đĩa. HTTP endpoint `GET /status` trả về ảnh chụp được lưu trong bộ nhớ đệm gần nhất.

Bên dưới là phân tích từng chỉ số và từng phần tử điều khiển trên trang.

Trong menu bên của bảng điều khiển (và trong ngăn kéo trên di động), kể từ phiên bản 3.4.2 có nút «**Tài liệu**» (biểu tượng quyển sách) – mở tài liệu chính thức <https://docs.sanaei.dev/>. Ngoài ra, trong 3.4.2 đã thực hiện cải tiến **khả năng truy cập** xuyên suốt: các nút biểu tượng và các phần tử có thể nhấp đã nhận được nhãn aria và khả năng kích hoạt bằng Enter/Space (cho trình đọc màn hình và điều hướng bằng bàn phím).

3.1. Nguyên tắc chung thu thập dữ liệu

- Ảnh chụp được thu thập bằng thư viện `gopsutil`. Nếu một phép đo cụ thể thất bại, trường đó sẽ giữ giá trị bằng không và một cảnh báo được ghi vào nhật ký (`get cpu percent failed`, `get uptime failed`, v.v.) – điều này không làm sập toàn bộ bảng điều khiển, chỉ là ô tương ứng sẽ hiển thị 0/N-A.
- Tốc độ "tức thời" (CPU %, mạng, I/O đĩa) được tính là hiệu số giữa ảnh chụp hiện tại và ảnh chụp trước đó, chia cho khoảng thời gian tính bằng giây. Do đó, khi tải trang lần đầu, giá trị tốc độ có thể bằng 0 cho đến khi phép đo thứ hai được tích lũy.
- Lịch sử có thể xem trong mục "Lịch sử hệ thống" (*System History*) – các biểu đồ được xây dựng dựa trên cùng các hàng dữ liệu được mô tả dưới đây (xem mục 3.12).

3.2. CPU

Ô "CPU" (*CPU*) hiển thị mức sử dụng bộ xử lý hiện tại theo phần trăm, cũng như các thông số của bộ xử lý.

Chỉ số	Mô tả
Mức sử dụng CPU, %	Tỷ lệ thời gian xử lý bị chiếm dụng trong khoảng thời gian cuối cùng. Được làm mịn bằng trung bình hàm mũ (EMA, hệ số $\alpha = 0.3$) để các biến động không làm giật chỉ báo. Giá trị luôn được giới hạn trong khoảng 0–100 %. Tại lần đo đầu tiên, trả về 0 (khởi tạo điểm cơ sở).
Bộ xử lý logic	Số nhân logic – tức là có tính đến Hyper-Threading.
Nhân vật lý	Số nhân vật lý.
Tần số	Tần số cơ sở của bộ xử lý tính bằng MHz. Được truy vấn theo cơ chế lazy và được lưu vào bộ nhớ đệm: phép đo thành công đầu tiên được lưu lại, lần thử lại không thực hiện thường xuyên hơn 5 phút một lần, và bản thân truy vấn bị giới hạn bởi thời gian chờ 1,5 giây (trên một số hệ thống, truy vấn tần số phản hồi chậm).

Mức sử dụng CPU được tính theo thuật toán như sau: nếu có triển khai nền tảng gốc, nó sẽ được sử dụng; nếu không – tính toán theo các delta của bộ đếm thời gian bộ xử lý (busy / total). Thời gian Guest và GuestNice bị loại trừ để tránh tính hai lần.

3.3. Bộ nhớ (RAM)

Ô "Bộ nhớ" (RAM) hiển thị đã sử dụng và tổng cộng. Được hiển thị dưới dạng "đã sử dụng / tổng cộng" và/hoặc phần trăm lấp đầy. Phần trăm được ghi vào lịch sử.

3.4. Bộ nhớ đệm trao đổi (Swap)

Ô "Swap" (Swap) hiển thị đã sử dụng và tổng cộng. Nếu tệp/phân vùng swap không được cấu hình (tổng cộng = 0), chỉ số bằng không; khi không có swap, giá trị 0 được ghi vào hàng lịch sử.

3.5. Đĩa (Storage)

Ô "Đĩa" (Storage) hiển thị đã sử dụng và tổng cộng, trong đó chỉ tính đến **phân vùng gốc** /. Phần trăm lấp đầy được ghi vào lịch sử "Sử dụng đĩa" (Disk Usage). I/O đĩa (đọc / ghi, byte/s) được thu thập riêng dưới dạng delta của bộ đếm theo khoảng thời gian – được hiển thị trên tab "Disk I/O" của lịch sử.

3.6. Thời gian hoạt động của hệ thống (Uptime)

Chỉ số "Thời gian hoạt động của hệ thống" (Uptime). Đây là thời gian kể từ khi khởi động **toàn bộ máy chủ** (tính bằng giây), không phải thời gian hoạt động của panel hoặc Xray. Thời gian hoạt động của tiến trình Xray được lưu riêng (xem mục 3.9), cũng như số luồng của panel (trong bản dịch – "Luồng" / Threads).

Bộ nhớ do panel chiếm dụng

Cùng với các chỉ số của tiến trình panel, lượng RAM mà chính tiến trình 3X-UI chiếm dụng cũng được hiển thị. Giá trị này được lấy từ RSS thực tế của tiến trình (như hệ điều hành nhìn thấy) và khớp với những gì các công cụ hệ thống hiển thị. Con số giảm xuống khi bộ nhớ được giải phóng. Trước đây, panel hiển thị bộ đếm nội bộ của Go, vốn phóng đại mức sử dụng bộ nhớ (ví dụ: ~300 MB trên máy chủ nhàn rỗi với một khách hàng) và không bao giờ giảm – hiện tại hiện tượng này không còn nữa. Ngoài ra, một tiến trình nền định kỳ trả lại bộ nhớ không sử dụng cho hệ điều hành để chỉ số phản ánh mức tiêu thụ thực tế.

3.7. Tải hệ thống (Load average)

Khối "Tải hệ thống" (System Load) – mảng gồm ba số [Load1, Load5, Load15]. Chú thích gợi ý: "Trung bình tải hệ thống trong 1, 5 và 15 phút vừa qua" (System load average for the past 1, 5, and 15 minutes). Biểu đồ lịch sử được gọi là "Trung bình tải hệ thống (1 / 5 / 15 phút)". Các giá trị được ghi riêng vào các hàng lịch sử: load1, load5, load15.

Đây là chỉ số Unix tiêu chuẩn: số trung bình các tiến trình đang chờ trong hàng đợi thực thi. Mốc tham chiếu – so sánh với số nhân: tải liên tục vượt quá số nhân vật lý cho thấy máy chủ đang bị quá tải.

3.8. Mạng: tốc độ và tổng lưu lượng

Chỉ tính đến **các giao diện vật lý**. Các giao diện ảo và đường hầm bị loại trừ: đó là lo / lo0, cũng như tất cả những gì bắt đầu bằng loopback, docker, br-, veth, virbr, tun, tap, wg, tailscale, zt. Các giá trị được tổng hợp trên tất cả các giao diện còn lại.

Tốc độ tổng thể (*Overall Speed*) – tốc độ tức thời, delta của bộ đếm theo khoảng thời gian:

Chỉ số	Mô tả
Tải lên (nhấn "Tải lên" / <i>Upload</i>)	Tốc độ gửi đi, byte/s.
Tải xuống (nhấn "Tải xuống" / <i>Download</i>)	Tốc độ nhận vào, byte/s.

Tổng lưu lượng (*Total Data*) – bộ đếm tích lũy từ khi hệ thống khởi động:

Chỉ số	Mô tả
Đã gửi (nhấn "Đã gửi" / <i>Sent</i>)	Tổng số byte đã gửi.
Đã nhận (nhấn "Đã nhận" / <i>Received</i>)	Tổng số byte đã nhận.

Tốc độ gói tin (gói/s) và tổng bộ đếm gói tin cũng được thu thập riêng – chúng được hiển thị trên tab "Gói tin mạng" (*Network Packets*) của lịch sử. Các hàng lịch sử mạng: `netUp` , `netDown` , `pktUp` , `pktDown` .

3.9. Địa chỉ IP của máy chủ

Khối "Địa chỉ IP của máy chủ" (*IP Addresses*) hiển thị `IPv4` và `IPv6` . Các địa chỉ bên ngoài được xác định qua các dịch vụ bên thứ ba (`api4.ipify.org` , `ipv4.icanhazip.com` , `v4.api.ipinfo.io/ip` , `ipv4.myexternalip.com/raw` , `4.ident.me` , `check-host.net/ip` cho `IPv4` và các dịch vụ tương tự cho `IPv6`). Danh sách được duyệt tuần tự cho đến phản hồi thành công đầu tiên; thời gian chờ cho mỗi yêu cầu là 3 giây.

Đặc điểm:

- Kết quả được **lưu vào bộ nhớ đệm** trong suốt vòng đời của tiến trình: địa chỉ đã xác định thành công sẽ không được truy vấn lại.
- Nếu không có dịch vụ nào phản hồi, trường sẽ hiển thị `N/A` . Đối với `IPv6`, sau lần `N/A` đầu tiên, các yêu cầu `IPv6` bị tắt hoàn toàn để không lãng phí thời gian trên các mạng không có `IPv6`.
- Bên cạnh có nút "mắt" để ẩn/hiển thị địa chỉ – gợi ý "Ẩn hoặc hiện địa chỉ IP của máy chủ" (*Toggle visibility of the IP*). Đây chỉ là ẩn hiển thị trong giao diện (ví dụ: để chụp màn hình), không ảnh hưởng đến bản thân các địa chỉ.

3.10. Kết nối TCP/UDP

Khối "Số lượng kết nối" (*Connection Stats*) hiển thị tổng số kết nối TCP và UDP đang hoạt động trên máy chủ (toàn bộ hệ thống, không chỉ Xray). Biểu đồ lịch sử – "Kết nối đang hoạt động (TCP / UDP)" (*Active Connections*), các hàng `tcpCount` , `udpCount` .

3.11. Trạng thái Xray và quản lý tiến trình

Thẻ "Xray" hiển thị trạng thái của tiến trình Xray-core và cho phép quản lý nó.

Các trạng thái

Giá trị	Nhãn	Khi nào được thiết lập
running	"Đang chạy" (Running)	Tiến trình Xray đang chạy.
stop	"Đã dừng" (Stopped)	Tiến trình không chạy và không có lỗi khởi động nào được ghi nhận.
error	"Lỗi" (Error)	Tiến trình không chạy, nhưng đã ghi nhận lỗi khởi động. Văn bản lỗi được hiển thị trong cửa sổ bật lên với tiêu đề "Đã xảy ra lỗi khi chạy Xray" (An error occurred while running Xray).
–	"Không xác định" (Unknown)	Hiển thị khi trạng thái chưa được nhận.

Phiên bản Xray được hiển thị bên cạnh trạng thái.

Các nút điều khiển

- **Dừng (Stop).** Gọi `POST /stopXrayService`. Khi thành công, panel phát trạng thái `stop` qua WebSocket và thông báo "Xray đã dừng thành công" (*Xray service has been stopped*), khi lỗi – trạng thái `error` kèm văn bản. Lưu ý: nếu panel được truy cập *thông qua* chính Xray, việc dừng Xray có thể ngắt kết nối đến panel – khi kết nối trực tiếp đến panel thì không có vấn đề.
- **Khởi động lại (Restart).** Gọi `POST /restartXrayService`. Trước khi thực hiện, hộp thoại xác nhận "Khởi động lại xray?" được hiển thị với giải thích "Tải lại dịch vụ xray với cấu hình đã lưu". Khi thành công – trạng thái `running` và thông báo "Xray đã khởi động lại thành công" (*Xray service has been restarted successfully*). Khởi động lại áp dụng cấu hình đã lưu hiện tại – hãy sử dụng sau khi thay đổi cài đặt.

Lưu ý. Trong fork này, bảng điều khiển đã thêm tính năng quản lý đầy đủ Start / Stop / Restart cho tất cả các loại xác thực; trong UI 3x-ui gốc không có nút "khởi động" riêng biệt – việc khởi động được thực hiện bằng cách khởi động lại.

Nút xem nhật ký Xray

Trên thẻ Xray có nút xem nhật ký Xray (*Logs*). Nó chỉ xuất hiện khi nhật ký truy cập (access log) được cấu hình trong cấu hình Xray: trình xem tích hợp đọc chính xác tệp đó, vì vậy nếu không có access log thì nút bị ẩn. Khả năng hiển thị của nút được liên kết với cờ riêng `accessLogEnable` và không còn phụ thuộc vào giới hạn IP – danh sách trực tuyến và giới hạn địa chỉ IP tiếp tục hoạt động ngay cả khi không có access log (xem mục 8).

Chọn phiên bản Xray

Mục "Chọn phiên bản" (*Version*) cho phép chuyển Xray-core sang một bản phát hành khác. Danh sách phiên bản được tải qua `GET /getXrayVersion` :

- Nguồn – GitHub API của kho `XTLS/Xray-core` (`/releases`). Các yêu cầu được lưu vào bộ nhớ đệm trong **15 phút**; khi GitHub gặp sự cố, danh sách nhận được thành công gần nhất sẽ được trả về để picker không bị trống.
- Chỉ các bản phát hành dạng `X.Y.Z` và **không cũ hơn 26.4.25** mới được đưa vào danh sách.

Gợi ý: "Chọn phiên bản bạn muốn chuyển sang" (*Choose the version you want to switch to.*) và cảnh báo "Quan trọng: các phiên bản cũ có thể không hỗ trợ các cài đặt hiện tại" (*Choose carefully, as older versions may not be compatible with current configurations.*).

Chuyển đổi: `POST /installXray/:version` . Kịch bản:

Ví dụ. Chuyển sang một phiên bản Xray-core cụ thể (cookie phiên phải đã được lấy khi xác thực):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Trong đó `v25.6.8` – thẻ từ danh sách do `GET /getXrayVersion` trả về. Phiên bản phải có trong danh sách này, nếu không panel sẽ từ chối. Kể từ phiên bản 3.4.2, phiên bản Xray tối thiểu được phép cài đặt được nâng lên **26.6.27**, còn đi kèm 3.5.0 là lỗi **Xray 26.7.11**. Các thay đổi đi kèm của lỗi: các phương thức mã hóa Shadowsocks `none` / `plain` và VMess `none` / `zero` đã bị xóa (các cấu hình đã lưu được ghi lại tự động: SS – sang phương thức mã hóa được hỗ trợ, VMess – sang `auto`), còn VLESS/Trojan-outbound không mã hóa đến địa chỉ công khai bị từ chối khi lưu – lỗi với cấu hình như vậy sẽ không khởi động được.

1. Phiên bản đã chọn được kiểm tra xem có trong danh sách phát hành hiện tại không (nếu không – từ chối).
2. Xray dừng lại.
3. Tập lưu trữ `Xray-<os>-<arch>.zip` được tải xuống từ GitHub phù hợp với hệ điều hành và kiến trúc hiện tại (hỗ trợ amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x; đối với Windows – `xray.exe`). Kích thước tập lưu trữ và tệp nhị phân được giới hạn ở 200 MB.
4. Tệp nhị phân được thay thế nguyên tử (thông qua tệp tạm thời + đổi tên) và được đánh dấu là có thể thực thi.
5. Xray khởi động lại.

Trước khi chuyển đổi, hộp thoại "Chuyển phiên bản Xray" (*Do you really want to change the Xray version?*) được hiển thị với mô tả "Điều này sẽ thay đổi phiên bản Xray thành #version#". Khi thành công – thông báo "Xray đã cập nhật thành công" (*Xray updated successfully*).

3.12. Cập nhật panel (3X-UI)

Khởi kiểm tra cập nhật panel. Dữ liệu đến qua `GET /getPanelUpdateInfo` :

Trường	Mô tả
Phiên bản panel hiện tại	Phiên bản panel đang được cài đặt.
Phiên bản panel mới nhất	Bản phát hành 3x-ui mới nhất lấy từ GitHub.
Có bản cập nhật	Dấu hiệu cho thấy phiên bản mới nhất mới hơn phiên bản hiện tại. Nếu không cần cập nhật – hiển thị "Panel đã được cập nhật" / "Đã cập nhật".

Nút **"Cập nhật panel"** (*Update Panel*) khởi động `POST /updatePanel`. Gợi ý: "Thao tác này sẽ cập nhật 3X-UI lên bản phát hành mới nhất và khởi động lại dịch vụ panel". Trước khi khởi động – xác nhận "Bạn có thực sự muốn cập nhật panel không?" với văn bản "Thao tác này sẽ cập nhật 3X-UI lên phiên bản #version# và khởi động lại dịch vụ panel".

Đặc điểm và giới hạn:

- Tự cập nhật chỉ được hỗ trợ **trên Linux** (trên các hệ điều hành khác sẽ trả về lỗi).
- Script cập nhật được tải xuống từ kho chính thức (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh` , giới hạn 2 MB) và được chạy thông qua `bash` , nếu có thể thì chạy biệt lập thông qua `systemd-run` .
- Khi khởi động thành công, hiển thị "Quá trình cập nhật panel đã bắt đầu" (*Panel update started*); nếu kiểm tra cập nhật thất bại – "Kiểm tra cập nhật panel thất bại". Trong quá trình cài đặt, cảnh báo "Đang cài đặt. Đừng làm mới trang" được hiển thị.

3.13. Cập nhật tệp địa lý (GeoIP / GeoSite)

Nút/hộp thoại cập nhật cơ sở dữ liệu địa lý gọi `POST /updateGeofile` (tất cả tệp) hoặc `POST /updateGeofile/:fileName` (một tệp). Việc cập nhật hoạt động theo danh sách trắng nghiêm ngặt về tên và nguồn:

Tệp	Nguồn
<code>geoip.dat</code> , <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> , <code>geosite_IR.dat</code>	<code>chocolate4u/iran-v2ray-rules</code> (latest)
<code>geoip_RU.dat</code> , <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> (latest)

Hành vi:

- Tên tệp được xác thực: cấm `..` , dấu gạch chéo, đường dẫn tuyệt đối; chỉ cho phép `[a-zA-Z0-9._-]+.dat` . Các tệp ngoài danh sách trắng sẽ không được tải xuống.
- Yêu cầu có điều kiện `If-Modified-Since` được sử dụng: nếu tệp trên máy chủ nguồn không thay đổi (HTTP 304), tệp sẽ không được tải lại, chỉ cập nhật dấu thời gian.
- Sau khi tải xuống, Xray **khởi động lại** (để nhận cơ sở dữ liệu mới).

Ví dụ. Chỉ cập nhật cơ sở dữ liệu địa lý của Nga mà không ảnh hưởng đến các tệp khác:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoiP_RU.dat' -b
cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b
cookie.txt
```

Để cập nhật tất cả tệp trong danh sách trắng cùng lúc – gọi `POST /updateGeofile` không có tên tệp.

- Hộp thoại: "Bạn có thực sự muốn cập nhật tệp địa lý không?" với "Thao tác này sẽ cập nhật tệp #filename#" cho một tệp và "Thao tác này sẽ cập nhật tất cả tệp địa lý" cho nút "Cập nhật tất cả". Thành công – "Tệp địa lý đã được cập nhật thành công".

3.14. Sao lưu và khôi phục cơ sở dữ liệu

Khối "Sao lưu & Khôi phục" (*Backup & Restore*). Hành vi phụ thuộc vào DBMS đang sử dụng (SQLite mặc định hoặc PostgreSQL).

Xuất cơ sở dữ liệu (Sao lưu)

Nút "Xuất cơ sở dữ liệu" / "Sao lưu" (*Back Up*) gọi `GET /getDb`. Tệp được trả về dưới dạng tệp đính kèm:

- SQLite**: trước tiên thực hiện checkpoint (xả WAL), sau đó tải xuống tệp `x-ui.db`. Gợi ý: "Nhấn để tải xuống tệp .db chứa bản sao lưu cơ sở dữ liệu hiện tại của bạn..."
- PostgreSQL**: tải xuống dump `x-ui.dump` ở định dạng tùy chỉnh (`pg_dump --format=custom --no-owner --no-privileges`). Các công cụ client PostgreSQL phải được cài đặt trên máy chủ; nếu không – lỗi về việc thiếu `pg_dump`.

Nhập cơ sở dữ liệu (Khôi phục)

Nút "Nhập cơ sở dữ liệu" / "Khôi phục" (*Restore*) tải lên tệp qua `POST /importDB` (trường biểu mẫu `db`). Gợi ý: "Nhấn để chọn và tải lên tệp .db... để khôi phục cơ sở dữ liệu từ bản sao lưu".

Kịch bản cho **SQLite** an toàn, có khả năng rollback:

- Tệp được kiểm tra định dạng SQLite và lưu vào tệp tạm thời, sau đó kiểm tra tính toàn vẹn.
- Xray dừng lại, DB hiện tại được đóng và đổi tên thành `*.backup` (fallback).
- Tệp mới được đặt vào vị trí DB làm việc, thực hiện khởi tạo và di chuyển. Nếu có sự cố – fallback được khôi phục.
- Xray khởi động lại.

Đối với **PostgreSQL**, từ 3.5.0 "Khôi phục" chấp nhận **ba loại tệp** (loại được xác định theo nội dung, không theo phần mở rộng): tệp lưu trữ `pg_dump` (`PGDMP`) – được áp dụng qua `pg_restore --clean --if-exists --single-transaction ...`; **cơ sở dữ liệu SQLite** `.db` (bản sao lưu thông thường) và `.dump` **di chuyển SQLite** – chúng được kiểm tra, khi cần được dừng lại và nhập vào PostgreSQL **trong một giao dịch duy nhất** bằng chính engine của `x-ui migrate-db --dsn` (khi có lỗi, PostgreSQL vẫn nguyên vẹn). Hộp thoại chọn tệp chấp nhận `.dump`, `.db` trên cả hai DBMS; việc tải tệp lưu trữ `pg_dump` lên panel SQLite trả về lỗi rõ ràng. Gợi ý cảnh báo thẳng thắn: "Thao tác này sẽ thay thế tất cả dữ liệu hiện tại".

Thông báo: "Cơ sở dữ liệu đã được nhập thành công", "Đã xảy ra lỗi khi nhập cơ sở dữ liệu", "...khi đọc cơ sở dữ liệu", "...khi nhận cơ sở dữ liệu".

Tập di chuyển (giữa SQLite và PostgreSQL)

Nút "Tải xuống tập di chuyển" (*Download Migration*) gọi `GET /getMigration`. Từ 3.5.0 nút này chỉ hiển thị **trên các panel PostgreSQL** và tải xuống `x-ui.db` – cơ sở dữ liệu SQLite sẵn sàng, được tạo từ dữ liệu PostgreSQL (con đường "quay lại SQLite"). Trên các panel SQLite, nút đã bị bỏ vì thừa: bản sao lưu `.db` thông thường nay có thể được khôi phục thẳng vào panel PostgreSQL.

3.15. Các phần tử giao diện bổ sung

- **Chỉ báo khách hàng trực tuyến.** Bảng điều khiển duy trì hàng `online` (*Online Clients* / "Khách hàng trực tuyến") – số lượng khách hàng có kết nối đang hoạt động. Được tính khi Xray đang chạy (nếu không thì 0) và được ghi vào lịch sử theo cùng chu kỳ 2 giây. Biểu đồ – tab "Trực tuyến".
- **Lịch sử hệ thống (biểu đồ).** Nút/mục "Biểu đồ" → "Lịch sử hệ thống" với các tab: "Băng thông", "Gói tin", "Disk I/O", "Trực tuyến", "Tải", "Kết nối", "Sử dụng đĩa". Dữ liệu được lấy qua `GET /history/:metric/:bucket`; các khoảng tổng hợp hợp lệ (bucket, giây): **2, 30, 60, 180, 360, 720, 1440, 2880, 10080**, mỗi tab nhận tối đa 60 điểm. Trong selector phạm vi trên trang, các nút **2m, 1h, 3h, 6h, 12h, 24h, 2d, 7d** (bucket 2, 60, 180, 360, 720, 1440, 2880, 10080 tương ứng) đều có sẵn. Ở các phạm vi dài **2d** và **7d**, nhãn thời gian trên trục được bổ sung ngày theo định dạng `MM-DD HH:MM`. Lưu trữ được tổ chức theo ba cấp độ làm mỏng dữ liệu (rollup): dữ liệu mới được giữ với bước 2 giây trong **một giờ** cuối, sau đó được trung bình hóa đến bước 1 phút trong **48 giờ** và đến bước 10 phút trong **7 ngày**. Do đó, các biểu đồ (CPU, RAM, lưu lượng, gói tin, kết nối, đĩa, trực tuyến, tải) có thể xem trong khoảng thời gian **lên đến 7 ngày** (trước đây – đến 48 giờ), và càng lùi về quá khứ thì độ chi tiết càng thô hơn. Các chỉ số hợp lệ: `cpu`, `mem`, `swap`, `netUp`, `netDown`, `pktUp`, `pktDown`, `diskRead`, `diskWrite`, `diskUsage`, `tcpCount`, `udpCount`, `online`, `load1`, `load5`, `load15`. Nhãn "2 phút cuối" tương ứng với bucket = 2 (chế độ thời gian thực).

Ví dụ. Lấy chuỗi tải CPU trong khoảng ~2 phút cuối (bucket = 2 giây, tối đa 60 điểm) và cùng chuỗi đó được tổng hợp theo 5 phút (bucket = 300 giây):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

Chỉ số có thể được thay thế bằng bất kỳ chỉ số hợp lệ nào (`mem`, `netUp`, `tcpCount`, `load1`, v.v.). Bucket ngoài danh sách trắng **2, 30, 60, 180, 360, 720, 1440, 2880, 10080** sẽ bị từ chối.

- **Chỉ số Xray** – khối riêng với mức sử dụng bộ nhớ và thu gom rác của Xray (các hàng `xrAlloc`, `xrSys`, `xrHeapObjects`, `xrNumGC`, `xrPauseNs`) và "Observatory" (trạng thái các kết nối đầu ra). Chỉ hoạt động khi khối `metrics` được cấu hình trong cấu hình Xray (`listen 127.0.0.1:11111`, thẻ `metrics_out`); nếu không hiển thị "Endpoint chỉ số Xray chưa được cấu hình". Trong cửa sổ chỉ số Xray có selector phạm vi riêng với các nút **2m, 1h, 3h, 6h, 12h** (bucket 2, 60, 180, 360, 720).

Ví dụ khởi kích hoạt ô chỉ số Xray. Trong phần cài đặt Xray phải có đồng thời `metrics` (có thể) và inbound lắng nghe thẻ đó:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

Địa chỉ `127.0.0.1:11111` cổ tình không được công bố ra ngoài – panel truy vấn nó cục bộ.

- **Bộ chuyển đổi chủ đề tối.** Nằm trong menu chung/tiêu đề, không phải trong bảng điều khiển. Các tùy chọn: "Chủ đề" (*Theme*) với các lựa chọn "Tối" và "Rất tối" (*Ultra Dark*). Đây là cài đặt giao diện thuần túy, không ảnh hưởng đến hoạt động của panel.
- **Các liên kết khác** trong môi trường bảng điều khiển (từ menu/thanh dưới): "Nhật ký", "Cấu hình" – xem JSON cuối cùng của Xray (`GET /getConfigJson`), "Tài liệu".

4. Inbounds: tạo mới và các tham số chung

Mục **«Входящие»** (tiếng Anh: *Inbounds*) – là danh sách tất cả các điểm đầu vào của Xray mà các client kết nối đến. Mỗi inbound lưu trữ cả các trường «bảng điều khiển» (ghi chú, giới hạn lưu lượng, lịch đặt lại) lẫn các khối JSON thô của cấu hình Xray (`settings` , `streamSettings` , `sniffing`).

Việc tạo mới thực hiện qua nút **«Создать подключение»** (*Add Inbound*), chỉnh sửa – qua **«Изменить подключение»** (*Modify Inbound*). Cả hai thao tác gửi yêu cầu đến các API endpoint `POST /add` và `POST /update/:id`.

Dưới đây trình bày tất cả các trường của form **không** liên quan đến cài đặt giao thức cụ thể (client, mã hóa, REALITY/TLS) và **không** liên quan đến transport/stream (các tab **«Поток»**, **«Безопасность»**) – đây là chủ đề của các mục riêng biệt.

4.1. Các trường chung của form

Remark (Ghi chú)

Tham số	Giá trị
Trường	<code>remark</code>
Kiểu	chuỗi
Mặc định	rỗng

Tên dễ đọc của inbound, hiển thị trong danh sách và trong tiêu đề các hộp thoại (**«Удалить подключение "{remark}"?»** v.v.). Nhân trường – **«Примечание»**. Không ảnh hưởng đến hoạt động của Xray, chỉ phục vụ quản trị; nên đặt tên duy nhất và có ý nghĩa vì chúng được dùng trong tên file xuất và trong xác nhận các thao tác hàng loạt.

Protocol (Giao thức)

Tham số	Giá trị
Trường	<code>protocol</code>
Nhãn	«Протокол»
Xác thực	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

Danh sách thả xuống chọn giao thức inbound. Các giá trị hợp lệ:

Giá trị	Ghi chú
<code>vmess</code>	
<code>vless</code>	

Giá trị	Ghi chú
trojan	
shadowsocks	
wireguard	
hysteria	Hysteria v2 – là hysteria với streamSettings.version = 2 , không có giao thức riêng
http	
mixed	socks/http trên cùng một cổng
tunnel	
tun	được validator chấp nhận, không có hằng số giao thức riêng

Trường bắt buộc (required). Việc chọn giao thức xác định các trường cài đặt client và transport nào sẽ khả dụng (xem các mục theo từng giao thức cụ thể).

Lưu ý quan trọng: khi lưu, dịch vụ chuẩn hóa streamSettings . Cài đặt transport chỉ được giữ lại cho các giao thức vmess , vless , trojan , shadowsocks , hysteria ; với các giao thức còn lại (http , mixed , tunnel , wireguard , tun) trường streamSettings **bị xóa cưỡng bức**.

Đối với inbound kiểu tunnel /TPProxy, mà khối streamSettings không có khóa security (biên thể không transport), form mở và lưu mà không gặp lỗi xác thực streamSettings.security Invalid input .

Listen IP (IP lắng nghe)

Tham số	Giá trị
Trường	listen
Kiểu	chuỗi
Mặc định	rỗng → Xray lắng nghe trên 0.0.0.0 (tất cả IP)

Địa chỉ IP mà inbound nhận kết nối. Gợi ý trường:

«Оставьте пустым для прослушивания всех IP-адресов».

Khi tạo cấu hình Xray, giá trị rỗng được thay bằng 0.0.0.0 . Ngoài IP, trường cũng nhận **đường dẫn Unix socket** – gợi ý:

«Можно также указать путь Unix-сокета (например, /run/xray/in.sock) или имя абстрактного сокета с префиксом @ (например, @xray/in.sock), чтобы слушать сокет вместо TCP-порта – в этом случае задайте порт 0».

Như vậy, trường nhận hai dạng Unix socket: đường dẫn trong hệ thống file (`/run/xray/in.sock`) và tên socket trừu tượng với tiền tố `@` (`@xray/in.sock`). Trong cả hai trường hợp, hãy đặt `Port` bằng `0` .

Trường này được thay đổi khi cần giới hạn inbound chỉ trên một interface (ví dụ: `127.0.0.1` cho inbound chỉ hoạt động như đích fallback sau Nginx) hoặc khi inbound lắng nghe Unix socket.

Ví dụ. Inbound lắng nghe chỉ trên interface cục bộ (đích fallback điển hình sau Nginx) và Unix socket:

```
listen = 127.0.0.1    port = 8443
listen = /run/xray/in.sock    port = 0
```

Port (Cổng)

Tham số	Giá trị
Trường	<code>port</code>
Nhân	«Порт»
Xác thực	<code>gte=0, lte=65535</code>
Mặc định	— (do người dùng đặt)

Cổng TCP/UDP lắng nghe. Giá trị hợp lệ từ `0` đến `65535` . Giá trị `0` chỉ dùng kết hợp với lắng nghe trên Unix socket (xem trên).

Khi lưu, dịch vụ kiểm tra xung đột cổng: hai inbound không thể đồng thời chiếm `listen:port` trùng nhau cho cùng một transport (TCP/UDP). Transport được suy ra từ giao thức và `streamSettings / settings` : ví dụ, `hysteria` và `wireguard` luôn chiếm UDP, `kcp / quic` — UDP, còn phần lớn các giao thức khác — TCP. Khi có xung đột, việc lưu bị từ chối với lỗi.

Ngoài ra, bảng điều khiển không cho phép chiếm **cổng dành riêng của Xray API nội bộ** (tag `api` , mặc định `62789` trên `127.0.0.1`): inbound TCP cục bộ mà địa chỉ lắng nghe trùng với cổng này trên loopback sẽ bị từ chối với cùng lỗi xung đột cổng. Cổng API thực tế được đọc từ template cấu hình Xray (với giá trị dự phòng `62789`). Trên các node, hạn chế này không áp dụng — chúng có Xray riêng.

Tag Xray (Tag , duy nhất) được tạo tự động từ cổng và transport theo định dạng `in-<port>-<tcp|udp|tcpudp|any>` ; đối với inbound triển khai trên node, thêm tiền tố `n<nodeId>-` . Khi trùng, tag được thêm `-2` , `-3` v.v. Người dùng thường không chỉnh sửa tag.

Total traffic (Tổng lưu lượng, GB)

Tham số	Giá trị
Trường	<code>total</code> (tính bằng <code>byte</code>)
Nhân	«Общий расход»
Mặc định	<code>0</code>

Giới hạn lưu lượng tổng của inbound. Trong form giá trị nhập bằng gigabyte, trong cơ sở dữ liệu lưu bằng byte. Gợi ý trường:

«= Безлимит. (единица: ГБ)».

Tức là **0** có nghĩa là không giới hạn. Đây là giới hạn ở cấp toàn bộ inbound (không phải từng client); lưu lượng thực tế đã dùng được lưu trong các trường `up` (đã gửi) và `down` (đã nhận) và so sánh với `total`.

Expiry date / Duration (Ngày hết hạn / thời hạn)

Tham số	Giá trị
Trường	<code>expiryTime</code> (Unix timestamp)
Nhân	«Дата окончания» (tiếng Anh: <i>Duration</i>)
Mặc định	rỗng / 0

Thời hạn hiệu lực của inbound. Gợi ý:

«Оставьте пустым, чтобы было бесконечным».

Giá trị rỗng (**0**) có nghĩa là inbound không có thời hạn. Giá trị được lưu dưới dạng Unix timestamp; form cho phép đặt cả ngày cụ thể lẫn số ngày (đếm tương đối từ thời điểm hiện tại – nhân tiếng Anh *Duration*).

Enabled (Bật)

Tham số	Giá trị
Trường	<code>enable</code>
Nhân	«Включить» (tiếng Anh: <i>Enabled</i>)
Mặc định	đặt khi tạo

Cờ trạng thái hoạt động của inbound. Việc chuyển cờ này trong danh sách được xử lý bởi một endpoint «nhẹ» riêng `POST /setEnabled/:id`, không phải cập nhật đầy đủ – điều này được thực hiện có chủ đích để không phải tuần tự hóa lại toàn bộ khối `settings` (tất cả client) mỗi khi nhân công tắc trên inbound có hàng nghìn client. Khi tắt, inbound bị xóa khỏi Xray đang chạy; khi bật – được thêm lại.

Node / Deploy to (Node / Triển khai lên)

Tham số	Giá trị
Trường	<code>nodeId</code>
Nhân	«Развернуть на», «Локальная панель»

Tham số	Giá trị
Mặc định	rỗng (bảng điều khiển cục bộ)

Chọn nơi inbound hoạt động vật lý: trên bảng điều khiển cục bộ hoặc trên một trong các node đã đăng ký. Từ 3.5.0, danh sách «Развернуть на» (cũng như bộ lọc «Node» phía trên danh sách inbound) hỗ trợ **tìm kiếm khi nhập**, còn phía trên chính danh sách inbound xuất hiện **ô tìm kiếm** (theo ghi chú, cổng và giao thức; hoạt động cùng với bộ lọc node). Đặc điểm thực thi: `nodeId = 0` được chuẩn hóa thành `nil`, vì `0` không phải id node hợp lệ mà là tạo phẩm của binding form; `nil / 0` có nghĩa là bảng điều khiển cục bộ. Khi lưu inbound trên node offline, có thể xuất hiện thông báo «thay đổi sẽ được đồng bộ khi node kết nối lại». Kể từ phiên bản 3.4.2, thông báo này chỉ hiển thị khi node thực sự offline hoặc đã tắt (trước đây nó có thể xuất hiện cả khi lưu trên node trực tuyến).

Chiến lược địa chỉ cho liên kết (Share address strategy)

Tham số	Giá trị
Trường	chiến lược + (tùy chọn) địa chỉ tùy chỉnh
Nhãn	«Стратегия адреса для ссылок» (tiếng Anh: <i>Share address strategy</i>)
Mặc định	«Адрес прослушивания inbound» (<i>Inbound listen</i>)

Danh sách thả xuống xác định địa chỉ nào được chèn vào **các liên kết chia sẻ và mã QR xuất ra** của inbound này. Các giá trị:

Giá trị	Nhãn	Nội dung chèn vào
<code>node</code>	«Адрес узла» (<i>Node address</i>)	địa chỉ node mà inbound đang chạy trên đó
<code>listen</code>	«Адрес прослушивания inbound» (<i>Inbound listen</i>)	địa chỉ lắng nghe của chính inbound
<code>custom</code>	«Пользовательская» (<i>Custom</i>)	địa chỉ tùy chỉnh từ trường «Пользовательский адрес для ссылок» (<i>Custom share address</i>)

Khi chọn «Пользовательская», xuất hiện trường «Пользовательский адрес для ссылок»; nhập host hoặc IP **không có scheme và cổng** (giá trị được xác thực). Tùy chọn «Адрес узла» chỉ hiển thị trong danh sách nếu tồn tại node đang bật mà inbound này có thể chạy trên đó; nếu không, nó bị ẩn và giá trị được đặt về «Адрес прослушивания inbound».

Chiến lược này chỉ ảnh hưởng đến các liên kết chia sẻ trực tiếp và mã QR. Nó **không** ảnh hưởng đến đầu ra subscription – ở đó địa chỉ vẫn được xác định theo logic thông thường của bảng điều khiển.

4.2. Sniffing (Nghe lén)

Tab «Сниффинг» chỉnh sửa khối `sniffing` của cấu hình Xray, được lưu dưới dạng JSON thô. Sniffing cho phép Xray «nghe lén» tên miền/giao thức thực sự bên trong kết nối phục vụ mục đích định tuyến.

Trường con	Nhân	Mục đích
<code>enabled</code>	(công tắc tab)	Bật/tắt sniffing cho inbound
<code>destOverride</code>	—	Danh sách giao thức mà địa chỉ đích bị chặn: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
<code>metadataOnly</code>	«Только метаданные»	Chỉ sử dụng metadata kết nối, không đọc payload
<code>routeOnly</code>	«Только маршрутизация»	Áp dụng kết quả sniffing chỉ cho định tuyến, không ghi đè địa chỉ đích
<code>domainsExcluded</code>	«Исключённые домены»	Các domain bị loại trừ khỏi sniffing
(IP bị loại trừ)	«Исключённые IP»	Các địa chỉ IP bị loại trừ khỏi sniffing

- **`destOverride`** — tập hợp các sniffer: `http` (xác định domain từ HTTP header Host), `tls` (từ SNI), `quic` (từ QUIC ClientHello), `fakedns` (so khớp với pool FakeDNS). Thông thường để xác định domain, bật `http` và `tls`.

Ví dụ khối `sniffing` (xác định domain theo HTTP và TLS, chỉ dùng kết quả cho định tuyến, không động đến mạng cục bộ):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** — khi bật, Xray không đọc nội dung gói đầu tiên và chỉ dựa vào metadata; hữu ích để không phá vỡ các giao thức mà dữ liệu không thể «nghe lén».
- **`routeOnly`** — kết quả sniffing chỉ được dùng bởi các quy tắc định tuyến; địa chỉ kết nối trong outbound không bị ghi đè bằng domain đã nhận dạng.

Lưu ý: bảng điều khiển lưu `sniffing` như một khối JSON không trong suốt và khi lưu không thêm gì vào đó — tất cả giá trị mặc định cho các ô checkbox này được tạo phía ứng dụng client. Ở dạng thô, khối có thể chỉnh sửa qua mục «JSON входящего» (xem bên dưới).

4.3. Allocate (chiến lược phân bổ cổng)

Khối `allocate` trong `streamSettings` kiểm soát cách Xray phân bổ các cổng lắng nghe. Đây là một phần cấu hình Xray; bảng điều khiển lưu và truyền nó như một phần của `streamSettings` /JSON inbound. Các tham số (theo thuật ngữ Xray-core):

Trường con	Mục đích	Giá trị / mặc định
<code>strategy</code>	Chiến lược phân bổ cổng	<code>always</code> — luôn lắng nghe trên cổng đã chỉ định (mặc định); <code>random</code> — định kỳ thay đổi các cổng lắng nghe trong phạm vi

Trường con	Mục đích	Giá trị / mặc định
refresh	Khoảng thời gian thay đổi cổng (phút) khi random	số nguyên phút (khuyến nghị 5; tối thiểu – 2)
concurrency	Số cổng giữ mở đồng thời khi random	số nguyên (mặc định 3; không quá một phần ba độ rộng dải cổng)

strategy: always giữ inbound trên một cổng (chế độ tiêu chuẩn). strategy: random dùng cho các tình huống chống chặn khi inbound định kỳ «nhảy» trong phạm vi cổng; trong trường hợp này refresh và concurrency có ý nghĩa. Chỉ thay đổi các giá trị này khi cố ý sử dụng chế độ cổng ngẫu nhiên.

Ví dụ khối allocate trong streamSettings (chế độ cổng ngẫu nhiên: giữ 3 cổng mở, thay đổi mỗi 5 phút):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Để điều này hoạt động, port của inbound được đặt theo dải (ví dụ: 20000–20100).

4.4. External Proxy (Proxy bên ngoài)

Trường «**External Proxy**» thuộc cài đặt tạo liên kết mời và được lưu trong streamSettings của inbound. Nó xác định danh sách các địa chỉ bên ngoài thay thế (host/port, nếu cần với TLS bắt buộc – «**Принудительный TLS**»), được chèn vào liên kết client thay vì listen:port thực tế của inbound.

Được sử dụng khi client cần kết nối không trực tiếp đến server mà thông qua proxy/reverse/CDN bên ngoài: khi đó trong các liên kết chung, địa chỉ công khai của frontend đó được chỉ định. Điều này không ảnh hưởng đến quá trình nhận kết nối của Xray – đây là «trang điểm» của các liên kết được tạo ra. Các trường form liên quan: «**Принудительный TLS**», «**Fingerprint**», nhãn của mỗi bản ghi.

4.5. Fallbacks (Các fallback)

Mục «**Fallback'и**» xác định các quy tắc chuyển hướng kết nối không khớp với bất kỳ client nào của inbound. Khả dụng cho master inbound trên TLS transport (VLESS/Trojan TCP-TLS). Được quản lý qua các endpoint GET /:id/fallbacks / POST /:id/fallbacks .

Gợi ý mục:

«Когда соединение на этом инбаунде не совпадает ни с одним клиентом, оно перенаправляется в другое место. Выберите дочерний инбаунд ниже, чтобы поля маршрутизации (SNI / ALPN / Path / xver) заполнились автоматически из его транспорта, либо оставьте выбор пустым и задайте Dest напрямую (например, 8080 или 127.0.0.1:8080), чтобы перенаправить на внешний сервер, такой как Nginx. Каждый дочерний инбаунд должен слушать на 127.0.0.1 с security=none».

Mục fallback chỉ hiển thị cho inbound VLESS/Trojan qua RAW (TCP) với bảo mật TLS hoặc REALITY. Inbound mới khởi động với `security=none`, do đó mục này ban đầu có thể trông như vắng mặt. Ở trạng thái này (VLESS/Trojan, RAW/TCP, bảo mật chưa được cấu hình), thay vì mục đó hiển thị gợi ý tích hợp: các fallback sẽ khả dụng sau khi chọn TLS hoặc Reality trong tab **«Безопасность»**.

Các trường của dòng fallback

Trường	Mặc định	Mô tả
(inbound con)	–	Chọn inbound con (nhấn «Выберите инбаунд»). Nếu được chọn, các trường Name/Alpn/Path/Dest có thể tự điền từ transport của nó
Name	rỗng (= bất kỳ)	Điều kiện khớp theo tên (SNI/tên). Nhấn «bất kỳ» – «любой»
Alpn	rỗng	Điều kiện khớp theo ALPN
Path	rỗng	Điều kiện khớp theo đường dẫn (cho transport WS/HTTP của inbound con)
Dest	tự động	Chuyển hướng đến đâu. Placeholder «авто (listen:порт дочернего)» . Có thể chỉ định cổng (<code>8080</code>) hoặc <code>host:port</code> (<code>127.0.0.1:8080</code>)
Xver	<code>0</code>	Phiên bản PROXY protocol («Xver»): <code>0</code> – tắt, <code>1</code> hoặc <code>2</code> – phiên bản PROXY protocol tương ứng
(thứ tự)	theo vị trí	Thứ tự áp dụng quy tắc; đặt bằng nút «Вверх»/«Вниз»

Logic lưu: toàn bộ danh sách fallback của master được thay thế nguyên tử. Dòng không có inbound con được chọn (`childId <= 0`) lần `Dest` được đặt **bị bỏ qua**. Nếu inbound con được chọn trùng với id của chính master, nó được đặt về không. Khi tạo JSON đầu ra: nếu `Dest` rỗng, nó được tính từ inbound con là `listen:port`, trong đó `0.0.0.0 / :: / ::0` được thay bằng `127.0.0.1`; các trường `name / alpn / path` rỗng không được đưa vào JSON đầu ra; `xver` chỉ được thêm nếu lớn hơn 0.

Ví dụ `settings.fallbacks` đầu ra (lưu lượng với `alpn=h2` đến đích WS theo đường dẫn `/ws`, tắt cả còn lại – đến Nginx cục bộ trên cổng 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

Dòng cuối không có `name / alpn / path` – đây là quy tắc «mặc định», bắt tất cả còn lại.

Các nút và gợi ý của mục fallbacks

- **«Добавить фолбэк»** – thêm dòng; **«Фолбэков пока нет»** – trạng thái rỗng.
- **«Быстро добавить все подходящие» / «Добавить все»** – thêm dòng fallback cho mỗi inbound phù hợp chưa được kết nối. Kết quả: **«Добавлено {n} фолбэк(ов)»** hoặc **«Нет новых подходящих инбаундов»**.

- **«Заполнить из дочернего»** – lấy lại các trường định tuyến (SNI/ALPN/Path/xver) từ transport của inbound con đã chọn; sau khi thực hiện – «Заполнено из дочернего».
- **«Изменить поля маршрутизации»** / **«Скрыть расширенные»** – hiện/ẩn các trường chi tiết của dòng.
- Nhấn **«Маршрутизирует, когда»** và **«По умолчанию – ловит всё остальное»** giải thích điều kiện kích hoạt của mỗi dòng.

Sau khi lưu các fallback, server gọi khởi động lại Xray để `settings.fallbacks` mới có hiệu lực.

4.6. Đặt lại lưu lượng định kỳ

Khởi **«Сброс трафика»** cấu hình tự động đặt lại bộ đếm lưu lượng của inbound theo lịch. Mô tả:

«Автоматический сброс счетчика трафика через указанные интервалы».

Tham số	Giá trị
Trường	<code>trafficReset</code>
Xác thực	<code>omitempty,oneof=never hourly daily weekly monthly</code>
Mặc định	<code>never</code>
Trường đi kèm	<code>lastTrafficResetTime</code> – mốc thời gian lần đặt lại cuối (nhấn «Последний сброс»)

Danh sách thả xuống:

Giá trị	Nhãn
<code>never</code>	«Никогда»
<code>hourly</code>	«Ежечасно»
<code>daily</code>	«Ежедневно»
<code>weekly</code>	«Еженедельно»
<code>monthly</code>	«Ежемесячно»

Cho mỗi chu kỳ, một cron job được đăng ký chạy theo lịch tương ứng (`@hourly` , `@daily` , `@weekly` , `@monthly`). Job chọn tất cả inbound có `trafficReset` đã đặt và đối với mỗi inbound đặt lại bộ đếm của chính inbound đó (`up=0` , `down=0`) và lưu lượng của tất cả client của nó. Tức là việc đặt lại định kỳ ảnh hưởng đến cả inbound lẫn các client của nó.

Ví dụ giá trị trường. Để bộ đếm được đặt về không vào ngày đầu tiên mỗi tháng, chọn **«Ежемесячно»** trong form, được lưu dưới dạng:

```
{ "trafficReset": "monthly" }
```

Giá trị `never` (mặc định) tắt hoàn toàn tự động đặt lại.

4.7. JSON входящего (nâng cao)

Mục «Разделы JSON входящего» cung cấp truy cập trực tiếp vào các khối JSON thô của inbound. Mô tả:

«Полный JSON входящего и отдельные редакторы для settings, sniffing и streamSettings».

Các trình soạn thảo khả dụng:

Tab	Nhân	Chỉnh sửa gì
Всё	«Полный объект входящего со всеми полями в одном редакторе»	toàn bộ đối tượng Inbound
Настройки	«Обёртка блока settings Xray»	trường settings
Sniffing	«Обёртка блока sniffing Xray»	trường sniffing
Stream	«Обёртка блока stream Xray»	trường streamSettings

Các trường này được tuần tự hóa dưới dạng đối tượng JSON lồng nhau: các khối rỗng được trả về dưới dạng null, còn văn bản không phải JSON hợp lệ được bọc trong chuỗi để dữ liệu không bị mất. Lỗi phân tích khi lưu được hiển thị với tiền tố «Расширенный JSON».

Cửa sổ xem «JSON входящего», cũng như cửa sổ nhập inbound, sử dụng trình soạn thảo mã đầy đủ tính năng với tô sáng cú pháp JSON (thay vì ô văn bản thông thường): xem câu hình – ở chế độ chỉ đọc với tô sáng, còn nhập – ở chế độ chỉnh sửa được, giúp đọc và chỉnh sửa dễ dàng hơn.

4.8. Các thao tác với inbound: QR / Edit / Reset / Delete và thống kê

Trong danh sách và trong card inbound có các thao tác sau (menu «Меню»):

Thống kê lưu lượng

Hiển thị lưu lượng tổng hợp của inbound: «Отправлено/получено» (các trường up / down), «Всего трафика», «Всего подключений». Trong card còn có – «Создано», «Обновлено», «Дата окончания».

Trong danh sách inbounds có cột **Speed** với tốc độ lưu lượng hiện tại theo mỗi inbound (gửi/nhận), được tính từ mức tăng bộ đếm giữa các lần truy vấn; tốc độ thực tế tương tự được hiển thị trong cửa sổ thống kê inbound. Khi lần truy vấn tiếp theo không có mức tăng, giá trị tốc độ được đặt lại.

Trong tóm tắt client trên trang inbounds, trạng thái được xác định theo ưu tiên «cạn kiệt/kết thúc»: các client đã hết hạn hoặc hết lưu lượng (và bị tác vụ tự động tắt enable) được xếp vào trạng thái «Исчерпан/завершён» (Depleted/Ended), không phải màu xám «Отключён» (Disabled), và không được tính hai lần. Phân loại trùng khớp với phân loại hiển thị trong card của chính client, và tính đúng các client được gắn với nhiều inbound.

Mã QR và sao chép liên kết

- «Подробнее» – mở rộng các liên kết kết nối và subscription.

- Mã QR của client: gợi ý **«Нажмите на QR-код, чтобы скопировать»**.
- **«Копировать ссылку»** (tiếng Anh: *Copy URL*), **«Экспорт ссылок»**. Kể từ phiên bản 3.4.2, thao tác toàn trang **«Xuất tất cả liên kết»** (GET /panel/api/inbounds/allLinks ; cửa sổ «Xuất tất cả liên kết inbound», tệp «All-Inbounds») tạo liên kết qua engine đăng ký – áp dụng mẫu nhận xét cho từng khách hàng và ưu tiên các endpoint Host được quản lý (trước đây sử dụng nhận xét cố định inbound-email).

Edit (Chỉnh sửa)

«Изменить подключение» – mở form chỉnh sửa (POST /update/:id). Khi cập nhật, dịch vụ đọc lại bản ghi hiện có, chuyển các trường đã thay đổi, nếu cần tạo lại tag (nếu tag cũ được tạo tự động) và đồng bộ runtime Xray. Thành công – thông báo **«Подключение успешно обновлено»**.

Reset Traffic (Đặt lại lưu lượng)

«Сбросить трафик» – đặt lại bộ đếm up / down của chính inbound này về không (POST /:id/resetTraffic , đặt up=0, down=0). Xác nhận:

«Сбросить трафик "{remark}"?» / «Сбрасывает счётчики отправки/получения этого подключения до 0».

Đặt lại lưu lượng inbound **không** ảnh hưởng đến bộ đếm của các client của nó (chúng có thao tác «Đặt lại lưu lượng client» riêng). Sau khi đặt lại, Xray được khởi động lại. Thành công – thông báo **«Входящий трафик сброшен»**. Còn có biến thể hàng loạt – **«Сброс трафика всех подключений»** (POST /resetAllTraffics).

Delete (Xóa)

«Удалить подключение» (POST /del/:id). Xác nhận:

«Удалить подключение "{remark}"?» / «Подключение и все его клиенты будут удалены. Это действие нельзя отменить».

Việc xóa gỡ inbound khỏi Xray đang chạy (nếu cần với khởi động lại). Thành công – thông báo **«Подключение успешно удалено»**. Xóa hàng loạt – POST /bulkDel , với báo cáo từng phần tử và không quá một lần khởi động lại Xray.

Các thao tác khác với client của inbound

Trong menu cũng có: **«Клонировать»** (bản sao inbound với cổng mới và danh sách client rỗng), **«Удалить всех клиентов»** (POST /:id/deleteAllClients – xóa tất cả client, bản thân inbound được giữ lại), **«Удалить отключенных клиентов»**, **«Привязать/Отвязать клиентов»**, **«Импортировать»/«Экспорт подключений»** (POST /import). Chi tiết các thao tác client thuộc mục về client.

5. Giao thức

Khi tạo inbound, điều đầu tiên cần chọn là **Giao thức** («Protocol»). Giao thức xác định phương thức xác thực và mã hóa lưu lượng mà Xray-core sẽ áp dụng cho inbound đó, tập hợp các trường cần điền trong `settings`, cũng như các loại transport (`network`) và bảo mật (TLS / REALITY) khả dụng.

Trường giao thức được thiết lập một lần khi tạo inbound và **không thể thay đổi khi chỉnh sửa** (trong biểu mẫu chỉnh sửa, danh sách thả xuống bị khóa). Để thay đổi giao thức, cần tạo inbound mới.

5.1. Danh sách giao thức được hỗ trợ

Máy chủ chấp nhận tập hợp giá trị sau cho trường `Protocol` :

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

Kể từ phiên bản **3.3.0**, giá trị `mtproto` (proxy Telegram) được bổ sung vào danh sách.

Giá trị trong cấu hình	Mục đích	Mô hình client
<code>vless</code>	Giao thức proxy chính (mặc định khi tạo inbound)	Client với UUID, hỗ trợ flow và mã hóa hậu lượng tử
<code>vmess</code>	Giao thức proxy Xray truyền thống	Client với UUID và tham số <code>security</code>
<code>trojan</code>	Proxy giả mạo HTTPS thông thường	Client với mật khẩu
<code>shadowsocks</code>	Proxy Shadowsocks (bao gồm SIP022 / 2022-blake3)	Một hoặc nhiều người dùng (2022)
<code>wireguard</code>	Inbound WireGuard	Peer (không phải client)
<code>hysteria</code>	Inbound Hysteria (mặc định phiên bản 2)	Client với token <code>auth</code>
<code>http</code>	Proxy HTTP truyền thống (forward proxy)	Tài khoản user/pass, không tính lưu lượng
<code>mixed</code>	Proxy kết hợp SOCKS + HTTP	Tài khoản user/pass
<code>tunnel</code>	Bộ chuyển tiếp trong suốt (xray dokodemo-door)	Không có client
<code>tun</code>	Giao diện TUN (chỉ hiển thị những cái đã tồn tại)	Không có client
<code>mtproto</code>	Proxy Telegram (MTPROTO), thêm vào từ 3.3.0; được phục vụ bởi tiến trình riêng <code>mtg</code> , không phải Xray	Không có client (truy cập bằng secret)

Ghi chú về `tun` : giá trị này được giữ lại trong danh sách để tương thích và **hiển thị** các inbound đã lưu trước đó, nhưng trong phiên bản hiện tại, backend không khuyến nghị tạo mới loại này – hỗ trợ đã bị coi là lỗi thời. Việc tạo inbound mới thuộc loại này không còn ý nghĩa.

Ghi chú về Hysteria 2: không có giao thức riêng tên «hysteria2». Đây là giao thức `hysteria` với trường `streamSettings.version = 2`. Scheme liên kết `hysteria2://` khi tạo share link được chọn tự động khi phiên bản stream bằng 2.

Không phải tất cả giao thức đều hỗ trợ phân phối theo node. Chỉ có thể triển khai lên node: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. Các giao thức `http`, `mixed`, `tunnel`, `tun`, `mtproto` chỉ hoạt động trên panel chính.

5.2. Giao thức nào hỗ trợ TLS / REALITY / transport

Khả năng bật lớp bảo mật hay transport nào đó phụ thuộc vào giao thức và mạng đã chọn (`streamSettings.network`):

Tính năng	Khả dụng cho giao thức	Mạng được phép (network)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (và luôn có với <code>hysteria</code>)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (xtls-rprx-vision)	chỉ <code>vless</code>	chỉ <code>tcp</code> , khi <code>security = tls</code> hoặc <code>reality</code>
Stream / transport (tab «Luồng»)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	–

Đối với các giao thức `http`, `mixed`, `tunnel`, `tun`, `wireguard`, tab transport không khả dụng – chúng không có cài đặt stream của Xray.

5.3. VLESS

Mục đích: giao thức proxy hiện đại chính. Hỗ trợ XTLS-Vision (`flow`), REALITY, cũng như mã hóa hậu lượng tử ở cấp độ bản thân VLESS (trường `decryption` / `encryption`). Được sử dụng mặc định cho các inbound mới.

Các trường trong khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>clients</code>	<code>[]</code>	Danh sách client. Mỗi client có: <code>id</code> (UUID), <code>email</code> (bắt buộc), <code>flow</code> , giới hạn (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Tham số giải mã phía máy chủ. Nhận trong UI: «Giải mã» (tiếng Anh «Decryption»)
<code>encryption</code>	<code>none</code>	Tham số mã hóa cặp đôi (đưa vào link client). Nhận: «Mã hóa» (tiếng Anh «Encryption»)
<code>fallbacks</code>	<code>[]</code>	Danh sách fallback (xem phần về fallback); khả dụng khi <code>network = tcp</code> và <code>security = TLS</code> hoặc REALITY

Trường	Giá trị mặc định	Mô tả
testseed	(4 số: 900, 500, 900, 256)	«Vision testseed» – 4 số nguyên dương cho XTLS-Vision padding. Chỉ áp dụng cho client có flow <code>xtls-rprx-vision</code> , ngược lại bị bỏ qua

flow (`xtls-rprx-vision`)

flow được thiết lập **trên client**, không phải trên inbound, và nhận một trong ba giá trị:

Giá trị	Ý nghĩa
`` (trống)	Không có XTLS-flow (mặc định)
<code>xtls-rprx-vision</code>	XTLS-Vision – chế độ khuyến nghị cho VLESS qua TCP+TLS/REALITY
<code>xtls-rprx-vision-udp443</code>	Vision tương tự nhưng xử lý UDP/443 (QUIC)

Trường flow chỉ khả dụng khi đáp ứng đủ các điều kiện: giao thức `vless`, `network = tcp` và `security = tls` hoặc `reality`. Trường **Vision testseed** trong biểu mẫu chỉ hiển thị trong cùng điều kiện đó.

Ngoại lệ cho XHTTP: khi VLESS qua `network = xhttp` với xác thực hậu lượng tử VLESS được bật (`encryption / decryption, vlessenc`), flow `xtls-rprx-vision` cũng được phép – bất kể lớp bảo mật, kể cả với REALITY. Trong trường hợp này, panel truyền đúng `xtls-rprx-vision` vào share link và subscription (bao gồm định dạng Clash/Mihomo), vì vậy client nhận cấu hình chính xác với Vision.

Giải mã / Mã hóa (xác thực hậu lượng tử VLESS)

Các trường `decryption` và `encryption` là xác thực ở cấp độ bản thân VLESS (tách biệt với TLS/REALITY trên transport). Mặc định cả hai đều bằng `none`. Trong biểu mẫu, bên dưới các trường này có khối **«Tạo khóa»** – danh sách thả xuống chế độ và nút **«Tạo»** (bên cạnh – nút **«Xóa»**). Danh sách thả xuống chứa sáu tùy chọn: **X25519 (native)**, **X25519 (xorpub)**, **X25519 (random)**, **ML-KEM-768 (native)**, **ML-KEM-768 (xorpub)**, **ML-KEM-768 (random)** – tức là hai loại khóa (X25519 cổ điển và ML-KEM-768 hậu lượng tử), mỗi loại có ba chế độ:

- **native** – cặp khóa cơ bản của loại được chọn;
- **xorpub** – chế độ dẫn xuất với xử lý bổ sung phần công khai;
- **random** – chế độ dẫn xuất có thành phần ngẫu nhiên.

Chọn chế độ cần thiết trong danh sách và nhấn **«Tạo»**: panel sẽ điền **cả hai** trường (`decryption` và `encryption`) với cặp giá trị phù hợp cho chế độ này. Nút **«Xóa»** đặt lại cả hai trường về `none`.

Bên dưới khối hiển thị dòng trạng thái **«Đã chọn: ...»**, nhận dạng từ chuỗi đã tạo cả loại khóa (X25519 hoặc ML-KEM-768) lẫn chế độ (`native / xorpub / random`) và hiển thị chúng. Các trường trống hoặc `none` được hiển thị là «None».

Về mặt kỹ thuật, các nút gọi `GET /panel/api/server/getNewVlessEnc` (tạo khóa qua `xray vlessenc`) và điền **cả hai** trường với các giá trị cặp đôi dạng `mlkem768x25519plus.native.<rtt>.<role>` (ví dụ:

`decryption = mlkem768x25519plus.native.600s.server-x25519` , `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). Tham số `decryption` ở lại phía máy chủ, `encryption` đi vào link client.

Quan trọng: khi tạo cấu hình inbound cho Xray, panel loại bỏ phần dư thừa: nếu `settings` còn lại `encryption` (thuộc về phía client), nó **bị cắt bỏ** khỏi cấu hình máy chủ. Trên máy chủ thực tế chỉ còn lại `decryption` .

Khi nào nên chọn VLESS: đây là lựa chọn khuyến nghị mặc định cho inbound mới, đặc biệt kết hợp với REALITY (không cần chứng chỉ riêng) hoặc TLS + XTLS-Vision.

Ví dụ: khối `settings` của VLESS-inbound với một client và XTLS-Vision. Trường `flow` đặt trên client, `decryption` ở lại trên máy chủ:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

Đối với kết hợp REALITY, khối `streamSettings` tương ứng (tab «Transport» → Security: REALITY) trông như sau:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<приватный ключ X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Mục đích: giao thức proxy Xray truyền thống. Xác thực bằng UUID, phía client có thêm cấu hình phương thức mã hóa payload (`security`).

Các trường trong khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>clients</code>	<code>[]</code>	Danh sách client

Mỗi client VMess (ngoài các trường chung `email` , giới hạn, `enable` , `tgId` , `subId` , `comment` , `reset`):

Trường client	Giá trị mặc định	Mô tả
<code>id</code>	—	UUID của client
<code>security</code>	<code>auto</code>	Phương thức mã hóa payload VMess. Giá trị được phép: <code>aes-128-gcm</code> , <code>chacha20-poly1305</code> , <code>auto</code> , <code>none</code> , <code>zero</code>

Giá trị `security` :

- `auto` — Xray tự chọn cipher tùy theo nền tảng (khuyến nghị);
- `aes-128-gcm` , `chacha20-poly1305` — cipher AEAD cố định;
- `none` — không mã hóa payload (chỉ có nghĩa khi chạy trên TLS);
- `zero` — không mã hóa và không xác thực payload.

Tương thích lịch sử: các bản ghi cũ có thể lưu `security: ""` — khi đọc, chuỗi rỗng được chuyển thành `auto` . Khi tạo cấu hình máy chủ, trường `security` của các client VMess **bị xóa** khỏi `settings` , vì với inbound nó không cần thiết.

Khi nào nên chọn VMess: để tương thích với client cũ hoặc cấu hình hiện có. Với các triển khai mới, thường VLESS được ưu tiên hơn.

5.5. Trojan

Mục đích: proxy giả mạo lưu lượng HTTPS thông thường. Xác thực bằng mật khẩu. Giống VLESS, hỗ trợ fallback và (khi `network = tcp`) REALITY/TLS.

Các trường trong khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>clients</code>	<code>[]</code>	Danh sách client
<code>fallbacks</code>	<code>[]</code>	Danh sách fallback (khả dụng khi <code>network = tcp</code> và TLS/REALITY)

Trường quan trọng của mỗi client Trojan:

Trường client	Giá trị mặc định	Mô tả
<code>password</code>	—	Mật khẩu client (bắt buộc, tối thiểu 1 ký tự)
<code>email</code>	—	Định danh duy nhất của client

Các trường còn lại của client là chung (`limitIp` , `totalGB` , `expiryTime` , `enable` , `tgId` , `subId` , `comment` , `reset`).

Khi nào nên chọn Trojan: khi cần giả mạo HTTPS trên cổng 443, kể cả với fallback đến máy chủ web (Nginx) cho các kết nối không xác thực.

Ví dụ: khối `settings` của Trojan với fallback đến máy chủ web cục bộ. Các kết nối không xác thực (không có mật khẩu hợp lệ) được chuyển đến Nginx đang lắng nghe `127.0.0.1:8080` :

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Để sử dụng fallback cần có `network = tcp` và `Security = TLS` hoặc `REALITY`; ngược lại trường `fallbacks` không khả dụng.

5.6. Shadowsocks

Mục đích: proxy Shadowsocks nhẹ. Hỗ trợ cả cipher AEAD cũ lẫn các phương thức SIP022 mới (`2022-blake3-*`). Có thể hoạt động ở chế độ một người dùng hoặc nhiều người dùng.

Các trường trong khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>method</code>	<code>2022-blake3-aes-256-gcm</code>	Phương thức mã hóa inbound. Nhân trong UI: «Phương thức mã hóa» (tiếng Anh «Encryption method»)
<code>password</code>	''	Mật khẩu inbound (với các phương thức 2022 được tạo tự động theo phương thức đã chọn)
<code>network</code>	<code>tcp,udp</code>	Transport. Nhân: «Mạng» (tiếng Anh «Network»). Tùy chọn: <code>tcp,udp</code> (TCP, UDP), <code>tcp</code> , <code>udp</code>
<code>clients</code>	<code>[]</code>	Danh sách client
<code>ivCheck</code>	<code>false</code> (tắt)	Công tắc «ivCheck» – bảo vệ chống tái sử dụng IV

Phương thức mã hóa (`method`)

Tập hợp được phép:

Phương thức	Danh mục
<code>aes-256-gcm</code>	AEAD lỗi thời

Phương thức	Danh mục
chacha20-poly1305	AEAD lỗi thời
chacha20-ietf-poly1305	AEAD lỗi thời
xchacha20-ietf-poly1305	AEAD lỗi thời
2022-blake3-aes-128-gcm	SS 2022 (khuyến nghị)
2022-blake3-aes-256-gcm	SS 2022 (mặc định)
2022-blake3-chacha20-poly1305	SS 2022, một người dùng

Logic của panel theo phương thức:

- **Phương thức 2022** (2022-blake3-*) được coi là «SS 2022». Phương thức 2022-blake3-chacha20-poly1305 – **một người dùng** (không hỗ trợ multi-user); các phương thức 2022 khác cho phép nhiều client. Trường mật khẩu (với nút tạo tự động điều chỉnh độ dài khóa theo phương thức) hiển thị trong biểu mẫu chính xác cho các phương thức 2022.
- **Cipher lỗi thời** (aes-*, chacha20-*) hoạt động theo mô hình cổ điển «một phương thức + một mật khẩu».

Chuẩn hóa trước khi chạy Xray: với cipher lỗi thời, mỗi client phải mang method trùng với phương thức của inbound (nếu không Xray sẽ báo lỗi «unsupported cipher method:»). Với phương thức 2022 ngược lại – trường method của client phải **rỗng** (nếu không Xray từ chối inbound với «users must have empty method»). Panel tự điều chỉnh dữ liệu khi chuyển đổi phương thức.

Tạo lại khóa client khi thay đổi kích thước khóa: với Shadowsocks-2022, khi thay đổi phương thức mã hóa sang phương thức có kích thước khóa khác (ví dụ giữa 2022-blake3-aes-256-gcm và 2022-blake3-aes-128-gcm), panel tự động tạo lại PSK client theo độ dài mới khi lưu inbound. Nếu không, các khóa cũ sẽ giữ nguyên độ dài và Xray sẽ từ chối chúng. Hệ quả: các client bị ảnh hưởng cần lấy lại subscription – các link cũ sẽ không kết nối được nữa.

Khi nào nên chọn Shadowsocks: cho các triển khai đơn giản không cần giả mạo TLS; lựa chọn hiện đại – phương thức 2022-blake3.

Ví dụ: khối settings của Shadowsocks cho phương thức 2022-blake3 (chế độ nhiều người dùng). Inbound có mật khẩu riêng (khóa base64 với độ dài phù hợp), mỗi client có mật khẩu riêng, trường method của client **rỗng**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Với cipher legacy (aes-256-gcm v.v.) – ngược lại: một mật khẩu cho inbound, còn `method` của client phải trùng với phương thức inbound.

5.7. Dokodemo-door / Tunnel (bộ chuyển tiếp trong suốt)

Mục đích: bộ chuyển tiếp trong suốt (trong panel – giao thức `tunnel`, thực hiện hành vi `dokodemo-door`). Nhận lưu lượng và chuyển hướng đến địa chỉ/cổng được chỉ định, không xác thực và không có client.

Các trường trong khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>rewriteAddress</code>	(không có)	«Viết lại địa chỉ» (tiếng Anh «Rewrite address») – địa chỉ đích để chuyển hướng lưu lượng
<code>rewritePort</code>	(không có)	«Viết lại cổng» (tiếng Anh «Rewrite port») – cổng đích (0-65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	«Mạng được phép» (tiếng Anh «Allowed network»). Tùy chọn: <code>tcp,udp</code> , <code>tcp</code> , <code>udp</code>
<code>portMap</code>	<code>{}</code>	«Ánh xạ cổng» – bảng ánh xạ cổng→cổng (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (tắt)	«Theo redirect» (tiếng Anh «Follow redirect») – sử dụng địa chỉ đích gốc từ kết nối bị chặn

Tab «Transport» cho Tunnel: inbound loại này có tab «**Transport**» giới hạn ở cài đặt `sockopt` – đủ để hoạt động ở chế độ **TProxy** (proxy trong suốt/redirect qua `sockopt.tproxy`). Danh sách thả xuống chọn transport (`network`) và tab «Security» cho Tunnel bị ẩn vì TLS/REALITY không được hỗ trợ bởi loại này.

Khi nào nên chọn: để proxy trong suốt/chuyển hướng cổng đến các dịch vụ nội bộ.

Trường «Viết lại cổng» (`rewritePort`) có thể để trống: khi xóa giá trị, nó chỉ đơn giản bị loại khỏi cài đặt inbound mà không gây ra lỗi lưu. (Trước đây việc xóa trường này gây ra lỗi xác thực `settings.rewritePort` và chặn việc lưu, kể cả qua tab JSON.)

5.8. SOCKS / HTTP (giao thức `mixed`)

Trong bản dựng này không có giao thức `socks` riêng – SOCKS và HTTP proxy được kết hợp trong giao thức `mixed` (SOCKS + HTTP kết hợp). Ngoài ra còn có `http` -proxy thuần riêng biệt.

5.8.1. Mixed (SOCKS + HTTP)

Các trường trong khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>auth</code>	<code>password</code>	«Auth» – chế độ xác thực. Tùy chọn: <code>password</code> (bằng login/pass) hoặc <code>noauth</code> (không xác thực)
<code>accounts</code>	(tùy chọn)	«Tài khoản» – danh sách cặp user/pass. Khi <code>auth = noauth</code> , trường này không ghi vào cấu hình
<code>udp</code>	<code>false</code> (tắt)	Công tắc «UDP» – hỗ trợ UDP qua SOCKS
<code>ip</code>	<code>127.0.0.1</code>	«UDP IP» – địa chỉ cục bộ cho UDP association. Trường này chỉ hiển thị khi <code>udp</code> được bật

Tài khoản được thêm bằng nút «Thêm»; khi thêm, login ngẫu nhiên (8 ký tự) và mật khẩu (12 ký tự) được tạo tự động, có thể chỉnh sửa.

5.8.2. HTTP (proxy thuần)

Mục đích: forward proxy HTTP truyền thống. Ở cấp độ Xray không theo dõi client như «billing» (không có email/giới hạn) – chỉ có danh sách tài khoản.

Các trường trong khối `settings` :

Trường	Giá trị mặc định	Mô tả
<code>accounts</code>	<code>[]</code>	«Tài khoản» – danh sách cặp user/pass (cả hai trường đều bắt buộc)
<code>allowTransparent</code>	<code>false</code> (tắt)	«Cho phép trong suốt» (tiếng Anh «Allow transparent») – chuyển tiếp yêu cầu với header Host gốc

Khi nào nên chọn SOCKS/HTTP: cho proxy cục bộ hoặc nội bộ không cần giả mạo phức tạp. `mixed` tiện lợi vì một cổng phục vụ cả client SOCKS và HTTP.

5.9. WireGuard (inbound)

Mục đích: inbound WireGuard – tunnel VPN WireGuard, chứ không phải proxy có giả mạo. Transport và TLS/REALITY không áp dụng được cho nó.

Kể từ phiên bản 3.4.2, WireGuard chuyển sang mô hình đa khách hàng (như VLESS/VMess/Trojan/Shadowsocks/Hysteria). Bản thân inbound chỉ lưu phần máy chủ; các thiết bị peer nay được thêm dưới dạng **khách hàng** thông thường (xem phần 8) – với việc tự động cấp địa chỉ trong tunnel, hỗ trợ đăng ký, giới hạn lưu lượng/thời hạn và nhóm. Danh sách inline «Peer / Thêm peer» cũ trong biểu mẫu inbound đã bị xóa: inbound mới được tạo không có peer, còn mỗi khách hàng được tự động cấp địa chỉ.

Các trường inbound (khối settings):

Trường	Giá trị mặc định	Mô tả
secretKey	–	Khóa riêng của máy chủ (bắt buộc). Có nút tạo bên cạnh; khóa công khai (Public Key) được suy ra từ nó tự động (chỉ đọc; publicKey lưu riêng trước đây đã bị xóa)
mtu	1420	MTU của giao diện
dns	1.1.1.1, 1.0.0.1	Mới trong 3.4.2. DNS được ghi vào dòng DNS = của .conf khách hàng
noKernelTun	false (tắt)	«TUN không kernel» – userspace-TUN thay vì kernel
domainStrategy	(tùy chọn)	«Domain Strategy»: ForceIP, ForceIPv4, ForceIPv4v6, ForceIPv6, ForceIPv6v4

Tham số WireGuard của khách hàng – tab «Thông tin đăng nhập» trong cửa sổ thêm/sửa khách hàng (hiển thị khi inbound được liên kết là WireGuard):

Trường	Mô tả
«Khóa riêng WireGuard»	Khóa riêng của khách hàng (có thể chỉnh sửa, có nút «Tạo lại»); khi nhập, khóa công khai được suy ra từ nó tự động
«Khóa công khai WireGuard»	Chỉ đọc; được tính từ khóa riêng
«Khóa chung WireGuard» (PSK)	Khóa chia sẻ bổ sung tùy chọn
«IP được phép WireGuard»	Từ 3.5.0 – có thể chỉnh sửa (placeholder 10.0.0.2/32, gợi ý «Để trống để tự động cấp; các mục nhập phân tách bằng dấu phẩy»). Để trống – địa chỉ được cấp tự động; máy chủ kiểm tra từng mục nhập (IP hoặc CIDR), chuẩn hóa các địa chỉ đơn lẻ thành /32 và từ chối địa chỉ đã bị khách hàng khác của inbound này chiếm dụng

Địa chỉ trong tunnel được máy chủ cấp tự động từ mạng con của inbound (mặc định 10.0.0.0/24 : máy chủ chiếm .1, khách hàng – bắt đầu từ .2); nếu các khách hàng hiện có dùng /24 khác, các địa chỉ mới được cấp trong cùng mạng con.

Domain Strategy và tab Transport

Ngoài những trường đã liệt kê, inbound WireGuard còn có trường **Domain Strategy** (chiến lược phân giải domain: ForceIP, ForceIPv4, ForceIPv4v6, ForceIPv6, ForceIPv6v4). Trường không bắt buộc và chỉ ghi vào cấu hình khi được thiết lập.

Trường **Workers** (workers , số luồng làm việc) đã bị xóa khỏi biểu mẫu WireGuard (cả inbound và outbound): kể từ xay-core v26.6.22, engine không còn sử dụng nó và dựa vào cơ chế nội bộ của wireguard-go. Các cấu hình đã lưu trước đó hoạt động không thay đổi – khi phân tích, trường chỉ đơn giản bị bỏ qua, không cần migration.

Cho WireGuard cũng có sẵn tab **«Transport»** – nhưng ở dạng rút gọn: trên đó chỉ cấu hình `sockopt` và obfuscation **Finalmask**. Danh sách thả xuống chọn transport (network) bị ẩn vì WireGuard luôn lắng nghe qua UDP. Trong các bản ghi noise (noise), Finalmask có trường riêng **Rand Range** (khoảng byte 0–255, có xác thực), và phương thức obfuscation **Salamander** khả dụng cho WireGuard và Hysteria.

Thao tác inbound và xuất (từ 3.5.0)

Menu của dòng WireGuard-inbound nay chứa bộ thao tác «đa khách hàng» đầy đủ – **«Xuất tất cả liên kết»**, xuất theo đăng ký, **liên kết/hủy liên kết khách hàng**, thêm vào nhóm và xóa tất cả khách hàng (trước đây chỉ có xuất/đặt lại/nhân bản/xóa), còn trong danh sách hiển thị bộ đếm khách hàng. Với WireGuard, cửa sổ «Xuất tất cả liên kết» được chia thành hai tab: **Config** (các khối `.conf` ghép lại, mỗi khách hàng một khối) và **Links** (các liên kết cá nhân `wireguard://`); «Sao chép» và «Tải xuống» tác động lên tab đang mở. Với inbound đặt trên node, `Endpoint` trong `.conf` /QR của khách hàng nay trở nên **địa chỉ của node**, chứ không phải của bảng điều khiển chính.

Cấu hình và chia sẻ khách hàng WireGuard

Với khách hàng WireGuard, trong các cửa sổ «Thông tin»/QR có **thẻ cấu hình có thể thu gọn**: `.conf` đầy đủ ([Interface] với PrivateKey / Address / DNS / MTU và [Peer] với PublicKey / PresharedKey / AllowedIPs = 0.0.0.0/0, ::/0 / Endpoint / PersistentKeepalive) cùng các nút **Sao chép**, **Tải xuống** và **QR**. Cũng hỗ trợ liên kết dạng `wireguard:// wg://`, được ứng dụng client phân tích thành `.conf` sẵn sàng.

Khi nào nên chọn WireGuard: khi cần đúng tunnel VPN WireGuard, không phải proxy có giả mạo.

5.10. Hysteria (mặc định v2)

Mục đích: inbound Hysteria qua QUIC. Panel mặc định làm việc với phiên bản 2. Mỗi client xác thực bằng token `auth` thay vì UUID/mật khẩu. TLS cho Hysteria luôn khả dụng (xem bảng tính năng trong 5.2).

Các trường trong khối settings :

Trường	Giá trị mặc định	Mô tả
version	2	Phiên bản giao thức (tối thiểu 1; panel mặc định 2)
clients	[]	Danh sách client

Trường quan trọng của mỗi client – `auth` (token, bắt buộc) cộng với các trường chung (`email` , giới hạn, `enable` , `tgId` , `subId` , `comment` , `reset`).

Các tham số bổ sung được thiết lập trong `streamSettings.hysteriaSettings` :

Trường	Giá trị / tùy chọn	Mô tả
<code>version</code>	cố định 2 (trường bị khóa)	«Phiên bản» (tiếng Anh «Version»)
<code>udpIdleTimeout</code>	(số nguyên ≥ 1, giây)	«UDP idle timeout (giây)» – thời gian chờ không hoạt động UDP
<code>masquerade</code>	tắt mặc định	«Masquerade» – giả mạo máy chủ web thông thường với các yêu cầu không xác thực

Khi bật `masquerade`, có thể chọn loại (`type`):

- ```` – default (trang 404);
- `proxy` – reverse proxy (các trường «Upstream URL», «Viết lại Host», «Bỏ qua TLS verify»);
- `file` – phục vụ thư mục (trường «Thư mục», ví dụ `/var/www/html`);
- `string` – phản hồi cố định (các trường «Mã trạng thái», «Body», «Headers»).

Khi nào nên chọn Hysteria: khi cần transport QUIC và độ ổn định trên các kênh không ổn định/di động; `masquerade` tăng tính ẩn của điểm vào.

5.11. MTProto (proxy cho Telegram)

Được thêm vào phiên bản **3.3.0**. Giá trị giao thức – `mtproto`.

MTProto – giao thức proxy riêng của Telegram. Trong 3X-UI, inbound như vậy **được phục vụ không phải bởi Xray mà bởi tiến trình riêng `mtg`**, do panel quản lý. Panel định kỳ đối chiếu các inbound MTProto được bật với các tiến trình `mtg` đang chạy: khởi động những cái thiếu, dừng những cái thừa và lấy số liệu lưu lượng từ metrics của `mtg`. Do đó **theo dõi lưu lượng** cho inbound này hoạt động như các giao thức thông thường.

Kể từ phiên bản 3.5.0, MTProto là giao thức đa khách hàng (engine được đổi sang fork `mtg-multi`): một inbound phục vụ nhiều người dùng, mỗi người là một **client** thông thường (xem phần 8) với secret FakeTLS riêng, hạn mức lưu lượng, thời hạn, công tắc bật/tắt, ad-tag tùy chọn và liên kết `tg://proxy` cá nhân. Mô hình cũ «một inbound = một secret» đã bị loại bỏ: trường «Secret» ở cấp inbound bị xóa khỏi biểu mẫu, còn khi cập nhật panel, secret đơn lẻ hiện có tự động trở thành client đầu tiên của inbound đó (không cần thao tác thủ công).

Hệ quả:

- Tab **«Transport» (Stream Settings)** **không áp dụng cho inbound này**; còn client nay được quản lý như ở các giao thức khác – liên kết/hủy liên kết, giới hạn, nhóm, đăng ký.
- Các thay đổi của client (thêm, xóa, đổi secret, bật/tắt, ad-tag) được áp dụng **«ngay lập tức»** qua management-API của `mtg`, không làm gián đoạn phiên Telegram của những người dùng còn lại; chỉ các thay đổi cấu trúc của inbound (địa chỉ, domain fronting, giới hạn kết nối, IP công khai) mới khởi động lại tiến trình.

- Inbound MTProto chỉ chạy **trên panel chính**; không được triển khai lên node con (node có **NodeID** được chỉ định sẽ bị bỏ qua).
- Tab «**Sniffing**» cho MTProto bị ẩn — giao thức này được phục vụ bởi tiến trình `mtg`, không phải Xray, vì vậy sniffing không áp dụng được cho nó.

Các trường. Được lưu trong `settings` của inbound:

Trường trong UI	Khóa	Mô tả
Remark	<code>remark</code>	Nhãn inbound.
Listen IP	<code>listen</code>	IP để lắng nghe (trống = tất cả giao diện).
Port	<code>port</code>	Cổng proxy.
Domain giả mạo (FakeTLS)	<code>settings.fakeTlsDomain</code>	Mặc định <code>www.cloudflare.com</code> . Từ 3.5.0 — domain mặc định để tạo secret cho các client mới (gợi ý: «Default FakeTLS domain used to generate a new client's secret. Each client can front its own domain»); mỗi client có thể giả mạo bằng domain riêng của mình qua secret của họ.
Max connections	<code>settings.throttleMaxConnections</code>	Mới trong 3.5.0. Giới hạn số kết nối đồng thời cho tất cả người dùng với việc chia đều; <code>0</code> — không giới hạn.
Public IPv4 / Public IPv6	<code>settings.publicIpv4</code> / <code>settings.publicIpv6</code>	Mới trong 3.5.0. Địa chỉ công khai của máy chủ cho ad-tag middle proxy (placeholder <code>1.2.3.4</code> / <code>2001:db8::1</code>); để trống — <code>mtg</code> sẽ tự xác định.

Định dạng secret (FakeTLS). Secret nay thuộc về **từng client** (trường «**MTProto secret**» trong biểu mẫu client, với nút tạo lại; bên cạnh — trường tùy chọn «**Ad-tag (kênh tài trợ)**», đúng 32 ký tự hex). Dạng secret vẫn như cũ: `ee` + 32 ký tự hex + mã hex của domain giả mạo (`ee<hex32><hex(domain)>`); khi liên kết client mới với MTProto-inbound, secret được tạo tự động từ domain mặc định của inbound. Hạn mức lưu lượng và thời hạn hiệu lực của client từ 3.5.0 **thực sự được áp dụng** (qua các giới hạn của `mtg-multi`): client đã hết hạn mức hoặc hết hạn sẽ bị ngắt, còn việc đặt lại lưu lượng khôi phục quyền truy cập ngay lập tức.

Domain-fronting và các tùy chọn nâng cao của mtg

Inbound MTProto có các tham số bổ sung của tiến trình `mtg`. Các trường **Domain fronting IP**, **Domain fronting port** và **Domain fronting PROXY protocol** xác định nơi `mtg` gửi lưu lượng không phải Telegram (ví dụ đến trang NGINX giả mạo): nếu để IP trống, domain FakeTLS được sử dụng qua DNS, cổng mặc định — `443`. Ngoài ra còn có **Accept PROXY protocol** (cho listener), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`) và **Debug logging**. Mỗi giá trị được ghi vào file `mtg-<id>.toml` chỉ khi nó được thiết lập.

Định tuyến lưu lượng Telegram qua Xray

Công tắc «**Route through Xray**» (mặc định tắt) và trường tùy chọn **Outbound** cho phép đưa egress Telegram vào router Xray. Khi bật, panel nhúng vào cấu hình Xray một cầu SOCKS cục bộ với tag của chính inbound đó, và `mtg` gửi lưu lượng Telegram qua nó. Sau đó lưu lượng có thể được đổi chiều bằng các quy tắc trong tab

«Routing» hoặc được chuyển bắt buộc đến outbound hay balancer đã chọn qua trường **Outbound** (nếu trường trống, các quy tắc định tuyến sẽ quyết định).

Cách chia sẻ với người dùng. Cho inbound MTProto, panel tạo link mời:

Ví dụ: secret FakeTLS và link sẵn sàng. Nếu domain giả mạo là `www.cloudflare.com`, secret được tổng hợp thành `ee` + 32 ký tự hex + mã hex của domain, ví dụ:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Link mời sẵn sàng (link này và mã QR được gửi cho người dùng qua Telegram):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<адрес>&port=<порт>&secret=<секрет>
```

(tương đương – `https://t.me/proxy?server=...&port=...&secret=...`). Từ 3.5.0, link là **cá nhân** – được tạo từ secret của từng client cụ thể, còn fragment `#remark` đã bị loại khỏi link (các trình phân tích Telegram không chặt chẽ từng dán nó vào secret và làm hỏng việc nhập); ghi chú được hiển thị dưới dạng nhãn riêng trong cửa sổ thông tin. Link này và mã QR cần được gửi cho người dùng Telegram – khi mở, proxy sẽ được thêm ngay vào ứng dụng. Link cũng được cung cấp qua máy chủ subscription.

Khi nào nên sử dụng. Phương pháp tiêu chuẩn để vượt chặn Telegram; giả mạo FakeTLS (domain giả mạo) làm cho lưu lượng trông giống như đang truy cập thông thường vào trang được chỉ định.

5.12. Bảng tóm tắt nhanh về chọn giao thức

- **VLESS** – lựa chọn mặc định; phương án tốt nhất với REALITY hoặc TLS + XTLS-Vision, hỗ trợ xác thực hậu lượng tử.
- **Trojan** – giả mạo HTTPS với fallback đến máy chủ web.
- **VMess** – tương thích với client cũ.
- **Shadowsocks** – proxy đơn giản không có TLS; lựa chọn hiện đại – phương thức `2022-blake3-*`.
- **Hysteria** – QUIC, độ ổn định trên kênh kém.
- **mixed / http** – SOCKS/HTTP proxy nội bộ.
- **WireGuard** – tunnel VPN đầy đủ tính năng.
- **tunnel** – chuyển hướng cổng trong suốt.
- **MTProto** – proxy để vượt chặn Telegram (FakeTLS); tiến trình riêng `mtg`.

6. Truyền tải (Stream Settings)

Truyền tải (trong giao diện panel – trường «**Truyền tải**», tiếng Anh *Transmission*) xác định cách Xray-core truyền dữ liệu bên trong inbound: giao thức mạng nào được sử dụng bên trên TLS/Reality và cách đóng khung lưu lượng. Các thông số này được lưu vào đối tượng `streamSettings` của cấu hình Xray và được thiết lập trong tab truyền tải của trình soạn thảo inbound. Mã hóa (TLS / Reality) được trình bày trong mục riêng – ở đây chỉ mô tả việc chọn mạng và các thông số của nó.

6.1. Chọn mạng truyền tải

Mạng được chọn trong danh sách thả xuống «**Truyền tải**» (`streamSettings.network`). Giá trị mặc định là `tcp` (hiển thị trong danh sách là **RAW**). Các tùy chọn có sẵn:

Giá trị trong danh sách	Trường <code>network</code>	Truyền tải
RAW	<code>tcp</code>	TCP thông thường (trong các phiên bản Xray mới được đổi tên thành RAW), tùy chọn có che giấu HTTP
mKCP	<code>kcp</code>	Truyền tải UDP đáng tin cậy mKCP
WebSocket	<code>ws</code>	WebSocket qua HTTP(S)
gRPC	<code>grpc</code>	Đường hầm gRPC (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP – truyền tải ghép kênh hiện đại

Khi thay đổi giá trị, panel xóa khối cài đặt của mạng cũ và điền khối của mạng mới bằng các giá trị mặc định từ schema của nó, do đó mỗi trường của biểu mẫu con luôn có giá trị ban đầu có ý nghĩa.

Lưu ý. Trong bản build này của panel **truyền tải HTTP/2 (h2) không có trong danh sách** – nó đã bị loại khỏi tập hợp các mạng; để tạo đường hầm hai chiều giống HTTP/2, hãy sử dụng gRPC, còn để có truyền tải HTTP giả mạo hiện đại – XHTTP. Truyền tải **Hysteria** (`hysteria`) không được chọn qua danh sách này: nó được gán cứng với giao thức Hysteria và xuất hiện tự động khi inbound được tạo với giao thức Hysteria (xem mục 6.8).

Bên dưới từng mạng và từng trường của nó được phân tích riêng.

6.2. RAW / TCP (`tcpSettings`)

Truyền tải TCP cơ bản. Theo mặc định, lưu lượng được truyền «nguyên dạng»; tùy chọn có thể giả mạo thành trao đổi HTTP/1.1 thông thường.

Trường	Giá trị mặc định	Mô tả
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (tắt)	Chấp nhận tiêu đề PROXY protocol từ bộ cân bằng tải/proxy phía trước
Che giấu HTTP (<code>header.type</code>)	<code>none</code> (tắt)	Bật che giấu lưu lượng dưới dạng HTTP/1.1

Proxy Protocol

Công tắc «**Proxy Protocol**» (`acceptProxyProtocol`). Khi bật, Xray chờ tiêu đề PROXY protocol trên kết nối đến và trích xuất IP thực của khách hàng từ đó. Chỉ bật khi trước panel có proxy ngược/bộ cân bằng tải (ví dụ HAProxy hoặc nginx với `send-proxy`) thêm tiêu đề này. Tắt theo mặc định.

Che giấu HTTP (camouflage)

Công tắc «**Che giấu HTTP**». Quản lý trường `header` :

- **Tắt** → `header.type = "none"` (trên đường truyền trường `header` đơn giản là vắng mặt). TCP thuần túy.
- **Bật** → `header.type = "http"` . Lưu lượng được đóng khung dưới dạng yêu cầu và phản hồi HTTP/1.1. Khi bật, panel ngay lập tức điền các đối tượng con `request` và `response` bằng các giá trị mặc định.

Khi bật che giấu HTTP, các trường cài đặt yêu cầu và phản hồi giả mạo sẽ xuất hiện.

Tiêu đề yêu cầu (`header.request`):

Trường	Khóa	Giá trị mặc định	Mô tả
Phiên bản yêu cầu	<code>request.version</code>	<code>1.1</code>	Phiên bản HTTP trong dòng bắt đầu yêu cầu
Phương thức yêu cầu	<code>request.method</code>	<code>GET</code>	Phương thức HTTP của yêu cầu giả mạo
Đường dẫn yêu cầu	<code>request.path</code>	<code>/</code>	Đường dẫn. Nhập danh sách giá trị cách nhau bằng dấu phẩy; trên đường truyền đây là mảng chuỗi. Nếu để trống, mặc định là <code>/</code>
Tiêu đề yêu cầu	<code>request.headers</code>	<code>{}</code> (trống)	Bảng «Tên/Giá trị» của các tiêu đề HTTP. Lưu dưới dạng map <code>tên → [giá trị]</code> (một tên có thể có nhiều giá trị)

Tiêu đề phản hồi (`header.response`):

Trường	Khóa	Giá trị mặc định	Mô tả
Phiên bản phản hồi	<code>response.version</code>	<code>1.1</code>	Phiên bản HTTP trong dòng bắt đầu phản hồi
Trạng thái phản hồi	<code>response.status</code>	<code>200</code>	Mã trạng thái HTTP của phản hồi giả mạo

Trường	Khóa	Giá trị mặc định	Mô tả
Lý do phản hồi	<code>response.reason</code>	OK	Mô tả văn bản của trạng thái (reason-phrase)
Tiêu đề phản hồi	<code>response.headers</code>	<code>{}</code> (trống)	Bảng «Tên/Giá trị» của các tiêu đề phản hồi (map <code>tên</code> → <code>giá trị</code>)

Các trường tiêu đề được chỉnh sửa theo từng dòng – mỗi dòng xác định tên tiêu đề (`Tên`) và giá trị của nó (`Giá trị`). Các thông số này chỉ dùng để che giấu diện mạo lưu lượng; chúng không ảnh hưởng đến mật mã học. Các giá trị mặc định (GET / HTTP/1.1 , phản hồi 200 OK) phù hợp với hầu hết các tình huống – chỉ nên thay đổi khi cần giả mạo một trang web/dịch vụ cụ thể.

Ví dụ `streamSettings` cho RAW với che giấu HTTP:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Lưu ý: `path` trên đường truyền là mảng chuỗi, và mỗi tiêu đề là mảng giá trị (`Host` → `["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP – truyền tải đáng tin cậy qua UDP. Hữu ích trên các kênh có mật gói và độ trễ cao, nhưng tạo ra lưu lượng phục vụ cao hơn. Tất cả các giá trị mặc định đều khớp với các giá trị được khuyến nghị trong xray-core.

Trường	Khóa	Mặc định	Cho phép	Mô tả
MTU	<code>mtu</code>	1350	576–1460	Kích thước gói tối đa (byte). Giảm khi có vấn đề phân mảnh

Trường	Khóa	Mặc định	Cho phép	Mô tả
TTI (ms)	<code>tti</code>	<code>20</code>	10–100	Khoảng thời gian truyền (ms). Nhỏ hơn – độ trễ thấp hơn nhưng chi phí cao hơn
Uplink (MB/s)	<code>uplinkCapacity</code>	<code>5</code>	≥ 0	Bảng thông tải lên ước tính (MB/s)
Downlink (MB/s)	<code>downlinkCapacity</code>	<code>20</code>	≥ 0	Bảng thông tải xuống ước tính (MB/s)
Hệ số CWND	<code>cwndMultiplier</code>	<code>1</code>	≥ 1	Hệ số cửa sổ tắc nghẽn (congestion window)
Cửa sổ gửi tối đa	<code>maxSendingWindow</code>	<code>2097152</code>	≥ 0	Kích thước cửa sổ gửi tối đa

Ghi chú về các trường:

- **Uplink / Downlink capacity** xác định mức độ tích cực mKCP chiếm dụng kênh. Đặt theo bảng thông thực tế của kênh: giá trị quá cao dẫn đến lưu lượng thừa, quá thấp – khai thác kênh không hiệu quả.
- **TTI** trực tiếp ảnh hưởng đến sự đánh đổi «độ trễ [?] chi phí»: giá trị nhỏ hơn giảm độ trễ nhưng tăng lượng gói phục vụ.
- **MTU** giới hạn kích thước một gói mKCP; giảm giá trị giúp trên các kênh mà các gói UDP lớn bị cắt hoặc mất.

Trong bản build này trường «seed» (mật khẩu che giấu mKCP) và danh sách thả xuống **loại tiêu đề/che giấu** (`none` , `srtcp` , `utp` , `wechat-video` , `dtls` , `wireguard`) trong biểu mẫu con mKCP **không có dưới dạng các trường riêng biệt** – che giấu tầng truyền tải đã được tách ra thành cơ chế chung «FinalMask» (bao gồm chế độ `mkcp-legacy`), được mô tả trong mục tương ứng. Thông số «congestion» dưới dạng hộp kiểm riêng biệt cũng không được hiển thị; kiểm soát tắc nghẽn được thiết lập thông qua `cwndMultiplier` và `maxSendingWindow` .

6.4. WebSocket (`wsSettings`)

Truyền tải WebSocket qua HTTP(S). Đi qua CDN và proxy ngược tốt, giả mạo thành lưu lượng web thông thường.

Trường	Khóa	Mặc định	Mô tả
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Chấp nhận tiêu đề PROXY protocol từ proxy phía trước (xem mục 6.2)
Host	<code>host</code>	<code>""</code> (trống)	Giá trị tiêu đề HTTP <code>Host</code> . Chỉ định khi làm việc qua CDN/ domain fronting
Đường dẫn	<code>path</code>	<code>/</code>	Đường dẫn trong yêu cầu bắt tay WebSocket
Chu kỳ heartbeat	<code>heartbeatPeriod</code>	<code>0</code>	Khoảng thời gian gửi khung heartbeat (giây). <code>0</code> tắt heartbeat

Trường	Khóa	Mặc định	Mô tả
Tiêu đề	headers	{ } (trống)	Các tiêu đề HTTP bắt tay bổ sung. Map «Tên → Giá trị» (chỉ giá trị chuỗi, không có mảng)

Ghi chú:

- **Đường dẫn** phải khớp trên máy chủ (inbound) và ở phía khách hàng. Thường đường dẫn này được dùng để che giấu điểm truy cập ở phía máy chủ web.
- **Host** có ý nghĩa khi inbound đứng sau CDN hoặc sử dụng domain fronting; nếu không có thể để trống.
- **Chu kỳ heartbeat** giữ kết nối «còn sống» khi đi qua proxy/CDN cắt đứt phiên không hoạt động một cách tích cực. Theo mặc định (0) heartbeat bị tắt.
- Khác với RAW, bảng tiêu đề WebSocket sử dụng định dạng «phẳng» tên → giá trị (một dòng giá trị cho mỗi tiêu đề).

Ví dụ `streamSettings` cho WebSocket sau CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

Các giá trị `host` và `path` phải khớp ở phía khách hàng; khác với RAW, giá trị tiêu đề ở đây là chuỗi thông thường, không phải mảng.

6.5. gRPC (`grpcSettings`)

Truyền tải «nhẹ nhàng» nhất về số lượng thông số. Tạo đường hầm lưu lượng bên trong các lời gọi gRPC (qua HTTP/2); tương thích tốt với CDN hỗ trợ gRPC. Không có che giấu tiêu đề.

Trường	Khóa	Mặc định	Mô tả
Tên dịch vụ (<code>Service Name</code>)	<code>serviceName</code>	"" (trống)	Tên dịch vụ gRPC (thực tế là «đường dẫn» của đường hầm). Phải khớp ở máy chủ và khách hàng
Authority	<code>authority</code>	"" (trống)	Giá trị pseudo-header <code>:authority</code> (tương tự <code>Host</code> cho HTTP/2). Chỉ định khi làm việc qua CDN/tên miền
Multi Mode	<code>multiMode</code>	<code>false</code> (tắt)	Bật ghép kênh nhiều luồng gRPC song song bên trong một kết nối

Ghi chú:

- **Service Name** — định danh chính của kênh gRPC; phải giống nhau ở cả hai phía. Giá trị trống được phép, nhưng thường đặt một chuỗi không rõ ràng để che giấu.
- **Authority** ảnh hưởng đến `:authority` nào được gửi trong các khung HTTP/2; cần thiết chủ yếu khi proxy qua CDN.
- **Multi Mode** cho phép nhiều luồng logic đi qua một kết nối vật lý; bật để cải thiện hiệu năng khi cả máy chủ và khách hàng đều hỗ trợ điều này.

Ví dụ `streamSettings` cho gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (ở đây `GunService`) đóng vai trò «đường dẫn» của đường hầm và phải khớp ở máy chủ và khách hàng.

6.6. HTTPUpgrade (`httpupgradeSettings`)

Truyền tải dựa trên cơ chế HTTP Upgrade (như WebSocket, nhưng không có bản thân giao thức WebSocket). Cũng đi qua proxy và CDN tốt. Tập hợp trường lặp lại WebSocket, nhưng **không có** chu kỳ heartbeat.

Trường	Khóa	Mặc định	Mô tả
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Chấp nhận tiêu đề PROXY protocol từ proxy phía trước
Host	<code>host</code>	<code>""</code> (trống)	Giá trị tiêu đề HTTP Host
Đường dẫn	<code>path</code>	<code>/</code>	Đường dẫn của yêu cầu HTTP với tiêu đề Upgrade
Tiêu đề	<code>headers</code>	<code>{}</code> (trống)	Các tiêu đề HTTP bổ sung. Map «phẳng» tên → giá trị (như WebSocket)

Mục đích của các trường **Host**, **Đường dẫn** và **Tiêu đề** giống với WebSocket (mục 6.4). Heartbeat không được cung cấp cho HTTPUpgrade — đây là đặc điểm riêng của WebSocket.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (còn gọi là SplitHTTP) — truyền tải HTTP ghép kênh hiện đại của xray-core. Phân tách luồng đi lên và đi xuống thành các yêu cầu HTTP riêng biệt, phù hợp tốt cho CDN và môi trường có giới hạn về thời lượng

kết nối. Không phải tất cả các trường đều hiển thị trong trình soạn thảo cùng một lúc: một số xuất hiện tùy thuộc vào chế độ được chọn (`mode`) và các công tắc đã bật.

Các trường cơ bản (luôn hiển thị)

Trường	Khóa	Mặc định	Mô tả
Host	<code>host</code>	<code>""</code> (trống)	Giá trị tiêu đề HTTP <code>Host</code>
Đường dẫn	<code>path</code>	<code>/</code>	Đường dẫn cơ bản của các yêu cầu HTTP
Chế độ (<code>Mode</code>)	<code>mode</code>	<code>auto</code>	Chế độ truyền (xem bên dưới)
Server Max Header Bytes	<code>serverMaxHeaderBytes</code>	<code>0</code>	Giới hạn kích thước tiêu đề yêu cầu trên máy chủ (byte). <code>0</code> – giá trị mặc định của xray-core
Padding Bytes	<code>xPaddingBytes</code>	<code>100–1000</code>	Phạm vi đệm ngẫu nhiên (byte, định dạng <code>min–max</code>) để gây khó khăn cho việc phân tích kích thước
Tiêu đề	<code>headers</code>	<code>{}</code> (trống)	Các tiêu đề HTTP bổ sung. Map «phẳng» tên → giá trị
Phương thức HTTP Uplink	<code>uplinkHTTPMethod</code>	<code>""</code> (Default = POST)	Phương thức HTTP của các yêu cầu đi lên. Tùy chọn: trống (mặc định POST), <code>POST</code> , <code>PUT</code> , <code>GET</code> (cái sau chỉ khả dụng trong chế độ <code>packet–up</code>)
Padding Obfs Mode	<code>xPaddingObfsMode</code>	<code>false</code> (tắt)	Bật che giấu đệm nâng cao và mở các trường bổ sung (xem bên dưới)
No SSE Header	<code>noSSEHeader</code>	<code>false</code> (tắt)	Không gửi tiêu đề <code>Content–Type: text/event–stream</code> (SSE). Bật nếu nó cản trở việc đi qua các nút trung gian

Trường «Chế độ» (`mode`)

Danh sách thả xuống với các giá trị:

Giá trị	Mô tả
<code>auto</code>	Tự động chọn chế độ (mặc định)
<code>packet–up</code>	Luồng đi lên được chia thành các yêu cầu HTTP riêng biệt (một gói mỗi yêu cầu)
<code>stream–up</code>	Luồng đi lên được truyền trong một yêu cầu phát trực tiếp kéo dài
<code>stream–one</code>	Một yêu cầu phát trực tiếp hai chiều chung

Việc chọn chế độ xác định các trường bổ sung nào trở nên hiển thị.

Các trường chỉ hiển thị khi `mode` = `packet–up` :

Trường	Khóa	Mặc định	Mô tả
Tải lên được đệm tối đa	<code>scMaxBufferedPosts</code>	<code>30</code>	Số yêu cầu POST đi lên được đệm đồng thời tối đa
Kích thước tải lên tối đa (byte)	<code>scMaxEachPostBytes</code>	<code>1000000</code>	Kích thước tối đa của một yêu cầu POST đi lên (byte)
Uplink Data Placement	<code>uplinkDataPlacement</code>	<code>""</code> (Default = body)	Nơi đặt dữ liệu luồng đi lên: <code>body</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Uplink Data Key	<code>uplinkDataKey</code>	<code>""</code>	Tên khóa/tiêu đề cho dữ liệu uplink. Chỉ xuất hiện nếu <code>uplinkDataPlacement</code> được đặt và không bằng <code>body</code>

Trường chỉ hiển thị khi `mode = stream-up` :

Trường	Khóa	Mặc định	Mô tả
Stream-Up Server	<code>scStreamUpServerSecs</code>	<code>20-80</code>	Phạm vi thời gian giữ kết nối phát trực tiếp phía máy chủ (giây, định dạng <code>min-max</code>)

Các trường che giấu đệm (hiển thị khi `xPaddingObfsMode = bật`)

Trường	Khóa	Mặc định	Mô tả
Padding Key	<code>xPaddingKey</code>	<code>""</code> (placeholder <code>x_padding</code>)	Tên khóa cho đệm
Padding Header	<code>xPaddingHeader</code>	<code>""</code> (placeholder <code>X-Padding</code>)	Tên tiêu đề HTTP truyền đệm
Padding Placement	<code>xPaddingPlacement</code>	<code>""</code> (Default = <code>queryInHeader</code>)	Nơi đặt đệm: <code>queryInHeader</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Padding Method	<code>xPaddingMethod</code>	<code>""</code> (Default = <code>repeat-x</code>)	Phương thức tạo đệm: <code>repeat-x</code> hoặc <code>tokenish</code>

Vị trí phiên và chuỗi (luôn hiển thị)

Trường	Khóa	Mặc định	Mô tả
Session ID Placement	<code>sessionIDPlacement</code>	<code>""</code> (Default = <code>path</code>)	Nơi truyền định danh phiên: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Session ID Key	<code>sessionIDKey</code>	<code>""</code> (placeholder <code>x_session</code>)	Tên khóa phiên. Chỉ xuất hiện nếu <code>sessionIDPlacement</code> được đặt và không bằng <code>path</code>

Trường	Khóa	Mặc định	Mô tả
Session ID Table	<code>sessionIDTable</code>	"" (placeholder Base62)	Bộ ký tự để tạo định danh phiên. Có thể chọn từ danh sách tự động hoàn thành được xác định trước (ALPHABET , Alphabet , BASE36 , Base62 , HEX , alphabet , base36 , hex , number) hoặc nhập chuỗi ASCII tùy ý. Trống – giá trị mặc định của xray-core
Session ID Length	<code>sessionIDLength</code>	"" (trống)	Độ dài hoặc phạm vi (ví dụ 8–16) của định danh được tạo. Chỉ hiển thị khi Session ID Table được đặt; giá trị tối thiểu phải lớn hơn 0
Sequence Placement	<code>seqPlacement</code>	"" (Default = path)	Nơi truyền số thứ tự gói: path , header , cookie , query
Sequence Key	<code>seqKey</code>	"" (placeholder x_seq)	Tên khóa chuỗi. Chỉ xuất hiện nếu seqPlacement được đặt và không bằng path

Các trường phiên được đổi tên theo xray-core v26.6.22: trước đây chúng được gọi là **Session Placement / Session Key** (`sessionPlacement` / `sessionKey`) – bây giờ là **Session ID Placement / Session ID Key** (`sessionIDPlacement` / `sessionIDKey`); phần lỗi không còn hiểu tên cũ nữa. Các inbound được lưu trước khi cập nhật sẽ được tự động chuyển sang các khóa mới – không cần lưu lại.

Khuyến nghị:

- Đối với hầu hết các cài đặt, chỉ cần để **Chế độ = auto** , đặt **Đường dẫn/Host** và (khi làm việc qua CDN) đồng bộ chúng với khách hàng.
- Các trường vị trí (`*Placement` / `*Key`) và che giấu đệm chỉ cần thiết để tinh chỉnh cho tình huống anti-DPI/CDN cụ thể; khi để trống, các giá trị mặc định của xray-core được ghi trong ngoặc sẽ được sử dụng.
- Các thông số liên quan đến phía khách hàng/outbound (ví dụ: khoảng thời gian POST lặp lại, kích thước chunk) không hiển thị trong biểu mẫu inbound – máy chủ lắng nghe bỏ qua chúng. Bộ ghép kênh XMUX, ngược lại, khả dụng trong biểu mẫu inbound (xem bên dưới).
- **Các giá trị mặc định phục vụ không được đặt.** Panel không còn ghi các giá trị mặc định phục vụ `scMaxEachPostBytes` và `scMinPostsIntervalMs` vào cấu hình XHTTP – các giá trị nội bộ của xray-core được áp dụng. Điều này loại bỏ chữ ký DPI cố định (ký tự `scMinPostsIntervalMs=30`) mà trước đây có thể bị chặn lưu lượng. Đối với các inbound đã lưu, các giá trị khớp với mặc định của xray-core không được xuất trong liên kết và đăng ký (không cần lưu lại inbound); các giá trị được đặt thủ công vẫn được lưu giữ.

Ví dụ `streamSettings` cho XHTTP (chế độ `auto`):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Đối với hầu hết các cài đặt, bốn trường này là đủ; các trường vị trí phiên/chuỗi và che giấu đệm để trống – khi đó các giá trị mặc định của xray-core sẽ được sử dụng.

XMUX (ghép kênh kết nối)

Công tắc **XMUX** (`enableXmux`) bật lớp ghép kênh phân phối các yêu cầu song song qua một nhóm nhỏ kết nối vật lý. Khi bật, sáu trường có thể cấu hình sẽ được mở (lưu trong `xhttpSettings.xmux`):

Trường	Khóa	Mặc định	Mô tả
Max Concurrency	<code>maxConcurrency</code>	16–32	Số yêu cầu đồng thời tối đa trên một kết nối (phạm vi <code>min–max</code>)
Max Connections	<code>maxConnections</code>	6	Số kết nối vật lý tối đa (<code>0</code> – không giới hạn). Kể từ phiên bản 3.4.2, các inbound mới được tạo với giá trị <code>6</code> ; các inbound được tạo trước đó giữ nguyên giá trị của mình.
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (trống)	Số lần tái sử dụng kết nối
Max Request Times	<code>hMaxRequestTimes</code>	600–900	Số yêu cầu tối đa trên một kết nối (phạm vi)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800–3000	Thời gian kết nối có thể tái sử dụng (giây, phạm vi)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (trống)	Chu kỳ keep-alive để duy trì kết nối

Lưu ý. Không thể đặt đồng thời **Max Connections** và **Max Concurrency** – xray-core sẽ từ chối cấu hình đó. Theo mặc định khi bật XMUX, panel đặt `Max Concurrency = 16–32` ; nếu bạn đặt **Max Connections** (giá trị lớn hơn `0`), panel sẽ xóa giá trị mặc định `Max Concurrency` để tránh xung đột.

6.8. Truyền tải Hysteria (`hysteriaSettings`)

Truyền tải **Hysteria** không được chọn trong danh sách «Truyền tải»: nó tự động được kích hoạt khi inbound được tạo với giao thức Hysteria, và bị ẩn đối với các giao thức khác (khi rời khỏi giao thức Hysteria, mạng bị buộc trở về `tcp`). Các thông số:

Trường	Khóa	Mặc định	Mô tả
Phiên bản	version	2 (cố định, trường bị khóa)	Phiên bản Hysteria. Chỉ hỗ trợ Hysteria 2
UDP idle timeout (s)	udpIdleTimeout	60	Thời gian chờ không hoạt động của phiên UDP (giây). Phạm vi cho phép – 2–600; xray-core từ chối các giá trị ngoài khoảng này khi khởi động
Masquerade	masquerade	tắt (văng mặt)	Bật che giấu trình lắng nghe thành máy chủ HTTP/3 khi thăm dò

Khi **Masquerade** được bật, sẽ xuất hiện lựa chọn loại (type) và các trường phụ thuộc vào nó:

- **"" – default (404 page)**: trả về trang 404 tiêu chuẩn (không cần trường bổ sung).
- **proxy (reverse proxy)**: proxy ngược đến trang web bên ngoài.
 - url (**Upstream URL**, placeholder `https://www.example.com`) – địa chỉ đích;
 - rewriteHost (**Ghi lại Host**, mặc định `false`) – thay thế tiêu đề `Host` ;
 - insecure (**Bỏ qua xác minh TLS**, mặc định `false`) – không xác minh chứng chỉ TLS của upstream.
- **file (serve directory)**: phục vụ tệp từ thư mục.
 - dir (**Thư mục**, placeholder `/var/www/html`).
- **string (fixed body)**: phản hồi HTTP cố định.
 - statusCode (**Mã trạng thái**, mặc định `0` , phạm vi 0–599);
 - content (**Body**) – nội dung phản hồi;
 - headers (**Tiêu đề**) – map tên → giá trị.

Masquerade cho phép inbound dựa trên Hysteria trông như một máy chủ HTTP/3 thông thường đối với các thăm dò chủ động, giúp tăng tính ẩn dật. Theo mặc định, che giấu bị tắt.

Ví dụ hysteriaSettings với proxy ngược (masquerade → proxy):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Ở đây khi thăm dò, trình lắng nghe trả về phản hồi từ `https://www.example.com` , giả mạo thành một trang web HTTP/3 thông thường.

6.9. Các thông số đi kèm

Ngoài việc chọn mạng, trong cùng tab còn có hai khối chung không phụ thuộc vào truyền tải cụ thể (chi tiết – trong các mục tương ứng):

- **External Proxy** (`externalProxy`) – danh sách địa chỉ/cổng bên ngoài được thay thế vào liên kết đăng ký thay vì địa chỉ của panel.
- **Sockopt** (`sockopt`) – các tùy chọn socket cấp thấp (TCP Fast Open, mark, chiến lược tên miền, proxy trong suốt, v.v.).

Real client IP (xác định IP thực sau CDN/relay)

Khi inbound đứng sau trung gian (CDN như Cloudflare, đường hầm/relay L4 hoặc panel khác), Xray thấy địa chỉ của trung gian, không phải khách truy cập thực. Địa chỉ này xuất hiện trong danh sách khách hàng trực tuyến và theo đó số lượng IP trên mỗi khách hàng được đếm, khiến cả hai trở nên vô dụng sau proxy. Để khôi phục IP thực, trong phần **Sockopt** của biểu mẫu inbound có lựa chọn preset **Real client IP**, kết hợp các cài đặt `acceptProxyProtocol` và `trustedXForwardedFor` :

Preset	Tác dụng	Khi nào áp dụng
Off / direct	Xóa cả hai trường.	Inbound được khách hàng truy cập trực tiếp
Cloudflare CDN	Đặt <code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> .	WebSocket / HTTPUpgrade / XHTTP / gRPC sau CDN Cloudflare (đám mây màu cam)
L4 relay / Spectrum (PROXY)	Bật <code>acceptProxyProtocol = true</code> .	Đường hầm/relay L4 trước inbound hoặc Cloudflare Spectrum

Các preset loại trừ lẫn nhau: chọn một cái sẽ xóa trường của cái kia, do đó `trustedXForwardedFor` cũ không ghi đè IP được khôi phục qua giao thức PROXY. Bên dưới preset, công tắc «thô» **Proxy Protocol** và danh sách **Trusted X-Forwarded-For** vẫn hiển thị – preset chỉ điền chúng cho bạn, và khi cần thiết có thể chỉnh sửa thủ công. Nếu preset được chọn không được hỗ trợ bởi truyền tải hiện tại (ví dụ PROXY-protocol trên mKCP), biểu mẫu hiển thị cảnh báo. Các trường này chỉ liên quan đến phía máy chủ và **không bao giờ được gửi đến khách hàng trong đăng ký**.

Chỉ dùng một cái. `acceptProxyProtocol` đọc IP thực từ tiêu đề L4 của giao thức PROXY, còn `trustedXForwardedFor` – từ tiêu đề HTTP của yêu cầu; kết hợp chúng thủ công chỉ nên làm khi chuỗi trung gian của bạn yêu cầu điều đó.

- **FinalMask** (`finalmask`) – cơ chế che giấu tầng truyền tải chung (bao gồm che giấu legacy mKCP), thay thế các trường riêng biệt «seed»/«header type» bên trong các biểu mẫu con của mạng.

7. Bảo mật kết nối: TLS, XTLS và REALITY

Mỗi inbound hỗ trợ truyền tải qua luồng transport (VMess, VLESS, Trojan, Shadowsocks, Hysteria) đều có tab «**Bảo mật**» trong trình chỉnh sửa. Tab này cấu hình cách kênh transport được mã hóa và ngụy trang. Có ba chế độ, chuyển đổi bằng các nút radio:

Chế độ	Nhãn trong UI	Khi nào khả dụng
none	Không	Luôn luôn (trừ Hysteria, nơi TLS là bắt buộc)
tls	TLS	Cho VMess/VLESS/Trojan/Shadowsocks trên mạng tcp, ws, http, grpc, httpupgrade, xhttp; cho Hysteria – luôn luôn
reality	Reality	Chỉ cho VLESS/Trojan trên mạng tcp, http, grpc, xhttp

Nút **Không** không hiển thị nếu giao thức là Hysteria (TLS bắt buộc đối với nó). Nút **Reality** chỉ xuất hiện với tổ hợp giao thức và mạng hợp lệ (xem bảng trên).

Khi thay đổi chế độ, panel tái tạo hoàn toàn khối `streamSettings`: các `tlsSettings` và `realitySettings` của chế độ trước bị xóa và giá trị mặc định cho chế độ được chọn được điền vào. Cụ thể, khi chọn **Reality**, panel ngay lập tức tự động: điền cặp `target` + `serverNames` (SNI) ngẫu nhiên từ danh sách tích hợp các tên miền phổ biến, tạo `shortIds` ngẫu nhiên, và gửi yêu cầu đến máy chủ để lấy cặp khóa X25519 mới (`privateKey/publicKey`).

7.1. Sự khác biệt: TLS vs XTLS vs REALITY

- **TLS** – mã hóa transport cổ điển theo giao thức TLS 1.2/1.3. Máy chủ phải có chứng chỉ hợp lệ (tên miền riêng + chuỗi chứng chỉ). Lưu lượng trông giống HTTPS thông thường, nhưng đối với cơ quan kiểm duyệt chủ động, TLS-handshake đến tên miền của bạn có thể nhận dạng được; nếu bị chặn theo SNI hoặc không có chứng chỉ đáng tin cậy, kết nối bị chặn/hiển thị lỗi.
- **XTLS (Vision)** – đây không phải là chế độ riêng biệt trong danh sách «Bảo mật», mà là cơ chế *flow* bên trên TLS **hoặc** Reality. Được kích hoạt phía client của inbound thông qua trường **Flow** = `xtls-rprx-vision` (hoặc `xtls-rprx-vision-udp443`). Vision khả dụng cho VLESS trên mạng tcp với `security = tls` hoặc `security = reality`, cũng như cho VLESS qua transport xhttp khi bật mã hóa VLESS (`vlessenc` / ML-KEM) – trong trường hợp này, trường **Flow** cũng có thể đặt thành `xtls-rprx-vision`, và giá trị được truyền đúng vào liên kết `vless:// (flow=xtls-rprx-vision)`. Vision giảm «mã hóa kép» (TLS-in-TLS), truyền tải payload trực tiếp sau bắt tay, giúp tăng tốc truyền tải và cải thiện ngụy trang. Vì vậy, tổ hợp **VLESS + Reality + Flow xtls-rprx-vision** được coi là cấu hình hiện đại được khuyến nghị.

Tự động khôi phục flow Vision. Nếu mã hóa (ML-KEM, các trường decryption/encryption) của VLESS/XHTTP-inbound được bật sau khi các client đã được thêm vào, inbound trở nên đủ điều kiện để sử dụng flow. Trong tình huống này, panel tự khôi phục `flow = xtls-rprx-vision` cho những client được phép dùng nó nhưng trường **Flow** của họ bị bỏ trống. Trước đây trong trường hợp như vậy, Vision âm thầm biến mất khỏi cấu hình, liên kết chia sẻ và đăng ký (đặc biệt rõ ràng trên các inbound nút trung gian). Không cần thao tác thủ công: bản sửa lỗi được áp dụng tự động khi lưu inbound và một lần khi

cập nhật panel. Hành vi này có tính bảo thủ – panel không bịa flow và không ghi đè giá trị mà client đã đặt trước.

- **REALITY** – cơ chế nguy trang không cần chứng chỉ riêng. Máy chủ «mượn» TLS-handshake của một trang web bên thứ ba thực sự (`target / serverNames`), vì vậy đối với người quan sát, kết nối không thể phân biệt được với việc truy cập trang web đó, và không cần chứng chỉ. Xác thực dựa trên cặp khóa X25519 và tập hợp `shortIds`. REALITY kháng cự các thăm dò chủ động (`active probing`) và chặn theo SNI, vì SNI trở nên một tên miền bên ngoài thực sự. Đánh đổi – yêu cầu cấu hình chặt chẽ hơn (`target` đúng với cổng, đồng bộ khóa với client).

Quy tắc lựa chọn ngắn gọn:

- có tên miền riêng và chứng chỉ hợp lệ, cần dạng HTTPS đơn giản → **TLS** (nếu có thể thì kèm Vision);
- không có tên miền/chứng chỉ hoặc cần ẩn danh tối đa khỏi DPI → **REALITY** (kèm Vision cho VLESS/TCP).

7.2. Chế độ «Không» (`none`)

Transport được truyền không có lớp bọc TLS: các khối `tlsSettings` và `realitySettings` bị loại khỏi `streamSettings`. Chế độ này không có thêm trường nào. Phù hợp khi:

- inbound chỉ lắng nghe trên `127.0.0.1` và dùng làm đích fallback (theo quy tắc panel, inbound con cho fallback phải lắng nghe trên `127.0.0.1` với `security=none`);
- mã hóa/ngụy trang được thực hiện bởi lớp bên ngoài (ví dụ, reverse proxy Nginx trước panel);
- sử dụng giao thức có mã hóa riêng (Shadowsocks) trong mạng nội bộ.

Đối với các inbound có thể truy cập từ bên ngoài, chế độ «Không» không được khuyến nghị.

Ví dụ: khối `streamSettings` cho TLS trên mạng `tcp` (VLESS/Trojan/VMess). Đây là kết quả sau khi chọn chế độ **TLS** và điền SNI và đường dẫn đến chứng chỉ:

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. Chế độ TLS

Các trường của khối `tlsSettings`. Giá trị mặc định được lấy từ schema của panel.

Tham số chính

Trường (nhân)	Giá trị mặc định	Mô tả
SNI (<code>serverName</code>)	<code>""</code> (trống)	Server Name Indication – tên miền được trình bày trong TLS-handshake. Phải khớp với tên miền của chứng chỉ. Gợi ý placeholder tiếng Anh: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Tự động	Danh sách các bộ mã hóa được phép. Mặc định trống – việc lựa chọn được giao cho Xray/Go (tùy chọn Tự động). Chỉ thay đổi khi cần giới hạn rõ ràng các bộ mã hóa.
Phiên bản Tối thiểu/Tối đa (<code>minMaxVersion</code>)	min = <code>1.2</code> , max = <code>1.3</code>	Phiên bản TLS tối thiểu và tối đa. Các giá trị khả dụng: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . Nên giữ <code>1.2</code> – <code>1.3</code> ; không nên hạ tối thiểu xuống 1.0/1.1 (các phiên bản lỗi thời, không an toàn).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (trong biểu mẫu – mục None = <code>""</code> khả dụng)	Dấu vân tay TLS được giả lập của client hello (uTLS fingerprint), để bắt tay trông giống trình duyệt phổ biến. Xem danh sách bên dưới. Trong TLS, mục đầu tiên của danh sách là None (<code>""</code>), tắt giả lập.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	Danh sách các giao thức tầng ứng dụng được thương lượng trong TLS (chọn nhiều). Các giá trị khả dụng: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . Mặc định đề xuất <code>h2</code> và <code>http/1.1</code> .

Các giá trị có thể có của **uTLS fingerprint** (giống nhau cho TLS và REALITY): `chrome` , `firefox` , `safari` , `ios` , `android` , `edge` , `360` , `qq` , `random` , `randomized` , `randomizednoalpn` , `unsafe` . Trong biểu mẫu TLS, có thêm tùy chọn trống **None** (không áp dụng giả lập dấu vân tay).

Các giá trị khả dụng của **Cipher Suites** (TLS 1.3 và các bộ ECDHE): `TLS_AES_128_GCM_SHA256` , `TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256` .

Công tắc TLS

Công tắc	Mặc định	Mô tả
Từ chối SNI không xác định (<code>rejectUnknownSni</code>)	tắt (<code>false</code>)	Nếu bật, máy chủ ngắt kết nối khi SNI mà client trình bày không khớp với tên trong chứng chỉ. Tăng tính ẩn danh (máy chủ không phản hồi các yêu cầu «lạ»), nhưng yêu cầu SNI của client phải khớp chính xác.
Tắt System Root (<code>disableSystemRoot</code>)	tắt (<code>false</code>)	Tắt việc sử dụng kho chứng chỉ gốc đáng tin cậy của hệ thống.

Công tắc	Mặc định	Mô tả
Tiếp tục phiên (<code>enableSessionResumption</code>)	tắt (<code>false</code>)	Bật tiếp tục phiên TLS (session resumption / session tickets).

Tham số TLS bổ sung (vcn, đường cong, nhật ký khóa, ECH Sockopt)

Bên dưới cài đặt TLS chính có các trường bổ sung.

Trường (nhân)	Mặc định	Mô tả
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	Tên (phân cách bằng dấu phẩy) mà client dùng để xác minh chứng chỉ máy chủ thay vì SNI. Đây là thay thế hiện đại cho trường <code>allowInsecure</code> đã bị xóa khỏi Xray sau ngày 2026-06-01. Giá trị chỉ dùng cho panel: không được ghi vào cấu hình xray của máy chủ, nhưng được truyền vào các liên kết mời và đăng ký (<code>vcn=...</code>) để client tự áp dụng. Placeholder: <code>example.com</code> .
Curve Preferences (<code>curvePreferences</code>)	""	Giới hạn và thứ tự ưu tiên của các đường cong trao đổi khóa TLS (ví dụ <code>X25519MLKEM768</code> , <code>X25519</code>). Trống – dùng giá trị mặc định của Xray-core.
Master Key Log (<code>masterKeyLog</code>)	""	Đường dẫn để ghi TLS master keys theo định dạng <code>SSLKEYLOGFILE</code> (để giải mã lưu lượng trong Wireshark khi gỡ lỗi). Placeholder: <code>/path/to/sslkeylog.txt</code> . Trong môi trường production để trống – tệp này cho phép giải mã toàn bộ lưu lượng.
ECH Sockopt (<code>echSockopt</code>)	tắt	Công tắc với các tham số socket cho kết nối mà Xray dùng để truy vấn ECH config list. Khi bật, có các tùy chọn: Dialer Proxy (<code>dialerProxy</code> – chuyển yêu cầu qua outbound được chỉ định theo tag), Domain Strategy (<code>domainStrategy</code>), TCP Fast Open (<code>tcpFastOpen</code>), Multipath TCP (<code>tcpMptcp</code>). Để tắt nếu không cần thiết.

Các trường `verifyPeerCertByName` , `curvePreferences` , `masterKeyLog` và `echSockopt` nằm ở cấp độ cao nhất của `tlsSettings` và được giữ nguyên khi panel cắt bớt các trường khi lưu cấu hình.

Chứng chỉ

Phần **Chứng chỉ SSL** (trong UI có tiêu đề «Chứng chỉ SSL») được định nghĩa dưới dạng danh sách: nút + thêm bản ghi chứng chỉ mới, nút – **Xóa** loại bỏ (nút xóa chỉ khả dụng khi có nhiều hơn một bản ghi). Mặc định khi bật TLS sẽ tạo một bản ghi trống.

Đối với mỗi bản ghi, công tắc chế độ nhập (`useFile`):

- **Đường dẫn đến chứng chỉ** (giá trị `useFile = true` , mặc định) – chỉ định đường dẫn đến tệp trên máy chủ:
 - **Khóa công khai** (`certificateFile`) – đường dẫn đến tệp chứng chỉ (`.crt` / `.pem`);
 - **Khóa riêng tư** (`keyFile`) – đường dẫn đến tệp khóa riêng tư (`.key`).

- **Nội dung chứng chỉ** (giá trị `useFile = false`) – nội dung được dán trực tiếp vào các trường (vùng văn bản nhiều dòng):
 - **Khóa công khai** (`certificate`) – nội dung PEM của chứng chỉ;
 - **Khóa riêng tư** (`key`) – nội dung PEM của khóa.

Bên dưới các trường của chế độ «Đường dẫn đến chứng chỉ» có hai nút:

- **Đặt chứng chỉ panel** – điền vào các trường đường dẫn đến chứng chỉ SSL của chính panel. Đối với inbound trên panel trung tâm, chứng chỉ của nó được lấy (`POST /panel/setting/all` → `webCertFile / webKeyFile`); đối với inbound được gán cho node – chứng chỉ của node đó (`GET /panel/api/nodes/webCert/{nodeId}`), vì các đường dẫn của panel trung tâm không tồn tại trên node. Nếu chứng chỉ chưa được cấu hình, sẽ hiển thị cảnh báo: «*Chưa cấu hình chứng chỉ cho panel. Vui lòng cài đặt trước trong Cài đặt.*» (chứng chỉ của chính panel được đặt trong phần «Cài đặt → Bảo mật»).
- **Xóa** – xóa cả hai đường dẫn.

Các trường bổ sung của mỗi bản ghi chứng chỉ:

Trường	Mặc định	Mô tả
OCSF Stapling (<code>ocspStapling</code>)	<code>0</code> (tắt)	Khoảng thời gian cập nhật OCSF stapling tính bằng giây (tối thiểu <code>0</code>). Đối với các inbound mới, mặc định tắt (<code>0</code>): điều này loại bỏ lỗi trong nhật ký xray cho các chứng chỉ không có OCSF responder (ví dụ, Let's Encrypt đã ngừng hỗ trợ OCSF). Chỉ bật cho các chứng chỉ hỗ trợ stapling.
Tải một lần (<code>oneTimeLoading</code>)	tắt (<code>false</code>)	Nếu bật, chứng chỉ được đọc từ đĩa một lần khi khởi động và không đọc lại khi tệp thay đổi.
Tùy chọn sử dụng (<code>usage</code>)	<code>encipherment</code>	Mục đích của chứng chỉ. Các giá trị khả dụng: <code>encipherment</code> (mã hóa – chứng chỉ máy chủ thông thường), <code>verify</code> (xác minh), <code>issue</code> (phát hành – máy chủ tự ký/phát hành chứng chỉ).
Build Chain (<code>buildChain</code>)	tắt (<code>false</code>)	Chỉ hiển thị khi <code>usage = issue</code> . Xây dựng chuỗi chứng chỉ.

Không có nút riêng để tạo chứng chỉ tự ký trong trình chỉnh sửa inbound: panel không tạo chứng chỉ tự ký ngay lập tức cho inbound. Chứng chỉ được chỉ định bằng đường dẫn/nội dung hoặc được lấy từ cài đặt panel bằng nút «Đặt chứng chỉ panel». Việc phát hành/lấy chứng chỉ SSL của chính panel (bao gồm tải tệp lên và liên kết với tên miền) được thực hiện trong phần **Cài đặt** → **Bảo mật**; không có endpoint ACME/Let's Encrypt cho inbound ở đây.

ECH và ghim chứng chỉ (các trường TLS nâng cao)

Trường	Mặc định	Mô tả
ECH key (<code>echServerKeys</code>)	<code>""</code>	Khóa máy chủ Encrypted Client Hello.
ECH config (<code>settings.echConfigList</code>)	<code>""</code>	ECH config list (phần client, được đưa vào liên kết).

Trường	Mặc định	Mô tả
SHA-256 của chứng chỉ peer (<code>settings.pinnedPeerCertSha256</code>)	<code>[]</code>	Các hash SHA-256 của chứng chỉ peer (chuỗi hex, phân cách bằng dấu phẩy). Gợi ý nguyên văn: « <i>Các hash SHA-256 của chứng chỉ peer dưới dạng chuỗi thập lục phân (ví dụ e8e2d3...), phân cách bằng dấu phẩy. Chỉ dùng cho panel – không được ghi vào cấu hình xray của máy chủ, nhưng được đưa vào liên kết mời để client có thể ghim chứng chỉ.</i> »

Các nút: Bên cạnh trường **SHA-256 của chứng chỉ peer** có hai nút tự động điền:

- **Fill from this inbound's certificate** (biểu tượng khiên) – điền hash SHA-256 của chứng chỉ của chính inbound này (lấy cục bộ qua endpoint `getCertHash`).
- **Fetch the hash by pinging the SNI (xray tls ping)** (biểu tượng tải xuống) – lấy hash của chứng chỉ máy chủ trực tiếp bằng cách thực hiện kết nối TLS theo SNI được chỉ định (trên máy chủ gọi `getRemoteCertHash`). Trường **SNI** (`serverName`) phải được điền – nếu không sẽ hiển thị gợi ý «*Set the SNI (serverName) first to ping the remote certificate.*»

Các hash thu được được thêm vào trường (phân cách bằng dấu phẩy) và được đưa vào liên kết mời để client có thể ghim chứng chỉ.

- **Lấy chứng chỉ ECH mới** – yêu cầu máy chủ cấp ECH mới cho SNI hiện tại (`POST /panel/api/server/getNewEchCert` , trên máy chủ thực thi `xray tls ech --serverName <SNI>`); điền vào các trường **ECH key** và **ECH config**.
- **Xóa** – đặt lại cả hai trường ECH.

7.4. Chế độ REALITY

Các trường của khối `realitySettings` .REALITY không sử dụng chứng chỉ SSL: thay vào đó là TLS-handshake mượn từ tên miền bên ngoài và cặp khóa X25519.

Tham số ngụy trang

Trường (nhân)	Giá trị mặc định	Mô tả
Hiển thị (<code>show</code>)	tắt (<code>false</code>)	Đầu ra gỡ lỗi REALITY vào nhật ký Xray. Thường để tắt.
Xver (<code>xver</code>)	<code>0</code>	Phiên bản giao thức PROXY được truyền đến backend (<code>0</code> – tắt). Tối thiểu <code>0</code> .
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code>	Dấu vân tay TLS được giả lập (danh sách giống TLS, nhưng không có tùy chọn None trống).

Trường (nhân)	Giá trị mặc định	Mô tả
Đích (target)	"" (từ 3.4.2 khi bật REALITY vẫn để trống)	Trường bắt buộc. Tên miền thực sự mà REALITY mượn TLS-handshake. Gợi ý nguyên văn: « <i>Bắt buộc. Phải chứa cổng (ví dụ, example.com:443). Không có cổng, Xray-core sẽ không khởi động.</i> » Xác thực trong panel kiểm tra sự có mặt và tính đúng đắn của cổng; nếu không sẽ hiển thị lỗi «Đích REALITY là bắt buộc» / «Đích REALITY phải chứa cổng...» / «Đích REALITY có cổng không hợp lệ». Bên cạnh trường – các nút « Quét » (kiểm tra đích hiện tại «trực tiếp») và « Tìm đích » (mở trình quét đích REALITY); xem bên dưới.
SNI (serverNames)	[] (điền cùng với đích)	Danh sách SNI được phép (nhập nhiều giá trị bằng tag). Phải tương ứng với tên miền trong Đích . Khi quét đích thành công, SNI được điền theo chứng chỉ của nó.
Chênh lệch thời gian tối đa (ms) (maxTimediff)	0	Chênh lệch đồng hồ tối đa được phép giữa client và máy chủ tính bằng mili giây (0 – không giới hạn). Tối thiểu 0 .
Phiên bản client tối thiểu (minClientVer)	""	Phiên bản client Xray tối thiểu (placeholder từ 3.5.0 – 26.3.27). Trống – không giới hạn.
Phiên bản client tối đa (maxClientVer)	""	Phiên bản client Xray tối đa. Trống – không giới hạn.
Short IDs (shortIds)	[] (được tạo khi bật)	Danh sách các định danh ngắn (hex), phân biệt các client. Nhập nhiều giá trị bằng tag; nút làm mới tạo tập hợp ngẫu nhiên.
SpiderX (settings.spiderX)	/	Đường dẫn «spider» (phần client của REALITY), được sử dụng khi giả lập truy cập đến trang web bên ngoài. Được đưa vào liên kết mời.

Kể từ phiên bản 3.4.2, **Đích** (target) và **SNI** (serverNames) khi bật REALITY **không** còn được điền tự động – cả hai trường vẫn để trống, còn đích được chọn bằng trình quét trực tiếp (xem bên dưới). Hãy chọn một trang HTTPS bên thứ ba ổn định, «nặng cân», có hỗ trợ TLS 1.3 và HTTP/2, không nằm trên cùng máy chủ của bạn.

Tìm đích REALITY (trình quét trực tiếp)

Kể từ phiên bản 3.4.2, nút «đích ngẫu nhiên» cũ (với danh sách tích hợp tĩnh) được thay bằng hai thao tác bên cạnh trường **Đích**:

- «**Quét**» – kiểm tra «trực tiếp» đích hiện tại: thiết lập kết nối TLS và, nếu đích phù hợp, điền SNI theo chứng chỉ của nó. Thông báo: «Đích phù hợp – đã điền đích và SNI.» / «Đích khả dụng, nhưng không phù hợp cho REALITY.» / «Không quét được đích REALITY.»
- «**Tìm đích**» – mở cửa sổ «**Trình quét đích REALITY**»: có thể chỉ định tên miền, **IP hoặc dải CIDR** (khi đó panel tìm các đích theo chứng chỉ của các host đang hoạt động) hoặc để trống trường (kiểm tra các ứng viên tích hợp). Kết quả được xếp hạng; các cột – «Trạng thái»/«Phù hợp», «Trao đổi khóa», «Chứng chỉ», TLS , ALPN , «Độ trễ». Bằng nút «**Sử dụng**», dòng đã chọn được điền vào các trường inbound.

Đích được coi là **phù hợp** nếu máy chủ thương lượng **TLS 1.3, HTTP/2 (ALPN h2)**, trao đổi khóa **X25519/X25519MLKEM768** và xuất trình chứng chỉ **được tin cậy** (không phải wildcard). Việc quét được thực hiện ở phía máy chủ của panel (POST /panel/api/server/scanRealityTarget và .../scanRealityTargets).

Ví dụ: khối `streamSettings` cho REALITY trên mạng `tcp` (VLESS). Không cần chứng chỉ – thay vào đó là tên miền mượn và cặp khóa X25519:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": ["", "6ba85179e30d4fc2"],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Ở đây trường **Đích** (`target`) của panel tương ứng với `dest` trong cấu hình Xray hoàn chỉnh. Nếu REALITY-inbound được tạo với đích trong khóa `dest` (bởi các phiên bản panel cũ hơn, qua API hoặc các công cụ bên ngoài), panel khi phân tích sẽ chuẩn hóa `dest` → `target` khi `target` trống – vì vậy inbound như vậy được tải đúng, trường **Đích** không bị trống, và lưu lại không phá vỡ REALITY đang hoạt động.

Khóa REALITY (X25519)

Trường	Mặc định	Mô tả
Khóa công khai (<code>settings.publicKey</code>)	""	Khóa công khai X25519 (client đặt nó vào cấu hình/liên kết của mình).
Khóa riêng tư (<code>privateKey</code>)	""	Khóa riêng tư X25519 (chỉ lưu trên máy chủ).

Các nút bên dưới khóa:

- **Lấy chứng chỉ mới** – yêu cầu máy chủ cặp khóa mới (`GET /panel/api/server/getNewX25519Cert` ; trên máy chủ thực thi `xray x25519`), điền **Khóa riêng tư** và **Khóa công khai**. Cặp này cũng được tự động tạo khi lần đầu bật chế độ REALITY.

Ví dụ: lấy cặp khóa X25519 qua API (ngoài biểu mẫu, ví dụ cho script). Yêu cầu trả về khóa riêng tư và công khai:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# 0Tbet:
# {"success":true,"obj":{"privateKey":"...", "publicKey":"..."}}
```

`cookie.txt` – tệp cookie phiên, lấy được sau khi đăng nhập qua `POST /login`.

- **Xóa** – đặt lại cả hai khóa.

Chữ ký hậu lượng tử ML-DSA-65 (mlds65)

Lớp xác thực hậu lượng tử REALITY bổ sung (tùy chọn):

Trường	Mặc định	Mô tả
mlds65 Seed (<code>mlds65Seed</code>)	<code>""</code>	Seed khóa ML-DSA-65 của máy chủ.
mlds65 Verify (<code>settings.mlds65Verify</code>)	<code>""</code>	Giá trị xác minh (phần client, được đưa vào liên kết).

Các nút:

- **Lấy Seed mới** – yêu cầu cập mới (`GET /panel/api/server/getNewmlds65` ; trên máy chủ thực thi `xray mlds65`), điền **mlds65 Seed** và **mlds65 Verify**.
- **Xóa** – đặt lại cả hai trường.

Giới hạn tốc độ fallback và nhật ký khóa REALITY

Trong cài đặt REALITY có giới hạn tốc độ lưu lượng fallback – nó ngăn các thăm dò chủ động sử dụng máy chủ như một kênh miễn phí đến tên miền mượn. Cài đặt được chỉ định riêng cho hai hướng – **Limit Fallback Upload** và **Limit Fallback Download** (`limitFallbackUpload` / `limitFallbackDownload`), mỗi hướng có cùng tập hợp trường:

Trường (nhân)	Mặc định	Mô tả
After Bytes (<code>afterBytes</code>)	<code>0</code>	Số byte được phép ở tốc độ đầy đủ trước khi bắt đầu giới hạn. <code>0</code> – giới hạn từ byte đầu tiên.
Bytes Per Sec (<code>bytesPerSec</code>)	<code>0</code>	Giới hạn tốc độ lưu lượng fallback tính bằng byte trên giây sau ngưỡng. <code>0</code> – không giới hạn (tắt hướng này).
Burst Bytes Per Sec (<code>burstBytesPerSec</code>)	<code>0</code>	Dự trữ cho các đợt tăng tốc ngăn vượt quá tốc độ ổn định (kích thước token-bucket). Nếu nhỏ hơn Bytes Per Sec , sẽ được nâng lên giá trị đó.

Ở đây cũng có trường **Master Key Log** (`masterKeyLog`) – đường dẫn để ghi TLS master keys theo định dạng `SSLKEYLOGFILE` để gỡ lỗi trong Wireshark; trong môi trường production để trống.

7.5. Khuyến nghị thực tế về cấu hình

1. **VLESS + Reality (được khuyến nghị)**: tạo VLESS-inbound trên mạng `tcp`, trong tab «Bảo mật» chọn **Reality** – panel sẽ tự động điền `target /SNI`, `shortIds` ngẫu nhiên và tạo khóa X25519. Nếu cần, nhấn «Lấy chứng chỉ mới» để có cặp khóa riêng của bạn. Đối với các client VLESS, bật **Flow** = `xtls-rprx-vision` (XTLS Vision) – điều này mang lại hiệu suất và tính ẩn danh tối đa.

Ví dụ: liên kết client cuối cùng VLESS + Reality + Vision. Đây là liên kết mời mà panel cung cấp cho inbound như vậy (giá trị khóa/ID chỉ mang tính minh họa):

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Ở đây **pbk** — khóa công khai X25519, **sni** — tên miền mượn từ **Đích**, **sid** — một trong các **Short IDs**, **flow=xtls-rprx-vision** — XTLS Vision đã bật. 2. **TLS với tên miền riêng:** chọn **TLS**, điền **SNi** bằng tên miền, thêm chứng chỉ (bằng đường dẫn đến tệp hoặc nội dung), hoặc nhấn «Đặt chứng chỉ panel» nếu tên miền và chứng chỉ đã được cấu hình trong «Cài đặt → Bảo mật». Giữ **Phiên bản Tối thiểu/Tối đa** = **1.2** – **1.3** và **uTLS** = **chrome** để ngụy trang thành trình duyệt thông thường. 3. Không để chế độ **Không** cho các inbound mở ra bên ngoài — chỉ dùng cho các đích fallback cục bộ (**127.0.0.1**) hoặc khi TLS được đảm bảo bởi proxy bên ngoài. 4. Lời khuyên từ giao diện: đối với hầu hết các trường nâng cao, có gợi ý «*Nên giữ cài đặt mặc định*» — chỉ thay đổi chúng khi hiểu rõ hậu quả.

8. Clients

Client là tài khoản người dùng VPN: một tập thông tin xác thực (UUID hoặc mật khẩu) được liên kết với một hoặc nhiều inbound, có hạn mức lưu lượng riêng, thời hạn hiệu lực và giới hạn số kết nối đồng thời. Trong bản fork này, client là một thực thể độc lập (bảng `clients`): cùng một client có thể được liên kết với nhiều inbound cùng lúc, dùng chung UUID/mật khẩu và bộ đếm lưu lượng chung. Mục **Clients** hiển thị tất cả tài khoản trong panel bất kể inbound, với chức năng tìm kiếm, lọc, sắp xếp và thao tác hàng loạt.

8.1. Các trường của client

Dưới đây là giải thích chi tiết từng trường trong trình soạn thảo **Thêm client** / **Sửa client**.

Form client được chia thành hai tab: **Cơ bản** (email, liên kết đến inbound, giới hạn, thời hạn, nhóm, ghi chú, reverse tag) và **Thông tin xác thực** (UUID/mật khẩu/auth, Flow, VMess Security). Trong nhãn trường, hạn mức được ghi là **Giới hạn lưu lượng (GB)**, còn thời gian được ghi là **Thời hạn (ngày)** và **Tự gia hạn (ngày)**; các trường **Giới hạn lưu lượng (GB)** và **Giới hạn IP** có chú thích giải thích rằng `0` nghĩa là «không giới hạn». Khi chỉnh sửa client đã tồn tại, nút tạo email ngẫu nhiên bị ẩn, còn nút nhật ký IP được đặt ngay bên cạnh trường **Giới hạn IP** và hiển thị số địa chỉ đã ghi nhận.

Trường	Khóa JSON	Mặc định	Mô tả
Email	<code>email</code>	– (bắt buộc)	Định danh duy nhất của client
UUID	<code>id</code>	được tạo tự động	Định danh cho VMess/VLESS
Mật khẩu	<code>password</code>	được tạo tự động	Mật khẩu cho Trojan/Shadowsocks
Xác thực	<code>auth</code>	được tạo tự động	Mật khẩu cho Hysteria
Flow	<code>flow</code>	rỗng	Flow control (XTLS), chỉ dùng cho VLESS
VMess Security	<code>security</code>	<code>auto</code>	Phương thức mã hóa VMess
Giới hạn IP	<code>limitIp</code>	<code>0</code> (không giới hạn)	Số IP đồng thời tối đa
Tổng đã gửi/nhận (GB)	<code>totalGB</code>	<code>0</code> (không giới hạn)	Hạn mức lưu lượng
Thời hạn hiệu lực	<code>expiryTime</code>	<code>0</code> (vĩnh viễn)	Ngày hết hạn
Tự gia hạn	<code>reset</code>	<code>0</code> (tắt)	Chu kỳ đặt lại lưu lượng, ngày
ID người dùng Telegram	<code>tgId</code>	<code>0</code> (không có)	ID Telegram dạng số
ID đăng ký	<code>subId</code>	được tạo tự động	Định danh đăng ký
Nhóm	<code>group</code>	rỗng	Nhãn logic để nhóm
Ghi chú	<code>comment</code>	rỗng	Ghi chú tùy ý
Đã bật	<code>enable</code>	<code>true</code>	Tài khoản có đang hoạt động không

Email (định danh)

Trường **Email** là định danh chính và bắt buộc của client. Dù có tên gọi như vậy, đây không nhất thiết phải là địa chỉ thư điện tử: bất kỳ nhân văn bản nào cũng được (tên người dùng, số thứ tự). Giá trị phải **duy nhất** trong toàn bộ panel; việc tạo client thứ hai với email đã dùng sẽ bị từ chối (`email already in use`), trừ trường hợp `subId` cũng trùng khớp (điều này được hiểu là liên kết cùng một client).

Email **không được để trống** (`client email is required`) và **không được chứa dấu cách, ký tự `/`, `\` hay ký tự điều khiển** («Email không được chứa dấu cách, `'`, `"` hay ký tự điều khiển»). Email tham gia vào thống kê lưu lượng, nhật ký IP, danh sách trực tuyến và tên các thao tác – không nên thay đổi email sau khi đã tạo.

UUID / Mật khẩu / Xác thực (thông tin xác thực)

Trường thông tin xác thực cụ thể phụ thuộc vào giao thức của inbound mà client được liên kết. Các giá trị được điền tự động nếu để trống trường:

- **UUID** (trường `id`) – dành cho giao thức **VMess** và **VLESS**. Nếu không được đặt, một UUID v4 ngẫu nhiên sẽ được tạo.
- **Mật khẩu** (trường `password`) – dành cho **Trojan** và **Shadowsocks**. Với Trojan, mặc định tạo UUID không có dấu gạch ngang. Với Shadowsocks, tạo khóa có độ dài phù hợp dạng Base64 tùy theo phương thức mã hóa của inbound: 16 byte cho `2022-blake3-aes-128-gcm`, 32 byte cho `2022-blake3-aes-256-gcm` và `2022-blake3-chacha20-poly1305`; với các phương thức khác – UUID không có dấu gạch ngang. Nếu khóa nhập thủ công không phù hợp với phương thức `2022-blake3`, nó sẽ được thay bằng khóa được tạo tự động.
- **Xác thực** (trường `auth`) – mật khẩu cho **Hysteria**. Mặc định là UUID không có dấu gạch ngang.

Vì một client có thể được liên kết với các inbound của các giao thức khác nhau, bản ghi client có thể đồng thời có UUID, mật khẩu và auth – mỗi giao thức sử dụng trường riêng của mình.

Ví dụ: thông tin xác thực của client trông như thế nào trong `settings` của các inbound khác nhau. Cùng một client trong VLESS inbound được xác định qua `id`, trong Trojan – qua `password`, trong Shadowsocks – qua `password` (khóa Base64):

```
// фпармент settings.clients y VLESS-inbound
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// тот же клиент в Trojan-inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// тот же клиент в Shadowsocks-inbound (метод 2022-blake3-aes-256-gcm)
{ "password": "GPY0aA3f7C00az53eaQ8eqMfRDjmbL0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (trường `flow`) – điều khiển luồng XTLS. Chỉ áp dụng **cho VLESS** và chỉ khi inbound được cấu hình cho XTLS Vision: VLESS qua transport **TCP** với security **tls** hoặc **reality**. Giá trị hợp lệ là `xtls-rprx-vision` (cũng như `xtls-rprx-vision-udp443` từ trước); giá trị rỗng nghĩa là không có flow.

Nếu inbound không hỗ trợ XTLS-flow, flow đã đặt sẽ **bị xóa âm thầm** khi lưu client: với cùng một client được liên kết với nhiều inbound, flow chỉ được áp dụng ở những nơi cho phép. Chỉ nên thay đổi nếu bạn đang chủ động sử dụng VLESS-Vision.

VMess Security

VMess Security (trường `security`) – phương thức mã hóa payload cho VMess. Giá trị mặc định là `auto` (Xray tự chọn mật mã). Các giá trị hợp lệ là tiêu chuẩn cho VMess: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. Trường này không dùng cho các giao thức khác.

Giới hạn IP (kết nối đồng thời)

Giới hạn IP (trường `limitIp`) – số lượng **địa chỉ IP khác nhau** tối đa mà client có thể kết nối đồng thời. Giá trị mặc định là `0`, nghĩa là **không có giới hạn**. Khi giá trị dương, panel theo dõi các IP đang hoạt động của client và, khi vượt quá giới hạn, tắt tài khoản bằng tác vụ nền. (Từ phiên bản **3.3.1** trở đi, việc đếm IP được thực hiện qua API online-stats của nhân Xray và **không yêu cầu** nhật ký truy cập; trên các phiên bản nhân cũ hơn, panel quay lại đọc nhật ký truy cập, và nhật ký đó khi đó phải được bật.) Sử dụng để ngăn chia sẻ một đăng ký cho nhiều thiết bị: ví dụ, `2` – cho phép hai thiết bị.

Giới hạn IP được áp dụng bằng **Fail2ban**, do đó trường **Giới hạn IP** chỉ hoạt động khi Fail2ban được cài đặt và hoạt động (panel kiểm tra trạng thái của nó qua `GET /panel/api/server/fail2banStatus`). Nếu Fail2ban chưa được cài đặt, trường trong trình soạn thảo client (và form thêm hàng loạt) bị khóa, và khi di chuột vào sẽ hiện chú thích đề nghị cài Fail2ban từ menu bash `x-ui` («Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option.»); trên Windows chú thích thông báo rằng Fail2ban không khả dụng ở đó («Fail2ban is not available on Windows, so the IP limit cannot be enforced.»), và nếu tính năng bị tắt trên máy chủ – «The IP limit feature is disabled on this server.». Khi cập nhật panel, giới hạn IP đã lưu của các client trên máy chủ không có Fail2ban sẽ bị đặt về 0 bằng một lần migration, vì dù sao nó cũng không được áp dụng ở đó.

Ví dụ giá trị. `limitIp: 0` – không giới hạn; `limitIp: 1` – chỉ đúng một thiết bị cùng lúc; `limitIp: 3` – tối đa ba IP khác nhau. Khi có IP hoạt động thứ tư, tác vụ nền sẽ tắt client (`enable = false`) cho đến khi bạn thực hiện **Đặt lại giới hạn IP**.

Các thao tác liên quan: **Nhật ký IP** hiển thị danh sách các IP đã ghi nhận của client; mỗi bản ghi chứa, ngoài bản thân địa chỉ IP, thời gian truy cập cuối cùng và nhãn node (`@ tên_node`), qua đó kết nối được ghi nhận – trong cấu hình đa panel có thể thấy client kết nối qua node nào. **Đặt lại giới hạn IP** xóa nhật ký IP tích lũy để client có thể kết nối lại mà không cần chờ các bản ghi hết hạn tự nhiên.

Tổng đã gửi/nhận (GB) – hạn mức lưu lượng

Tổng đã gửi/nhận (GB) (trường `totalGB`) – hạn mức lưu lượng tổng cộng (gửi + nhận). Giá trị mặc định là `0` – nghĩa là **không giới hạn**. Khi đạt hạn mức (`up + down >= total`), client được coi là **đã hết** (depleted) và bị tắt. Trong giao diện thường nhập bằng gigabyte; trong cơ sở dữ liệu lưu bằng byte.

Trong danh sách client, cột **Lưu lượng** hiển thị thanh màu thể hiện mức sử dụng: lượng lưu lượng đã dùng, nhãn giới hạn (hoặc ký hiệu ∞ khi không giới hạn) và chú thích khi di chuột với phân tích theo gửi/nhận và phần còn lại. Cùng chỉ báo nhỏ gọn đó được hiển thị trong thẻ client trên điện thoại.

Thời hạn hiệu lực (Expiry)

Thời hạn hiệu lực (trường `expiryTime`) xác định thời điểm tài khoản hết hạn. Trường có ba chế độ:

- **Vĩnh viễn** — `0`. Client không bao giờ hết hạn theo thời gian.
- **Ngày cụ thể** — Unix-timestamp dương (tính bằng mili giây). Khi đến thời điểm đó (`expiryTime <=` `hiện tại`), client được coi là đã hết hạn (expired) và bị tắt. Trong giao diện thường đặt bằng cách chọn ngày hoặc độ dài thời gian theo ngày (**Thời hạn**, đơn vị — **Ngày**).
- **Bắt đầu sau lần sử dụng đầu tiên** — giá trị âm, mã hóa độ dài thời gian. Cho đến khi client chưa truyền một byte nào, thời hạn vẫn ở giá trị âm («khởi động trễ»). Ngay ở lần đếm lưu lượng đầu tiên, panel chuyển đổi nó thành ngày tuyệt đối: `hiện tại + |độ dài|`. Điều này cho phép bán, ví dụ, «30 ngày kể từ lần kết nối đầu tiên» mà không cần biết trước client sẽ kích hoạt khi nào. Việc chuyển đổi được thực hiện một lần cho mỗi email, để tất cả các inbound được liên kết nhận cùng một thời hạn.

Ví dụ mã hóa thời hạn. Ngày cố định 1 tháng 3 năm 2026, 00:00 UTC → `expiryTime: 1772323200000` (timestamp dương tính bằng mili giây). «30 ngày kể từ lần kết nối đầu tiên» → `expiryTime: -2592000000` (giá trị âm, $30 \times 24 \times 60 \times 60 \times 1000$); khi có byte lưu lượng đầu tiên, panel sẽ thay nó bằng `hiện tại + 2592000000`. Vĩnh viễn → `expiryTime: 0`.

Tự gia hạn (chu kỳ đặt lại lưu lượng của client)

Trường **Tự gia hạn** (trường `reset`) — chu kỳ gia hạn/đặt lại tự động tính bằng ngày. Chú thích: «Tự động gia hạn sau khi hết hạn. (0 = tắt) (đơn vị: ngày)».

- `0` — tự gia hạn **tắt** (giá trị mặc định). Khi hết hạn, client đơn giản trở nên đã hết.
- `> 0` — tác vụ nền khi hết hạn sẽ **đặt lại bộ đếm lưu lượng về không** (`up = down = 0`), **dịch chuyển thời hạn hiệu lực về phía trước** theo số ngày `reset` (nếu cần — nhiều chu kỳ, cho đến khi thời hạn mới nằm trong tương lai) và nếu cần **bật lại** client. Điều này thực hiện đăng ký định kỳ (ví dụ, hàng tháng). Tự gia hạn **không áp dụng cho các inbound trên các node** (`node_id IS NOT NULL`).

Hệ quả quan trọng: các client có `reset > 0` **bị loại trừ** khỏi khái niệm «đã hết» trong các thao tác xóa hàng loạt — lưu lượng/thời hạn của họ dự kiến sẽ được đặt lại về không bởi tự gia hạn, chứ không phải làm cho tài khoản trở thành ứng viên để xóa.

ID người dùng Telegram

ID người dùng Telegram (trường `tgId`) — định danh Telegram dạng số của người dùng để liên kết với bot Telegram tích hợp của panel (thông báo, tự xem thống kê). Chú thích: «ID người dùng Telegram dạng số (0 = không có)». Giá trị mặc định `0` — không có liên kết. Có thể lọc theo trường này (**Có / Không**).

ID đăng ký (subId)

ID đăng ký (trường `subId`) — định danh mà client được đưa vào **đăng ký** (subscription). Tất cả các client có cùng `subId` được trả về qua một liên kết đăng ký. Nếu trường được để trống khi tạo, panel **tự động tạo** `subId` ngẫu nhiên (UUID). Giá trị phải **duy nhất** trong số các client có email khác (`subId already in use`) và tuân theo các hạn chế ký tự tương tự như email («ID đăng ký không được chứa dấu cách, '/', '' hay ký tự điều khiển»).

Không có `subId`, liên kết đăng ký cho client không khả dụng («Client này không có subId, liên kết chia sẻ không khả dụng.»).

Tab Links (liên kết ngoài và đăng ký)

Ngoài các tab **Cơ bản** và **Thông tin xác thực**, trong trình soạn thảo client còn có tab thứ ba **Links** (chú thích: «Add third-party share links and remote subscription URLs to include in this client's subscription.»). Trên tab này, nút **Add External Link** thêm các share-link của bên thứ ba (`vless://`, `vmess://`, `trojan://`, `ss://`, `hysteria2://`, `wireguard://`), còn nút **Add External Subscription** thêm URL đăng ký từ xa (ví dụ, `https://provider.example/sub/...`).

Tất cả những thứ được liệt kê sẽ được trộn vào đầu ra đăng ký của client này (các định dạng raw, JSON và Clash): các liên kết được thêm nguyên văn, còn các đăng ký từ xa panel định kỳ tải về (với bộ nhớ cache và timeout ngắn) và kết hợp cấu hình của chúng với cấu hình của chính mình. Như vậy trong một liên kết đăng ký của client có thể cung cấp cùng với các máy chủ của mình cả các cấu hình bên ngoài.

Nhóm

Nhóm (trường `group`) – nhãn logic để nhóm các client có liên quan. Chú thích: «Nhãn logic để nhóm các client có liên quan (ví dụ: đội nhóm, khách hàng, khu vực). Có thể lọc từ thanh công cụ.», placeholder – «ví dụ: customer-a». Trường không bắt buộc (mặc định rỗng). Có thể lọc danh sách theo nhóm và thực hiện thao tác hàng loạt; để xóa nhãn của client, sử dụng thao tác **Bỏ nhóm**.

Có thể xóa nhóm ngay trong trình soạn thảo của một client: nếu xóa trường **Nhóm** và lưu, nhãn sẽ được xóa đúng cách và client sẽ không còn hiển thị dưới nhóm cũ nữa.

Ghi chú

Ghi chú (trường `comment`) – ghi chú văn bản tùy ý cho quản trị viên (mặc định rỗng). Nội dung tham gia vào tìm kiếm và có thể lọc (**Có / Không** ghi chú).

Đã bật

Đã bật (trường `enable`) – cờ hoạt động của tài khoản. Mặc định **đã bật** (`true`); khi tạo, ngay cả khi cờ không được truyền, panel bắt buộc đặt `true`. Client đã tắt (`enable = false`) không thể kết nối và trong tổng quan thuộc danh mục **không hoạt động** (deactive). Panel tự tắt các client đã hết hạn mức, đã hết hạn hoặc vượt quá giới hạn IP.

Các trường chỉ đọc

Trong thẻ client cũng hiển thị các trường hệ thống: **Ngày tạo** (`created_at`) và **Cập nhật lúc** (`updated_at`) – đầu thời gian tạo và lần thay đổi cuối, được điền tự động và không chỉnh sửa được. Trường **Reverse tag** (`reverse`) – Reverse tag tùy chọn cho reverse proxy VLESS đơn giản («Reverse tag tùy chọn»).

8.2. Liên kết với inbound

Mỗi client phải được liên kết với ít nhất một inbound – khi tạo cần có tối thiểu một (`at least one inbound is required`). Trong trình soạn thảo đây là trường **Inbound được liên kết** với chú thích **Chọn một hoặc nhiều inbound**.

- **Liên kết** – thêm client vào các inbound đã chọn (cùng UUID/mật khẩu và lưu lượng chung). Các liên kết hiện có được giữ nguyên.
- **Hủy liên kết** – xóa client khỏi các inbound đã chọn. Bản ghi client được giữ lại (để xóa hoàn toàn hãy dùng **Xóa**). Các cặp mà client chưa được liên kết sẽ được bỏ qua âm thầm.

Khi lưu client được liên kết với nhiều inbound, các trường không tương thích với giao thức/transport cụ thể (ví dụ, Flow ngoài VLESS-Vision) sẽ tự động được điều chỉnh về các giá trị hợp lệ cho từng inbound.

Phía trên danh sách chọn inbound (trong form client, khi thêm hàng loạt và trong các cửa sổ liên kết/hủy liên kết hàng loạt) có các nút **Chọn tất cả** và **Xóa tất cả**. Trong các danh sách này, mỗi inbound được ký bằng ghi chú của nó (remark), nếu được đặt, nếu không – bằng tag của inbound.

8.3. Các thao tác trên client

Với một client riêng lẻ (qua thẻ **Thông tin client** hoặc menu ngữ cảnh **Hành động**) có sẵn:

Xem thông tin, mã QR và liên kết

- **Thông tin client** – thẻ với tất cả các trường, lưu lượng đã dùng/còn lại (**Còn lại**), thời hạn hiệu lực và các inbound được liên kết. Từ 3.5.0, thẻ này còn hiển thị **nhóm** của client (hàng «Nhóm» với nhãn, phía trên «Ghi chú»), khi nhóm được đặt.

Yêu cầu client qua API (`GET /panel/api/clients/get/:email`) bên cạnh các trường `client` và `inboundIds` còn trả về thêm `usedTraffic` – lưu lượng thực tế đã dùng (gửi + nhận, tính cả dữ liệu của các node), giúp dễ dàng so sánh mức tiêu thụ với hạn mức `totalGB` .

- **Mã QR và Liên kết** – liên kết cấu hình của client để nhập vào ứng dụng client. Được tạo từ tất cả các inbound được liên kết với giao thức được hỗ trợ (`GET /links/:email`). Nếu không có liên kết phù hợp: «Không có liên kết chia sẻ – trước tiên hãy liên kết client với inbound có giao thức được hỗ trợ.».
- **Liên kết đăng ký** – URL đăng ký theo `subId` (`GET /subLinks/:subId`). Chỉ khả dụng nếu client có `subId` và dịch vụ đăng ký được bật trong **Cài đặt panel** → **Đăng ký** (nếu không sẽ là «Dịch vụ đăng ký bị tắt.»). Ngoài ra còn có **URL đăng ký JSON**.

Đặt lại lưu lượng

Đặt lại lưu lượng (`POST /resetTraffic/:email`) đặt lại bộ đếm gửi/nhận (`up` , `down`) của client cụ thể về không. Hạn mức (`totalGB`) và thời hạn hiệu lực **không bị ảnh hưởng** – chỉ lượng đã dùng được đặt về không. Thông báo: «Lưu lượng đã được đặt lại». Nếu client không được liên kết với bất kỳ inbound nào: «Trước tiên hãy liên kết client này với inbound.».

Nút **Đặt lại lưu lượng** cũng có trong form chỉnh sửa client – ở bảng dưới cùng, cạnh **Hủy / Lưu** (trước khi đặt lại sẽ yêu cầu xác nhận). Nếu client bị tắt do hết lưu lượng, việc đặt lại (cả đơn lẻ lẫn hàng loạt) sẽ tự động **bật lại** client (`enable = true`) và ngay lập tức phát tán thay đổi này đến các node – không cần phải bật lại client thủ công trên master và các node nữa.

Đặt lại giới hạn IP

Xóa nhật ký IP tích lũy của client (`POST /clearIps/:email`) để gỡ bỏ lệnh chặn tạm thời do vượt quá giới hạn số kết nối đồng thời. Thông báo: «Nhật ký đã được xóa».

Xóa

Xóa (`POST /del/:email`) – xóa hoàn toàn client. Hộp thoại xác nhận: tiêu đề «Xóa client {email}?», nội dung «Client sẽ bị xóa khỏi tất cả các inbound được liên kết, và bản ghi lưu lượng của nó sẽ bị hủy. Thao tác này không thể hoàn tác.». Xóa gỡ client khỏi **tất cả** inbound và hủy bản ghi lưu lượng của nó. Thông báo: «Client đã được xóa».

8.4. Thao tác hàng loạt

Trong danh sách client có thể chọn nhiều bản ghi (**Chọn tất cả**, **Bỏ chọn tất cả**); bộ đếm – «{count} đã chọn». Trên các bản ghi đã chọn có sẵn:

- **Xóa ({count})** (`POST /bulkDel`) – xóa theo nhóm. Xác nhận: «Xóa {count} client?», «Mỗi client đã chọn sẽ bị xóa khỏi tất cả các inbound được liên kết, bản ghi lưu lượng của nó bị hủy. Thao tác này không thể hoàn tác.». Thông báo: «Đã xóa client: {count}», khi có lỗi một phần – «Đã xóa: {ok}, thất bại: {failed}».
- **Sửa ({count}) / Điều chỉnh** (`POST /bulkAdjust`) – thay đổi hàng loạt thời hạn và/hoặc hạn mức. Hộp thoại «Sửa {count} client» với chú thích «Giá trị dương thêm vào, giá trị âm giảm đi. Các client với thời hạn hoặc lưu lượng không giới hạn bị bỏ qua cho trường tương ứng.». Các trường: **Thêm ngày**, **Thêm lưu lượng (GB)** và **Set flow**. Logic:
 - **Thời hạn:** các client với thời hạn vĩnh viễn (`expiryTime == 0`) bị bỏ qua («unlimited expiry»); với các client có ngày hết hạn, thời hạn được dịch chuyển theo số ngày đã chỉ định; với các client ở chế độ «sau lần sử dụng đầu tiên» (thời hạn âm), độ dài thời gian chờ được điều chỉnh. Việc giảm vượt quá phần còn lại bị bỏ qua («reduction exceeds remaining time/delay window»).
 - **Lưu lượng:** các client không giới hạn (`totalGB == 0`) bị bỏ qua («unlimited traffic»); nếu không, hạn mức thay đổi theo lượng đã chỉ định, không xuống dưới không.
 - **Flow:** danh sách thả xuống **Set flow** cho phép đặt hoặc xóa XTLS flow cho tất cả các client đã chọn cùng lúc. Mặc định chọn **No change** (không thay đổi). Tùy chọn **Disable (clear flow)** xóa flow, còn các giá trị `xtls-rprx-vision` và `xtls-rprx-vision-udp443` đặt vision-flow tương ứng. Việc đặt vision-flow chỉ áp dụng cho các inbound hỗ trợ flow; các inbound không phù hợp vẫn không thay đổi và được đánh dấu là đã bỏ qua, trong khi việc xóa flow luôn được phép.
 - **Tự động bật (từ 3.4.2):** client đã bị tắt **chỉ vì hết hạn mức** (hết hạn hoặc vượt quá hạn mức), khi gia hạn sẽ tự động được bật lại – cục bộ và trên node của nó – nếu việc điều chỉnh đưa nó trở lại trong giới hạn. Những client bị tắt thủ công hoặc vẫn còn hết hạn mức vẫn bị tắt (trường `enable` nay được ghi rõ ràng, nên trạng thái được giữ cả với các client không có inbound).
 - Nếu không chỉ định ngày, lưu lượng hay flow: «Hãy chỉ định ngày, lưu lượng hoặc flow trước khi áp dụng.». Thông báo: «Đã sửa: {count}» / «Đã sửa: {ok}, đã bỏ qua: {skipped}».

Ví dụ: gia hạn các client đã chọn thêm 30 ngày và thêm 50 GB. Trong hộp thoại **Sửa**, nhập **Thêm ngày** = 30 , **Thêm lưu lượng (GB)** = 50 . Để ngược lại, trừ đi một tuần và giảm hạn mức 10 GB, hãy nhập giá trị âm: **Thêm ngày** = -7 , **Thêm lưu lượng (GB)** = -10 (các client với thời hạn vĩnh viễn hoặc không giới hạn cho trường tương ứng sẽ bị bỏ qua).

- **Liên kết ({count}) / Hủy liên kết ({count})** (POST /bulkAttach / bulkDetach) – liên kết/hủy liên kết hàng loạt các client đã chọn với các inbound đã chọn. Mục tiêu – chỉ các inbound đa người dùng. Kết quả hủy liên kết: «Đã hủy liên kết {detached}, đã bỏ qua {skipped}». Từ 3.5.0, bộ chọn inbound trong biểu mẫu client **ấn các inbound đã tắt** (trừ những inbound mà client đã được liên kết), còn trong cửa sổ «Hủy liên kết» đã loại bỏ việc tích lũy các bản ghi trùng lặp của cùng một client (dữ liệu được sửa tự động khi panel khởi động).
- **Liên kết đăng ký ({count})** – bảng tổng hợp URL đăng ký và URL đăng ký JSON của các client đã chọn với nút **Sao chép tất cả**. Nếu không ai có subId: «Không có client nào trong số đã chọn có ID đăng ký».
- **Thêm vào nhóm và Bỏ nhóm** – gán và xóa nhãn nhóm.
- **Bật ({count}) / Tắt ({count})** (POST /bulkEnable / bulkDisable) – bật và tắt hàng loạt các client đã chọn. **Bật** kích hoạt từng client đã chọn trên tất cả các inbound được liên kết; các client đã hết hạn mức lưu lượng hoặc hết hạn sẽ tự động bị tắt lại. **Tắt** ngay lập tức tước quyền truy cập của client, nhưng các bản ghi và lưu lượng tích lũy của họ được giữ lại. Trước khi thực hiện, panel yêu cầu xác nhận, và sau thao tác hiển thị thông báo với số lượng client đã xử lý và, nếu có, với số lượng những client mà thao tác không thành công.

Đặt lại lưu lượng và xóa theo trạng thái

- **Đặt lại lưu lượng của tất cả client** (POST /resetAllTraffics) – đặt lại bộ đếm up / down của **tất cả** client trong panel về không. Xác nhận: «Đặt lại lưu lượng của tất cả client?» và «Bộ đếm gửi/nhận của tất cả client được đặt lại về không. Hạn mức và thời hạn hiệu lực không bị ảnh hưởng. Thao tác này không thể hoàn tác». Thông báo: «Lưu lượng của tất cả client đã được đặt lại».
- **Xóa đã hết** (POST /delDepleted) – xóa tất cả client đã **hết hạn mức** (total > 0 and up + down >= total) **hoặc hết hạn** (expiry_time > 0 and expiry_time <= hiện tại), với điều kiện reset = 0 (các client có tự gia hạn không bị động đến). Xác nhận: «Xóa các client đã hết?», «Xóa tất cả client đã hết hạn mức lưu lượng hoặc hết hạn. Thao tác này không thể hoàn tác». Thông báo: «Đã xóa client đã hết: {count}».

Xuất, nhập và xóa các client không được liên kết

Khi không có gì được chọn, trong menu **Thêm** trên trang **Clients** có sẵn ba thao tác.

Xuất client (GET /clients/export) mở trình xem với danh sách JSON của tất cả client theo định dạng {client, inboundIds} với các nút sao chép và tải xuống (tệp clients-export.json). **Nhập client** (POST /clients/import) mở trình soạn thảo, nơi dán JSON tương tự và nhấn **Import**: các client có inboundIds được tạo và liên kết với inbound, các client không có liên kết được khôi phục dưới dạng các bản ghi «trần» riêng lẻ, còn các email đã tồn tại **không bao giờ bị ghi đè** – chúng được đưa vào danh sách đã bỏ qua. Thông báo: «{count} clients imported», «{ok} imported, {failed} skipped».

Xóa các client không được liên kết (`POST /clients/orphans`) – thao tác nguy hiểm: xóa tất cả client không được liên kết với bất kỳ inbound nào, cùng với bản ghi lưu lượng, nhật ký IP và các liên kết ngoài của họ. Xác nhận: «Delete clients without an inbound?», «Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone». Thông báo: «{count} unattached clients deleted». Thao tác không thể hoàn tác.

8.5. Tìm kiếm, lọc và sắp xếp

Phía trên danh sách có thanh tìm kiếm («Tìm kiếm email, ghi chú, sub ID, UUID, mật khẩu, auth, Telegram ID...») – tìm theo email, ghi chú, subId, UUID, mật khẩu, auth và (một lần nữa từ 3.5.0) theo **Telegram ID**. Bộ đếm kết quả: «Hiển thị {shown} trong {total}».

Danh sách client cập nhật tự động: panel mỗi vài giây lấy trang hiện tại cập nhật, vì vậy các client mới kết nối và thứ tự sắp xếp thay đổi xuất hiện mà không cần làm mới thủ công (chỉ báo tải không nhấp nháy khi đang thăm dò nền).

Từ 3.5.0, trong bảng có cột «**Tốc độ**» (giữa «Lưu lượng» và «Còn lại»): tốc độ trực tiếp của client – nhân màu xanh dương ↑ tải lên / ↓ tải xuống (trung bình trượt ~5 giây, cập nhật mỗi 5 giây); khi không có lưu lượng – dấu gạch ngang màu xám. Trên di động, tốc độ hiển thị dưới dạng một dòng trong thẻ client.

Bảng **Lọc client** cho phép lọc theo trạng thái (danh mục), giao thức, inbound được liên kết, phạm vi thời hạn, phạm vi lưu lượng đã dùng, sự hiện diện của tự gia hạn (**Có/Không**), sự hiện diện của ID Telegram và ghi chú, cũng như theo nhóm. Trên các panel có node xuất hiện bộ chọn đa **Node**: có thể giới hạn danh sách chỉ các client của các node đã chọn; mục riêng **Panel cục bộ** lọc các client của inbound không liên kết với node (bộ lọc chỉ hiển thị khi có node). Sắp xếp: **Cũ nhất/Mới nhất, Cập nhật gần đây, Trực tuyến gần đây, Email A→Z / Z→A, Lưu lượng nhiều hơn, Còn lại nhiều hơn, Sắp hết hạn nhất**.

8.6. Biểu tượng và trạng thái

Thứ tự ưu tiên trạng thái: đã hết/hết hạn → không hoạt động → sắp hết hạn → đang hoạt động.

- **Trực tuyến / Ngoại tuyến** – client có kết nối đang hoạt động (có trong danh sách trực tuyến hiện tại) và **đã bật**. Danh sách trực tuyến được cập nhật bằng các yêu cầu riêng (`/onlines` , `/onlinesByGuid`).
- **Đã hết** (depleted) – hạn mức đã dùng hết (`up + down >= totalGB`) **hoặc** hết hạn (`expiryTime <=` hiện tại). Client như vậy tự động bị tắt và rơi vào phạm vi của **Xóa đã hết**.
- **Sắp hết hạn / sắp hết** (expiring) – client đã bật có thời gian còn lại đến hết hạn ít hơn ngưỡng khoảng thời gian **hoặc** lưu lượng còn lại đến hết hạn mức ít hơn ngưỡng khối lượng (các ngưỡng được đặt trong cài đặt panel). Không tính nếu client đã hết/bị tắt.
- **Không hoạt động** (deactive) – client có `enable = false` (bị tắt thủ công hoặc bởi tác vụ nền).
- **Đang hoạt động** (active) – đã bật, chưa hết hạn mức, thời hạn chưa hết và vẫn còn cách xa các ngưỡng.

9. Nhóm khách hàng

Đây là tính năng của bản fork 3X-UI này. Trong dự án gốc 3x-ui (MHSanaei) không có khái niệm "nhóm khách hàng" — bản fork này bổ sung bảng nhóm riêng, trang **Nhóm** trong menu panel và các phương thức API tương ứng. Nếu bạn chuyển cấu hình sang 3x-ui gốc, nhân nhóm sẽ không được xử lý ở đâu cả.

9.1. Nhóm khách hàng là gì và dùng để làm gì

Nhóm là một nhân logic (label) có tên, có thể gán cho một hoặc nhiều khách hàng. Nhóm không tạo ra phương thức kết nối mới và không phải là inbound hay node — đây thuần túy là một nhân tổ chức, giúp lọc và xử lý hàng loạt khách hàng một cách thuận tiện.

Ý tưởng cốt lõi của mô hình khách hàng trong bản fork này: **khách hàng là thực thể cấp cao nhất, được nhận dạng bằng email** (trường `email` trong bảng `clients` có chỉ mục duy nhất). Cùng một khách hàng (một email với cùng thông tin xác thực) có thể đồng thời thuộc nhiều inbound và thậm chí nhiều node, kể cả với các giao thức khác nhau. Nhân nhóm được lưu **một lần trên mỗi khách hàng**, do đó nó tự động áp dụng cho tất cả các liên kết của khách hàng đó với các inbound cùng một lúc.

Nhân nhóm là nhân logic để phân nhóm:

Lớp	Nơi lưu trữ	Trường
Bản ghi khách hàng (CSDL)	bảng <code>clients</code>	<code>group_name</code> (mặc định là chuỗi rỗng <code>''</code>)
Danh mục nhóm (CSDL)	bảng <code>client_groups</code>	<code>name</code> (chỉ mục duy nhất, không được rỗng)
Cài đặt inbound (Xray)	JSON <code>settings.clients[].group</code>	được sao chép vào từng đối tượng khách hàng của mỗi inbound mà khách hàng thuộc về

Tại sao điều này cần thiết trong thực tế:

- **Một khách hàng trên nhiều inbound/node.** Nếu khách hàng được "bán" như quyền truy cập vào nhiều inbound cùng lúc (ví dụ: các giao thức khác nhau hoặc các node khác nhau), nhóm cho phép quản lý khách hàng đó như một thể thống nhất: đặt lại lưu lượng, xóa, đổi tên nhân — một thao tác áp dụng cho tất cả inbound của khách hàng đó.
- **Thao tác hàng loạt và lọc.** Trên trang **Khách hàng**, danh sách có thể được lọc theo nhóm; trên trang **Nhóm**, có sẵn các thao tác hàng loạt đối với tất cả thành viên của nhóm.
- **Quản lý số lượng lớn khách hàng.** Các nhân như `vip`, `trial`, `team-A` giúp sắp xếp hàng nghìn khách hàng vào các nhóm logic mà không cần tạo thêm inbound riêng biệt.

9.2. Môi liên hệ của nhóm với khách hàng, inbound, node và giao thức

Đây là phần quan trọng nhất cần hiểu, vì việc đồng bộ nhân không hề đơn giản.

Nhóm mô tả khách hàng, không phải inbound. Nhân tồn tại trong bản ghi khách hàng

(`clients.group_name`). Khi một khách hàng được liên kết với nhiều inbound, bất kỳ khi nào nhóm thay đổi, panel sẽ duyệt qua **tất cả** các inbound mà khách hàng đó thuộc về và gán/xóa trường `group` bên trong cài đặt Xray (`settings.clients[]`). Về mặt kỹ thuật, điều này được thực hiện như sau: tìm tất cả inbound chứa khách hàng đó theo email, sau đó sửa đổi trường khách hàng có email đó trong cài đặt JSON của mỗi inbound như vậy. Do đó:

- Nhóm **không phụ thuộc vào giao thức**. Một email có thể là khách hàng VLESS trong inbound này và là khách hàng Hysteria trong inbound khác – nhân nhóm của khách hàng đó vẫn là một và được áp dụng cho cả hai (trong khi thông tin xác thực của mỗi giao thức là riêng và được lưu riêng biệt).
- Nhóm **bao gồm các node**. Các inbound thuộc về node khác với inbound của panel chính ở trường `nodeId` (đối với inbound của panel chính, trường này là `null / 0`). Nhân nhóm được áp dụng cho các đối tượng khách hàng trong inbound bất kể đó là inbound chính hay inbound node – miễn là khách hàng có email đó tồn tại ở đó.

Nhân nhóm ổn định khi đồng bộ từ node và khi tái cấu hình cài đặt. Hành vi này được thiết kế đặc biệt:

- Khi một node gửi snapshot lưu lượng, dữ liệu của node đó **không ghi đè** `group_name` và `comment` cục bộ của khách hàng trong CSDL panel. Nhóm và chú thích được coi là các trường "cục bộ của panel" – node không quản lý chúng.
- Khi tái cấu hình cài đặt inbound, giá trị `group` rỗng trong dữ liệu đến **không đặt lại** nhân đã lưu. Nhóm được quản lý qua trang **Nhóm** (chứ không phải qua chỉnh sửa cài đặt Xray của inbound), do đó "nhóm rỗng" khi tái cấu hình thông thường được hiểu là "không thay đổi", chứ không phải "xóa sạch".

Kết luận thực tế: các thao tác duy nhất **có ý xóa** nhân là xóa nhóm và xóa khách hàng khỏi nhóm một cách rõ ràng (xem bên dưới). Việc chỉnh sửa inbound thông thường hoặc đồng bộ nền với node sẽ không làm mất nhóm.

9.3. Danh mục nhóm và các nhóm "rỗng"

Danh sách nhóm trên trang được tạo bằng cách hợp nhất hai nguồn:

- Nhóm dẫn xuất (derived)** – tất cả các giá trị `group_name` không rỗng thực sự xuất hiện ở các khách hàng, cùng với số lượng khách hàng.
- Nhóm đã lưu (stored)** – các bản ghi trong bảng `client_groups` .

Sự hợp nhất này mang lại một hiệu ứng quan trọng: một nhóm có thể tồn tại **mà không có một khách hàng nào**. Nhóm như vậy được tạo bằng nút "Thêm nhóm" (bản ghi trong `client_groups`) và hiển thị trong danh sách với bộ đếm `0` . Các bản ghi này được gọi là **nhóm rỗng**. Danh sách luôn được sắp xếp theo tên không phân biệt chữ hoa/thường.

Các bộ đếm tổng quan trên trang:

Trường	Ý nghĩa
Tổng số nhóm	Tổng số nhóm (đã lưu và dẫn xuất gộp lại).
Khách hàng có nhóm	Số khách hàng có nhân nhóm không rỗng.

Trường	Ý nghĩa
Nhóm rỗng	Số nhóm tồn tại mà không có khách hàng (bộ đếm 0).
Khách hàng trong nhóm	Số khách hàng trong một nhóm cụ thể (cột bảng).

9.4. Các trường và cột của nhóm

Bản ghi nhóm trong bảng `client_groups` chứa:

Trường	Kiểu	Mặc định	Mô tả
<code>Id</code>	int	tự tăng	Khóa chính của bản ghi nhóm.
<code>Name</code>	string	– (bắt buộc)	Tên nhóm. Chỉ mục duy nhất, không được rỗng. Trong UI – cột Tên .
<code>CreatedAt</code>	int64 (ms)	thời gian tạo	Thời điểm tạo bản ghi nhóm.
<code>UpdatedAt</code>	int64 (ms)	thời gian sửa	Thời điểm sửa đổi lần cuối.

Bảng trên trang hiển thị ít nhất các cột **Tên** và **Khách hàng trong nhóm**, cùng với các nút hành động (xem bên dưới).

9.5. Tạo nhóm

Nút **Thêm nhóm**.

Các bước:

1. Nhấn **Thêm nhóm**.
2. Nhập tên nhóm.
3. Xác nhận.

Hành vi backend (`POST /panel/api/clients/groups/create` , body `{"name": "..."}`):

- Tên được cắt bỏ khoảng trắng đầu/cuối. Tên rỗng bị từ chối với lỗi «group name is required».
- Nếu nhóm có tên như vậy đã tồn tại – lỗi «group already exists».
- Khi thành công, một bản ghi được tạo trong `client_groups` (ban đầu không có khách hàng – đây là nhóm rỗng).

Thông báo thành công: **«Nhóm «{name}» đã được tạo.»**

Ví dụ: tạo nhóm rỗng qua API. Chuẩn bị sẵn bộ nhân trước khi thêm khách hàng:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

Phản hồi khi thành công:


```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

Gọi lại với cùng tên sẽ trả về `"success": false` và thông báo `group already exists`.

Tạo nhóm rỗng trước rất tiện khi bạn muốn chuẩn bị sẵn bộ nhân rồi sau đó thêm khách hàng vào qua «Thêm khách hàng...».

9.6. Đổi tên nhóm

Nút **Đổi tên**, tiêu đề hộp thoại – «**Đổi tên {name}**».

Hành vi (`POST /panel/api/clients/groups/rename`, body `{"oldName": "...", "newName": "..."}:`

- Cả hai tên đều được cắt bỏ khoảng trắng. Tên cũ rỗng – lỗi «old group name is required», tên mới rỗng – «new group name is required».
- Nếu tên mới trùng với tên cũ – không làm gì (0 khách hàng bị ảnh hưởng).
- Nếu khác, việc đổi tên được thực hiện nguyên tử:
 - bản ghi trong `client_groups` được đổi tên;
 - tất cả khách hàng có `group_name = oldName` được cập nhật thành `newName`;
 - trong **tất cả inbound** mà các khách hàng bị ảnh hưởng thuộc về (kể cả inbound node), giá trị `group` trong cài đặt Xray được sửa từ tên cũ sang tên mới.
- Sau khi đổi tên, panel đánh dấu Xray cần khởi động lại và gửi thông báo về việc thay đổi khách hàng.

Thông báo:

- Thành công: «**Nhóm đã được đổi tên cho {count} khách hàng.**»
- Xung đột tên trong UI: «**Nhóm có tên «{name}» đã tồn tại.**»

9.7. Thêm khách hàng vào nhóm

Nút **Thêm khách hàng...**, tiêu đề – «**Thêm khách hàng vào nhóm «{name}»**».

Gợi ý trong hộp thoại:

«Chọn khách hàng để thêm vào nhóm này. Các liên kết inbound hiện có được giữ nguyên; chỉ nhân nhóm thay đổi. Những khách hàng đã thuộc nhóm này sẽ không hiển thị.»

Nếu không có ai để thêm, hiển thị «**Không có khách hàng nào khác để thêm.**»

Hành vi (`POST /panel/api/clients/groups/bulkAdd`, body `{"emails": [...], "group": "..."}:`

- Tên nhóm là bắt buộc (nếu không có sẽ báo lỗi «group name is required»); danh sách email rỗng – thao tác không làm gì.
- Nếu nhóm như vậy chưa tồn tại trong `client_groups` hay trong các nhóm dẫn xuất – nhóm sẽ được tạo tự động.

- Đối với các email được chọn, khách hàng được gán `group_name = group`; **các liên kết của khách hàng với inbound không thay đổi** – chỉ nhân bị ảnh hưởng. Sau đó trường `group` được gán trong tất cả inbound của các khách hàng đó.
- Trả về số bản ghi khách hàng bị ảnh hưởng; Xray được đánh dấu cần khởi động lại.

Thông báo thành công: **«Đã thêm {count} khách hàng vào {name}.»**

Ví dụ: gán nhóm cho nhiều khách hàng bằng một yêu cầu. Khách hàng được chỉ định bằng email, các liên kết inbound không thay đổi:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

Nếu nhóm `vip` chưa tồn tại, nó sẽ được tạo tự động. Sau yêu cầu, các khách hàng này trong bản ghi sẽ được gán `group_name = "vip"`, và đối tượng khách hàng trong cài đặt Xray của mỗi inbound của họ sẽ nhận trường `"group": "vip"`:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Xóa khách hàng khỏi nhóm (không xóa bản thân khách hàng)

Nút **Xóa khách hàng...**, tiêu đề – **«Xóa khách hàng khỏi nhóm «{name}»»**.

Gợi ý:

«Chọn thành viên để xóa khỏi nhóm này. Bản thân các khách hàng được giữ lại (sử dụng «Xóa khách hàng của nhóm» để xóa hoàn toàn).»

Hành vi (`POST /panel/api/clients/groups/bulkRemove`, body `{"emails": [...]}`): về mặt kỹ thuật, đây tương đương với «Thêm vào nhóm» với nhóm rỗng. Đối với các khách hàng được chọn, `group_name` được xóa, và trường `group` bị loại bỏ khỏi cài đặt Xray trong các inbound của họ. Bản thân các khách hàng và các liên kết inbound của họ được giữ nguyên.

Thông báo thành công: **«Đã xóa {count} khách hàng khỏi {name}.»**

9.9. Đặt lại lưu lượng của nhóm

Nút **Đặt lại lưu lượng**.

Hộp thoại xác nhận:

- Tiêu đề: **«Đặt lại lưu lượng của nhóm {name}?»**
- Nội dung: **«Chỉ đặt lại bộ đếm lưu lượng của nhóm. Bộ đếm của từng khách hàng không bị ảnh hưởng.»**

Hành vi: chỉ **bộ đếm hiển thị của chính nhóm** được đặt về 0 – bảng điều khiển ghi nhớ tổng up/down hiện tại của nhóm làm mốc cơ sở và sau đó trong danh sách nhóm hiển thị $\max(0, \text{tổng} - \text{mốc cơ sở})$. Các bộ đếm up / down, hạn mức và trường enable của từng khách hàng **không bị thay đổi**, không cần khởi động lại Xray. Đây là thay đổi hành vi so với các phiên bản trước, nơi việc đặt lại xóa lưu lượng của tất cả khách hàng trong nhóm.

Yêu cầu: POST /panel/api/clients/groups/resetTraffic với body {"name": "<tên nhóm>"}

Thông báo thành công: «**Đã đặt lại lưu lượng của nhóm {name}.**»

9.10. Xóa nhóm và xóa khách hàng của nhóm

Trên trang có **hai thao tác xóa về bản chất khác nhau** – dễ nhầm lẫn, vì vậy sự khác biệt rất quan trọng.

9.10.1. Xóa nhóm (giữ lại khách hàng)

Nút «**Xóa nhóm (giữ lại khách hàng)**».

Hộp thoại:

- Tiêu đề: «**Xóa nhóm {name}?**»
- Nội dung: «**Thao tác này xóa nhóm và xóa nhân của nó khỏi {count} khách hàng. Bản thân các khách hàng không bị xóa.**»

Hành vi (POST /panel/api/clients/groups/delete , body {"name": "..."}): bản ghi nhóm bị xóa khỏi client_groups , group_name của tất cả khách hàng trong nhóm được xóa, và trường group bị loại bỏ khỏi các inbound của họ. **Các khách hàng, kết nối và lưu lượng của họ được giữ lại.** Xray được đánh dấu cần khởi động lại.

Thông báo thành công: «**Đã xóa nhân nhóm khỏi {count} khách hàng.**»

9.10.2. Xóa khách hàng của nhóm (xóa hoàn toàn)

Nút «**Xóa khách hàng của nhóm**».

Hộp thoại:

- Tiêu đề: «**Xóa tất cả khách hàng trong {name}?**»
- Nội dung: «**Thao tác này xóa vĩnh viễn {count} khách hàng cùng với bản ghi lưu lượng của họ. Nhân nhóm cũng sẽ bị xóa. Không thể hoàn tác.**»

Đây là thao tác hủy diệt: nó xóa bản thân các khách hàng (qua xóa hàng loạt theo email, endpoint POST /panel/api/clients/bulkDel), bao gồm bản ghi lưu lượng của họ, và do đó loại bỏ họ khỏi tất cả inbound.

Thông báo:

- Thành công: «**Đã xóa {count} khách hàng.**»
- Kết quả một phần: «**{ok} đã xóa, {failed} đã bỏ qua**»

Nếu nhóm rỗng, các thao tác đối với thành viên sẽ không khả dụng – hiển thị **«Nhóm này hiện chưa có khách hàng nào.»**

9.11. Liên kết với trang «Khách hàng»

Nhân nhóm hiển thị và được sử dụng cả ngoài trang **Nhóm**:

- Trong bản ghi gọn của khách hàng có trường `group`, vì vậy trong danh sách khách hàng sẽ hiển thị nhóm mà khách hàng thuộc về.
- Danh sách khách hàng (`/panel/api/clients/list/paged`) nhận tham số lọc `group`: có thể truyền một tên hoặc nhiều tên cách nhau bằng dấu phẩy. Việc so khớp được thực hiện theo nguyên tắc "HOẶC" trong trường, không phân biệt chữ hoa/thường. Trường hợp đặc biệt: phần tử rỗng trong danh sách nhóm lọc có nghĩa là "khách hàng không có nhóm" (những khách hàng có `group` rỗng).
- Trong phản hồi trang khách hàng, mảng `groups` được trả về – danh sách đầy đủ tên các nhóm hiện có, để UI có thể xây dựng danh sách thả xuống cho bộ lọc.

Ví dụ: lọc khách hàng theo nhóm. Yêu cầu chỉ trả về khách hàng có nhãn `vip` hoặc `trial` (nhiều tên cách nhau bằng dấu phẩy, «HOẶC»):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Để lấy khách hàng **không có** nhóm, hãy truyền phần tử rỗng trong danh sách – ví dụ, giá trị lọc `group=` (chuỗi rỗng) hoặc `group=vip,` (nhãn `vip` cộng khách hàng không có nhóm).

9.12. Tổng hợp các endpoint API

Tất cả các route nhóm được gắn dưới `/panel/api/clients`:

Phương thức và đường dẫn	Mục đích	Body yêu cầu
GET <code>/panel/api/clients/groups</code>	Danh sách nhóm với số lượng khách hàng	–
GET <code>/panel/api/clients/groups:name/emails</code>	Email của tất cả thành viên nhóm (sắp xếp theo email)	–
POST <code>/panel/api/clients/groups/create</code>	Tạo nhóm rỗng	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Đổi tên nhóm	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Xóa nhóm, giữ lại khách hàng (xóa nhân)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Thêm khách hàng vào nhóm (theo email)	<code>{"emails": [...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Xóa khách hàng khỏi nhóm (xóa nhân)	<code>{"emails": [...]}</code>

Phương thức và đường dẫn	Mục đích	Body yêu cầu
POST /panel/api/clients/bulkDel	Xóa hoàn toàn khách hàng (được dùng bởi «Xóa khách hàng của nhóm»)	<code>{"emails": [...], "keepTraffic"}</code>

Ví dụ: kích bản vòng đời nhóm điển hình qua API.

```
# 1. Tạo nhãn trial
curl -s .../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. Gắn nhãn đó cho hai khách hàng
curl -s .../panel/api/clients/groups/bulkAdd -d '{"emails": ["u1@example.com", "u2@example.com"], "group":"trial"}'

# 3. Đặt lại lưu lượng của tất cả thành viên (theo email từ /groups/trial/emails)
curl -s .../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Xóa nhóm, nhưng giữ lại khách hàng (chỉ xóa nhãn)
curl -s .../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

Bước 4 xóa bản ghi nhóm và xóa `group_name` của các khách hàng trong nhóm, nhưng bản thân các khách hàng, kết nối và lưu lượng của họ vẫn được giữ lại. Để xóa vĩnh viễn bản thân các khách hàng, hãy sử dụng `bulkDel` thay thế.

Các thao tác thay đổi nhãn của khách hàng (`rename` , `delete` , `bulkAdd` , `bulkRemove`) đánh dấu Xray cần khởi động lại và gửi thông báo về việc thay đổi khách hàng.

9.13. Lưu lượng theo nhóm

Tính năng mới trong phiên bản 3.3.0: trong phần **Nhóm** (trang «Khách hàng», tab quản lý nhóm), bảng nhóm giờ đây hiển thị không chỉ số lượng khách hàng trong mỗi nhóm mà còn cả tổng lưu lượng đã sử dụng của nhóm. Cột được đặt tên là «**Lưu lượng đã sử dụng**».

Cột hiển thị gì

Đối với mỗi hàng nhóm, tổng lưu lượng được hiển thị theo tất cả khách hàng thuộc nhóm đó – tức là tổng `up + down` (lưu lượng gửi + nhận) của tất cả thành viên. Điều này cho phép trả lời nhanh câu hỏi «cả nhóm đã tải về/gửi đi tổng cộng bao nhiêu», mà không cần mở từng khách hàng và cộng tay.

Bên cạnh đó trong bảng nhóm còn hiển thị:

Cột	Ý nghĩa
Tên	Tên nhóm
Khách hàng	Số khách hàng được gắn nhãn nhóm này (trước đây cột có tên «Khách hàng trong nhóm»)
Đã gửi	Tổng <code>up</code> (lưu lượng gửi) của tất cả khách hàng trong nhóm
Đã nhận	Tổng <code>down</code> (lưu lượng nhận) của tất cả khách hàng trong nhóm
Lưu lượng đã sử dụng	Tổng <code>up + down</code> của tất cả khách hàng trong nhóm

Lưu lượng gửi và nhận được hiển thị trong các cột riêng biệt **Đã gửi** và **Đã nhận**, còn cột **Lưu lượng đã sử dụng** hiển thị tổng của chúng. Cột số lượng khách hàng đơn giản có tên là **Khách hàng**.

Phần tổng quan trên bảng còn hiển thị các tổng hợp cho tất cả nhóm – «**Tổng số nhóm**» và «**Khách hàng có nhóm**», còn tổng lưu lượng được chia thành hai thẻ: «**Tổng đã gửi / đã nhận**» (với mũi tên lên/xuống – lưu lượng gửi và nhận riêng biệt của tất cả nhóm) và «**Tổng lưu lượng**» (với biểu tượng biểu đồ – tổng cộng của chúng).

Cách tính

Việc tính toán được thực hiện bằng một truy vấn SQL đến bảng khách hàng với phép kết hợp (`LEFT JOIN`) bảng theo dõi lưu lượng:

- theo trường nhân nhóm (`group_name`), khách hàng được nhóm lại, số lượng được đếm – đó chính là «Khách hàng trong nhóm»;
- lưu lượng được lấy là tổng `up + down` từ bảng `client_traffics` được kết hợp. Tức là cả byte gửi đi (`up`) lẫn byte nhận về (`down`) đều được cộng lại cho mỗi khách hàng;
- vì email là duy nhất cả trong bảng khách hàng lẫn bảng lưu lượng, phép kết hợp không nhân đôi lưu lượng của một khách hàng.

Đặc điểm của các giá trị:

- Khách hàng không có bản ghi lưu lượng** được tính vào bộ đếm thành viên, nhưng cộng 0 vào tổng, do đó nhóm mới tạo hiển thị lưu lượng `0`.
- Nhóm rỗng** (đã được tạo nhưng không có khách hàng) cũng xuất hiện trong danh sách với bộ đếm và lưu lượng bằng 0: ngoài các nhóm «được suy ra» từ nhân khách hàng, các nhóm được lưu rõ ràng cũng được đưa vào kết quả, sau đó danh sách được sắp xếp theo tên không phân biệt chữ hoa/thường.
- Khách hàng không có nhân nhóm (`group_name` rỗng) không được tính vào phép tính.

Các thao tác liên quan

Từ bảng nhóm, các thao tác đối với toàn bộ nhóm vẫn khả dụng, bao gồm «**Đặt lại lưu lượng**» – chỉ đặt về 0 **bộ đếm của chính nhóm** (mốc cơ sở), không động đến lưu lượng của từng khách hàng (xem 9.9). Sau khi đặt lại như vậy, cột «Lưu lượng đã sử dụng» cho nhóm đó hiển thị `0`.

10. Đăng ký (Subscription)

Đăng ký (subscription) là cơ chế cho phép cấp cho client một liên kết (URL) cố định duy nhất, qua đó ứng dụng VPN tự tải về và định kỳ cập nhật toàn bộ bộ cấu hình. Thay vì gửi thủ công từng liên kết riêng lẻ cho từng inbound đến người dùng, họ chỉ nhận một địa chỉ duy nhất dạng `https://domain:port/sub/<subId>`. Qua địa chỉ này, panel tự động tổng hợp tất cả cấu hình liên kết với client đó và trả về theo định dạng mà client yêu cầu. Khi thay đổi cài đặt trên server (địa chỉ mới, xoay vòng khóa Reality, thêm inbound), client sẽ nhận được cấu hình mới nhất vào lần cập nhật tự động tiếp theo mà không cần thao tác nào từ người dùng.

Đăng ký được phục vụ bởi một server HTTP/HTTPS riêng biệt bên trong panel, chạy độc lập với giao diện web và lắng nghe trên cổng riêng. Điều này được thực hiện vì lý do bảo mật: cổng đăng ký có thể mở ra ngoài mà không cần mở cổng panel.

10.1. subId là gì và cách tạo liên kết

Mỗi client trong inbound có trường `subId` (trong giao diện — «ID đăng ký»). Giá trị này chính là khóa đăng ký: panel tìm kiếm trong tất cả inbound các client có `subId` khớp với yêu cầu và gộp các cấu hình của họ thành một phản hồi.

- Nếu nhiều client (trong cùng một hoặc các inbound khác nhau) có cùng `subId`, các cấu hình của họ sẽ được gộp vào một đăng ký. Đây là cách tiêu chuẩn để cấp cho một người dùng nhiều server/giao thức qua một liên kết duy nhất.

Ví dụ: một người dùng — hai server qua một liên kết. Giả sử có hai inbound (VLESS trên server A và Trojan trên server B). Để cấp cho người dùng cả hai cấu hình qua một liên kết, hãy đặt cùng `subId` cho cả hai client của họ:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Khi đó tại địa chỉ `https://sub.example.com:2096/sub/ivan2025`, panel sẽ trả về cả hai cấu hình cùng lúc. Nếu sau này thêm inbound thứ ba với cùng `subId` đó — nó sẽ xuất hiện cho người dùng vào lần tự cập nhật đăng ký tiếp theo mà không cần gửi liên kết mới.

- Nếu trường `subId` của client để trống, không thể chia sẻ liên kết để truy cập chung. Trong giao diện điều này được ghi chú: «Client này không có subId, liên kết truy cập chung không khả dụng.»

Liên kết ngoài và đăng ký của client (tab «Links»)

Trong form client có tab «Links», nơi cho từng client riêng lẻ có thể đính kèm các nguồn cấu hình bổ sung được trộn vào đúng đăng ký của client đó (các định dạng RAW, JSON và Clash):

- **Add External Link** — liên kết chia sẻ bên ngoài (`vless://`, `trojan://`, `ss://` v.v.). Được thêm vào đầu ra nguyên vẹn, và còn được phân tích thành cấu hình cho JSON/Clash.
- **Add External Subscription** — địa chỉ đăng ký bên ngoài. Panel tự tải về (có bộ nhớ cache và thời gian chờ ngắn) và đưa các cấu hình nhận được vào danh sách chung của client.

Điều này tiện lợi để cung cấp cho client các server bổ sung ngoài các inbound của bạn qua cùng một liên kết duy nhất đó. Nếu phản hồi đăng ký từ xa quá lớn, nó không còn bị cắt ngắn nữa: panel trả về lỗi và tiếp tục sử dụng giá trị cache thành công cuối cùng.

- Giá trị `subId` không thể đặt tùy ý: khi lưu sẽ kiểm tra xem có chứa dấu cách, ký tự `/`, `\` và ký tự điều khiển hay không. Gợi ý xác thực tương ứng: «ID đăng ký không được chứa dấu cách, '/', '' hoặc ký tự điều khiển».

Liên kết cuối cùng được xây dựng theo dạng `< cơ sở >/<subPath>/<subId>` (xem phần cài đặt server đăng ký và trường «URI proxy ngược»). Nếu không tìm thấy client nào theo `subId` (client đã bị xóa, `subId` không tồn tại), server trả về HTTP 404 không có nội dung. Khi có lỗi nội bộ – HTTP 500. Các ứng dụng VPN chỉ dựa vào mã phản hồi, vì vậy nội dung lỗi được để trống có chủ ý.

Thứ tự liên kết inbound trong đăng ký

Mỗi inbound có trường «**Thứ tự trong đăng ký**» (`subSortIndex`) – số từ 1, xác định vị trí liên kết của inbound đó trong đầu ra đăng ký. Giá trị nhỏ hơn đứng trước; khi bằng nhau thì giữ nguyên thứ tự tạo ban đầu (theo id). Thứ tự được áp dụng cho tất cả định dạng đầu ra – văn bản thuần, trang đăng ký, JSON và Clash. Trường này không ảnh hưởng đến thứ tự inbound trong panel.

Trường được chỉnh sửa trong form inbound bên cạnh cài đặt địa chỉ trong liên kết (share address) và được đồng bộ đến các node theo quy tắc thông thường. Nếu ít nhất một inbound có thứ tự khác 1, cột gọn «**Thứ tự**» sẽ xuất hiện trong danh sách Inbounds.

10.2. Cài đặt server đăng ký

Tất cả các thông số đăng ký nằm trong phần cài đặt panel tại tab «**Đăng ký**». Dưới đây là giải thích từng thông số; trong ngoặc là khóa cài đặt nội bộ và giá trị mặc định.

Phần này được chia thành các tab: «**Cài đặt panel**», «**Thông tin**», «**Hồ sơ**», «**Chứng chỉ**», «**Happ**» và «**Clash / Mihomo**». Các trường tiêu đề đăng ký, URL hỗ trợ, trang hồ sơ, thông báo và thư mục chủ đề nằm ở tab «**Hồ sơ**»; quy tắc định tuyến Happ và Clash/Mihomo – ở các tab tương ứng; khoảng thời gian cập nhật đăng ký – ở tab «**Thông tin**».

Thông số chính

Trường (UI)	Khóa	Giá trị mặc định	Mô tả
Bật đăng ký	<code>subEnable</code>	<code>true</code> (bật)	Khởi động server đăng ký riêng biệt. Gợi ý: «Tính năng đăng ký với cấu hình riêng biệt». Nếu tắt – server đăng ký không khởi động và không có liên kết nào hoạt động.
IP lắng nghe	<code>subListen</code>	trống	Địa chỉ IP mà server đăng ký nhận kết nối. Gợi ý: «Để trống theo mặc định để theo dõi tất cả địa chỉ IP».
Cổng đăng ký	<code>subPort</code>	<code>2096</code>	Cổng TCP của server đăng ký. Gợi ý: «Số cổng phục vụ dịch vụ đăng ký không được sử dụng trên server» – cổng phải trống và không xung đột với panel hoặc Xray.

Trường (UI)	Khóa	Giá trị mặc định	Mô tả
Đường dẫn URI	subPath	/sub/	Đường dẫn để phục vụ đăng ký thông thường. Gợi ý: «Phải bắt đầu bằng '/' và kết thúc bằng '/'».
Tên miền lắng nghe	subDomain	trống	Tên miền được phép truy cập đăng ký (xác thực Host). Gợi ý: «Để trống theo mặc định để lắng nghe tất cả tên miền và địa chỉ IP». Nếu được đặt – các yêu cầu có Host khác sẽ bị từ chối.

Lưu ý về bảo mật: đường dẫn mặc định /sub/ (và /json/ cho JSON) được biết đến rộng rãi và dễ đoán. Panel hiển thị cảnh báo: «Đường dẫn đăng ký mặc định "/sub/" được biết đến rộng rãi – hãy thay đổi nó.» và tương tự cho JSON. Nên đặt đường dẫn tùy chỉnh không rõ ràng.

TLS / Chứng chỉ

Trường (UI)	Khóa	Mặc định	Mô tả
Đường dẫn file khóa công khai chứng chỉ đăng ký	subCertFile	trống	Đường dẫn đầy đủ đến file chứng chỉ (.crt / fullchain). Gợi ý: «Nhập đường dẫn đầy đủ bắt đầu bằng '/'».
Đường dẫn file khóa riêng tư chứng chỉ đăng ký	subKeyFile	trống	Đường dẫn đầy đủ đến file khóa riêng tư. Gợi ý: «Nhập đường dẫn đầy đủ bắt đầu bằng '/'».

Nếu cả hai đường dẫn được đặt và chứng chỉ tải thành công, server đăng ký hoạt động qua **HTTPS**. Nếu các trường trống hoặc chứng chỉ không đọc được – server quay lại dùng **HTTP** (lỗi được ghi vào log). Sự có mặt của TLS hợp lệ cũng ảnh hưởng đến việc tạo URL cơ sở: với cổng 443 có TLS và cổng 80 không có TLS, số cổng sẽ bị bỏ qua trong liên kết.

Khoảng thời gian cập nhật

Trường (UI)	Khóa	Mặc định	Mô tả
Khoảng thời gian cập nhật đăng ký	subUpdates	12	Tần suất (tính bằng giờ) mà ứng dụng client nên yêu cầu lại đăng ký. Gợi ý: «Khoảng thời gian giữa các lần cập nhật trong ứng dụng client (tính bằng giờ)». Từ 3.5.0, trường chấp nhận 0–525600 và hiển thị phạm vi (giới hạn giao diện cũ 168 không khớp với máy chủ và sau khi nâng cấp từ 2.x từng chặn việc lưu mọi cài đặt).

Giá trị được truyền đến client trong HTTP header `Profile-Update-Interval` ; các client hiện đại sử dụng nó làm chu kỳ tự động cập nhật cấu hình.

Định dạng và thông tin trong phản hồi

Trường (UI)	Khóa	Mặc định	Mô tả
Mã hóa	subEncrypt	true	Gợi ý: «Mã hóa các câu hình được trả về trong đăng ký». Về mặt kỹ thuật đây không phải mã hóa mà là mã hóa Base64 toàn bộ nội dung đăng ký thông thường (định dạng mà hầu hết client mong đợi). Khi tắt, các liên kết được trả về dạng văn bản thuần, mỗi liên kết một dòng.
Hiển thị thông tin sử dụng	subShowInfo	true	Gợi ý: «Hiển thị lưu lượng còn lại và ngày hết hạn sau tên câu hình». Khi bật, tên (remark) của mỗi câu hình được thêm các nhãn lưu lượng còn lại () và thời hạn (ví dụ 5D, 3H); với client đã hết hạn/không khả dụng hiển thị N/A .
Bao gồm Email vào tên	subEmailInRemark	true	Gợi ý: «Bao gồm email của client vào tên hồ sơ đăng ký». Thêm email của client vào remark của hồ sơ.

Mẫu ghi chú (Remark Template)

Tên hiển thị (remark) của mỗi câu hình trong đăng ký được tạo theo **mẫu ghi chú** – trường «**Mẫu ghi chú**» (remarkTemplate) tại tab «**Thông tin**» trong cài đặt đăng ký. Trình tạo mô hình ghi chú cũ (chọn riêng lẻ các phần inbound/email/external proxy và ký tự phân cách) đã bị xóa khỏi giao diện; giờ đây bạn viết định dạng tên tùy ý và chèn biên vào đó. Giá trị mặc định – {{INBOUND}}-{{EMAIL}}| {{TRAFFIC_LEFT}}| {{DAYS_LEFT}}D (tức là theo mặc định tên hồ sơ chứa email của client). Nếu để trống, mô hình ghi chú cũ (không thể câu hình qua giao diện) sẽ được áp dụng.

Các biến được nhóm theo các phần **Client**, **Traffic** và **Time & status** và hiển thị bên cạnh trường dưới dạng chip có thể click {{VAR}} với gợi ý khi di chuột; click sẽ chèn token vào mẫu, có hỗ trợ xem trước trực tiếp. Mỗi biến được thay thế riêng lẻ cho từng client tại thời điểm tạo đăng ký. Cũng cho phép ký pháp rút gọn trong ngoặc đơn ({DATA_LEFT} , {EXPIRE_DATE} , {PROTOCOL} , {TRANSPORT} v.v.) – panel tự chuyển đổi sang định dạng nội bộ {{...}} .

Các biến khả dụng:

- **Nhận dạng client:** {{EMAIL}} (đồng nghĩa – {{USERNAME}}), {{INBOUND}} (remark của chính inbound), {{HOST}} (remark của host), {{ID}} (UUID), {{SHORT_ID}} (8 ký tự đầu của UUID), {{SUB_ID}} , {{COMMENT}} , {{TELEGRAM_ID}} , {{PROTOCOL}} , {{TRANSPORT}} .
- **Lưu lượng:** {{TRAFFIC_USED}} , {{TRAFFIC_LEFT}} , {{TRAFFIC_TOTAL}} (và các biến thể *_BYTES tương ứng tính byte chính xác), {{UP}} , {{DOWN}} , {{USAGE_PERCENTAGE}} .
- **Thời hạn và trạng thái:** {{DAYS_LEFT}} , {{TIME_LEFT}} , {{EXPIRE_DATE}} (YYYY-MM-DD), {{JALALI_EXPIRE_DATE}} (ngày theo lịch jalali), {{EXPIRE_UNIX}} , {{CREATED_UNIX}} , {{RESET_DAYS}} , {{STATUS}} (active / expired / disabled / depleted), {{STATUS_EMOJI}} .
- **Kết nối (Connection):** {{PROTOCOL}} – giao thức (VLESS, VMess, Trojan v.v.), {{TRANSPORT}} – mạng truyền tải (tcp, ws, grpc v.v.), {{SECURITY}} – bảo mật truyền tải (TLS, REALITY, NONE; hiển thị chữ hoa). Cũng như các biến lưu lượng và thời hạn, ba biến này chỉ hoạt động trong nội dung đăng ký và được tự động loại bỏ khỏi remark trên các liên kết hiển thị trong panel (QR/«Thông tin») và trên trang thông tin đăng ký.

Mẫu có thể được chia thành các phân đoạn bằng dấu gạch đứng `|`. Phân đoạn trong đó biến cho giá trị «không giới hạn» (∞) – ví dụ `{{TRAFFIC_LEFT}}` hoặc `{{DAYS_LEFT}}` với client không có giới hạn – sẽ tự động bị ẩn. Ngoài ra, khối lưu lượng và thời hạn chỉ hiển thị một lần, trên liên kết đầu tiên của client, để không bị lặp lại trong mỗi cấu hình.

Ví dụ. Mẫu `{{EMAIL}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` với client còn 42 GB và 7 ngày sẽ cho tên dạng `ivan@vpn 42.00GB 7D`, còn với client không giới hạn – chỉ `ivan@vpn` (các phân đoạn có ∞ bị bỏ qua).

Kể từ phiên bản 3.4.2, biến `{{EMAIL}}` (và đồng nghĩa `{{USERNAME}}`) chỉ hiển thị trên liên kết **đầu tiên** của client trong nội dung đăng ký – giống như khối lưu lượng/thời hạn còn lại; trên các liên kết khác của cùng client nó bị bỏ qua.

Trên các liên kết hiển thị trong panel (mã QR và cửa sổ «Thông tin» trên trang «Clients») và trên trang thông tin đăng ký, email của client có mặt trong tên hồ sơ: dạng «inbound-host-email» khi có host hoặc «inbound-email» khi không có host. Thông tin lưu lượng và thời hạn (cũng như các biến nhóm «Kết nối») không được điền vào các tên hiển thị này – chúng chỉ hoạt động trong nội dung đăng ký mà ứng dụng VPN nhận được.

Nếu chuỗi thống kê lưu lượng của client bị «mờ côi» sau khi xóa và tạo lại inbound, biến `{{TRAFFIC_USED}}` (và các chỉ số sử dụng khác) không còn hiển thị `0.00B` nữa: panel bổ sung tìm kiếm thống kê theo email của client và điền lưu lượng đã sử dụng chính xác. | Mẫu ghi chú | `remarkTemplate | {{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` | Mẫu tùy do người dùng định nghĩa cho tên hiển thị (remark) của mỗi cấu hình với việc thay thế biến `{{VAR}}`. Được thay thế riêng lẻ cho mỗi client khi tạo đăng ký. Trình tạo «mô hình ghi chú» cũ (chọn inbound/email/external proxy và ký tự phân cách) đã bị xóa khỏi giao diện và chỉ được dùng như phương án dự phòng nếu để trống trường này. Chi tiết – xem «Mẫu ghi chú (Remark Template)» bên dưới. |

Metadata hồ sơ (response headers)

Các chuỗi này được truyền đến client trong các HTTP header của phản hồi và hiển thị trong ứng dụng VPN dưới dạng metadata hồ sơ. Tất cả đều trống theo mặc định.

Trường (UI)	Khóa	Header	Mô tả
Tiêu đề đăng ký	<code>subTitle</code>	<code>Profile-Title</code> (trong Base64)	«Tên đăng ký mà client thấy trong ứng dụng VPN». Đối với Clash còn được dùng làm tên hồ sơ được nhập qua <code>Content-Disposition</code> .
URL hỗ trợ	<code>subSupportUrl</code>	<code>Support-Url</code>	«Liên kết hỗ trợ kỹ thuật hiển thị trong ứng dụng VPN».
URL hồ sơ	<code>subProfileUrl</code>	<code>Profile-Web-Page-Url</code>	«Liên kết đến trang web của bạn hiển thị trong ứng dụng VPN». Nếu không đặt, URL thực tế của yêu cầu đăng ký sẽ được dùng thay.
Thông báo	<code>subAnnounce</code>	<code>Announce</code> (trong Base64)	«Nội dung thông báo hiển thị trong ứng dụng VPN». Từ 3.5.0, nội dung này còn được hiển thị dưới dạng banner thông tin ở đầu trang thông tin đăng ký (khi không trống), còn các mẫu tùy chỉnh có sẵn biến <code>announce</code> .

Ngoài ra, mỗi phản hồi còn có header `Subscription-Userinfo` với dữ liệu lưu lượng tổng hợp của client: `upload`, `download`, `total` và `expire` (thời điểm hết hạn tính bằng giây). Qua đó client hiển thị lưu lượng còn lại và thời hạn.

Định tuyến (chỉ dành cho client Happ)

Trường (UI)	Khóa	Mặc định	Mô tả
Bật định tuyến	<code>subEnableRouting</code>	<code>false</code>	«Cài đặt toàn cục để bật định tuyến trong ứng dụng VPN client. (Chỉ dành cho Happ)». Được truyền trong header <code>Routing-Enable</code> .
Quy tắc định tuyến	<code>subRoutingRules</code>	trống	«Quy tắc định tuyến toàn cục cho ứng dụng VPN client. (Chỉ dành cho Happ)». Được truyền trong header <code>Routing</code> .

| Ẩn cài đặt server | `subHideSettings` | `false` | «Ẩn cài đặt server trong đăng ký (chỉ dành cho Happ)». Khi bật, client Happ ẩn khả năng xem và thay đổi thông số server. Tùy chọn chỉ có tác dụng với client Happ. |

Định tuyến Incy (chỉ dành cho client Incy)

Đối với ứng dụng VPN **Incy**, trong cài đặt đăng ký có tab riêng **«Incy»** với hai trường: công tắc **«Bật định tuyến»** (`subIncyEnableRouting`, mặc định tắt) và trường văn bản **«Quy tắc định tuyến»** (`subIncyRoutingRules`) định dạng `incy://routing/onadd/<base64>`. Khi định tuyến được bật và trường được điền, chuỗi này được thêm vào như một dòng riêng trong nội dung đăng ký (định dạng raw) – do đó hồ sơ định tuyến được giao đến client Incy mà không xung đột với header `Routing` của client Happ. Cài đặt chỉ có tác dụng với client Incy.

URI proxy ngược

Trường (UI)	Khóa	Mặc định	Mô tả
URI proxy ngược	<code>subURI</code>	trống	«Thay đổi URI cơ sở của URL đăng ký để sử dụng đằng sau các proxy server».

Nếu trường trống, panel tự tạo địa chỉ cơ sở của liên kết từ tên miền và cổng đăng ký (có tính đến TLS). Nếu đăng ký được phân phối qua proxy ngược/CDN bên ngoài trên tên miền hoặc đường dẫn khác, URI cơ sở cuối cùng được đặt vào trường này và tất cả liên kết sẽ được xây dựng từ đó. Các trường riêng lẻ tương tự cũng có cho JSON (`subJsonURI`) và Clash (`subClashURI`).

Nếu chỉ đặt `subURI` chung mà để trống các trường riêng lẻ cho JSON và Clash, các liên kết của các định dạng này trên trang đăng ký sẽ kế thừa scheme và host từ `subURI` (chứ không phải cổng sub-server và `http`) – nhờ vậy chúng khớp với địa chỉ proxy ngược.

Ví dụ: đăng ký đằng sau proxy ngược. Chính đăng ký lắng nghe trên `2096`, nhưng từ bên ngoài có thể truy cập qua nginx/CDN tại `https://cfg.example.com/u/`. Để các liên kết trong phản hồi được xây dựng từ địa chỉ bên ngoài chứ không phải `domain:2096` nội bộ, URI cơ sở cuối cùng được đặt vào trường «Reverse proxy URI»:

Reverse proxy URI: `https://cfg.example.com/u`

Khi đó liên kết cuối cùng sẽ có dạng `https://cfg.example.com/u/ivan2025`. Với các định dạng JSON và Clash nếu cần thiết thì điền các trường riêng `subJsonURI` và `subClashURI` theo cách tương tự.

10.3. Định dạng đầu ra

Đăng ký có thể được phục vụ ở ba định dạng độc lập, mỗi định dạng có endpoint riêng có thể bật/tắt độc lập.

Địa chỉ server và node trong đầu ra

Địa chỉ server trong các liên kết đăng ký được điền theo cùng chiến lược địa chỉ trong liên kết như các liên kết thông thường và mã QR trong panel: «listen» – địa chỉ liên kết có thể định tuyến, «custom» – địa chỉ tùy chỉnh do người dùng đặt (`shareAddr`), «node» (mặc định) – địa chỉ của node. Với inbound không có chiến lược được đặt rõ ràng, đầu ra đăng ký không thay đổi. Điều này cho phép inbound của node được gắn với IP công khai cụ thể trả về địa chỉ có thể tiếp cận cho client. Chiến lược được áp dụng cho các định dạng raw, JSON và Clash.

Tên node không được thêm vào tên (remark) hồ sơ trong đăng ký: ứng dụng client chỉ hiển thị remark của inbound do quản trị viên đặt, không có hậu tố nội bộ dạng `@tên-node`. Để phân biệt các mục trùng tên trong đăng ký đa node, hãy đặt cho chúng các remark khác nhau theo cách thủ công hoặc sử dụng Hosts được quản lý với Remark riêng.

Nếu do mất đồng bộ giữa các node mà cùng một client xuất hiện hai lần trong inbound JSON nội bộ, đầu ra đăng ký tự động loại bỏ các bản sao đó theo email trong cả ba định dạng, vì vậy các hồ sơ trùng lặp không xuất hiện trong đầu ra.

Hosts được quản lý (Hosts)

Phần **Hosts** (mục menu bên; trang tổng hợp với số lượng Total/Enabled/Disabled và danh sách) đặt các ghi đề địa chỉ cho các liên kết đăng ký. Cho mỗi inbound có thể thêm một hoặc nhiều **host** – endpoint được điền vào các liên kết subscription giao cho client **thay thế địa chỉ, cổng và thông số TLS của chính inbound đó**. Điều này tiện lợi để phân phối lưu lượng qua CDN hoặc relay mà không thay đổi bản thân inbound.

Kể từ phiên bản 3.5.0, host là một «nhóm»: một bản ghi bao phủ **nhiều inbound** và **nhiều địa chỉ** cùng lúc. Các thao tác của danh sách (bật/tắt/xóa/sắp xếp lại) và API làm việc với các nhóm (`groupId` dạng chuỗi); xuất hiện endpoint hàng loạt `POST /panel/api/hosts/bulk/add ({"inboundIds": [...], "hosts": [...], ...})`, còn `list / get / update / del / setEnable` thao tác trên các đối tượng nhóm.

Mỗi host (nhóm) có:

- **Remark** và mô tả (Description), liên kết đến **Inbounds** (chọn nhiều với tìm kiếm; tối thiểu một), công tắc **Enable** và gán cho các node (**Nodes**).
- **Address** – từ 3.5.0 đây là **danh sách địa chỉ** (nhập dạng thẻ; dấu phân tách – dấu phẩy, chấm phẩy, dấu cách; placeholder `cdn.example.com`, `cdn2.example.com:443`). Mỗi mục nhập có thể có `:cổng` nhưng riêng (kể cả IPv6 trong ngoặc vuông `[::1]:443`); gợi ý thả xuống để xuất các địa chỉ đã được các host khác sử dụng. Danh sách trống – kế thừa địa chỉ của chính inbound (trong danh sách đây là nhân màu

cam **Inherits**). **Port** (`0` — kế thừa cổng inbound) đóng vai trò cổng mặc định cho các mục nhập không có cổng nhúng; **Tags** (chỉ được tính trong đăng ký RAW).

- Tab **Security** — `same` / `tls` / `none` / `reality` với SN1, fingerprint, ALPN, pinned-cert, `allowInsecure` và ECH.
- Tab **Advanced** — Host header, Path, tuyến đường VLESS, Mux, Sockopt, Final Mask và loại trừ host khỏi các định dạng đăng ký riêng lẻ (`raw` / `json` / `clash`). Từ 3.5.0, **Final Mask của host cũng được đưa vào các liên kết raw** (`fm=` ; trước đây — chỉ vào JSON/Clash): các mặt nạ TCP/UDP của host được thêm vào các mặt nạ của inbound, còn các tham số QUIC của host chỉ được lấy khi inbound không có chúng. Công tắc **Allow insecure** nay cũng có hiệu lực cho **Hysteria/Hysteria2** (`insecure=1` trong liên kết, `skip-cert-verify: true` trong Clash).
- Tab **Clash (mihomo)** — phiên bản IP, Mihomo X25519, xáo trộn host (Shuffle host).

Trong danh sách, cột **Endpoint** hiển thị các địa chỉ dưới dạng chip (địa chỉ đầu tiên hiển thị, số còn lại — trong popover «+N»), còn cột **Inbounds** — các chip inbound được tô màu theo giao thức. Từ 3.5.0, các host được sắp xếp **toàn cục** (theo thứ tự sắp xếp, sau đó theo remark), chứ không trong phạm vi inbound; bật, tắt và xóa hàng loạt vẫn được giữ. Hosts được quản lý thay thế mảng External Proxy trước đây.

VLESS route. Kể từ phiên bản 3.4.2, đây là một số duy nhất `0-65535` (chứ không phải danh sách cổng; gợi ý — «một giá trị VLESS route (0-65535) được nhúng vào UUID, ví dụ 443; để trống — không có nó», placeholder `443`). Giá trị đã chỉ định thực sự được «nhúng» vào UUID của mỗi đăng ký được tạo (`raw` / `JSON` / `Clash`): Xray đọc các byte 6-7 của UUID và che chúng trước khi xác thực, do đó client vẫn khớp. Giá trị trống hoặc không hợp lệ không làm thay đổi UUID.

Liên kết thông thường (SUB) — Base64 / văn bản thuần

Định dạng cơ bản, endpoint `subPath` (mặc định `/sub/`). Luôn bật (khi đăng ký tổng thể được bật). Trả về danh sách liên kết Xray (`vless://` , `vmess://` , `trojan://` , `ss://` v.v.) — mỗi liên kết một dòng. Khi bật tùy chọn «Mã hóa» (`subEncrypt`), toàn bộ danh sách được mã hóa Base64; khi tắt — trả về dạng văn bản thuần. Định dạng này được hầu hết các client hiểu (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ v.v.).

Ví dụ: nội dung phản hồi khi tắt «Mã hóa». Với `subEncrypt = false` , endpoint `/sub/` trả về văn bản thuần — mỗi liên kết một dòng:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

Với `subEncrypt = true` (mặc định), cùng danh sách đó được mã hóa Base64 toàn bộ và trả về một chuỗi duy nhất — đây chính xác là dạng mà hầu hết client mong đợi.

Đăng ký JSON (sing-box và tương thích)

Endpoint `subJsonPath` (mặc định `/json/`), được bật bằng checkbox riêng biệt.

Trường (UI)	Khóa	Mặc định	Mô tả
Đăng ký JSON	<code>subJsonEnable</code>	<code>false</code>	«Bật/tắt JSON endpoint đăng ký độc lập.».

Trả về cấu hình JSON đầy đủ (định dạng tương thích với sing-box và các client dẫn xuất – Podkop, OpenWRT sing-box, Karing, NekoBox). Từ 3.5.0, các client của **WireGuard-inbound gốc** cũng được đưa vào đăng ký JSON (`secretKey` , `address` , `peers[]` với `publicKey` / `endpoint` / `preSharedKey` / `keepAlive` / `allowedIPs` , `mtu`) – trước đây chúng bị bỏ qua một cách âm thầm. Đối với định dạng này có thêm các thông số (tab `subFormats`):

- **Mux** (`subJsonMux` , mặc định trống) – cài đặt JSON đa luồng (Mux) được nhúng vào outbound của mỗi luồng đăng ký JSON. «Truyền nhiều luồng dữ liệu độc lập trong một kết nối.».
- **Final Mask** (`subJsonFinalMask` , mặc định trống) – «Mask finalmask xray (TCP/UDP) và cài đặt QUIC được thêm vào mỗi luồng đăng ký JSON. Yêu cầu phiên bản xray mới nhất trên client.». Được cấu hình qua các trường con: «Gói» (`packets`), «Độ dài» (`length`), «Khoảng» (`interval`), «Phân tách tối đa» (`maxSplit`), «Nhiều» (`noises` : «Loại»/ `type` , «Gói»/ `packet` , «Độ trễ (ms)»/ `delayMs` , «Áp dụng cho»/ `applyTo` , nút «+ Nhiều», cũng như «Đồng thời» (`concurrency`), «Đồng thời xudp» (`xudpConcurrency`) và «xudp UDP 443» (`xudpUdp443`).
- **Quy tắc định tuyến** (`subJsonRules` , mặc định trống) – các quy tắc toàn cục được thêm vào cấu hình JSON.

Đăng ký Clash / Mihomo (YAML)

Endpoint `subClashPath` (mặc định `/clash/`), được bật bằng checkbox riêng biệt.

Trường (UI)	Khóa	Mặc định	Mô tả
Đăng ký Clash / Mihomo	<code>subClashEnable</code>	<code>false</code>	Bật tạo cấu hình YAML cho các client Clash và Mihomo.
Bật định tuyến	<code>subClashEnableRouting</code>	<code>false</code>	«Thêm các quy tắc định tuyến Clash/Mihomo toàn cục vào các đăng ký YAML được tạo.».
Quy tắc định tuyến toàn cục	<code>subClashRules</code>	trống	«Quy tắc Clash/Mihomo được thêm vào đầu mỗi đăng ký YAML trước MATCH,PROXY.».

Phản hồi được trả về với kiểu `application/yaml; charset=utf-8` . Nếu «Tiêu đề đăng ký» (`subTitle`) được đặt, nó cũng được truyền trong header `Content-Disposition` (`attachment; filename*=UTF-8'<title>`) để client Clash đặt tên hồ sơ được nhập theo tên này.

Định dạng các liên kết và YAML được tạo ra được duy trì ở trạng thái phù hợp với các client hiện đại: Shadowsocks-2022 (SS2022) không còn mã hóa userinfo trong Base64; các liên kết Shadowsocks với obfuscation http được trả về ở định dạng SIP002 với plugin `obfs-local` ; cho đăng ký Clash/Mihomo đã triển khai bộ trường XHTTP đầy đủ. Từ 3.5.0, các client của **WireGuard-inbound gốc** cũng được đưa vào đăng ký Clash/Mihomo (các trường `private-key` , `public-key` , `pre-shared-key` , `persistent-keepalive` , `ip` / `ipv6` , `mtu` , `dns`) – trước đây chúng bị bỏ qua một cách âm thầm. Điều này không yêu cầu cài đặt riêng – các liên kết chỉ đơn giản được client nhận dạng chính xác hơn.

Lưu ý: phiên bản này hỗ trợ đúng ba định dạng – liên kết thông thường (Base64/văn bản), JSON (tương thích sing-box) và Clash/Mihomo (YAML). Không có định dạng Outline riêng biệt trong server đăng ký.

10.4. Trang thông tin đăng ký và mã QR

Nếu mở **bất kỳ liên kết nào trong ba liên kết đăng ký** – raw,JSON hoặc Clash – trong trình duyệt (hoặc thêm tham số `?html=1` hay `?view=html` vào URL một cách rõ ràng, hoặc gửi header `Accept: text/html`), server thay vì phản hồi «thô» sẽ trả về **trang thông tin đăng ký** trực quan («Thông tin đăng ký»). Trước 3.5.0, chỉ liên kết chính `/sub/` hoạt động như vậy, còn `/json/` và `/clash/` trả về JSON/YAML thô cho trình duyệt. Các ứng dụng VPN vẫn nhận được phản hồi máy tính vì chúng không yêu cầu HTML.

Trang (ứng dụng một trang được xây dựng bằng Vite) hiển thị:

- **Thông tin đăng ký** (khối Descriptions):
 - «ID đăng ký» – giá trị `subId` ;
 - «Trạng thái» – «Hoạt động», «Không hoạt động» hoặc «Không giới hạn». Trạng thái «không hoạt động» được đặt nếu client bị tắt, đã hết giới hạn lưu lượng hoặc hết hạn;
 - «Tải xuống» và «Tải lên» – lượng lưu lượng;
 - «Giới hạn tổng» – giới hạn lưu lượng hoặc ∞ nếu không giới hạn;
 - «Thời hạn» – ngày hết hạn hoặc «Vĩnh viễn»;
 - lưu lượng còn lại và thời điểm online cuối cùng.
 - Ngày được hiển thị theo lịch Gregorian hoặc Jalali tùy thuộc vào cài đặt «Calendar Type» của panel (`datepicker` , mặc định `gregorian`).
- **Liên kết đăng ký**: cho mỗi định dạng được bật – một dòng riêng với thẻ màu (xanh lá **SUB**, tím **JSON**, vàng **CLASH**), nút sao chép và nút **mã QR** (cửa sổ bật lên, kích thước 240 px). Dòng với JSON và CLASH chỉ xuất hiện khi định dạng tương ứng được bật trong cài đặt.
- **Liên kết riêng lẻ** («Sao chép liên kết»): danh sách đầy đủ các cấu hình riêng lẻ có trong đăng ký, mỗi cái với thẻ giao thức, nút sao chép và mã QR (với các liên kết post-quantum mã QR không được tạo).
- **Nút «Sao chép tất cả cấu hình»** (phía trên danh sách liên kết riêng lẻ): một lần nhấn sao chép tất cả các liên kết cấu hình vào clipboard (mỗi liên kết trên một dòng mới) mà không cần sao chép từng cái; sau khi hoàn thành hiển thị thông báo «Đã sao chép tất cả cấu hình».
- **Nút nhập nhanh vào ứng dụng** (menu thả xuống theo nền tảng): cho Android – `v2box`, `v2rayNG` (deep-link `v2rayng://install-config?url=...`), `Sing-box`, `V2RayTun`, `NPV Tunnel`, `Happ` (`happ://add/...`), `Incy` (`incy://add/...`); cho iOS – `Shadowrocket` (qua tham số `flag=shadowrocket`), `v2box` (`v2box://install-sub?url=...&name=...`), `Streisand` (`streisand://import/...`), `V2RayTun`, `NPV Tunnel`, `Happ`, `Incy`. Các nút này hoặc mở deep-link của ứng dụng tương ứng với địa chỉ đăng ký đã được điền sẵn, hoặc sao chép liên kết vào clipboard.

Trang thông tin được trả về với các header cấm cache (`Cache-Control: no-cache`) để client luôn thấy dữ liệu mới nhất về lưu lượng và thời hạn.

10.5. Mẫu tùy chỉnh trang đăng ký

Kể từ 3.3.0 có thể thay thế trang landing đăng ký tiêu chuẩn bằng mẫu HTML tùy chỉnh. Theo mặc định, địa chỉ đăng ký trả về trang tích hợp sẵn, nhưng nếu chỉ định thư mục với mẫu của bạn, panel sẽ render nó và điền vào đó dữ liệu hiện tại của client (lưu lượng, thời hạn, liên kết v.v.).

Quan trọng: panel **không cung cấp** sẵn các mẫu – chủ đề của bạn cần được tạo từ đầu. Hướng dẫn tạo và danh sách các biến có sẵn nằm trong [docs/custom-subscription-templates.md](#).

Nơi bật

Thư mục chủ đề được đặt trong cài đặt panel:

Cài đặt → **Đăng ký** → phần «**Thông tin đăng ký**», trường «**Thư mục chủ đề đăng ký**» (`subThemeDir`).

Mô tả trường trong giao diện: «Đường dẫn tuyệt đối đến thư mục chứa mẫu tùy chỉnh (index.html/sub.html) cho trang đăng ký (ví dụ: /etc/3x-ui/sub_templates/my-theme/). Để trống để dùng trang mặc định.»

Bên cạnh trong cùng phần có các cài đặt liên quan, giá trị của chúng có thể truy cập trong mẫu:

Trong mô tả trường «Thư mục chủ đề đăng ký» có liên kết «**Hướng dẫn mẫu** ↗» đến tài liệu tạo mẫu giao diện trang đăng ký tùy chỉnh.

- «**Tiêu đề đăng ký**» (`subTitle`) – tên mà client thấy;
- «**URL hỗ trợ**» (`subSupportUrl`) – liên kết hỗ trợ kỹ thuật.

Thông số cài đặt

Thông số	Giá trị mặc định	Mục đích
<code>subThemeDir</code>	"" (trống)	Đường dẫn tuyệt đối đến thư mục chứa mẫu HTML của bạn. Trống = trang mặc định tích hợp sẵn.

Cách đặt mẫu tùy chỉnh

1. Tạo thư mục chủ đề trên server (bất kỳ đâu), ví dụ `/etc/3x-ui/sub_templates/my-theme/`.
2. Đặt vào đó file HTML với tên `index.html` hoặc `sub.html`.

Ví dụ: đường dẫn đến chủ đề. Bố cục cuối cùng trên server và giá trị trường trong cài đặt:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html          (hoặc sub.html – nó có ưu tiên)
```

Cài đặt → Đăng ký → «Thư mục chủ đề đăng ký»:
`/etc/3x-ui/sub_templates/my-theme/`

Đường dẫn phải là **tuyệt đối** (bắt đầu bằng `/`). Nếu thư mục không có `index.html` hay `sub.html`, panel sẽ trả về trang tích hợp sẵn. 3. Trong panel mở **Cài đặt** → **Đăng ký** và nhập đường dẫn **tuyệt đối** đến thư mục này vào trường «Thư mục chủ đề đăng ký». 4. Lưu cài đặt.

Hành vi lựa chọn file và render:

- Nếu thư mục có `sub.html`, chính nó được sử dụng; nếu không thì lấy `index.html`. Tức là `sub.html` có ưu tiên hơn `index.html`.
- Mẫu được render bằng engine Go tiêu chuẩn `html/template`.
- Mẫu đã parse được **cache** và chỉ đọc lại từ đĩa khi thời gian chỉnh sửa file thay đổi. Vì vậy các chỉnh sửa mẫu được nhận mà không cần khởi động lại panel, nhưng không tốn chi phí đọc/parse cho mỗi yêu cầu.
- Phản hồi được tạo đầy đủ vào buffer và chỉ sau đó mới được gửi đến client: nếu mẫu gặp lỗi trong quá trình thực thi, trang đã tạo một phần (bị hỏng) sẽ không đến tay người dùng.

Hành vi mặc định và fallback

- Trường trống → trang SPA tích hợp sẵn được trả về (dữ liệu được nhúng vào `window.__SUB_PAGE_DATA__`).
- Đường dẫn không tồn tại hoặc không phải thư mục → dùng trang mặc định.
- Thư mục không có `index.html` hay `sub.html` → ghi cảnh báo «subThemeDir set but no usable template found» vào log, trả về trang mặc định.
- File mẫu có nhưng không parse được → ghi lỗi «custom template parse failed» vào log, trả về trang mặc định.
- Lỗi khi thực thi mẫu → ghi «custom template execution failed» vào log, trả về trang mặc định.

Tức là bất kỳ sự cố nào với mẫu tùy chỉnh đều không «phá vỡ» đăng ký – panel luôn lùi về trang tích hợp sẵn. Tất cả các trang đăng ký (cả tùy chỉnh lẫn tiêu chuẩn) đều được trả về với các header cấm cache (`Cache-Control: no-cache, no-store, must-revalidate`) để client luôn nhận được dữ liệu mới nhất về lưu lượng và thời hạn.

Các biến mẫu khả dụng

Một tập dữ liệu của client đăng ký được truyền vào context của mẫu. Truy cập qua `{{ .tên }}`:

Biến	Kiểu	Mô tả
<code>{{ .sId }}</code>	string	ID đăng ký (UUID).
<code>{{ .enabled }}</code>	bool	Client/đăng ký có được bật không.
<code>{{ .download }}</code>	string	Lượng tải xuống được định dạng (ví dụ «2.5 GB»).
<code>{{ .upload }}</code>	string	Lượng tải lên được định dạng.
<code>{{ .total }}</code>	string	Giới hạn lưu lượng tổng được định dạng.
<code>{{ .used }}</code>	string	Lưu lượng đã dùng được định dạng (download + upload).
<code>{{ .remained }}</code>	string	Lưu lượng còn lại được định dạng.
<code>{{ .expire }}</code>	int64	Thời hạn – Unix-time tính bằng giây (<code>0</code> = vĩnh viễn). Để dùng cho <code>Date</code> trong JS hãy nhân với 1000.
<code>{{ .lastOnline }}</code>	int64	Thời điểm online cuối – Unix-time tính bằng mili giây (<code>0</code> = chưa bao giờ).
<code>{{ .downloadByte }}</code>	int64	Lượng tải xuống tính bằng byte chính xác.

Biên	Kiểu	Mô tả
<code>{{ .uploadByte }}</code>	int64	Lượng tải lên tính bằng byte chính xác.
<code>{{ .totalByte }}</code>	int64	Giới hạn tổng tính bằng byte chính xác.
<code>{{ .subUrl }}</code>	string	URL trang đăng ký.
<code>{{ .subJsonUrl }}</code>	string	URL cấu hình JSON đăng ký.
<code>{{ .subClashUrl }}</code>	string	URL cấu hình Clash/Mihomo.
<code>{{ .subTitle }}</code>	string	Tiêu đề đăng ký từ cài đặt (có thể trống).
<code>{{ .subSupportUrl }}</code>	string	URL hỗ trợ từ cài đặt (có thể trống).
<code>{{ .links }}</code>	[]string	Danh sách chuỗi cấu hình (VMess, VLESS v.v.). Duyệt qua: <code>{{ range .links }} ... {{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Danh sách email thuộc đăng ký.
<code>{{ .datepicker }}</code>	string	Định dạng lịch hiện tại của panel: <code>gregorian</code> hoặc <code>jalali</code> (lấy từ cài đặt «Loại lịch»; nếu trống – <code>gregorian</code>).

Ví dụ tối thiểu về nội dung mẫu sử dụng một số biên:

```
<h1>{{ .subTitle }}</h1>
<p>Đã dùng: {{ .used }} trong tổng {{ .total }} (còn lại {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

Ví dụ: ngày hết hạn từ `expire`. Trường `{{ .expire }}` – là Unix-time tính bằng **giây**, vì vậy để dùng trong JavaScript cần nhân với 1000; giá trị `0` có nghĩa là «không có thời hạn»:

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? 'Không có thời hạn'
    : 'Đến ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Lưu ý: `{{ .lastOnline }}` được tính bằng **mili giây** – không cần nhân với 1000.

11. Xray: định tuyến, outbounds, DNS và phần mở rộng

Mục **«Cài đặt Xray»** là trình chỉnh sửa mẫu cấu hình Xray-core, dựa trên đó bảng điều khiển tạo ra `config.json` cuối cùng để khởi động lõi. Gợi ý của mục này: *«Tập cấu hình Xray được tạo dựa trên mẫu.»* Khác với inbounds (được lưu riêng trong cơ sở dữ liệu và được chèn vào mẫu khi tạo cấu hình), tất cả phần còn lại – nhật ký, định tuyến, outbounds, DNS, chính sách, thống kê – đều được thiết lập tại đây.

Quan trọng: giá trị của mẫu được lưu trong cơ sở dữ liệu dưới khóa `xrayTemplateConfig`. Khi lưu, bảng điều khiển sẽ thực hiện một loạt các chuyển đổi tự động (xem 11.11). Bất kỳ JSON nào không hợp lệ về mặt cú pháp sẽ bị từ chối với lỗi *«xray template config invalid»*.

Vị trí trong menu: «Outbounds» và «Định tuyến»

«Outbounds» và **«Định tuyến» (Routing)** – đây là các mục riêng biệt trong menu bên (ngay dưới «Hosts», phía trên «Cài đặt bảng điều khiển»), mỗi mục có địa chỉ riêng: `/outbound` và `/routing`. Các liên kết trực tiếp đến các trang này và tải lại trang hoạt động như mong đợi. Trong submenu **«Cấu hình Xray»** chỉ còn lại: Cơ bản, Bộ cân bằng tải, DNS và Mẫu nâng cao. Trong phần mô tả bên dưới, các mục 11.3 và 11.4 tương ứng với các trang «Định tuyến» và «Outbounds».

11.1. Cấu trúc trình chỉnh sửa: tab/chế độ

Trình chỉnh sửa cung cấp một số chế độ hiển thị mẫu (bộ lọc theo phần JSON):

Chế độ	Nội dung hiển thị
Cơ bản	Các phần cơ bản (Nhật ký, định tuyến cơ bản, cài đặt chính)
Mẫu nâng cao	Toàn bộ mẫu JSON Xray
Tất cả	Tất cả các phần cùng một lúc

Các nhóm cài đặt logic trong trình chỉnh sửa:

- **Cài đặt chính** (gợi ý: *«Các tham số này mô tả các cài đặt chung»*).
- **Nhật ký** (xem 11.10).
- **Kết nối cơ bản**: chặn và định tuyến trực tiếp.
- **Inbounds** (gợi ý: *«Thay đổi mẫu cấu hình để kết nối các máy khách nhất định»*).
- **Outbounds** (xem 11.4).
- **Bộ cân bằng tải** (xem 11.5).
- **Định tuyến** (gợi ý: *«Thứ tự ưu tiên của mỗi quy tắc rất quan trọng!»*, xem 11.3).
- **DNS / Fake DNS** (xem 11.6).

11.2. Cài đặt chính (General)

Freedom Protocol Strategy

Trường	Nhãn	Mô tả	Mặc định
FreedomStrategy	Cài đặt chiến lược giao thức Freedom	Chiến lược xuất mạng cho outbound trực tiếp (freedom). Gợi ý: «Đặt chiến lược xuất mạng trong giao thức Freedom». Kiểm soát trường domainStrategy bên trong settings của outbound có giao thức freedom.	Trong mẫu tham chiếu, domainStrategy cho freedom-outbound direct bằng AsIs (địa chỉ không được phân giải, được truyền ở dạng ban đầu).

domainStrategy cho freedom (các giá trị Xray-core): AsIs (không phân giải tên miền phía máy chủ), cũng như nhóm UseIP / UseIPv4 / UseIPv6 và các biến thể «bắt buộc» của chúng ForceIP*, buộc máy chủ xuất phân giải tên miền và kết nối theo IP nhận được. Thay đổi thành UseIPv4 nếu máy chủ xuất không có IPv6 hoặc cần bắt buộc chỉ đi qua IPv4.

Freedom Happy Eyeballs (IPv4/IPv6)

Trường	Nhãn	Mô tả
FreedomHappyEyeballs	Freedom Happy Eyeballs (IPv4/IPv6)	Gợi ý: «Kết nối hai ngăn xếp cho outbound trực tiếp (freedom) – hữu ích trên các máy chủ xuất có cả IPv4 và IPv6.» Bật thuật toán Happy Eyeballs (thử đồng thời trên cả hai họ địa chỉ) cho freedom-outbound.

Happy Eyeballs riêng cũng có trong cài đặt dial (sockopt) của từng outbound cụ thể: từ 3.5.0 nó thực sự được bật (trước đây câu hình được tuần tự hóa ở trạng thái tắt và công tắc cứ tự «nhảy về»). «Try delay (ms)» mặc định nay là 250; giá trị 0 được đặt rõ ràng (= tắt) sẽ được giữ; cũng có sẵn Prioritize IPv6, interleave (1) và maxConcurrentTry (4). | try delay | (gợi ý) | «Mili giây trước khi thử họ địa chỉ khác. 150–250 ms là điểm khởi đầu tốt.» Độ trễ trước khi chuyển sang họ địa chỉ thay thế. Phạm vi khuyến nghị – 150–250 ms. |

Overall Routing Strategy

Trường	Nhãn	Mô tả	Mặc định
RoutingStrategy	Cài đặt định tuyến tên miền	Chiến lược phân giải DNS tổng thể cho định tuyến. Gợi ý: «Đặt chiến lược định tuyến phân giải DNS tổng thể». Kiểm soát trường routing.domainStrategy.	Trong mẫu tham chiếu, routing.domainStrategy = AsIs.

routing.domainStrategy xác định cách các quy tắc định tuyến IP được so khớp với các yêu cầu tên miền: AsIs (chỉ các quy tắc tên miền, không phân giải), IPIfNonMatch (nếu tên miền không khớp với quy tắc – phân giải và kiểm tra các quy tắc IP), IPOnDemand (phân giải ngay khi gặp quy tắc IP). Để các quy tắc IP (ví dụ: geoip:*) hoạt động với yêu cầu tên miền, thường cần IPIfNonMatch.

Outbound Test URL

Trường	Nhãn	Mô tả	Mặc định
outboundTestUrl	URL kiểm tra outbound	URL để kiểm tra kết nối khi kiểm tra outbound. Gợi ý: «URL để kiểm tra kết nối của outbound». Được lưu riêng khỏi mẫu, dưới khóa xrayOutboundTestUrl.	https://www.google.com/generate_204

Giá trị này được làm sạch. Khi kiểm tra outbound thực sự, nó được xác minh thêm là URL công khai – đây là biện pháp bảo vệ chống SSRF: người dùng không thể truyền vào một URL tùy ý (kể cả URL nội bộ) thông qua máy khách, URL kiểm tra luôn được lấy từ cài đặt phía máy chủ. Giá trị trống khi lưu/kiểm tra sẽ được thay bằng generate_204 mặc định.

Default Outbound (từ 3.5.0)

Mục đầu tiên của tab «**Định tuyến cơ bản**» – bộ chọn **Default Outbound**: «Lưu lượng không khớp với bất kỳ quy tắc định tuyến nào sẽ sử dụng outbound này (Xray lấy outbound đầu tiên trong danh sách)». Trong danh sách – direct, blocked và tất cả các thẻ hiện có; mặc định direct. Việc chọn sẽ **di chuyển outbound được chỉ định lên vị trí đầu tiên** của mẫu; các direct / blocked còn thiếu được tạo ngay khi cần, còn khi chuyển đổi chúng không bị xóa.

Block BitTorrent

Trường	Nhãn	Mô tả
Torrent	Chặn BitTorrent	Thêm vào routing.rules một quy tắc gửi lưu lượng với protocol: ["bittorrent"] đến outbound blocked. Trong mẫu tham chiếu, quy tắc này có mặt theo mặc định.

Giới hạn kết nối (Connection Limits)

Gợi ý: «*Chính sách cấp kết nối cho người dùng cấp 0. Để trống để sử dụng giá trị mặc định của Xray.*» Các tham số này được ghi vào policy.levels.0.

Trường	Nhãn	Mô tả	Mặc định
connIdle	Thời gian chờ không hoạt động (giây)	«Đóng kết nối sau khi không hoạt động trong số giây được chỉ định. Giảm giá trị này sẽ giải phóng bộ nhớ và bộ mô tả tệp nhanh hơn trên các máy chủ có tải cao (mặc định trong Xray: 300).»	trống → mặc định Xray 300
bufferSize	Kích thước bộ đệm (KB)	«Kích thước bộ đệm nội bộ mỗi kết nối tính bằng KB. Đặt 0 để giảm thiểu sử dụng bộ nhớ trên các máy chủ có RAM nhỏ (giá trị mặc định của Xray tùy thuộc vào nền tảng).» Placeholder: «auto».	trống → tùy thuộc nền tảng; 0 – giảm thiểu

Ví dụ (policy.levels.0). Các trường từ nhóm này được ghi vào chính sách cấp 0. Trên máy chủ có tải cao và RAM nhỏ, bạn có thể tăng tốc giải phóng tài nguyên như sau:

```
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

Ở đây kết nối được đóng sau 120 giây không hoạt động (thay vì mặc định 300), còn `bufferSize: 0` giảm thiểu mức tiêu thụ bộ nhớ cho bộ đệm. Trường để trống trong biểu mẫu sẽ không được đưa vào JSON – và Xray sẽ áp dụng giá trị mặc định của mình.

11.3. Các quy tắc định tuyến (routing)

Danh sách quy tắc `routing.rules`. **Thứ tự rất quan trọng** («*Thứ tự ưu tiên của mỗi quy tắc rất quan trọng!*»): các quy tắc được đánh giá từ trên xuống dưới, quy tắc khớp đầu tiên sẽ có hiệu lực. Gợi ý: «*Kéo để thay đổi thứ tự*». Các nút điều khiển thứ tự: **Đầu tiên**, **Cuối cùng**, **Di chuyển lên**, **Di chuyển xuống**.

Mỗi quy tắc có `type: "field"`. Các nút: **Tạo quy tắc**, **Chỉnh sửa quy tắc**. Gợi ý cho các trường danh sách: «*Các phần tử được phân tách bằng dấu phẩy*».

Trên trang «**Định tuyến**», các nút «**Nhập quy tắc**» và «**Xuất quy tắc**» được gộp vào menu thả xuống «**thêm**» (more) – cũng như trên trang «**Outbounds**». Nút «**Xuất quy tắc**» không tải xuống tệp ngay, mà mở cửa sổ modal với xem trước JSON và các nút «**Sao chép**» và «**Tải xuống**»: nội dung có thể được xem trước khi lưu. Xuất outbounds trên trang «**Outbounds**» cũng hoạt động tương tự.

Route Tester (bộ kiểm tra tuyến đường)

Trên tab Routing có sub-tab **Route Tester** – nó hỏi Xray đang chạy xem outbound nào sẽ xử lý một kết nối cụ thể, mà không gửi lưu lượng thực sự. Chỉ định tên miền hoặc IP, cổng, mạng (TCP/UDP) và nếu cần, inbound và giao thức bị chặn (`http` / `tls` / `quic` / `bittorrent`), sau đó nhấn **Test Route**. Quyết định được lấy trực tiếp từ bộ máy định tuyến đang hoạt động.

Phản hồi hiển thị outbound đã được chọn, và khi sử dụng bộ cân bằng tải – còn cả thẻ của bộ cân bằng tải. Nếu không có quy tắc nào khớp, bộ kiểm tra sẽ thông báo rằng lưu lượng đi đến outbound mặc định (đầu tiên trong danh sách `outbounds`). Điều này tiện lợi để kiểm tra thứ tự quy tắc trước khi dựa vào chúng.

Bật và tắt từng quy tắc riêng lẻ

Từng quy tắc định tuyến riêng lẻ có thể được **tắt** tạm thời bằng công tắc mà không cần xóa. Trong bảng quy tắc có cột «**Bật**» với công tắc (Switch), và trong biểu mẫu quy tắc có trường «**Bật**» – cũng là công tắc. Quy tắc bị tắt sẽ không được đưa vào câu hình Xray cuối cùng, nhưng vẫn được lưu trong mẫu và có thể bật lại bất kỳ lúc nào.

Quy tắc thống kê dịch vụ (`inboundTag: ["api"]` → `outboundTag: "api"`) không thể tắt – công tắc của nó bị khóa để không làm hỏng bộ đếm lưu lượng của bảng điều khiển (xem 11.11).

Các trường biểu mẫu quy tắc

Trường biểu mẫu	Nhân	Trường JSON	Mô tả
Nguồn	Nguồn	<code>source</code>	Địa chỉ IP/mạng con nguồn. Danh sách phân tách bằng dấu phẩy.
Cổng nguồn	Cổng nguồn	<code>sourcePort</code>	Cổng nguồn.
Đích	Đích	<code>domain + ip + port</code>	Tên miền, IP và cổng đích. Tên miền hỗ trợ tiền tố <code>domain:</code> , <code>full:</code> , <code>regexp:</code> , <code>keyword:</code> , cũng như <code>geosite:*</code> ; IP – <code>geoip:*</code> và CIDR.
Mạng	–	<code>network</code>	<code>tcp</code> , <code>udp</code> hoặc <code>tcp,udp</code> . Từ 3.4.2 được ghi vào cấu hình bằng chữ thường (<code>tcp</code> / <code>udp</code>), còn trong bảng định tuyến hiển thị bằng chữ hoa (<code>TCP</code> / <code>UDP</code>); để trống – mọi mạng.
Giao thức	–	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (xác định qua sniffing).
Người dùng	Người dùng	<code>user</code>	Lọc theo email/định danh người dùng.
Thuộc tính / Giá trị	Thuộc tính / Giá trị	<code>attrs</code>	Thuộc tính tiêu đề HTTP để so khớp.
VLESS route	VLESS route	–	Định tuyến theo trường route cho VLESS.
Thẻ inbounds	Thẻ inbounds	<code>inboundTag</code>	Một hoặc nhiều thẻ inbound mà quy tắc áp dụng (bao gồm cả thẻ tích hợp <code>api</code> và thẻ DNS từ cài đặt DNS). Trong danh sách inbound hiển thị dưới dạng «thẻ (ghi chú)» nếu inbound có ghi chú riêng, nếu không – chỉ là thẻ; trong các quy tắc đã lưu vẫn chỉ lưu thẻ.
Thẻ outbound	Thẻ outbound / Kết nối đi	<code>outboundTag</code>	Nơi định tuyến lưu lượng khớp.
Thẻ bộ cân bằng tải	Thẻ bộ cân bằng tải / Bộ cân bằng tải	<code>balancerTag</code>	Gợi ý: « <i>Định tuyến lưu lượng qua một trong các bộ cân bằng tải đã cấu hình</i> ».

Loại trừ lẫn nhau giữa `outboundTag` và `balancerTag`: «*Không thể sử dụng `balancerTag` và `outboundTag` cùng một lúc. Khi sử dụng đồng thời, chỉ `outboundTag` hoạt động*». Trong một quy tắc, hãy chỉ định thẻ outbound hoặc thẻ bộ cân bằng tải.

Các quy tắc tích hợp của mẫu tham chiếu

Trong `config.json` tiêu chuẩn, phần `routing` chứa ba quy tắc (theo thứ tự này):

1. `inboundTag: ["api"]` → `outboundTag: "api"` – quy tắc dịch vụ cho gRPC-API thống kê bảng điều khiển.

2. `ip: ["geoip:private"]` → `outboundTag: "blocked"` – chặn các dải địa chỉ riêng tư.
3. `protocol: ["bittorrent"]` → `outboundTag: "blocked"` – chặn BitTorrent.

Quy tắc `api` → `api` luôn được tự động đưa lên vị trí 0 khi lưu (xem 11.11), để yêu cầu thông kê không bị «nuốt» bởi quy tắc catch-all phía trên.

Ví dụ quy tắc. Gửi tất cả lưu lượng đến các trang web Nga và mạng riêng tư trực tiếp (bỏ qua proxy), còn lại – đến bộ cân bằng tải. Thứ tự quan trọng: quy tắc «định tuyến trực tiếp» phải đứng trước catch-all. Trong `routing.rules`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

Để các quy tắc IP (`geoip:ru`) cũng hoạt động với các yêu cầu tên miền, thường cần `routing.domainStrategy: "IPIfNonMatch"` ở cấp cao nhất của định tuyến (xem 11.2).

Các nhóm định tuyến được cấu hình sẵn (Kết nối cơ bản)

Ở chế độ «Kết nối cơ bản», bảng điều khiển giúp xây dựng các quy tắc điển hình từ các danh sách có sẵn:

Nhóm	Trường	Gợi ý
Chặn theo giao thức/ trang web	–	«Cấu hình để máy khách không có quyền truy cập vào một số giao thức nhất định»
Chặn theo quốc gia	Địa chỉ IP bị chặn, Tên miền bị chặn	«Các tham số này sẽ chặn lưu lượng dựa trên quốc gia đích.»
Kết nối trực tiếp	Địa chỉ IP trực tiếp, Tên miền trực tiếp	«Kết nối trực tiếp có nghĩa là một số lưu lượng nhất định sẽ không được chuyển hướng qua máy chủ khác.»
Quy tắc IPv4	–	«Các tham số này sẽ cho phép máy khách định tuyến đến các tên miền đích chỉ qua IPv4»
Quy tắc WARP	–	«Các tùy chọn này sẽ định tuyến lưu lượng qua WARP tùy theo đích cụ thể.»
Định tuyến NordVPN	–	«Các tùy chọn này sẽ định tuyến lưu lượng qua NordVPN tùy theo đích cụ thể.»

MTProto-inbound: định tuyến lưu lượng Telegram qua Xray

MTProto-inbound có công tắc «**Route through Xray**» (mặc định tắt) và tùy chọn chọn **Outbound**. Khi bật, bảng điều khiển thêm một cầu nối SOCKS vòng lặp vào cấu hình Xray với thẻ của inbound đó, và mtg định tuyến lưu lượng Telegram qua nó. Sau đó, lưu lượng Telegram đi ra được bộ định tuyến kiểm soát: nó có thể được so khớp với các quy tắc thông thường trên tab Routing theo thẻ inbound hoặc buộc chuyển hướng đến

outbound hoặc bộ cân bằng tải đã chọn thông qua trường **Outbound**. Để trống **Outbound** để các quy tắc định tuyến quyết định.

11.4. Outbounds (kết nối đi)

Danh sách outbounds. Các nút: **Tạo outbound**, **Chỉnh sửa outbound**. Gợi ý: «*Thay đổi mẫu cấu hình để xác định các kết nối đi cho máy chủ này*».

Trong mẫu tham chiếu có hai outbound bắt buộc:

- `protocol: "freedom", tag: "direct"` – đi thẳng ra internet (với `domainStrategy: "AsIs"` và `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole", tag: "blocked"` – «hố đen» cho lưu lượng bị chặn.

Các trường biểu mẫu outbound chung

Trường	Nhãn	Mô tả
Thẻ	Thẻ (gợi ý: « <i>Thẻ duy nhất</i> »)	Định danh duy nhất của outbound. Placeholder: « <i>thẻ-duy-nhất</i> ». Xác thực: « <i>Thẻ là bắt buộc</i> », « <i>Thẻ đã được sử dụng bởi outbound khác</i> ».
Giao thức	—	Loại outbound (xem bên dưới).
Địa chỉ / Cổng	Địa chỉ / Cổng	Đích kết nối. Địa chỉ và cổng là bắt buộc.
Gửi qua	Gửi qua	Địa chỉ IP cục bộ của giao diện đi (<code>sendThrough</code>). Placeholder: « <i>IP cục bộ</i> ».
Target Strategy	Target Strategy	Mới trong 3.5.0. Cách domain đích được phân giải trước khi kết nối. 11 giá trị: <code>AsIs</code> (mặc định, không phân giải), <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> , <code>UseIPv6v4</code> , <code>UseIPv4v6</code> (phân giải với fallback), <code>ForceIP</code> , <code>ForceIPv6v4</code> , <code>ForceIPv6</code> , <code>ForceIPv4v6</code> , <code>ForceIPv4</code> (yêu cầu phân giải thành công). Trống = <code>AsIs</code> (không được ghi vào cấu hình); với <code>freedom</code> , giá trị được đọc với fallback về <code>domainStrategy</code> cũ (khóa legacy vẫn được ghi cho các lỗi cũ).
Dialer proxy (chuỗi)	—	Gợi ý: « <i>Kết nối outbound này qua một outbound khác (theo thẻ) để xây dựng chuỗi proxy. Để trống để kết nối trực tiếp</i> » Placeholder: « <i>Chọn outbound để tạo chuỗi</i> ». Được thực hiện thông qua <code>streamSettings.sockopt.dialerProxy</code> .

Danh sách thả xuống **Dialer Proxy** hiển thị không chỉ các outbound cục bộ mà còn cả các thẻ outbound từ các đăng ký – do đó chuỗi cũng có thể được xây dựng qua một outbound nhận được từ đăng ký. Blackhole-outbound và outbound đang được chỉnh sửa vẫn bị loại khỏi danh sách. Để trống trường này để kết nối trực tiếp.

Các giao thức outbound được hỗ trợ

Các giao thức được biểu mẫu hỗ trợ:

- **freedom** – đi thẳng ra ngoài. Các trường `settings.domainStrategy`, `finalRules` (xem bên dưới), Happy Eyeballs. Không thể kiểm tra («*Outbound has no testable endpoint*»).

- **blackhole** — loại bỏ lưu lượng. Trường **Loại phản hồi**. Không thể kiểm tra.
- **socks**, **http** — danh sách `settings.servers[]` với `address / port`; trường **Mật khẩu xác thực**. Đối với giao thức **http**, bên dưới các trường **Username/Password** có trình chỉnh sửa **Headers** (Tiêu đề) — các cặp khóa/giá trị cho tiêu đề CONNECT được gửi đến proxy HTTP phía trên. Các tiêu đề này được lưu khi mở lại và lưu outbound (trước đây bị mất). Lưu ý: chỉ áp dụng tiêu đề ở cấp cài đặt (`settings.headers`); xray-core bỏ qua tiêu đề ở cấp máy chủ riêng lẻ.
- **vmess** — `settings.vnext[]` (`address / port`).
- **vless** — `settings.address / settings.port`.
- **trojan**, **shadowsocks** — `settings.servers[]`.
- **wireguard** — `settings.peers[]` với `endpoint`, cộng thêm các khóa (xem 11.8).
- **hysteria** — `settings.address / settings.port` (vận chuyển UDP).

Đối với outbound loại **loopback**, có khối **Sniffing** với cùng các tham số như của inbound: bật, **destOverride**, **Metadata Only**, **Route Only** và danh sách **tên miền bị loại trừ**.

Trong mặt nạ **UDP** (FinalMask) cho **Hysteria2**, có các chế độ bổ sung. Mặt nạ **Salamander** có bộ chọn **Mode** với các giá trị **Salamander** và **Gecko**: chế độ Gecko thêm phần đệm ngẫu nhiên vào các gói với các trường **Min/Max** kích thước (`packetSize`, phạm vi 1–2048, mặc định 512–1200) — điều này bảo vệ khỏi việc phân tích dấu vân tay theo độ dài gói. Mặt nạ **Realm** (UDP hole-punching) có thêm khối tùy chọn **TLS Config** với các trường **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS) và công tắc **Allow Insecure**.

Từ 3.5.0 đã thêm loại **mặt nạ TCP** mới — **XMC (Minecraft)** (`xmc`): luồng được ngụy trang thành giao thức Minecraft. Các trường: **Hostname** (tùy chọn; «địa chỉ máy chủ được mô phỏng trong bất tay»), **Username** (danh sách tùy chọn; «tên người chơi được đưa ra cho các trình thăm dò; mặc định lỗi dùng Dream») và **Password** bắt buộc (16 ký tự, tự động tạo với nút ; «mật khẩu obfuscation»). Quan trọng: **tổ hợp TCP-Finalmask với REALITY bị từ chối khi lưu** — nó từng làm Xray-core sập ở kết nối đầu tiên (lỗi: «Finalmask is not supported with REALITY security...»; xem Xray-core#6453). Mặt nạ UDP với REALITY được cho phép; các tổ hợp không hợp lệ đã lưu trước đây được tự động «chữa» (mặt nạ bị loại bỏ) khi dựng cấu hình.

Ví dụ: chuỗi qua SOCKS phía trên. Outbound `upstream` kết nối đến SOCKS5-proxy bên ngoài, còn `chained` gửi lưu lượng của mình qua nó (`dialerProxy`), tạo thành một chuỗi. Trong `outbounds`:

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Bây giờ quy tắc định tuyến với `outboundTag: "chained"` sẽ xuất lưu lượng ra internet qua `upstream`.

Nhập outbound từ liên kết chia sẻ

Outbound có thể được nhập từ liên kết chia sẻ (`vless://` , `vmess://` v.v.). Khi nhập, các cài đặt của bộ ghép kênh **xmux** (XHTTP) được truyền trong khối `extra=` của liên kết cũng được lưu: sau khi nhập, các giá trị của chúng được điền vào biểu mẫu con **XMUX** của outbound đã tạo.

Các trường Mux (ghép kênh)

Số song song tối đa, Số kết nối tối đa, Số lần tái sử dụng tối đa, Số yêu cầu tối đa, Số giây tái sử dụng tối đa, Chu kỳ keep alive. Các tham số này cấu hình hành vi mux/XUDP của outbound.

Sockopts (cài đặt socket)

Nhóm **Sockopts**: Khoảng thời gian keep alive, Mark (fwmark), Giao diện, Chỉ IPv6, Chấp nhận proxy protocol, Proxy protocol, TCP user timeout (ms), TCP keep-alive idle (s). Dialer-proxy của chuỗi cũng được đặt ở đây.

Freedom finalRules (ghi đè chặn IP riêng tư)

Đối với freedom-outbound, có nhóm **Quy tắc cuối cùng**:

Trường	Nhãn	Mô tả
<code>overrideXrayPrivateIp</code>	Ghi đè chặn IP riêng tư mặc định trong Xray	Xóa lệnh cấm tích hợp trong Xray đối với các kết nối đi đến IP riêng tư.
<code>action</code>	Hành động	<code>allow</code> (như trong mẫu tham chiếu: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect) hoặc các giá trị khác.
<code>blockDelay</code>	Độ trễ chặn (ms)	Độ trễ trước khi loại bỏ kết nối.
<code>redirect / fragment</code>	Redirect / Fragment	Hành động chuyển hướng và phân mảnh lưu lượng.

Mặt nạ fragment: Lengths và Delays theo từng đoạn

Trong mặt nạ **fragment** (loại fragment trong FinalMask, cho TCP), các trường đơn Length và Delay được thay thế bằng các danh sách **Lengths** và **Delays**: cho mỗi phân đoạn, bạn có thể chỉ định một phạm vi độ dài riêng (ví dụ `100-200`) và độ trễ tính bằng mili giây (ví dụ `10-20` hoặc `0`). Các dòng danh sách có thể được thêm và xóa; các giá trị đơn đã lưu trước đây được tự động chuyển thành mảng một phần tử.

Các trường biểu mẫu khác

- **UDP over TCP** và **Phiên bản UoT** — cho các giao thức tương tự shadowsocks.
- **Không có tiêu đề gRPC, Kích thước chunk Uplink** — các tham số vận chuyển gRPC.
- Các trường TLS/uTLS: **Xác minh tên peer, Pinned SHA256, Short ID, Vision testpre**, placeholder «tên máy chủ».

Kiểm tra outbound

Các nút: **Kiểm tra**, **Kiểm tra tất cả**. Trạng thái: **Đang kiểm tra kết nối...**, **Kiểm tra thành công**, **Kiểm tra thất bại**, **Không thể kiểm tra kết nối outbound**. Kết quả: **Kết quả kiểm tra**, độ trễ tính bằng mili giây.

Hai chế độ (gợi ý: «TCP: probe nhanh chỉ dial. HTTP: yêu cầu đầy đủ qua xray.»):

- **TCP** (mode=tcp) – dial đơn giản đến host:port , thực hiện song song trên tất cả các điểm cuối, ~timeout 5 giây. Chỉ kiểm tra khả năng tiếp cận TCP, không xác thực giao thức proxy. Với freedom / blackhole /thẻ blocked sẽ trả về «Outbound has no testable endpoint».
- **HTTP** (mode=http hoặc trống) – khởi động một phiên bản Xray tạm thời, thực hiện yêu cầu HTTP thực (URL thăm dò = outboundTestUrl phía máy chủ), đo độ trễ thực. Chính xác nhưng tốn kém: được tuần tự hóa bởi khóa toàn cục («Another outbound test is already running, please wait»). Timeout của một lần thử – 10 giây, cửa sổ chờ kết quả – 15 giây (tăng lên để các outbound khỏe mạnh trên các kênh chậm hoặc có tunnel không bị đánh dấu là «Failed»). Khi thất bại, lý do thực (lỗi DNS, connection refused, hết deadline, lỗi TLS v.v.) được ghi vào nhật ký bảng điều khiển/Xray, mà các thông báo timeout chung trở về đến.
- **Real delay (mới trong 3.5.0)** – chế độ thứ ba của bộ chuyển. Sử dụng cùng phiên bản Xray tạm thời đó, nhưng báo cáo **toàn bộ thời gian của yêu cầu «lạnh»**, bao gồm cả việc thiết lập tunnel – con số gần với con số mà các ứng dụng client hiển thị. Gợi ý của bộ chuyển: «TCP: probe dial nhanh. HTTP: yêu cầu đầy đủ qua xray. Real delay: toàn bộ thời gian, bao gồm thiết lập kết nối». Chế độ **HTTP** thông thường từ 3.5.0 được đo trên kết nối «âm» (keep-alive), vì vậy các con số của nó thấp hơn và gần với độ trễ của một yêu cầu đơn lẻ.

Các giao thức UDP (wireguard , hysteria) và vận chuyển UDP (kcp , quic , hysteria) **luôn** được kiểm tra ở chế độ HTTP, ngay cả khi yêu cầu TCP – dial UDP thuần không phân biệt được điểm cuối «đang hoạt động» với «đã chết». Đối với wireguard trong cấu hình kiểm tra, noKernelTun: true được bắt buộc đặt.

Các cột Egress và Country (từ 3.5.0)

Sau bài kiểm tra **HTTP** hoặc **Real delay** thành công, panel qua chính tuyến tạm thời đó truy vấn metadata Cloudflare trace và hiển thị trong danh sách outbound hai cột: **Egress** – IP đầu ra (IPv4/IPv6, mỗi cái được ẩn sau biểu tượng «con mắt»); khi chưa có bài kiểm tra – dấu gạch ngang với gợi ý «Chạy kiểm tra HTTP để hiển thị IP đầu ra và quốc gia») và **Country** – cờ và tên quốc gia đầu ra; nếu đầu ra đi qua Cloudflare WARP, nhãn WARP màu cam được thêm vào. Kiểm tra TCP không điền các cột này; trên di động, dữ liệu hiển thị trong thẻ outbound.

Kiểm tra hàng loạt và phân tích theo giai đoạn

Kiểm tra và **Kiểm tra tất cả** ở chế độ HTTP khởi động một phiên bản Xray tạm thời chung cho một nhóm outbounds, tạo SOCKS-inbound vòng lặp với quy tắc cho mỗi outbound và gửi song song yêu cầu HTTP thực qua đó; **Kiểm tra tất cả** kiểm tra các outbounds theo từng đợt. **Kiểm tra tất cả** cũng kiểm tra các outbounds

nhận được từ đăng ký (bảng «từ đăng ký», chỉ đọc) – các dòng của chúng cũng được tô màu theo kết quả kiểm tra. Đồng thời, các outbounds `freedom` («direct») và `dns` không được kiểm tra ở bất kỳ chế độ nào (đây không phải là proxy): nút kiểm tra ở chúng không khả dụng, **Kiểm tra tất cả** bỏ qua chúng, và bảo vệ phía máy chủ cấm kiểm tra HTTP của chúng ngay cả khi gọi API trực tiếp. Ngoài thành công/lỗi, popup kết quả hiển thị mã trạng thái HTTP của phản hồi và phân tích thời gian theo giai đoạn: **Proxy connect** (kết nối đến proxy), **TLS via outbound** (TLS qua outbound) và **First byte** (thời gian đến byte đầu tiên) – điều này giúp hiểu ở bước nào xảy ra độ trễ hoặc lỗi.

Thông kê lưu lượng outbounds

Bảng điều khiển duy trì bộ đếm lưu lượng theo thẻ (`up / down / total`). Nút đặt lại sẽ đặt lại bộ đếm cho thẻ cụ thể hoặc cho tất cả (`tag = "-alltags-"`). Các trường **Thông tin tài khoản** và **Trạng thái kết nối outbound** hiển thị tóm tắt.

11.5. Bộ cân bằng tải (Balancers)

Danh sách `routing.balancers` . Các nút: **Tạo bộ cân bằng tải**, **Chỉnh sửa bộ cân bằng tải**.

Trên tab Balancers có các cột trạng thái trực tiếp: **Live Target** hiển thị mục tiêu đang hoạt động hiện tại của bộ cân bằng tải trong Xray đang chạy, còn **Override** cho phép ghi đè thủ công lựa chọn mục tiêu (giá trị **Auto (strategy)** trả về lựa chọn theo chiến lược). Trạng thái được cập nhật bằng nút riêng. Nếu bộ cân bằng tải chưa hoạt động trong Xray đang chạy, bảng điều khiển sẽ đề nghị trước tiên lưu thay đổi hoặc khởi động Xray.

Trường	Nhãn	Mô tả
Thẻ	Thẻ (gợi ý: «Thẻ duy nhất»)	Định danh duy nhất. Placeholder: « <i>thẻ bộ cân bằng tải duy nhất</i> ». Xác thực: «Thẻ là bắt buộc», «Thẻ đã được sử dụng bởi bộ cân bằng tải khác».
Bộ chọn	Bộ chọn	Danh sách thẻ outbound (theo chuỗi con) trong đó bộ cân bằng tải chọn đầu ra. Phải chọn ít nhất một: « <i>Chọn ít nhất một outbound</i> ».
Dự phòng	Dự phòng	Thẻ dự phòng nếu không có bộ chọn nào phù hợp. Từ 3.5.0 có thể chọn cả một bộ cân bằng tải khác (trong danh sách, các tùy chọn như vậy được đánh dấu bằng nhãn xanh dương «Balancer», gợi ý «Chọn một bộ cân bằng tải khác làm fallback»). Xray không hỗ trợ điều này một cách tự nhiên, vì vậy panel tự xây dựng loopback-outbound ẩn <code>_bl_<đích></code> và quy tắc định tuyến (đường đi: bộ cân bằng tải → loopback → máy chủ → bộ cân bằng tải đích → outbound; cảnh báo về «hop» thừa được hiển thị trong biểu mẫu). Có bảo vệ khỏi vòng lặp (lỗi «không thể chọn – sẽ phát sinh phụ thuộc vòng», tùy chọn trong danh sách bị chặn với gợi ý về đường đi của vòng lặp) và khỏi việc xóa bộ cân bằng tải đang được sử dụng («Không thể xóa – đang được dùng làm fallback của: ...»). Tiền tố thẻ <code>_bl_</code> được dành riêng (bị cấm trong các thẻ); các đối tượng <code>_bl_</code> phục vụ nội bộ bị ẩn khỏi danh sách Outbounds/ Routing, còn các cột Fallback/Live Target/Override hiển thị tên bộ cân bằng tải đích, chứ không phải thẻ loopback.
Chiến lược	Chiến lược	Thuật toán lựa chọn (xem bên dưới).

Chiến lược và các tham số quan sát

Chiến lược (`strategy.type`) xác định cách bộ cân bằng tải chọn outbound từ các bộ chọn. Các giá trị Xray-core: `random` (ngẫu nhiên), `roundRobin` (lần lượt), `leastPing` (độ trễ nhỏ nhất theo kết quả observatory), `leastLoad` (tải nhỏ nhất). Đối với `leastLoad` / `leastPing`, sử dụng các tham số từ `strategy.settings`:

Trường	Nhân	Mô tả
<code>expected</code>	Mong đợi	Placeholder: «số nút tối ưu» – số lượng nút hoạt động mục tiêu.
<code>maxRtt</code>	RTT tối đa	Giới hạn trên của RTT chấp nhận được khi chọn ứng viên.
<code>tolerance</code>	Dung sai	Dung sai khi so sánh độ trễ/tải.
<code>baselines</code>	Baselines	Các ngưỡng độ trễ để nhóm các nút.
<code>costs</code>	Costs	Hệ số trọng số (cost) cho các thẻ riêng lẻ.

Ví dụ về chiến lược. Khởi `strategy` nằm bên trong bộ cân bằng tải (trong JSON – cạnh `tag` và `selector`):

```
"strategy": { "type": "random" } // lựa chọn ngẫu nhiên từ các bộ chọn
"strategy": { "type": "roundRobin" } // lần lượt, tuần tự
"strategy": { "type": "leastPing" } // độ trễ nhỏ nhất (cần bộ quan sát)
```

Đối với `leastLoad`, các tham số được đặt trong `settings`:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regexp": false, "match": "proxy-premium", "value": 0.1 },
      { "regexp": true, "match": "^proxy-cheap-.*$", "value": 5 }
    ]
  }
}
```

Cách hoạt động (theo ví dụ). Giả sử bộ quan sát đo được độ trễ cho các đầu ra: `A = 250 ms`, `B = 280 ms`, `C = 700 ms`, `D = 1500 ms`. Với các cài đặt trên, lựa chọn diễn ra như sau:

- maxRTT: "1s"** – các đầu ra có độ trễ trên 1 giây bị loại bỏ: `D` (1500 ms) bị loại. Còn lại `A`, `B`, `C`.
- baselines + expected** – các đầu ra được nhóm theo ngưỡng độ trễ, và ngưỡng **nhỏ nhất** chứa ít nhất `expected` đầu ra được chọn. Ngưỡng `500ms` đã chứa `A` và `B` – tức là 2 (= `expected`), vì vậy nhóm `{ A, B }` được chọn. `C` (700 ms) không vào lựa chọn khi còn đủ các đầu ra nhanh (nó là «dự phòng nóng»).
- tolerance: 0.05** – trong nhóm đã chọn, các đầu ra có độ trễ chênh lệch không quá 5% được coi là tương đương và tải được chia đều giữa chúng. `A` (250) và `B` (280) chênh nhau ~12% (> 5%), vì vậy nếu các yếu tố khác bằng nhau, ưu tiên thuộc về `A` nhanh hơn; nếu chênh lệch trong phạm vi 5% – lưu lượng sẽ đi qua cả `A` và `B`.

4. **costs** – trước khi so sánh, điều chỉnh «chi phí» của các đầu ra riêng lẻ: **value** nhỏ hơn làm cho đầu ra hấp dẫn hơn, lớn hơn – ngược lại. Trong ví dụ, **proxy-premium** nhận **0.1** (trở nên «rẻ hơn» và được chọn nhiều hơn), còn tất cả **proxy-cheap-*** (theo biểu thức chính quy, **regexp: true**) – **5** (trở nên «đắt hơn» và được sử dụng sau cùng). Điều này cho phép ưu tiên nhẹ nhàng các đầu ra mà không loại trừ chúng cứng nhắc.

Kết quả: lưu lượng chủ yếu đi qua **A** (khi độ trễ gần nhau – chia đều với **B**), **C** là dự phòng, **D** bị loại cho đến khi **RTT** của nó giảm xuống dưới **maxRTT**.

Bộ quan sát: **observatory** và **burstobservatory** (đo lường cho **leastPing** / **leastLoad**)

Các chiến lược **leastPing** và **leastLoad** bản thân không đo lường gì – chúng cần dữ liệu về độ trễ và tính khả dụng của từng outbound. Dữ liệu này được thu thập bởi **bộ quan sát** (observatory): nó định kỳ «ping» từng outbound đang được theo dõi và ghi lại thời gian phản hồi và tính khả dụng. Cùng dữ liệu đó được hiển thị trên tab «**Đài quan sát**» (trạng thái **Hoạt động** / **Không khả dụng**, «**Hoạt động lần cuối**», «**Thử lần cuối**»).

Kể từ phiên bản 3.4.2 trang «Bộ cân bằng tải» được chia thành các tab «**Cài đặt bộ cân bằng tải**» (bảng và thêm mới) và «**Observatory**», còn bộ quan sát được chỉnh sửa bằng **biểu mẫu có cấu trúc** (trình chỉnh sửa JSON thô cũ đã bị xóa). Bảng điều khiển tự tạo và xóa các bộ quan sát khi cần: **observatory** thông thường – cho chiến lược **Least Ping**, **burstobservatory** mở rộng – cho **Least Load**, cũng như cho **Random/Round-robin** với **fallbackTag** **đã đặt** (điều này do xay-core yêu cầu; khi xóa **fallbackTag** cuối cùng, bộ quan sát bị loại bỏ). Việc tạo hoặc xóa một bộ quan sát yêu cầu khởi động lại hoàn toàn Xray.

Có hai tùy chọn:

- **observatory** – đơn giản: **subjectSelector** + **probeURL** + **probeInterval**.
- **burstobservatory** – nâng cao, với cấu hình ping chi tiết qua **pingConfig**; tiện lợi cho nhiều đầu ra.

Ví dụ khối **burstobservatory**:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Ý nghĩa các trường:

Trường	Mô tả
subjectSelector	Danh sách tiền tố thẻ outbound để quan sát. Xray lấy tất cả outbound có thẻ bắt đầu bằng các chuỗi được chỉ định. Trong ví dụ, các đầu ra WS-SE... , WS-FR... , WS-PL... được quan sát. Các thẻ này phải khớp với những gì được chọn trong Bộ chọn của bộ cân bằng tải.

Trường	Mô tả
<code>pingConfig.destination</code>	URL được yêu cầu qua từng outbound để đo độ trễ. Sử dụng trang «nhẹ» với phản hồi 204 không có nội dung – ví dụ <code>https://www.google.com/generate_204</code> . Thời gian đến phản hồi chính là độ trễ đo được.
<code>pingConfig.interval</code>	Tần suất ping mỗi outbound. Chuỗi thời gian: <code>"1m"</code> – một lần mỗi phút, cũng có thể <code>"30s"</code> , <code>"5m"</code> v.v. Thường xuyên hơn – dữ liệu mới hơn, nhưng nhiều lưu lượng nền hơn.
<code>pingConfig.connectivity</code>	(tùy chọn) URL kiểm tra kết nối cơ bản của chính máy chủ. Nếu nó không khả dụng – có nghĩa là vấn đề nằm ở mạng của máy chủ, và bộ quan sát không đánh dấu outbound là không khả dụng (bảo vệ khỏi cảnh báo sai khi có sự cố cục bộ). Thường cũng là endpoint với phản hồi 204 .
<code>pingConfig.timeout</code>	Thời gian chờ phản hồi một ping trước khi coi lần thử là thất bại (ví dụ <code>"5s"</code>).
<code>pingConfig.sampling</code>	Số lượng phép đo gần nhất cần lưu và tính trung bình cho mỗi outbound. 2 – tính đến hai ping gần nhất (làm mịn các đột biến ngẫu nhiên).
<code>pingConfig.httpMethod</code>	Mới trong 3.4.2. Phương thức HTTP của các thăm dò: HEAD (mặc định) hoặc GET .

Trên tab «**Observatory**» các tham số tương tự được đặt bằng biểu mẫu (chứ không phải JSON). Đối với `observatory` thông thường, đó là **Watched Outbounds** (`subjectSelector` , chỉ đọc), **Probe URL** (`probeURL` , mặc định `https://www.google.com/generate_204`), **Probe Interval** (`probeInterval` , `1m`) và **Enable Concurrency** (`enableConcurrency` , mặc định bật). Đối với `burstObservatory` – các trường đã liệt kê ở trên cộng thêm trường mới «**Phương thức HTTP**». Nếu bộ cân bằng tải có cả hai bộ quan sát, một công tắc phân đoạn sẽ chuyển giữa các biểu mẫu Observatory và Burst.

Cách kết nối tất cả:

- Trên tab «**Observatory**» cấu hình các tham số của bộ quan sát (nó được tạo tự động theo chiến lược đã chọn; `subjectSelector` đi theo các bộ chọn của bộ cân bằng tải).
- Tạo bộ cân bằng tải: **Chiến lược** = `leastPing` , trong **Bộ chọn** chỉ định các thẻ outbound tương tự (`WS-SE` , `WS-FR` , `WS-PL`).
- Định tuyến lưu lượng đến nó bằng quy tắc định tuyến (trường **Thẻ bộ cân bằng tải**, xem 11.3).
- Khởi động lại Xray. Trên tab «**Đài quan sát**» sẽ xuất hiện các trạng thái đầu ra, và bộ cân bằng tải sẽ bắt đầu chọn cái nhanh nhất trong số những cái đang hoạt động.

Trong một quy tắc không thể đồng thời đặt `balancerTag` và `outboundTag` – chỉ `outboundTag` hoạt động.

Điều gì xảy ra khi xóa outbound hoặc bộ cân bằng tải (từ 3.4.2)

Khi xóa outbound hoặc bộ cân bằng tải, trong cùng thao tác bảng điều khiển **dọn sạch các tham chiếu đến nó trong định tuyến** và hiển thị **xem trước hậu quả** trong hộp thoại xác nhận (tiêu đề «Khi xóa, định tuyến của bạn cũng sẽ được cập nhật:», sau đó với mỗi quy tắc – «đã xóa (không còn đích)» hoặc «giữ lại (nay dùng ...)», và «Bộ cân bằng tải ... đã xóa (không còn mục tiêu)». Nhờ đó tránh được tham chiếu «treo» (`balancerTag` / `outboundTag` / `dialerProxy`) mà trước đây có thể khiến xray-core không khởi động được ở lần khởi động

lại tiếp theo. Xóa bộ cân bằng tải: các quy tắc giữ lại `outboundTag` của chúng, nếu không thì bị xóa. Xóa `outbound`: thẻ của nó được loại khỏi các bộ chọn và `fallbackTag` của các bộ cân bằng tải, `dialerProxy` của các `outbound` khác được xóa, các bộ cân bằng tải trống được xóa cùng với các quy tắc nay không còn đối tượng, các bộ quan sát được dừng lại.

11.6. DNS

Phần `dns`. **Bật: `Bật DNS`** (gợi ý: «*Bật máy chủ DNS tích hợp*»).

Từ 3.5.0, các máy chủ DNS trên **IP riêng tư** (ví dụ AdGuard/Pi-hole của riêng bạn trong cùng mạng docker) hoạt động «ngay lập tức»: panel tự động chèn và duy trì một quy tắc `allow` được quản lý (direct, giới hạn theo cổng của máy chủ DNS) **phía trước** quy tắc chặn `geoip:private` và đồng bộ nó khi `dns.servers` thay đổi. Trước đây, các truy vấn DNS của chính Xray bị quy tắc này âm thầm chặn, thêm ~4 giây độ trễ cho mỗi domain mới; các quy tắc thủ công không còn cần thiết nữa (những quy tắc đã tạo thủ công không bị động đến).

Các tham số DNS chung

Trường	Nhãn	JSON	Mô tả / gợi ý
<code>tag</code>	Tên thẻ DNS	<code>dns.tag</code>	«Thẻ này sẽ có sẵn như thẻ <code>inbound</code> trong các quy tắc định tuyến.» Cho phép định tuyến các yêu cầu DNS qua <code>inboundTag</code> .
<code>clientIp</code>	IP máy khách	<code>dns.clientIp</code>	«Được sử dụng để thông báo cho máy chủ về vị trí IP được chỉ định trong các yêu cầu DNS» (EDNS Client Subnet).
<code>strategy</code>	Chiến lược yêu cầu	<code>dns.queryStrategy</code>	«Chiến lược phân giải tên miền chung». Các giá trị: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .
<code>disableCache</code>	Tắt bộ nhớ đệm	<code>dns.disableCache</code>	«Tắt bộ nhớ đệm DNS».
<code>disableFallback</code>	Tắt DNS dự phòng	<code>dns.disableFallback</code>	«Tắt các yêu cầu DNS dự phòng».
<code>disableFallbackIfMatch</code>	Tắt DNS dự phòng khi khớp	<code>dns.disableFallbackIfMatch</code>	«Tắt các yêu cầu DNS dự phòng khi danh sách tên miền của máy chủ DNS khớp».
<code>enableParallelQuery</code>	Bật truy vấn song song	—	«Bật các truy vấn DNS song song đến nhiều máy chủ để phân giải nhanh hơn».
<code>useSystemHosts</code>	Sử dụng Hosts hệ thống	<code>dns.useSystemHosts</code>	«Sử dụng tệp <code>hosts</code> từ hệ thống đã cài đặt».

Ví dụ khối dns . Các yêu cầu đến tên miền Google được phân giải qua máy chủ DoH của Cloudflare, tất cả những cái khác – qua `1.1.1.1` ; đối với yêu cầu Google, chỉ mong đợi các IP không phải riêng tư. Ở cấp cao nhất của cấu hình:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

Chuỗi máy chủ (`"1.1.1.1"`) không có trường – đây là máy chủ mặc định cho tất cả các tên miền khác. Thẻ `dns-inbound` sau đó có thể được sử dụng như `inboundTag` trong các quy tắc định tuyến để định tuyến chính các yêu cầu DNS qua outbound cần thiết.

Bộ nhớ đệm bản ghi lỗi thời

Trường	Nhãn	Mô tả
<code>serveStale</code>	Sử dụng bản ghi lỗi thời	«Trả về kết quả lỗi thời từ bộ nhớ đệm trong khi cập nhật ở nền».
<code>serveExpiredTTL</code>	TTL lỗi thời	«Thời hạn (giây) của bản ghi bộ nhớ đệm lỗi thời; 0 = vô thời hạn».

Máy chủ DNS (danh sách `dns.servers`)

Các nút: **Tạo DNS**, **Chỉnh sửa DNS**, **Xóa tất cả** (xác nhận: «Tất cả máy chủ DNS sẽ bị xóa khỏi danh sách. Hành động này không thể hoàn tác.»). Mẫu: **Sử dụng mẫu**, cửa sổ **Mẫu DNS**, bao gồm cả cài đặt sẵn **Gia đình**.

Khi nhấn **Chỉnh sửa DNS** trên bản ghi máy chủ DNS (cũng như trên bản ghi Fake DNS), cửa sổ chỉnh sửa điền vào các giá trị đã lưu của máy chủ, chứ không phải giá trị mặc định.

Các trường máy chủ DNS:

Trường	Nhãn	Mô tả
<code>address</code>	—	Địa chỉ DNS (IP, URL DoH, <code>localhost</code> , <code>fakedns</code> v.v.).
<code>domains</code>	Tên miền	Danh sách tên miền sử dụng máy chủ này.
<code>expectIPs</code>	IP mong đợi	Chỉ chấp nhận phản hồi nếu IP nằm trong danh sách.
<code>unexpectIPs</code>	IP không mong đợi	Loại bỏ phản hồi với các IP được chỉ định.
<code>skipFallback</code>	Bỏ qua Fallback	Không sử dụng máy chủ này làm fallback.
<code>finalQuery</code>	Truy vấn cuối cùng	Đánh dấu máy chủ là cuối cùng trong chuỗi.

Trường	Nhãn	Mô tả
timeoutMs	Thời gian chờ (ms)	Timeout yêu cầu đến máy chủ.

Hosts (bản ghi tĩnh)

Nhóm **Hosts** (dns.hosts). Nút **Thêm Host**; trạng thái trống **Host chưa được xác định**. Các trường: tên miền (placeholder: «Tên miền (vd: domain.example.com)») và các giá trị (placeholder: «IP hoặc tên miền – nhập và nhấn Enter»).

Nhật ký DNS

Xem 11.10: cờ **Nhật ký DNS** (dnsLog) trong phần ghi nhật ký.

11.7. Fake DNS

Phần fakedns . Các nút: **Tạo Fake DNS**, **Chỉnh sửa Fake DNS**.

Trường	Nhãn	Mô tả
ipPool	Mạng con pool IP	Phạm vi CIDR từ đó cấp các IP giả (ví dụ 198.18.0.0/15).
poolSize	Kích thước pool	Số lượng địa chỉ giữ trong pool vòng tròn.

Fake DNS được sử dụng kết hợp với sniffing trên inbound: lỗi cấp cho máy khách IP giả, ghi nhớ ánh xạ tên miền[?]IP và khôi phục tên miền khi định tuyến. Để Fake DNS hoạt động, máy chủ DNS với địa chỉ fakedns phải được thêm vào danh sách máy chủ DNS.

Ví dụ: kết hợp Fake DNS + máy chủ DNS. Trước tiên xác định pool địa chỉ giả, sau đó thêm máy chủ DNS fakedns để các yêu cầu tên miền nhận IP từ pool này:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Ngoài ra, trên inbound cần bật sniffing với destOverride: ["fakedns"] , nếu không lỗi sẽ không có nguồn để lấy tên miền thực để khôi phục.

11.8. WireGuard / WARP / NordVPN

Các trường WireGuard (wireguard)

Trường	Nhãn	Mô tả
secretKey	Khóa bí mật	Khóa riêng tư của giao diện cục bộ.
publicKey	Khóa công khai	Khóa công khai của peer.
psk	Khóa chia sẻ	PreShared Key (tùy chọn).
allowedIPs	Địa chỉ IP được phép	Các dải được định tuyến vào tunnel.
endpoint	Điểm cuối	host:port của peer.
domainStrategy	Chiến lược tên miền	Chiến lược phân giải cho WireGuard-outbound.

Cloudflare WARP (warp)

Tích hợp sử dụng API <https://api.cloudflareclient.com/v0a4005> (client-version a-6.30-3596). Các hành động của bộ điều khiển (/xray/warp/:action): config , reg , license , data , del .

Từng bước:

1. **Tạo tài khoản WARP** → reg : bảng điều khiển tạo/nhận khóa riêng tư (privateKey) và công khai (publicKey), đăng ký thiết bị với Cloudflare và lưu access_token , device_id , license_key , private_key (cũng như client_id) trong cài đặt warp .
2. **Khóa giấy phép WARP / WARP+** → license : đặt khóa WARP+ 26 ký tự (placeholder: «Khóa WARP+ 26 ký tự»). Khi có lỗi: «Không thể đặt giấy phép WARP.» Nếu cấu hình chưa được nhận: «Trước tiên hãy nhận cấu hình WARP.»
3. **Thông tin tài khoản:** Tên thiết bị, Mẫu thiết bị, Thiết bị được bật, Loại tài khoản, Vai trò, Dữ liệu WARP+, Hạn mức, Mức sử dụng.
4. **Thêm outbound** – tạo WireGuard-outbound với các khóa và endpoint Cloudflare đã nhận.
5. **Xóa tài khoản** → del : xóa dữ liệu WARP đã lưu.

NordVPN (nord / nordvpn)

Tích hợp sử dụng NordLynx (= WireGuard). Các hành động của bộ điều khiển (/xray/nord/:action): countries , servers , reg , setKey , data , del .

Từng bước:

1. **Token truy cập** → reg : bảng điều khiển yêu cầu thông tin xác thực NordLynx từ api.nordvpn.com và trích xuất nordlynx_private_key . Lưu private_key và token trong cài đặt nord . Thay thế – setKey : nhập **Khóa riêng tư** trực tiếp (không thể để trống).
2. **Quốc gia** → countries tải danh sách quốc gia; **Thành phố** (hoặc **Tất cả thành phố**).
3. **Máy chủ** → servers tải các máy chủ của quốc gia đã chọn (countryId được xác thực là số – bảo vệ khỏi injection). Bộ lọc: chỉ hiển thị các máy chủ có **Tải** > 7%. Nếu không có máy chủ: «Không tìm thấy máy

chủ cho quốc gia đã chọn». Nếu máy chủ không có khóa công khai NordLynx: «Máy chủ đã chọn không báo cáo khóa công khai NordLynx.»

4. Tạo/cập nhật outbound: các toast «Đã thêm outbound NordVPN» / «Đã cập nhật outbound NordVPN».

Ưu tiên IPv4 và userspace TUN

Các WireGuard-outbounds được tạo bởi trình hướng dẫn WARP và NordVPN sử dụng `domainStrategy`: `"ForceIPv4v6"` (ưu tiên IPv4 với dự phòng sang IPv6 trên các máy chủ chỉ có v6) thay vì `ForceIP` – điều này loại bỏ tình trạng «treo» bắt tay trên các máy chủ có IPv6 được cấu hình một nửa, khi bản ghi AAAA của endpoint Cloudflare được chọn. Ngoài ra, `userspace TUN` (`noKernelTun: true`) được bật cho chúng thay vì `kernel TUN`: loại sau yêu cầu quyền và định tuyến fwmark, đồng thời âm thầm thất bại trên nhiều VPS, trong khi kiểm tra kết nối tích hợp của bảng điều khiển luôn kiểm tra qua `userspace TUN` – bây giờ lưu lượng thực và kiểm tra đi cùng một đường. Thay đổi chỉ áp dụng cho các outbounds mới được thêm hoặc đặt lại; các mẫu đã lưu giữ nguyên cài đặt của chúng.

11.9. Reverse-proxy và TUN

Reverse (reverse-proxy)

Phần `reverse` của cấu hình Xray. Trong biểu mẫu outbound có công tắc cho loại **Reverse-proxy**. Các nút: **Tạo reverse-proxy**, **Chỉnh sửa reverse-proxy**.

Trường	Nhãn	Mô tả
Loại	Loại	Bridge hoặc Portal – hai vai trò của reverse-proxy Xray.
Tên miền	Tên miền	Tên miền nhân dịch vụ cho cặp <code>bridge</code> <code>portal</code> .
Thẻ / Kết nối	Thẻ / Kết nối	Các thẻ để liên kết <code>bridge</code> và <code>portal</code> .
Reverse Tag	Thẻ reverse-proxy	Gợi ý: «Thẻ kết nối đi cho reverse-proxy VLESS đơn giản. Để trống để tắt.» Placeholder: «thẻ outbound (trống = tắt)». Thực hiện VLESS reverse đơn giản hóa.

Trong biểu mẫu outbound cũng có các trường của luồng ngược: **Sniffing ngược**, **Workers**, **Dành riêng**, **Khoảng thời gian tải tối thiểu (ms)**, **Kích thước tải tối đa (byte)**.

TUN (tun)

Trường	Nhãn	Mô tả	Mặc định
<code>name</code>	—	«Tên giao diện TUN.»	xray0
<code>mtu</code>	—	«Đơn vị truyền tối đa. Kích thước tối đa của các gói dữ liệu.»	1500
<code>userLevel</code>	Cấp người dùng	«Tất cả kết nối được thiết lập qua luồng inbound này sẽ sử dụng cấp người dùng này.»	0

11.10. Nhật ký và thông kê (Stats, metrics)

Nhật ký (log)

Gợi ý: «Nhật ký có thể làm chậm máy chủ. Chỉ bật các loại nhật ký bạn cần khi cần thiết!» Phần `log` của mẫu tham chiếu: `access: "none", error: "", loglevel: "warning", dnsLog: false, maskAddress: ""`.

Trường	Nhân	JSON	Mô tả	Mặc định
<code>logLevel</code>	Cấp độ nhật ký	<code>loglevel</code>	«Cấp độ nhật ký cho nhật ký lỗi...» Các giá trị: <code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code> , <code>none</code> .	<code>warning</code>
<code>accessLog</code>	Nhật ký truy cập	<code>access</code>	«Đường dẫn đến tệp nhật ký truy cập. Giá trị đặc biệt «none» tắt nhật ký truy cập.»	<code>none</code>
<code>errorLog</code>	Nhật ký lỗi	<code>error</code>	«Đường dẫn đến tệp nhật ký lỗi. Giá trị đặc biệt «none» tắt nhật ký lỗi.»	<code>""</code> (mặc định)
<code>dnsLog</code>	Nhật ký DNS	<code>dnsLog</code>	«Bật nhật ký yêu cầu DNS»	<code>false</code>
<code>maskAddress</code>	Che giấu địa chỉ	<code>maskAddress</code>	«Khi được bật, địa chỉ IP thực sẽ được thay thế bằng địa chỉ che giấu trong nhật ký.»	<code>""</code> (tắt)

Thông kê (stats / policy)

Nhóm **Thông kê**. Bật các bộ đếm trong `policy.system` và `policy.levels`. Trong mẫu tham chiếu: `statsInboundUplink: true, statsInboundDownlink: true, statsOutboundUplink: false, statsOutboundDownlink: false`; cho cấp 0 — `statsUserUplink: true, statsUserDownlink: true`.

Trường	Nhân	Mô tả	Mặc định
<code>statsInboundUplink</code>	Thông kê uplink inbound	«Bật thu thập thông kê cho lưu lượng đi của tất cả các proxy inbound.»	<code>true</code>
<code>statsInboundDownlink</code>	Thông kê downlink inbound	«Bật thu thập thông kê cho lưu lượng đến của tất cả các proxy inbound.»	<code>true</code>
<code>statsOutboundUplink</code>	Thông kê uplink outbound	«Bật thu thập thông kê cho lưu lượng đi của tất cả các proxy outbound.»	<code>false</code>
<code>statsOutboundDownlink</code>	Thông kê downlink outbound	«Bật thu thập thông kê cho lưu lượng đến của tất cả các proxy outbound.»	<code>false</code>

Thông kê theo máy khách và inbounds (uplink/downlink) — là cơ sở để hiển thị lưu lượng trong bảng điều khiển và ở máy khách; không nên tắt nó. Thông kê outbound bị tắt theo mặc định và chỉ cần nếu bạn theo dõi lưu lượng theo thẻ outbound.

Metrics

Trong mẫu tham chiếu có phần `metrics` (`listen: "127.0.0.1:11111"` , `tag: "metrics_out"`) và API `metrics_out` tương ứng. Bảng điều khiển sử dụng listener này để thu thập số liệu và ảnh chụp observatory: nó phân tích `metrics.listen` từ mẫu, truy vấn `/debug/vars` và tổng hợp lịch sử độ trễ theo thẻ. Nếu bạn thay đổi địa chỉ/cổng `metrics.listen` , bảng điều khiển sẽ truy cập địa chỉ mới; xóa phần `metrics` sẽ tắt thu thập biểu đồ observatory.

Kiểm tra outbound ở chế độ HTTP khởi động một phiên bản Xray tạm thời **riêng biệt** với listener `metrics` của chính nó trên cổng ngẫu nhiên – đây không phải là listener tương tự trong cấu hình chính.

11.11. Lưu, khởi động lại và các chuyển đổi tự động

Các nút

Nút	Hành động
Lưu	POST <code>/xray/update</code> : xác thực và lưu mẫu + <code>outboundTestUrl</code> .
Khởi động lại Xray	Tải lại dịch vụ với cấu hình đã lưu. Xác nhận: « <i>Khởi động lại xray?</i> » / « <i>Tải lại dịch vụ xray với cấu hình đã lưu.</i> »

Các toast: thành công – «*Xray đã khởi động lại thành công*», «*Xray đã dừng thành công*»; lỗi – «*Đã xảy ra lỗi khi khởi động lại Xray.*», «*Đã xảy ra lỗi khi dừng Xray.*» Cửa sổ **Đầu ra khởi động lại Xray** hiển thị đầu ra chẩn đoán của lỗi.

Áp dụng thay đổi nóng (không cần khởi động lại hoàn toàn)

Các thay đổi đối với inbounds, outbounds và quy tắc định tuyến được áp dụng «trực tiếp»: khi nhấn **Lưu**, bảng điều khiển tính toán sự khác biệt giữa cấu hình cũ và mới, và chỉ áp dụng các phần đã thay đổi thông qua gRPC-API của Xray (HandlerService/RoutingService), mà không khởi động lại tiến trình. Khởi động lại hoàn toàn chỉ được thực hiện tự động khi các phần không có API tải lại nóng thay đổi (`log` , `dns` , `policy` , `observatory` v.v.). Vì vậy, trên trang Xray không cần nút «Khởi động lại» riêng – **Lưu** tự áp dụng thay đổi. Khởi động lại lỗi khi cần thiết vẫn được thực hiện tự động (xem thêm về tải lại tự động khi cập nhật đăng ký và xoay vòng WARP).

Khôi phục mẫu mặc định

Endpoint GET `/xray/getDefaultJsonConfig` trả về mẫu tham chiếu (`config.json` , được tích hợp trong file nhị phân). Bạn có thể sử dụng nó để đặt lại cấu hình về trạng thái xuất xưởng.

Các chuyển đổi tự động khi lưu

Khi lưu cài đặt Xray, bảng điều khiển thực hiện (theo thứ tự này):

1. **Gỡ bỏ lớp bọc** – gỡ bỏ các lớp bọc dạng `{ "xraySetting": <cấu hình>, "inboundTags": ..., "outboundTestUrl": ... }`, nếu chúng vô tình lọt vào giá trị (nếu không, các lớp sẽ tích lũy mỗi lần lưu). Gỡ đến 8 lớp.
2. **Kiểm tra cấu hình** – JSON được phân tích thành cấu trúc cấu hình Xray; nếu có lỗi – từ chối với «*xray template config invalid*».
3. **Đảm bảo quy tắc thống kê** – quy tắc `inboundTag: ["api"] → outboundTag: "api"` được buộc đưa lên vị trí 0 trong `routing.rules` (hoặc được thêm vào nếu không có). Điều này đảm bảo rằng yêu cầu gRPC-thống kê của bảng điều khiển sẽ không bị chặn bởi quy tắc catch-all phía trên (nếu không, máy khách có thể hiển thị ngoại tuyến với lưu lượng bằng 0 trong khi proxy đang hoạt động).

Do điểm 3, đừng cố xóa hoặc di chuyển quy tắc `api → api` – bảng điều khiển sẽ đưa nó trở lại vị trí cũ trong lần lưu tiếp theo. Đây là cơ sở hạ tầng dịch vụ thống kê, không phải tuyến đường của người dùng.

11.12. Outbound từ đăng ký (với tự động cập nhật)

Kể từ phiên bản 3.3.0, bảng điều khiển có thể nhập `outbound` trực tiếp từ URL đăng ký – cùng định dạng mà các nhà cung cấp VPN phân phối cho các ứng dụng máy khách. Các đăng ký được đọc lại định kỳ ở nền, vì vậy bộ

`outbound` trên máy chủ luôn được cập nhật mà không cần chỉnh sửa thủ công mẫu cấu hình. Từ 3.5.0, các liên kết `ss://` (SIP002) có tham số query (`?plugin=`, `?type=`) hoặc dấu `/` ở cuối được phân tích cổng chính xác (trước đây cổng âm thầm trở thành `0`); liên kết có cổng thực sự không hợp lệ sẽ bị bỏ qua, thay vì được nhập ở trạng thái hỏng.

Trong giao diện, phần này được gọi là «**Đăng ký outbound**», mô tả: «Nhập outbound từ URL đăng ký từ xa (`vmess/vless/trojan/ss/...`). Các thẻ không thay đổi để sử dụng trong bộ cân bằng tải và quy tắc định tuyến. Cập nhật được thực hiện tự động.» Phần này nằm trên trang Xray, phía trên bảng cài đặt `outbound`.

Cách hoạt động

Các đăng ký được lưu riêng khỏi mẫu cấu hình Xray. Mẫu **không bao giờ bị ghi đè**: các `outbound` nhận được từ đăng ký được thêm vào cấu hình cuối cùng ngay khi tạo cấu hình Xray.

Thêm đăng ký

Trong biểu mẫu «Thêm đăng ký», có các trường sau:

Trường	Khóa	Mặc định	Mục đích
URL đăng ký	<code>url</code>	– (bắt buộc)	Địa chỉ đăng ký. Placeholder: « <code>https://...</code> (danh sách liên kết trong base64)». Chỉ HTTP(S) được chấp nhận; địa chỉ được kiểm tra về tính an toàn.

Trường	Khóa	Mặc định	Mục đích
Ghi chú	remark	trống	Nhân tùy ý (placeholder «vd: các nút HK»).
Tiền tố thẻ	tagPrefix	subN-	Tiền tố mà các thẻ của outbound được nhập bắt đầu bằng. Nếu để trống, bảng điều khiển sẽ tự chọn số nhỏ nhất còn trống dạng sub1- , sub2- v.v.
Khoảng thời gian cập nhật	updateInterval	600 giây (10 phút)	Tần suất đọc lại đăng ký. Trong UI được đặt bằng giờ/phút.
Đã bật	enabled	có (true)	Chỉ các đăng ký đã bật mới được đưa vào cấu hình và cập nhật tự động.
Cho phép địa chỉ riêng tư	allowPrivate	không (false)	Cho phép URL trên localhost, LAN và IP riêng tư. Bị tắt theo mặc định để bảo vệ khỏi SSRF — chỉ bật cho nguồn cục bộ đáng tin cậy.
Trước outbound thủ công	prepend	không (false)	Nếu được bật, outbound của đăng ký này được đặt trước các outbound thủ công của mẫu, và một trong số chúng có thể trở thành outbound mặc định. Nếu không, chúng được thêm sau .

Nút **«Xem trước»** (POST /outbound-subs/parse) cho phép tải xuống và phân tích URL trước khi lưu để xem outbound và thẻ nào sẽ nhận được; không có gì được ghi vào cơ sở dữ liệu. Nếu không có gì được nhận dạng theo URL, hiển thị «Không tìm thấy outbound nào theo URL này.»

Thứ tự của nhiều đăng ký trong danh sách outbound chung được đặt theo ưu tiên (priority) và thay đổi bằng các mũi tên lên/xuống (POST /outbound-subs/:id/move).

Các định dạng đăng ký được chấp nhận

Nội dung phản hồi theo URL được xử lý như sau:

- Nội dung trước tiên được thử như **base64** (tiêu chuẩn và biến thể an toàn URL, với tự động điền padding và xóa khoảng trắng/xuống dòng). Nếu đây là base64 — nó được giải mã; nếu không thì lấy nguyên như vậy.
- Sau đó nội dung được chia thành các dòng. Mỗi dòng không trống không bắt đầu bằng # được phân tích như một liên kết. Các dòng không được nhận dạng (bình luận, giao thức không được hỗ trợ) bị bỏ qua âm thầm.
- Các giao thức liên kết được hỗ trợ: vmess:// , vless:// , trojan:// , ss:// (Shadowsocks), hysteria2:// / hy2:// , wireguard:// / wg:// .

Tức là phù hợp với đăng ký thông thường dạng «danh sách liên kết được mã hóa base64», như của hầu hết các nhà cung cấp.

Thẻ ổn định

Mỗi liên kết được tính toán một «danh tính» ổn định (URI cốt lõi không có fragment ghi chú; đối với vmess — JSON nội bộ không có trường ps). Ánh xạ «danh tính → thẻ» được lưu, và trong lần cập nhật tiếp theo, cùng

một máy chủ nhận cùng một thẻ, ngay cả khi ghi chú hoặc các tham số phụ thay đổi. Điều này được thực hiện đặc biệt để các bộ cân bằng tải và quy tắc định tuyến tiếp tục hoạt động sau khi cập nhật:

- Thẻ chính xác trong bộ cân bằng tải/quy tắc sẽ tiếp tục trở đến cùng một máy chủ.
- Bộ chọn tiền tố/wildcard (ví dụ: `hk-*`) sẽ tự động bắt các máy chủ mới mà đăng ký trả về sau này – đây là cách được khuyến nghị để «đăng ký vào một pool».
- Nếu một máy chủ biến mất khỏi đăng ký, thẻ của nó đơn giản biến mất khỏi mảng `outbound` cuối cùng; nếu bộ cân bằng tải có `fallbackTag`, Xray sẽ sử dụng nó.
- Nếu nhà cung cấp thay đổi UUID/host/thông tin xác thực của máy chủ, danh tính thay đổi – đây được coi là `outbound` mới với thẻ mới.

Trong một lần xuất, các thẻ được loại trùng lặp bằng hậu tố `-N`. Các thẻ từ đăng ký giữ lại các ký tự không phải ASCII (ví dụ: chữ Cyrillic) và vẫn có thể đọc được: các chữ cái và chữ số Unicode được giữ trong slug, còn dấu câu được thay thế bằng dấu gạch ngang – các thẻ từ tên Cyrillic không còn được rút gọn thành chỉ các chữ số.

Cách hoạt động của tự động cập nhật

- Tác vụ nền cập nhật đăng ký chạy theo lịch **mỗi 5 phút**.
- Mỗi lần chạy, nó duyệt qua tất cả các đăng ký đã bật và chỉ cập nhật những đăng ký đã hết khoảng thời gian riêng của chúng: đăng ký được cập nhật nếu chưa bao giờ được cập nhật hoặc nếu ít nhất `updateInterval` của nó đã trôi qua kể từ lần cập nhật cuối cùng. Do đó tác vụ kiểm tra đăng ký thường xuyên, nhưng mỗi đăng ký cụ thể được đọc lại không thường xuyên hơn `updateInterval` của nó (mặc định 10 phút). Trong UI điều này được phản ánh với gợi ý tương ứng.
- Cập nhật: URL được kiểm tra lại về tính an toàn như URL công khai (địa chỉ riêng tư bị chặn nếu đăng ký không có `allowPrivate`), yêu cầu đi qua proxy-máy khách của bảng điều khiển với tiêu đề `User-Agent: 3x-ui-outbound-sub/1.0`. Chuỗi chuyển hướng được giới hạn 10 bước, và mỗi bước cũng được kiểm tra về tính riêng tư (bảo vệ chống SSRF). Mong đợi HTTP 200; nếu không thì ghi lại lỗi.
- Sau khi phân tích thành công, kết quả được lưu, thời gian cập nhật lần cuối được đặt, lỗi được xóa. Khi có lỗi, văn bản của nó hiển thị trong UI như «Lỗi lần cuối», còn các `outbound` đã nhận trước đó vẫn có hiệu lực.
- Nếu ít nhất một đăng ký được cập nhật thực sự, tác vụ đánh dấu Xray cần khởi động lại và gửi vô hiệu hóa UI để giao diện tải các `outbound` mới. Việc tải lại Xray thực sự xảy ra trong chu kỳ 30 giây tiếp theo của trình quản lý.

Cập nhật thủ công một đăng ký – nút «**Cập nhật ngay**» (`POST /outbound-subs/:id/refresh`); nó cũng đánh dấu Xray cần khởi động lại. Thêm, thay đổi, xóa đăng ký cũng kích hoạt chờ khởi động lại Xray (khi xóa, các `outbound` của nó rời khỏi cấu hình ở lần tải lại tiếp theo). UI gợi ý: «Sau khi thêm hoặc cập nhật, hãy khởi động lại Xray (hoặc đợi lần tự động tải lại tiếp theo) để các outbound trở nên hoạt động.»

Cách đưa vào cấu hình Xray

Khi tạo cấu hình Xray, các `outbound` đăng ký đang hoạt động được chia thành hai nhóm – `prepend` (còn «Trước outbound thủ công») và những cái còn lại – và được kết hợp với mẫu: `[đăng ký prepend] + [outbound của mẫu] + [đăng ký còn lại]`. Trong mỗi nhóm, các đăng ký đi theo ưu tiên. Các `outbound`

thủ công từ mẫu không bị ảnh hưởng; nếu vì lý do nào đó mảng `outbound` của mẫu không được phân tích, các `outbound` đăng ký sẽ không được trộn vào (để không mất các outbound thủ công).

Các `outbound` đã nhập cũng được hiển thị trong chính bảng `outbound` trong một khối riêng **«Tự đăng ký outbound (chỉ đọc)»** – chúng không thể được chỉnh sửa ở đó, chỉ quản lý qua phần «Đăng ký outbound».

11.13. Xoay vòng IP trong WARP

Trong 3X-UI, bạn có thể tạo WARP-outbound – kết nối WireGuard đi đến Cloudflare WARP (thẻ `warp` trong câu hình Xray). Bảng điều khiển tự đăng ký tài khoản thiết bị với máy chủ Cloudflare, nhận các khóa WireGuard và địa chỉ, đồng thời điền chúng vào outbound với thẻ `warp`. Lưu lượng qua outbound như vậy thoát ra internet dưới địa chỉ IP Cloudflare WARP. Tính năng mới của phiên bản 3.3.0 – khả năng thay đổi IP đi này theo cách thủ công hoặc theo lịch, mà không cần tạo lại tài khoản WARP thủ công.

Quản lý nằm trong phần **Xray** trong thẻ WARP (sau khi nhấn «Tạo tài khoản WARP» và nhận câu hình; trước đó các hành động không khả dụng – bảng điều khiển sẽ gợi ý «Trước tiên hãy nhận câu hình WARP»).

Điều gì xảy ra khi thay đổi IP

Nút **«Thay đổi IP»** khởi động quá trình thay đổi IP. Logic:

1. Một cặp khóa WireGuard mới được tạo.
2. Với khóa mới, thiết bị WARP được đăng ký lại với máy chủ Cloudflare (`device_id` , `access_token` , địa chỉ và dữ liệu peer mới).
3. Dữ liệu mới được ghi vào WARP-outbound của câu hình Xray: `secretKey` , `address` (v4 /32 và v6 /128), `reserved` (từ `client_id`), cũng như `publicKey` và `endpoint` của peer được cập nhật.
4. Nếu trước đó đã đặt khóa giấy phép WARP+ (dài ít nhất 26 ký tự), nó được tự động cài đặt lại cho tài khoản mới. Nếu thất bại, đây chỉ là cảnh báo trong nhật ký – việc thay đổi IP không bị hủy.
5. Sau khi thay đổi thành công, Xray được đánh dấu cần khởi động lại để outbound mới có hiệu lực.

Khi thành công, giao diện hiển thị «Địa chỉ IP WARP đã được thay đổi thành công!».

Xoay vòng tự động theo lịch

Trong thẻ WARP có công tắc **«Tự động cập nhật địa chỉ IP»** và trường **«Khoảng thời gian (ngày)»**. Gợi ý: «0 – tắt. Tự động thay đổi địa chỉ IP.»

Tham số	Giá trị
Cài đặt trong cơ sở dữ liệu	<code>warpUpdateInterval</code> (số nguyên, ≥ 0)
Giá trị mặc định	0 (tắt tự động xoay vòng)
Đơn vị đo	ngày
0	tắt thay đổi tự động
> 0	thay đổi IP mỗi N ngày

Lưu khoảng thời gian sẽ lưu `warpUpdateInterval` , và khi giá trị lớn hơn 0, đặt lại «thời gian cập nhật lần cuối» về thời điểm hiện tại – nếu không, bộ lập lịch sẽ thay đổi IP ngay tại lần tick tiếp theo.

Lịch được thực hiện bởi tác vụ nền chạy một lần mỗi giờ – tức là bảng điều khiển kiểm tra mỗi giờ xem đã đến lúc xoay vòng chưa. Thuật toán kiểm tra:

- nếu khoảng thời gian ≤ 0 – không làm gì;
- nếu «thời gian cập nhật lần cuối» bằng 0 (ví dụ: khoảng thời gian được đặt bằng cách chỉnh sửa trực tiếp cơ sở dữ liệu) – đây là lần chạy đầu tiên: tác vụ chỉ ghi lại mốc thời gian cơ sở và KHÔNG thay đổi IP ngay;
- nếu kể từ lần cập nhật cuối cùng đã trôi qua ít nhất `khoảng_thời_gian × 24 × 3600` giây – thực hiện cùng một thay đổi IP, cập nhật mốc thời gian và lên kế hoạch khởi động lại Xray.

Chi tiết quan trọng: thay đổi thủ công bằng nút «Thay đổi IP» cũng đặt lại mốc thời gian cập nhật lần cuối. Do đó, sau khi xoay vòng thủ công, đếm ngược của khoảng thời gian tự động bắt đầu lại và thay đổi theo lịch sẽ không xảy ra ngay sau đó.

«Qua proxy bảng điều khiển»

Đã thay đổi trong 3.3.1. Cài đặt riêng «Proxy mạng bảng điều khiển» (`panelProxy`) đã bị xóa. Lưu lượng đi của chính bảng điều khiển (bao gồm cả yêu cầu đến WARP API) giờ được định tuyến qua **outbound cho lưu lượng bảng điều khiển** đã chọn – Xray-outbound hoặc bộ cân bằng tải (xem phần 13). Mô tả bên dưới áp dụng cho các phiên bản trước 3.3.1.

Tất cả các yêu cầu đến API Cloudflare WARP (đăng ký, nhận câu hình, đặt giấy phép, thay đổi IP) không đi trực tiếp, mà qua HTTP-máy khách của bảng điều khiển với timeout 15 giây. Máy khách này tôn trọng cài đặt **«Proxy mạng bảng điều khiển»** (`panelProxy`) từ cài đặt bảng điều khiển.

Từ mô tả cài đặt: proxy định tuyến các yêu cầu đi của chính bảng điều khiển (cập nhật cơ sở dữ liệu geo, kiểm tra phiên bản Xray/bảng điều khiển, Telegram, và giờ là cả các yêu cầu đến WARP) – để vượt qua bộ lọc phía máy chủ. Chấp nhận các địa chỉ dạng `socks5://` hoặc `http(s)://` , ví dụ inbound SOCKS cục bộ của chính Xray. Nếu trường trống hoặc proxy được chỉ định không hợp lệ – sử dụng kết nối trực tiếp (hành vi không bị phá vỡ).

Lợi ích cho WARP: nếu máy chủ không thể kết nối trực tiếp đến `api.cloudflareclient.com` , việc đăng ký và xoay vòng trước đây bị thất bại. Giờ đây, bằng cách chỉ định trong `panelProxy` một proxy đang hoạt động (bao gồm cả inbound Xray của chính mình), bạn có thể đảm bảo khả năng truy cập WARP API và hoạt động của cả nút thủ công lẫn xoay vòng theo lịch.

Khi nào điều này hữu ích

- Thay đổi IP đi định kỳ cho outbound đi qua WARP – giảm nguy cơ bị chặn và theo dõi theo một địa chỉ.
- «Làm mới» IP thủ công nếu địa chỉ Cloudflare hiện tại bị đưa vào danh sách đen hoặc hoạt động chậm.
- Các máy chủ không có quyền truy cập trực tiếp đến Cloudflare WARP API: định tuyến các yêu cầu qua `panelProxy` làm cho việc đăng ký và xoay vòng hoạt động được.

12. Nút (đa bảng điều khiển, master/slave)

Phần **Nút** biến một cài đặt 3X-UI thông thường thành **bảng điều khiển trung tâm (master)**, có khả năng theo dõi và quản lý từ xa các bảng điều khiển 3X-UI khác (bảng con). Mỗi nút là một cài đặt 3X-UI riêng biệt trên máy chủ của nó; master kết nối với nút qua HTTP API riêng của nút đó, truy vấn trạng thái và đồng bộ các inbound cũng như client được chỉ định cho nút đó. Đây chính là tính năng **đa bảng điều khiển**: thay vì phải đăng nhập vào từng bảng riêng lẻ, bạn thấy tất cả các máy chủ trong một danh sách và quản lý chúng tập trung.

Nguyên tắc quan trọng: **nút không phải là agent, mà là một bảng điều khiển 3X-UI đầy đủ chức năng**. Master không "cài đặt" gì trên nút – nó chỉ kết nối với API của nút qua token. Xóa nút khỏi danh sách chỉ dừng việc giám sát; bản thân bảng điều khiển từ xa không bị ảnh hưởng (gợi ý: «Thao tác này sẽ dừng giám sát nút. Bảng điều khiển từ xa sẽ không bị ảnh hưởng»).

12.1. Tóm tắt ở đầu danh sách

Phía trên bảng nút hiển thị các bộ đếm tổng hợp:

Trường	Mô tả
Tổng số nút	Tổng số nút trong danh sách.
Trực tuyến	Số nút có trạng thái online .
Ngoại tuyến	Số nút có trạng thái offline .
Độ trễ trung bình	Độ trễ trung bình (ping) đến các nút, tính bằng mili giây.

12.2. Thêm và chỉnh sửa nút

Các nút **Thêm nút** và **Chỉnh sửa nút** mở biểu mẫu với các trường của nút.

Các trường **Tên**, **Địa chỉ**, **Cổng** và **API Token** là bắt buộc (gợi ý: «Tên, địa chỉ, cổng và token API là bắt buộc»).

Khi nhấn «Lưu» (cả khi thêm lẫn khi chỉnh sửa), bảng điều khiển **trước tiên kiểm tra khả năng tiếp cận của nút** với thời gian chờ 6 giây. Nếu nút không phản hồi, bản ghi sẽ không được lưu và hiển thị lỗi. Tức là không thể thêm nút mà rõ ràng không thể truy cập được.

Các trường trong biểu mẫu

Trường	Mặc định	Giá trị hợp lệ	Mô tả
Tên	– (bắt buộc)	chuỗi không rỗng, duy nhất	Tên nội bộ của nút. Cột tên áp dụng ràng buộc duy nhất – không thể tạo hai nút cùng tên. Placeholder gợi ý: napr. de-frankfurt-1 . Khi lưu, khoảng trắng ở đầu và cuối sẽ bị cắt bỏ.
Ghi chú	trống	bất kỳ chuỗi nào	Ghi chú/mô tả tùy chọn về nút. Không ảnh hưởng đến hoạt động.

Trường	Mặc định	Giá trị hợp lệ	Mô tả
Giao thức	<code>https</code>	<code>http / https</code>	Giao thức kết nối đến bảng điều khiển từ xa. Nếu để trống hoặc chỉ định giá trị không hợp lệ, quá trình chuẩn hóa sẽ đặt <code>https</code> . Nếu nút phản hồi qua HTTP thông thường nhưng giao thức được đặt là <code>https</code> , bảng điều khiển sẽ trả về gợi ý rõ ràng: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Địa chỉ	– (bắt buộc)	tên máy chủ hoặc IP	Địa chỉ của bảng điều khiển từ xa. Placeholder: <code>panel.example.com</code> hoặc <code>1.2.3.4</code> . Địa chỉ được chuẩn hóa; theo mặc định, các địa chỉ riêng tư/cục bộ bị cấm để bảo vệ khỏi SSRF – xem «Cho phép địa chỉ riêng».
Cổng	– (bắt buộc)	số nguyên 1–65535	Cổng bảng điều khiển web của nút từ xa. Các giá trị ngoài phạm vi bị từ chối («node port must be 1-65535»).
Đường dẫn cơ sở	<code>/</code>	chuỗi đường dẫn	Đường dẫn cơ sở (web base path) của bảng điều khiển từ xa nếu được đặt. Được chuẩn hóa: đảm bảo bắt đầu và kết thúc bằng <code>/</code> (giá trị rỗng → <code>/</code>). Bảng điều khiển thêm <code>panel/api/server/status</code> vào đó khi truy vấn.
API Token	– (bắt buộc)	token của bảng điều khiển từ xa	Bearer token để truy cập API của nút. Được truyền trong header <code>Authorization: Bearer <token></code> . Placeholder: «Token từ trang Cài đặt của bảng điều khiển từ xa». Gợi ý: «Bảng điều khiển từ xa hiển thị token API của nó trong phần Cài đặt → API Token». Tức là token phải được tạo trên chính nút đó (Cài đặt → API Token), sau đó dán vào đây.
Bật	<code>true</code>	có/không	Bật giám sát và đồng bộ nút. Các nút bị tắt không được truy vấn bởi các tác vụ nền (heartbeat và traffic-sync bỏ qua chúng) và không tham gia vào cập nhật bảng điều khiển hàng loạt.
Cho phép địa chỉ riêng	<code>false</code>	có/không	Vô hiệu hóa bảo vệ SSRF và cho phép kết nối với nút qua địa chỉ riêng tư/cục bộ. Gợi ý: «Chỉ bật cho các nút trong mạng riêng hoặc VPN». Chỉ bật khi nút thực sự nằm trong mạng riêng hoặc có thể truy cập qua VPN.

Lấy và tái tạo token phía nút

Token được lấy trên bảng điều khiển từ xa trong phần **Cài đặt** → **API Token**. Ở đó cũng có thể tái tạo token: nút **Tạo lại token** với cảnh báo: «Tạo lại token sẽ vô hiệu hóa token hiện tại. Bất kỳ bảng điều khiển trung tâm nào đang sử dụng nó sẽ mất quyền truy cập cho đến khi được cập nhật. Tiếp tục?». Sau khi tái tạo, token cũ trong bảng master sẽ ngừng hoạt động – cần cập nhật nó trong biểu mẫu nút.

Kết nối ra ngoài (Connection outbound)

Trường **Connection outbound** (Kết nối ra ngoài, `outboundTag`) xác định cách lưu lượng truy cập API của master đến nút này rời khỏi máy chủ. Nếu chọn tag của Xray outbound, các yêu cầu của bảng điều khiển đến nút sẽ không đi trực tiếp mà qua outbound được chỉ định; bảng điều khiển tự thêm bridge inbound trên loopback vào cấu hình đang chạy và áp dụng thay đổi trực tiếp, không cần khởi động lại. Gợi ý: «Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

Selector hoạt động như lựa chọn outbound của bảng: các tag được nhóm thành **Outbounds** (outbound thông thường) và **Balancers** (bộ cân bằng tải), các outbound blackhole bị ẩn khỏi danh sách. Giá trị rỗng (placeholder «Direct connection») = kết nối trực tiếp đến nút.

Nhập inbound (chọn inbound cần đồng bộ)

Biểu mẫu nút có cài đặt **Nhập inbound** (`inboundSyncMode`) với hai chế độ: **Tất cả inbound** (`all` , mặc định) và **Đã chọn** (`selected`). Theo mặc định, master đồng bộ tất cả inbound có chọn nút đó; các nút hiện có tiếp tục hoạt động ở chế độ «Tất cả inbound».

Ở chế độ **Đã chọn**, bên dưới trường xuất hiện bộ chọn đa tag inbound. Nhân **Tải inbound** – master sẽ sử dụng các thông số kết nối đã nhập (chưa lưu) để yêu cầu danh sách inbound từ nút (endpoint `POST /panel/api/nodes/inbounds`) và hiển thị các tag của chúng; đánh dấu những tag cần thiết. Bảng điều khiển sẽ chỉ đồng bộ và triển khai các tag đã đánh dấu lên nút, còn các inbound khác tồn tại trực tiếp trên nút sẽ không bị chạm vào – master không xóa và không quản lý chúng.

Ví dụ: yêu cầu danh sách inbound của nút để nhập có chọn lọc. Thân yêu cầu chứa các thông số kết nối chưa lưu; phản hồi chứa các tag của inbound có sẵn trên nút:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. Kiểm tra TLS (cho nút https)

Nhóm trường xác định cách master xác minh chứng chỉ HTTPS của nút. Các cài đặt này **chỉ áp dụng cho giao thức https** ; đối với nút `http` , chúng bị bỏ qua.

Kiểm tra TLS – danh sách thả xuống, gợi ý: «Cách bảng điều khiển xác minh chứng chỉ HTTPS của nút. Ghim hoặc Bỏ qua – dành cho chứng chỉ tự ký (chỉ nút https)».

Chế độ	Giá trị	Mặc định	Mô tả
Xác minh (CA tiêu chuẩn)	<code>verify</code>	có (mặc định)	Xác minh chuỗi chứng chỉ thông thường bởi CA tin cậy. Phù hợp với các nút có chứng chỉ công khai/Let's Encrypt. Cũng được sử dụng cho tất cả nút <code>http</code> .
Ghim chứng chỉ (SHA-256)	<code>pin</code>	–	Chuỗi CA không được xác minh, nhưng SHA-256 của chứng chỉ lá của nút được so sánh với dấu vân tay đã lưu (so sánh theo thời gian không đổi). Duy trì bảo vệ chống MITM cho chứng chỉ tự ký . Yêu cầu hiển thị trường dấu vân tay.
Bỏ qua xác minh	<code>skip</code>	–	Tắt hoàn toàn xác minh chứng chỉ. Cảnh báo: «Bỏ qua xác minh sẽ loại bỏ bảo vệ khỏi tấn công người đứng giữa – token API có thể bị chặn. Tốt hơn là ghim chứng chỉ».

Ngoài ba chế độ trên, trong 3.4.0 có thêm chế độ thứ tư – **Mutual TLS (client certificate)** (`mtls`), cũng chỉ khả dụng cho giao thức `https` .

Chế độ	Giá trị	Mặc định	Mô tả
Mutual TLS (chứng chỉ client)	<code>mtls</code>	—	Ngoài việc xác minh chứng chỉ của nút, master còn xác thực bản thân với nút bằng chứng chỉ client do CA của nút cấp. Đối với nút ở chế độ này, API token trở thành tùy chọn — nút nhận dạng master qua chứng chỉ. Khi chọn chế độ này, hiển thị gợi ý: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Để bật TLS lẫn nhau cho nút: trên phía nút, đặt chế độ **Mutual TLS**, sao chép CA của bảng điều khiển quản lý từ phần **Node mTLS** (xem bên dưới), đặt nó làm **trusted parent CA** trên nút và khởi động lại nút.

Nếu chọn bất kỳ giá trị nào khác ngoài `skip`, `pin` hoặc `mtls`, quá trình chuẩn hóa sẽ buộc đặt `verify`.

Ghim chứng chỉ

Khi chọn **Ghim chứng chỉ**, sẽ xuất hiện:

- **SHA-256 của chứng chỉ được ghim** — trường nhập. Chấp nhận dấu vân tay ở định dạng **base64** (định dạng `pinnedPeerCertSha256` từ Xray) hoặc **hex** với dấu hai chấm hoặc không có (kiểu `openssl -fingerprint`). Gợi ý: «SHA-256 của chứng chỉ nút ở dạng base64 hoặc hex. Nhấn «Lấy» để đọc từ nút ngay bây giờ». Placeholder: «SHA-256 ở dạng base64 hoặc hex». Khi chọn `pin`, dấu vân tay rỗng hoặc không hợp lệ sẽ gây lỗi xác thực khi lưu.

Ví dụ: cùng một dấu vân tay ở hai định dạng. Trường chấp nhận bất kỳ định dạng nào — cả hai đều đại diện cho cùng một chứng chỉ:

```
# base64 (формат pinnedPeerCertSha256 из Xray)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex с двоеточиями (стиль openssl x509 -fingerprint -sha256)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Nếu dấu vân tay chưa biết, nhấn **Lấy** — master sẽ tự đọc nó từ nút qua HTTPS và điền vào trường.

- Nút **Lấy** — kết nối đến nút qua HTTPS mà không xác minh chứng chỉ và đọc SHA-256 của chứng chỉ là hiện tại (endpoint `POST /certFingerprint`), điền vào trường. Sau khi thành công — «Đã lấy chứng chỉ hiện tại của nút»; khi thất bại — «Không thể lấy chứng chỉ». Chỉ khả dụng cho nút https.

Node mTLS (xác thực TLS lẫn nhau giữa các bảng điều khiển)

Trên trang **Nút** có phần riêng **Node mTLS** — cấu hình xác thực TLS lẫn nhau, thêm yếu tố thứ hai (chứng chỉ client) vào API token cho các cuộc gọi «bảng điều khiển → nút». TLS lẫn nhau là tùy chọn; nếu các trường của phần này để trống, các nút hoạt động theo sơ đồ cũ — **chỉ với API token** (gợi ý: «Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). Phần này có hai thao tác:

- **Sao chép CA của bảng này** (`POST /panel/api/nodes/mtls/ca`) — sao chép chứng chỉ gốc (CA) của bảng điều khiển này vào clipboard. CA này cần được chuyển cho các nút được quản lý để chúng tin tưởng

chứng chỉ client của bảng; trên chính các nút đó sau đó đặt chế độ xác minh TLS **Mutual TLS** (gợi ý: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). Sau khi sao chép – «CA certificate copied to clipboard».

- **Trusted parent CA** (Trusted parent CA , POST /panel/api/nodes/mtls/trustCA) – trường được sử dụng khi chính bảng điều khiển này đóng vai trò nút cho bảng điều khiển (quản lý) cấp trên. Dán CA của bảng quản lý vào đây để yêu cầu chứng chỉ client từ nó, rồi nhấn **Save trust CA**. Thay đổi yêu cầu **khởi động lại bảng điều khiển** (gợi ý: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. Thông tin hiển thị cho từng nút

Các cột của bảng và trường của thẻ nút (trạng thái quan sát được, được điền vào mỗi lần truy vấn heartbeat):

Trường	Mô tả
Trạng thái	online / offline / unknown – xem bên dưới.
CPU	Tải bộ xử lý của máy chủ từ xa theo phần trăm.
Bộ nhớ	Sử dụng RAM theo phần trăm (tính là $\text{current}/\text{total} \times 100$).
Uptime	Thời gian hoạt động liên tục của máy chủ (tính bằng giây).
Độ trễ	Thời gian phản hồi của nút trong lần truy vấn cuối (ms).
Ping cuối	Thời gian heartbeat thành công cuối cùng (giây unix; 0 = «chưa bao giờ»; giá trị gần đây hiển thị là «vừa xong»).
Phiên bản Xray	Phiên bản Xray core đang chạy trên nút.
Phiên bản bảng	Phiên bản 3X-UI trên nút – so sánh với phiên bản hiện tại để hiển thị chỉ báo cập nhật.
(inbounds)	Số lượng inbound được đặt trên nút này.
(client)	Số lượng client trên các inbound của nút.
(trực tuyến)	Số lượng client của nút đang trực tuyến.
(đã hết)	Số lượng client của nút đã hết hạn hoặc vượt giới hạn lưu lượng . Client bị tắt thủ công không được tính vào bộ đếm này.
(tốc độ)	Tốc độ truyền dữ liệu hiện tại (thực) trên các inbound được đặt trên nút.

Bộ đếm inbound/client/trực tuyến được gắn với nút theo GUID ổn định của nó (panelGuid), không phải theo id cục bộ – để client trên nút con được tính dưới nút con đó, không phải dưới nút trung gian qua đó nó được đồng bộ.

Đối với các inbound được đặt trên nút, trang hiển thị client trực tuyến, các bộ đếm và **tốc độ truyền dữ liệu hiện tại**. Gắn kết theo GUID ổn định giúp phân biệt chính xác cả các nút «nhân bản» có cùng panelGuid .

Trạng thái của nút

Trạng thái	Khi nào được đặt
online	Nút phản hồi <code>success=true</code> cho truy vấn <code>panel/api/server/status</code> .
offline	Nút không phản hồi, trả về lỗi HTTP, <code>success=false</code> hoặc phản hồi không nhận dạng được.
unknown	Giá trị ban đầu, trong khi nút chưa được truy vấn lần nào.

Khi truy vấn không thành công, văn bản lỗi được lưu và hiển thị dưới dạng thông báo rõ ràng, giúp chẩn đoán nguyên nhân «offline».

12.5. Các thao tác trên nút

- **Kiểm tra kết nối** (`POST /test`) – trong biểu mẫu nút, kiểm tra kết nối theo các thông số đã nhập (chưa lưu) với thời gian chờ 6 giây. Kết quả: «Kết nối ổn ({ms} ms)» hoặc «Không thể kết nối». Tiện lợi để gỡ lỗi địa chỉ/cổng/token/TLS trước khi lưu.
- **Kiểm tra ngay** (nút «Kiểm tra ngay», `POST /probe/:id`) – truy vấn ngoài kế hoạch cho nút đã lưu; cập nhật ngay trạng thái và các chỉ số (CPU/bộ nhớ/uptime/độ trễ/phiên bản) và ghi lại heartbeat. Khi thất bại – «Kiểm tra không thành công».

Ví dụ: kiểm tra và truy vấn nút qua API của master. «Kiểm tra kết nối» thử nghiệm các thông số chưa lưu từ biểu mẫu:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

Truy vấn ngoài kế hoạch cho nút đã lưu với id 7:

```
POST /panel/api/nodes/probe/7
```

- **Cập nhật bảng điều khiển** (`POST /updatePanel` với thân `{ids: [...]}`) – khởi chạy trình tự cập nhật tiêu chuẩn trên nút: nút tải xuống bản phát hành 3X-UI mới nhất và khởi động lại với nó. Nút **Cập nhật đã chọn ({count})** thực hiện điều này cho nhiều nút được đánh dấu cùng lúc. Bên cạnh mỗi nút hiển thị chỉ báo: **Có bản cập nhật** hoặc **Đã cập nhật**, dựa trên so sánh phiên bản bảng của nút với phiên bản mới nhất.

Ví dụ: cập nhật nhiều nút bằng một yêu cầu. Thân yêu cầu chứa id của các nút đã đánh dấu; chỉ các nút được bật và `online` mới được cập nhật, các nút khác sẽ được trả về là đã bỏ qua.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Phản hồi dạng «Đã khởi động cập nhật trên 2 nút, 1 thất bại»: nút 12, chẳng hạn, có thể đang offline và do đó bị bỏ qua.

- Xác nhận: «Cập nhật {count} nút lên phiên bản mới nhất? Mỗi nút đã chọn sẽ tải xuống bản phát hành mới nhất và khởi động lại. Chỉ các nút đang bật và trực tuyến mới được cập nhật».
- **Chỉ các nút được bật ở trạng thái online mới được cập nhật.** Nút bị tắt sẽ được đánh dấu «node is disabled» trong kết quả, nút offline – «node is offline». Tổng kết: «Đã khởi động cập nhật trên {ok} nút, {failed} thất bại». Nếu không có nút phù hợp nào được chọn – «Vui lòng chọn ít nhất một nút đang bật và trực tuyến».

Trong hộp thoại xác nhận cập nhật (cả cho nút đơn lẻ lẫn hàng loạt) có hộp kiểm **Cập nhật lên kênh phát triển (commit mới nhất)**. Nếu đánh dấu, các nút đã chọn sẽ cài đặt bản dựng rolling dev-latest (commit mới nhất của nhánh main) thay vì bản phát hành ổn định; khi bỏ đánh dấu, nút cập nhật theo kênh thông thường của nó. Khi hộp kiểm được bật, bên dưới hiển thị cảnh báo: «Bản dựng phát triển theo dõi từng commit trong main và không phải là bản phát hành ổn định – không có tự động rollback». Cờ dev được chuyển qua `POST /panel/api/nodes/updatePanel` đến nút, và nút khởi động cập nhật qua kênh dev.

- **Set Cert from Panel** (hỗ trợ, `GET /webCert/:id`) – khi tạo inbound trên nút, cho phép điền đường dẫn đến chứng chỉ web-TLS của chính nút (không phải bảng điều khiển trung tâm), để các tệp tồn tại chính xác trên nút. Yêu cầu nút phải được bật và có thể truy cập.
- **Xóa nút** (`POST /del/:id`) – xác nhận: «Xóa nút "{name}"? Thao tác này sẽ dừng giám sát nút. Bảng điều khiển từ xa sẽ không bị ảnh hưởng». Xóa bản ghi nút và thông kê lưu lượng tích lũy của nó; bảng điều khiển từ xa tiếp tục hoạt động bình thường. **Chỉ có thể xóa nút sau khi đã gỡ tất cả inbound khỏi nút đó.** Nếu ít nhất một inbound vẫn được gắn với nút (qua `node_id`), bảng điều khiển sẽ từ chối xóa với lỗi dạng «cannot delete node: N inbound(s) still attached to it; detach or delete them first» – trước tiên gỡ hoặc xóa các inbound đó, rồi mới xóa nút. Điều này ngăn chặn các inbound «mồ côi» với tham chiếu trỏ đến nút đã bị xóa.

12.6. Lịch sử số liệu

Nút/biểu đồ lịch sử truy cập `GET /history/:id/:metric/:bucket`. Các số liệu có sẵn: **cpu** và **mem** – chúng được tích lũy mỗi khi heartbeat thành công. Kích thước khoảng thời gian tổng hợp (`bucket`, tính bằng giây) bị giới hạn bởi danh sách trắng:

Ví dụ: yêu cầu lịch sử. Biểu đồ tải CPU của nút 7 với tổng hợp theo khoảng 60 giây (trả về tối đa 60 điểm):

```
GET /panel/api/nodes/history/7/cpu/60
```

Đối với bộ nhớ và chế độ «thời gian thực» (2 giây) – tương ứng là `.../7/mem/60` và `.../7/cpu/2`. Các giá trị ngoài danh sách trắng bị từ chối («invalid metric» / «invalid bucket»).

Bucket (giây)	Mục đích
2	Chế độ thời gian thực
30	Khoảng 30 giây
60	Khoảng 1 phút

Bucket (giây)	Mục đích
120	Khoảng 2 phút
180	Khoảng 3 phút
300	Khoảng 5 phút

Trả về tối đa 60 điểm. Số liệu hoặc bucket không hợp lệ bị từ chối («invalid metric» / «invalid bucket»).

12.7. Cách đồng bộ inbound và client

Inbound «thuộc về» nút thông qua trường `node_id` (trong trình chỉnh sửa inbound có thể chọn nút):

Ví dụ: token trong biểu mẫu nút. Token được lấy trên bảng điều khiển con (Cài đặt → API Token) và dán vào trường **API Token** của master. Trong mỗi lần truy vấn, master gửi token trong header:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Nếu bảng điều khiển con có **đường dẫn cơ sở** (web base path), ví dụ `/secret/`, master sẽ tự thêm nó trước `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

- Triển khai cấu hình (reconcile).** Khi có bất kỳ thay đổi nào đối với inbound/client được gắn với nút, nút được đánh dấu là «bẩn». Tác vụ nền cho mỗi nút được bật ở **trạng thái online** khi có thay đổi sẽ triển khai các inbound của nút (theo `node_id`) lên nút, sau đó đặt lại cờ «bẩn». Nút bị tắt, offline hoặc «bẩn» được coi là «đang chờ» – việc triển khai lên nút đó được hoãn lại cho đến khi kết nối được khôi phục.
- Thu thập lưu lượng.** Cùng tác vụ đó yêu cầu snapshot lưu lượng từ nút và hợp nhất nó vào thống kê cục bộ. Dựa trên lưu lượng đã hợp nhất, hệ thống kiểm tra việc vượt giới hạn/thời hạn và tắt client nếu cần; bộ đếm «đã hết» theo nút phản ánh chính xác điều này. Nếu nút không thể truy cập, các client trực tuyến của nó sẽ bị xóa.

Đối với client được gắn với nhiều bảng điều khiển, master trong cùng tác vụ đó còn gửi đến các nút **tổng lưu lượng qua tất cả bảng** của client đó (trong bảng riêng trên nút, khóa là GUID của master; được ghi đè mỗi lần gửi, do đó việc đặt lại phía master cũng được truyền đi). Trên nút, lưu lượng của client hiển thị giá trị lớn hơn trong hai giá trị – cục bộ hoặc được gửi đến – và khi vượt quota tổng, client bị ngắt kết nối **cục bộ ngay trên nút** (qua cơ chế khởi động lại Xray tương tự khi tự động ngắt, điều này cắt đứt các kết nối hiện có). Điều này loại bỏ tình huống khi nút chỉ thấy phần lưu lượng của mình, tính thiếu và tiếp tục phục vụ client đã vượt giới hạn tổng. Khi đặt lại lưu lượng, tự động gia hạn hoặc xóa client, các bộ đếm đã gửi sẽ bị xóa.

Trong lần **đầu tiên** đồng bộ inbound được đặt trên nút (thêm nút mới hoặc nhập lại inbound), master khởi tạo bộ đếm lưu lượng của client với các giá trị thực từ nút. Trước đây trong tình huống này, bộ đếm tổng của inbound được chuyển đúng, còn bộ đếm riêng lẻ của client bị đặt về không, và master tính thiếu lưu lượng client cho toàn bộ lịch sử tích lũy trước khi kết nối nút. Bây giờ, nếu inbound được tạo trong cùng lần đồng bộ, dòng `client_traffics` mới thừa kế giá trị bộ đếm từ nút (baseline được đặt bằng nó, do đó delta tiếp theo bằng không và lưu lượng không được tính hai lần). Việc gieo bộ đếm chỉ áp dụng cho inbound được tạo trong cùng lần xử lý: client xuất hiện lại dưới inbound đã tồn tại vẫn bắt

đầu từ không (bảo vệ khỏi lưu lượng «ma»), và client vừa bị xóa không «sống lại» khi inbound của nó được tạo lại.

3. **Heartbeat.** Tác vụ nền riêng biệt định kỳ truy vấn tất cả nút **được bật** (với giới hạn song song) qua `panel/api/server/status`, cập nhật trạng thái/số liệu/phiên bản và, khi có web-client, gửi cây nút đã cập nhật qua WebSocket.

12.8. Chuỗi nút (nút con / nút chuyển tiếp)

Cấu trúc liên kết có thể không phẳng: bản thân một nút có thể là master cho các nút của nó. Các bảng điều khiển cấp dưới như vậy hiển thị với bạn là **Nút con** — đây là **các chiều chỉ đọc**, nhận được từ nút trực tiếp.

- Gợi ý: «Chỉ đọc: nút cấp dưới có thể truy cập qua {parent}. Quản lý nó từ bảng điều khiển riêng của {parent}». Tức là không thể chỉnh sửa, xóa hoặc cập nhật nút con ở đây — tất cả các thao tác với nó được thực hiện từ bảng điều khiển của nút cha trực tiếp.
- Danh tính của nút con được xác định bởi GUID của nó; nhờ đó các client trực tuyến và inbound được tính dưới đúng nút vật lý đang host chúng, ngay cả trong chuỗi `Node1 → Node2 → Node3` (master «đi» một cấp sâu hơn qua mỗi nút trực tiếp).
- Nếu nút trực tiếp trở nên không thể truy cập, cache nút con của nó bị xóa và các nút con biến mất khỏi cây cho đến khi kết nối được khôi phục.

12.9. Các bản sửa lỗi 3.5.0 (độ ổn định của nút)

Gói các bản sửa lỗi trong 3.5.0 mà người vận hành nút có thể nhận thấy:

- **Lưu client không thay đổi không còn phá vỡ lưu lượng của nút.** Trước đây, bất kỳ lần lưu client nào (kể cả «mở ra — lưu lại») đều gửi đến nút bản cập nhật inbound đầy đủ với việc tạo lại handler Xray — nút vẫn được tính là trực tuyến nhưng không cho lưu lượng đi qua cho đến khi khởi động lại thủ công. Nay các lần lưu không thay đổi (no-op) không làm gì cả, còn các chỉnh sửa thực sự được gửi đi bằng một bản cập nhật nhẹ duy nhất không tạo lại handler.
- **Các ghi đề Host của nút được nhận vào master:** khi chấp nhận inbound của nút lần đầu, các dòng Hosts của nó (SNI, fingerprint, ALPN v.v.) được nhập vào, và các đăng ký mang đúng thông số TLS.
- **Tự động gia hạn trên nút mở cửa sổ hạn mức mới** (gói dạng «100 GB mỗi 30 ngày» lại cấp đủ dung lượng).
- **Xóa client trên master xóa hoàn toàn client đó trên các nút** (bản ghi + lưu lượng), chứ không chỉ hủy liên kết; phần còn sót của các lần xóa hàng loạt trước đây có thể dọn bằng thao tác «Xóa các client không được liên kết» trên nút.
- **Các inbound của nút không bị quét sạch trước lần chấp nhận đầu tiên:** việc lưu một nút vừa được thêm không còn có thể xóa các inbound đang hoạt động của nó.
- **Một inbound không hợp lệ không dừng việc đồng bộ lưu lượng của nút** (đồng thời các inbound legacy `socks` trên các nút được đổi tên thành `mixed`).
- **Kiểm tra xung đột cổng được giới hạn trong nút của chính nó** — việc sửa địa chỉ lắng nghe của inbound trên nút không còn bị từ chối vì cổng tương tự trên nút khác («port ... already used by inbound ...»).

12.10. Nút: điểm mới trong 3.3.0

Trong phiên bản 3.3.0, phần **Nút** nhận được ba cải tiến đáng chú ý: gán lưu lượng và client trực tuyến chính xác trong các cấu trúc đa chặng (multi-hop), đồng bộ hóa client-IP giữa các nút và chỉ báo trạng thái riêng biệt cho trường hợp bảng điều khiển nút hoạt động nhưng nhân Xray trên đó bị sập.

1. Multi-hop: gán lưu lượng chính xác theo chuỗi nút con

Trước đây, các bộ đếm (số inbound, client trực tuyến, đã hết) được tính ở cấp độ nút «trực tiếp». Nếu bạn có chuỗi dạng Master → Node1 → Node2 → Node3, tất cả những gì thực sự tồn tại trên Node2 / Node3 bị gán nhầm cho Node1 – qua đó nó đến với master. Trong 3.3.0, việc gán được thực hiện theo nguồn thực tế.

Cách hoạt động:

- **Nút con trở thành các dòng riêng biệt.** Mỗi bảng điều khiển công bố danh sách các nút trực tiếp của nó; chỉ bao gồm các nút có Guid đã biết – danh tính ổn định cần thiết để gán nút lên một «bước nhảy». Master định kỳ (từ tác vụ heartbeat) kéo các danh sách này và cache chúng, sau đó thêm các nút con «chuyển tiếp» vào các nút trực tiếp.
- **Các nút chuyển tiếp chỉ đọc.** Trong UI, chúng được đánh dấu là «**Nút con**» với gợi ý: «*Chỉ đọc: nút cấp dưới có thể truy cập qua {parent}. Quản lý nó từ bảng điều khiển riêng của {parent}.*» Dòng đó không có nút quản lý – nút được quản lý từ bảng điều khiển của nút cha trực tiếp.
- **Hệ thống phân cấp qua GUID.** Nút trực tiếp có ParentGuid là GUID của chính master; nút chuyển tiếp có GUID của nút cha của nó. Đây là cách xây dựng cây.
- **Nguồn thực về bộ đếm – origin_node_guid trên inbound.** Đây là panelGuid của nút thực sự đang giữ inbound đó. Nó được đặt khi đồng bộ inbound từ nút và **được giữ nguyên qua các bước nhảy tiếp theo**, do đó inbound được lồng sâu được gán cho nút thực tế, không phải nút trung gian. Dựa trên GUID này, các bộ đếm số inbound, client trực tuyến và client đã hết được tính lại. Logic chọn khóa:

Trạng thái inbound	Được gán cho
origin_node_guid đã đặt	GUID này (nút nguồn thực)
trống, nhưng node_id đã đặt	GUID tổng hợp của nút (bản dựng cũ, chưa báo panelGuid)
trống và node_id trống	GUID riêng của master (inbound trên Xray cục bộ)

Client trực tuyến cũng được nhóm theo GUID, do đó mỗi dòng nút chỉ hiển thị những người thực sự kết nối với nút đó.

Người dùng thấy gì: trong cấu trúc phẳng (các nút trực tiếp dưới master), không có gì thay đổi – bộ đếm theo GUID và theo id trùng khớp. Nhưng ngay khi xuất hiện chuỗi nút, trong danh sách sẽ xuất hiện các dòng «Nút con», và số inbound/trực tuyến/đã hết của mỗi nút bây giờ phản ánh đúng tải của chính nó, không phải tổng của tất cả những gì đi qua nó theo lộ trình chuyển tiếp.

2. Đồng bộ client-IP từ access.log giữa các nút

Giới hạn theo IP (`limitIp` của client) dựa vào các địa chỉ mà Xray ghi vào access.log của nó. Trước đây mỗi nút chỉ thấy các kết nối đến nó, do đó giới hạn «không quá N IP trên mỗi client» không hoạt động trong cluster: client có thể kết nối với các nút khác nhau và vượt qua giới hạn. Trong 3.3.0, các IP được quan sát được đồng bộ trên toàn cluster.

Cách hoạt động:

- Trên mỗi nút, tác vụ nền phân tích access.log, trích xuất từ mỗi dòng IP, email client và dấu thời gian, lưu chúng vào bảng cục bộ (một bản ghi cho mỗi email, IP được lưu dưới dạng mảng JSON `{ip, timestamp}`). Các địa chỉ cục bộ `127.0.0.1` và `::1` bị loại bỏ.
- Đồng bộ hóa **mỗi 10 giây** thực hiện trao đổi hai chiều cho mỗi nút được bật đang trực tuyến: kéo IP từ nút và hợp nhất vào bảng cục bộ, sau đó gửi cho nút bức tranh tổng hợp của master.
- Việc hợp nhất kết hợp các quan sát cũ và mới **không tính trùng** cùng một IP được nhìn thấy trên nhiều nút và **không làm sống lại** các bản ghi lỗi thời: áp dụng ngưỡng tuổi thọ giống như trong tác vụ cục bộ – **30 phút**. Dấu thời gian mới nhất được lưu cho mỗi IP. Các bản ghi từ các nút khác nhận id cục bộ mới (không gian id của các nút độc lập); việc chen đồng thời cùng một email được bảo vệ khỏi trùng lặp.
- Khi tính giới hạn, IP được coi là «sống» nếu nó được nhìn thấy trong lần quét cục bộ hiện tại hoặc có dấu thời gian rất mới từ cơ sở đã đồng bộ (**trong vòng 2 phút**). Chính điều này làm cho giới hạn hoạt động trên toàn bộ cluster, ngay cả khi địa chỉ được nhìn thấy trên nút khác. Khi vượt giới hạn, các IP «sống» cũ nhất được gửi vào nhật ký fail2ban và các kết nối bị ngắt buộc (remove/re-add client qua Xray API).

Người dùng thấy gì: giới hạn số IP bây giờ áp dụng cho toàn bộ cluster, không phải cho mỗi nút riêng lẻ; trong bảng theo client, các IP được nhìn thấy trên bất kỳ nút nào đều hiển thị (trong cửa sổ 30 phút). Không có nút/cài đặt riêng cho điều này – đồng bộ hóa diễn ra tự động trong nền, với điều kiện access.log của nút được bật và có thể truy cập (bản thân giới hạn cũng yêu cầu Fail2Ban trên nút).

3. Chỉ báo trạng thái riêng biệt: bảng điều khiển nút trực tuyến nhưng Xray bị sập

Trước đây, trạng thái nút về cơ bản là «trực tuyến / ngoại tuyến». Nếu bảng điều khiển nút phản hồi, nút được coi là trực tuyến – ngay cả khi nhân Xray trên đó không hoạt động và client thực sự không thể kết nối. Trong 3.3.0, tình trạng bảng điều khiển và tình trạng nhân Xray được tách biệt.

Cách hoạt động:

- Khi truy vấn nút, master lấy từ phản hồi của `/panel/api/server/status` từ xa các trường `xray.state` và `xray.errorMsg` và lưu chúng trong nút. Các trường này được điền ngay cả khi ping bảng điều khiển thành công nhưng nhân không khỏe mạnh – chính để phân biệt khả năng truy cập bảng với trạng thái Xray.
- Giá trị `xray.state` : `"running"` (đang chạy), `"stop"` (đã dừng), `"error"` (lỗi).
- Các giá trị này được chuyển thành trạng thái nút. Các trạng thái mới được thêm vào bên cạnh các trạng thái quen thuộc:

Khóa trạng thái	Khi nào hiển thị
online	bảng điều khiển phản hồi, Xray đang chạy (running)
offline	bảng điều khiển không thể truy cập / ping thất bại
unknown	trạng thái chưa được xác định
xrayError	bảng trực tuyến, nhưng nhân Xray ở trạng thái error (có errorMsg)
xrayStopped	bảng trực tuyến, nhưng Xray bị dừng (stop)

- Đối với trạng thái như vậy, UI sử dụng **chỉ báo màu tím riêng biệt** (màu khác với màu xanh «trực tuyến» và màu đỏ «ngoại tuyến»). Màu tím trực tiếp báo hiệu: có thể kết nối đến nút, nhưng vấn đề nằm ở chính nhân Xray, không phải mạng hay bảng điều khiển.

Người dùng thấy gì: thay vì «màu xanh» gây nhầm lẫn khi nhân bị sập, nút được tô sáng **màu tím** với trạng thái «**Lỗi Xray**» hoặc «**Đã dừng**». Điều này ngay lập tức cho thấy cần phải sửa Xray trên nút (khởi động lại nhân, xem errorMsg), thay vì xử lý khả năng truy cập của chính nút. Cùng xrayState / xrayError cũng được chuyển đến các nút con chuyển tiếp (xem điểm 1), vì vậy trạng thái nhân không chính xác có thể thấy được qua toàn bộ chuỗi.

13. Cài đặt bảng điều khiển

Phần «Cài đặt» (tiêu đề trang – **Cài đặt**, tiếng Anh *Panel Settings*) quản lý hành vi của chính bảng điều khiển web 3X-UI: địa chỉ và cổng nào nó lắng nghe, cách bảo mật, cách tương tác với Telegram bot và các dịch vụ bên ngoài, múi giờ nào để thực thi các tác vụ định kỳ. Mỗi thông số được lưu trong bảng `settings` của cơ sở dữ liệu dưới dạng cặp «khóa – giá trị»; nếu giá trị không có trong CSDL, giá trị mặc định sẽ được áp dụng.

Quan trọng – áp dụng thay đổi. Mọi thay đổi trên trang này cần được lưu bằng nút **Lưu** (*Save*), sau đó khởi động lại bảng điều khiển để thay đổi có hiệu lực. Gợi ý nguyên văn: «Lưu thay đổi và khởi động lại bảng điều khiển để áp dụng.» Khi lưu, thông báo «Cài đặt đã được thay đổi» sẽ hiển thị.

13.1. Lưu và khởi động lại bảng điều khiển

Phần tử	Mục đích
Lưu (<i>Save</i>)	Ghi tất cả các trường của biểu mẫu vào CSDL (<code>POST /panel/setting/update</code>). Trước khi ghi, các giá trị được xác thực – địa chỉ, cổng hoặc đường dẫn không hợp lệ sẽ bị từ chối và bảng điều khiển sẽ trả về lỗi.
Khởi động lại bảng điều khiển (<i>Restart Panel</i>)	Khởi động lại máy chủ web của bảng điều khiển (<code>POST /panel/setting/restartPanel</code>). Việc khởi động lại diễn ra sau độ trễ 3 giây. Gợi ý: «Bạn có chắc muốn khởi động lại bảng điều khiển không? Xác nhận và bảng điều khiển sẽ khởi động lại sau 3 giây. Nếu bảng điều khiển không phản hồi, hãy kiểm tra log máy chủ». Khi thành công – «Bảng điều khiển đã được khởi động lại thành công».
Khôi phục cài đặt mặc định (<i>Reset to Default</i>)	Xóa tất cả cài đặt đã lưu trong CSDL, sau đó bảng điều khiển sẽ sử dụng các giá trị mặc định. Thông tin đăng nhập của quản trị viên không bị đặt lại bởi thao tác này.

Việc khởi động lại được thực hiện bằng cách gửi tín hiệu `SIGHUP` đến tiến trình bảng điều khiển (hoặc thông qua hook khởi động lại đã đăng ký). Trên Windows, khởi động lại tự động qua tín hiệu không được hỗ trợ. **Các thay đổi về thông số lắng nghe (IP, cổng, đường dẫn, tên miền, chứng chỉ, múi giờ) chỉ có hiệu lực sau khi khởi động lại bảng điều khiển.**

13.2. Cài đặt chung (tab «Bảng điều khiển» / *General*)

Ngôn ngữ giao diện (*Language*)

Ngôn ngữ của giao diện web bảng điều khiển. Các ngôn ngữ có sẵn: `en-US` (tiếng Anh), `ru-RU` (tiếng Nga), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Đây là cài đặt hiển thị và không ảnh hưởng đến hoạt động của Xray.

Loại lịch (*Calendar Type*)

- **Khóa:** `datepicker`
- **Giá trị mặc định:** `gregorian` (lịch Gregory).

- **Mục đích:** loại lịch được sử dụng trong bộ chọn ngày (ví dụ: khi đặt ngày hết hạn của khách hàng). Gợi ý: «Các tác vụ định kỳ sẽ được thực thi theo lịch này.» Giá trị thay thế – lịch Ba Tư (jalali), được ưa chuộng trong cộng đồng Iran sử dụng bảng điều khiển.

Kích thước phân trang (*Pagination Size*)

- **Khóa:** `pageSize`
- **Giá trị mặc định:** `25`
- **Giá trị cho phép:** số nguyên từ `0` đến `1000`.
- **Mục đích:** số hàng trên mỗi trang trong các bảng (danh sách kết nối/inbound). Gợi ý: «Xác định kích thước trang cho bảng kết nối. Đặt 0 để tắt» – khi `0`, phân trang bị tắt và tất cả các bản ghi hiển thị trong một danh sách duy nhất.
- **Không cần khởi động lại bảng điều khiển** (cài đặt hiển thị).

Khởi động lại Xray sau khi tự động tắt (*Restart Xray After Auto Disable*)

- **Khóa:** `restartXrayOnClientDisable`
- **Giá trị mặc định:** `true`
- **Mục đích:** khi khách hàng bị tắt tự động (do hết hạn hoặc đạt giới hạn lưu lượng), Xray sẽ được khởi động lại để ngắt các kết nối đã thiết lập của khách hàng đó. Gợi ý: «Khi khách hàng bị tắt tự động do hết hạn hoặc vượt giới hạn lưu lượng, hãy khởi động lại Xray.» Bản thân chức năng không thay đổi – công tắc chỉ nằm ở tab «Bảng điều khiển» (*General*) cùng với các cài đặt chung khác.

Mô hình ghi chú và ký tự phân tách (*Remark Model & Separation Character*)

- **Khóa:** `remarkModel`
- **Giá trị mặc định:** `-ieo`
- **Mục đích:** xác định cách hình thành tên (remark) của cấu hình trong subscription. Chuỗi bao gồm **ký tự đầu tiên** – dấu phân tách, và tiếp theo là **chuỗi chữ cái thứ tự**:
 - `i` – ghi chú inbound (*inbound remark*);
 - `e` – email của khách hàng;
 - `o` – nhân bổ sung (*extra*).

Với giá trị mặc định `-ieo`, dấu phân tách là `-`, và thứ tự các phần: inbound → email → extra (ví dụ: `MyInbound-user@mail-extra`). Các phần trống sẽ bị bỏ qua. Trường «Ví dụ ghi chú» (*Sample Remark*) trong giao diện hiển thị bản xem trước tên được tạo. Việc đưa email vào tên còn phụ thuộc vào thông số «Bao gồm Email trong tên» trong cài đặt subscription (phần về subscription).

Ví dụ: cách giá trị `remarkModel` ảnh hưởng đến tên cấu hình. Giả sử inbound có tên `VLESS-Reality`, email của khách hàng là `alex@vpn`, và nhân bổ sung là `RU`. Khi đó:

Giá trị trường	Tên kết quả (remark)
<code>-ieo</code> (mặc định)	<code>VLESS-Reality-alex@vpn-RU</code>

Giá trị trường	Tên kết quả (remark)
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (dấu cách làm phân tách, chỉ nhận)	RU

Ký tự đầu tiên của chuỗi luôn là dấu phân tách; các chữ cái còn lại xác định phần nào và theo thứ tự nào sẽ có trong tên.

13.3. Quyền truy cập bảng điều khiển: IP, cổng, đường dẫn, tên miền, chứng chỉ

Nhóm này xác định điểm vào mạng của bảng điều khiển. **Tất cả các thay đổi ở đây chỉ có hiệu lực sau khi khởi động lại bảng điều khiển.**

Trường	Khóa	Giá trị mặc định	Mô tả
Địa chỉ IP để quản lý bảng điều khiển (<i>Listen IP</i>)	webListen	"" (trống)	IP mà bảng điều khiển web lắng nghe. Trống = lắng nghe trên tất cả IP. Gợi ý: «Để trống để cho phép kết nối từ bất kỳ IP nào». Nếu được đặt, phải là địa chỉ IP hợp lệ (nếu không, quá trình lưu sẽ bị từ chối).
Tên miền bảng điều khiển (<i>Listen Domain</i>)	webDomain	"" (trống)	Tên miền của bảng điều khiển để xác minh yêu cầu theo tên miền. Trống = chấp nhận kết nối từ bất kỳ tên miền và IP nào. Gợi ý: «Để trống để cho phép kết nối từ bất kỳ tên miền và IP nào.»
Cổng bảng điều khiển (<i>Listen Port</i>)	webPort	2053	Cổng mà bảng điều khiển hoạt động. Gợi ý: «Cổng mà bảng điều khiển hoạt động». Cho phép 1–65535. Cổng phải trống; bảng điều khiển và dịch vụ subscription không thể đồng thời sử dụng cùng một cặp IP: cổng.
Đường dẫn URI (<i>URI Path</i>)	webBasePath	/	Đường dẫn cơ sở URL của bảng điều khiển (basePath). Gợi ý: «Phải bắt đầu bằng '/' và kết thúc bằng '/'». Khi lưu, bảng điều khiển tự động thêm dấu gạch chéo đầu và cuối nếu thiếu. Các ký tự không được phép trong đường dẫn sẽ bị từ chối.

Chứng chỉ bảng điều khiển (TLS / HTTPS)

Trường	Khóa	Giá trị mặc định	Mô tả
Đường dẫn đến file khóa công khai của chứng chỉ bảng điều khiển (<i>Public Key Path</i>)	webCertFile	""	Đường dẫn đầy đủ đến file chứng chỉ (chuỗi). Gợi ý: «Nhập đường dẫn đầy đủ bắt đầu bằng '/'».
Đường dẫn đến file khóa riêng tư của chứng chỉ bảng điều khiển (<i>Private Key Path</i>)	webKeyFile	""	Đường dẫn đầy đủ đến file khóa riêng tư. Gợi ý: «Nhập đường dẫn đầy đủ bắt đầu bằng '/'».

Nếu **ít nhất một** trong các đường dẫn chứng chỉ/khóa được đặt, bảng điều khiển khi lưu sẽ cố tải cặp «chứng chỉ + khóa»; nếu xảy ra lỗi (file không tồn tại, khóa và chứng chỉ không khớp), quá trình lưu sẽ bị từ

chối. Khi cả hai đường dẫn hợp lệ được đặt, bảng điều khiển chuyển sang HTTPS. Cả hai trường trống = bảng điều khiển hoạt động qua HTTP thông thường.

Cảnh báo bảo mật (*Security warnings*). Bảng điều khiển hiển thị khối «Bảng điều khiển của bạn có thể bị lộ:» với các cảnh báo nếu phát hiện cấu hình không an toàn:

- hoạt động qua HTTP thông thường – «hãy cấu hình TLS cho môi trường production»;
- cổng mặc định 2053 – «hãy đổi sang cổng ngẫu nhiên»;
- đường dẫn cơ sở mặc định / – «hãy đổi sang đường dẫn ngẫu nhiên»;
- đường dẫn subscription mặc định /sub/ và JSON subscription /json/ – «hãy thay đổi». Đây là khuyến nghị, không phải chặn.

13.4. Phiên, proxy bảng điều khiển và proxy tin cậy (tab «Proxy và máy chủ» / *Proxy and Server*)

Thời gian phiên (*Session Duration*)

- **Khóa:** `sessionMaxAge`
- **Giá trị mặc định:** 360 (phút, tức là 6 giờ).
- **Giá trị cho phép:** từ 1 đến 525600 phút (1 năm).
- **Mục đích:** thời gian quản trị viên duy trì trạng thái đăng nhập mà không cần đăng nhập lại. Đơn vị – phút. Gợi ý: «Thời gian phiên trong hệ thống (đơn vị: phút)».

Outbound cho lưu lượng bảng điều khiển (*Panel Traffic Outbound*)

- **Khóa:** `panelOutbound`
- **Giá trị mặc định:** "" (trống = kết nối trực tiếp).
- **Mục đích:** chỉ định Xray-outbound mà qua đó bảng điều khiển gửi **các yêu cầu của chính mình** – kiểm tra phiên bản và tải xuống bảng điều khiển/Xray, kết nối với Telegram, cập nhật file geo thông thường – để vượt qua bộ lọc máy chủ cho GitHub/Telegram. Trường này là **danh sách thả xuống**: nó liệt kê các outbound từ template cấu hình Xray, các outbound từ subscription outbound, cũng như các **bộ cân bằng tải** định tuyến (thành nhóm riêng). Các outbound kiểu `blackhole` bị loại khỏi danh sách – định tuyến tải xuống vào «hố đen» là vô nghĩa. Gợi ý nguyên văn: «Định tuyến các yêu cầu của chính bảng điều khiển – kiểm tra phiên bản và tải xuống bảng điều khiển/Xray, Telegram và cập nhật file geo thông thường – qua outbound Xray này để vượt qua bộ lọc máy chủ GitHub/Telegram. Một loopback inbound cầu nối cục bộ được tự động thêm vào cấu hình đang chạy và áp dụng ngay lập tức. Tính năng tự động cập nhật Geodata tích hợp trong Xray không bị ảnh hưởng; nó có outbound riêng để tải xuống. Để trống để kết nối trực tiếp.»

Cách hoạt động. Khi chọn outbound, bảng điều khiển tự thêm vào cấu hình hoạt động một loopback inbound dịch vụ (SOCKS bridge với tag `panel-egress`) và quy tắc định tuyến chuyển hướng lưu lượng HTTP của chính bảng điều khiển đến outbound đã chọn. Nếu chọn bộ cân bằng tải, `balancerTag` được đưa vào quy tắc và lưu lượng bảng điều khiển được phân phối giữa các thành viên của nó. Bridge và quy tắc được áp dụng **ngay lập tức**, mà không cần khởi động lại toàn bộ bảng điều khiển. Để trống

trường để kết nối trực tiếp. Tính năng tự động cập nhật dữ liệu geo tích hợp trong Xray **không bị ảnh hưởng** bởi cài đặt này – nó có outbound riêng trong định tuyến Xray.

- **Định dạng:** `socks5://` (hoặc `socks5h://`) hoặc `http(s)://`, khi cần với thông tin xác thực dạng `socks5://user:pass@host:port`. Các scheme được hỗ trợ chính xác là: `socks5`, `socks5h`, `http`, `https` – các scheme khác được coi là không hợp lệ và bảng điều khiển sẽ quay lại kết nối trực tiếp. Ví dụ điển hình – SOCKS inbound cục bộ của chính Xray.
- Gợi ý nguyên văn: «Định tuyến các yêu cầu đi ra của chính bảng điều khiển (cập nhật geo, kiểm tra phiên bản Xray/bảng điều khiển, Telegram) qua proxy này để vượt qua bộ lọc máy chủ GitHub/Telegram. Chấp nhận `socks5://` hoặc `http(s)://`, ví dụ SOCKS inbound cục bộ của Xray. Để trống để kết nối trực tiếp.»
- Proxy không hợp lệ không dẫn đến lỗi lưu – bảng điều khiển chỉ đơn giản sử dụng kết nối trực tiếp và ghi cảnh báo vào log.

Ví dụ các giá trị trường. Nếu trên máy chủ đã có SOCKS inbound cục bộ của Xray trên cổng `10808`, hãy định tuyến các yêu cầu của chính bảng điều khiển qua nó:

```
socks5://127.0.0.1:10808
```

Đối với HTTP proxy bên ngoài có xác thực:

```
http://user:pass@proxy.example.com:8080
```

Sau khi lưu và khởi động lại, bảng điều khiển sẽ tải cập nhật cơ sở dữ liệu geo, kiểm tra phiên bản và kết nối Telegram qua proxy đã chỉ định.

CIDR proxy tin cậy (Trusted proxy CIDRs)

- **Khóa:** `trustedProxyCIDRs`
- **Giá trị mặc định:** `127.0.0.1/32, ::1/128` (chỉ máy chủ cục bộ).
- **Định dạng:** danh sách địa chỉ IP hoặc mạng con CIDR được phân tách bằng dấu phẩy (ví dụ `10.0.0.0/8, 192.168.1.5`). Mỗi phần tử được kiểm tra như IP hoặc CIDR – giá trị không hợp lệ sẽ bị từ chối khi lưu.
- **Mục đích:** liệt kê các nguồn được phép đặt các header `X-Forwarded-Host`, `X-Forwarded-Proto` và header IP thực của khách hàng. Gợi ý nguyên văn: «IP/CIDR cách nhau bằng dấu phẩy được phép đặt các header forwarded host, proto và IP khách hàng.» Cần cấu hình nêu bảng điều khiển hoạt động sau reverse proxy (nginx, Caddy, v.v.) để xác định chính xác IP khách hàng và scheme.

Ví dụ: bảng điều khiển sau reverse proxy. Nếu nginx nằm trên cùng máy chủ và proxy các yêu cầu đến bảng điều khiển, hãy giữ tin tưởng chỉ với máy chủ cục bộ (giá trị mặc định):

```
127.0.0.1/32, ::1/128
```

Nếu proxy nằm trên máy chủ riêng trong mạng nội bộ `10.0.0.0/8`, hãy thêm mạng con của nó, nếu không bảng điều khiển sẽ bỏ qua các header mà proxy đã gửi và sẽ thấy IP của proxy thay vì IP thực của khách hàng:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

Ví dụ về khối nginx tương ứng truyền IP thực và scheme:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram bot (tab «Telegram bot» / *Telegram Bot*)

Bật Telegram bot (*Enable Telegram Bot*)

- **Khóa:** `tgBotEnable`
- **Loại/mặc định:** `boolean`, `false`.
- **Mục đích:** bật hoạt động của Telegram bot. Gợi ý: «Truy cập các tính năng bảng điều khiển qua Telegram bot».

Token Telegram (*Telegram Token*)

- **Khóa:** `tgBotToken`
- **Mặc định:** `""`.
- **Mục đích:** token của bot. Gợi ý: «Cần lấy token từ trình quản lý bot Telegram @botfather».
- **Đặc điểm bảo mật:** token là giá trị bí mật. Nó không được trả về trong phản hồi của bảng điều khiển khi đọc cài đặt (trường bị xóa, chỉ trả về cờ «đã cấu hình/chưa cấu hình»). Nếu để trống trường khi lưu, token đã lưu trước đó **được giữ nguyên** (không bị xóa).

Ngôn ngữ Telegram bot (*Telegram Bot Language*)

- **Khóa:** `tgLang`
- **Mặc định:** `en-US`.
- **Mục đích:** ngôn ngữ của tin nhắn bot (độc lập với ngôn ngữ giao diện web). Danh sách các ngôn ngữ có sẵn trùng với ngôn ngữ của bảng điều khiển.

User ID của quản trị viên bot (*Admin Chat ID*)

- **Khóa:** `tgBotChatId`
- **Mặc định:** `""`.
- **Định dạng:** một hoặc nhiều Telegram User ID dạng số **phân tách bằng dấu phẩy**.
- **Mục đích:** người nhận thông báo và quản trị viên được phép quản lý bảng điều khiển qua bot. Gợi ý: «Một hoặc nhiều User ID của quản trị viên Telegram bot. Để lấy User ID, hãy sử dụng @userinfobot hoặc lệnh '/id' trong bot.»

Tần suất thông báo (*Notification Time*)

- **Khóa:** `tgRunTime`
- **Mặc định:** `@daily` (một lần mỗi ngày).
- **Định dạng:** chuỗi ở định dạng **Crontab** (hỗ trợ cả biểu thức cron tiêu chuẩn và viết tắt như `@daily`, `@hourly`, `@every 1h`). Gợi ý: «Chỉ định khoảng thời gian thông báo ở định dạng Crontab». Kiểm soát các báo cáo định kỳ của bot.

Ví dụ các giá trị trường.

Giá trị	Khi bot gửi báo cáo
<code>@daily</code>	một lần mỗi ngày vào nửa đêm (mặc định)
<code>@hourly</code>	mỗi giờ
<code>@every 6h</code>	mỗi 6 giờ
<code>0 9 * * *</code>	hàng ngày lúc 09:00
<code>30 8 * * 1</code>	mỗi thứ Hai lúc 08:30

Thời gian được tính theo múi giờ từ cài đặt «Múi giờ» (mục 13.6).

SOCKS proxy (*SOCKS Proxy*)

- **Khóa:** `tgBotProxy`
- **Mặc định:** `""`.
- **Mục đích:** SOCKS5 proxy riêng cho kết nối bot với Telegram. Gợi ý: «Nếu bạn cần proxy Socks5 để kết nối với Telegram, hãy cấu hình các thông số của nó theo hướng dẫn.» Áp dụng đặc biệt cho lưu lượng bot (khác với «Proxy mạng chung của bảng điều khiển» từ mục 13.4).

Telegram API Server (*Telegram API Server*)

- **Khóa:** `tgBotAPIServer`
- **Mặc định:** `""` (sử dụng máy chủ tiêu chuẩn `api.telegram.org`).
- **Định dạng:** URL `http(s)://...`; khi lưu sẽ kiểm tra tính hợp lệ của URL – địa chỉ không hợp lệ sẽ bị từ chối. Gợi ý: «Máy chủ API Telegram được sử dụng. Để trống để sử dụng máy chủ mặc định.» Cần thiết cho Telegram Bot API server tự triển khai.

Thông báo bot (nhóm «Thông báo» / *Notifications*)

Trường	Khóa	Mặc định	Mô tả
Sao lưu cơ sở dữ liệu (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	Gửi file sao lưu CSDL kèm báo cáo đến Telegram. Gợi ý: «Gửi thông báo kèm file sao lưu cơ sở dữ liệu».

Trường	Khóa	Mặc định	Mô tả
Thông báo đăng nhập (<i>Login Notification</i>)	tgBotLoginNotify	true	Thông báo khi có lần thử đăng nhập vào bảng điều khiển. Gợi ý: «Hiển thị tên người dùng, địa chỉ IP và thời gian khi có ai đó cố đăng nhập vào bảng điều khiển của bạn.»
Độ trễ thông báo hết hạn phiên (<i>Expiration Date Notification</i>)	expireDiff	0	Số ngày trước khi hết hạn của khách hàng để gửi thông báo. 0 — đã tắt. Cho phép ≥ 0 . Gợi ý: «Nhận thông báo về ngày hết hạn phiên trước khi đạt ngưỡng (đơn vị: ngày)».
Ngưỡng thông báo lưu lượng (<i>Traffic Cap Notification</i>)	trafficDiff	0	Ngưỡng lưu lượng còn lại để thông báo. Gợi ý: «Nhận thông báo về việc cạn kiệt lưu lượng trước khi đạt ngưỡng (đơn vị: GB)». Cho phép 0–100.
Ngưỡng tải CPU (<i>CPU Load Notification</i>)	tgCpu	80	Thông báo cho quản trị viên nếu tải CPU vượt quá ngưỡng (tính bằng %). Cho phép 0–100. Gợi ý: «Thông báo cho quản trị viên trong Telegram nếu tải CPU vượt quá ngưỡng này (đơn vị: %).»

13.6. Ngày và giờ (tab «Ngày và giờ» / *Date and Time*)

Múi giờ (*Time Zone*)

- **Khóa:** timeLocation
- **Giá trị mặc định:** Local (múi giờ hệ thống của máy chủ).
- **Định dạng:** tên vùng từ cơ sở dữ liệu IANA tz (ví dụ: Europe/Moscow, UTC, Asia/Tehran).
- **Mục đích:** múi giờ mà bảng điều khiển thực thi các tác vụ định kỳ (báo cáo bot, đặt lại/kiểm tra lưu lượng, hết hạn). Gợi ý: «Các tác vụ định kỳ được thực thi theo giờ trong múi giờ này».
- **Xác thực:** khi lưu, vùng được kiểm tra — vùng không tồn tại sẽ bị từ chối. Nếu sau đó CSDL chứa giá trị không hợp lệ, bảng điều khiển trong runtime sẽ quay lại Local, và nếu không khả dụng — về UTC.

13.7. Lưu lượng bên ngoài và hành vi Xray (tab «Lưu lượng bên ngoài» / *External Traffic*)

Trường	Khóa	Mặc định	Mô tả
Thông tin lưu lượng bên ngoài (<i>External Traffic Inform</i>)	externalTrafficInformEnable	false	Thông báo cho API bên ngoài khi mỗi lần cập nhật lưu lượng. Gợi ý: «Thông báo cho API bên ngoài khi mỗi lần cập nhật lưu lượng.»
URI thông tin lưu lượng bên ngoài (<i>External Traffic Inform URI</i>)	externalTrafficInformURI	""	URL mà bảng điều khiển gửi cập nhật lưu lượng đến. Kiểm tra tính hợp lệ của URL khi lưu. Gợi ý: «Cập nhật lưu lượng được gửi đến URI này.»

Trường	Khóa	Mặc định	Mô tả
Khởi động lại Xray sau khi tự động tắt (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	Khởi động lại Xray khi khách hàng bị tắt tự động do hết hạn hoặc vượt giới hạn lưu lượng. Gợi ý: «Khi khách hàng bị tắt tự động do hết hạn hoặc vượt giới hạn lưu lượng, hãy khởi động lại Xray.» Công tắc nằm ở tab «Bảng điều khiển» (General) – xem mục 13.2; đây đưa ra để đầy đủ.

13.8. Khác: template cấu hình Xray và URL kiểm tra

Template cấu hình Xray (*xrayTemplateConfig*)

- **Khóa:** `xrayTemplateConfig`
- **Mặc định:** template JSON tích hợp (embedded) được cung cấp cùng với bản build.
- **Mục đích:** template cấu hình JSON cơ sở của Xray-core, trên đó bảng điều khiển xây dựng thêm inbound/outbound. Giá trị này **không được trả về** trong đầu ra thông thường của tất cả cài đặt và được chỉnh sửa trên trang cấu hình Xray riêng biệt, không phải trong danh sách trường cài đặt chung của bảng điều khiển. Template tiêu chuẩn mặc định có thể truy cập qua `GET /panel/setting/getDefaultJsonConfig`.

URL kiểm tra outbound (*xrayOutboundTestUrl*)

- **Khóa:** `xrayOutboundTestUrl`
- **Mặc định:** `https://www.google.com/generate_204`
- **Mục đích:** URL được sử dụng khi kiểm tra khả năng hoạt động của các kết nối outbound. Khi thiết lập, nó được làm sạch như HTTP(S)-URL.

13.9. Tài khoản quản trị viên và API token

Các thông số này nằm ở tab liên kế («Tài khoản» / *Authentication*) và được xem xét chi tiết trong phần về bảo mật; đây là tóm tắt ngắn gọn về các khóa.

- **Thay đổi thông tin đăng nhập** (các trường «Đăng nhập hiện tại», «Mật khẩu hiện tại», «Đăng nhập mới», «Mật khẩu mới») được lưu bằng yêu cầu riêng `POST /panel/setting/updateUser`. Yêu cầu đăng nhập và mật khẩu hiện tại chính xác; đăng nhập và mật khẩu mới không được trống. Thông báo: «Bạn đã thay đổi thông tin đăng nhập quản trị viên thành công.» / «Tên người dùng hoặc mật khẩu không đúng».
- **Xác thực hai yếu tố (2FA)** – các khóa `twoFactorEnable` (mặc định `false`) và bí mật `twoFactorToken`. Token là bí mật: khi 2FA được bật, trường trống khi lưu không xóa token hiện có. Khi **lần đầu** bật 2FA, bảng điều khiển vô hiệu hóa các phiên hiện tại (tăng «epoch đăng nhập»).
- **API token** được quản lý bởi các endpoint riêng (`/panel/setting/apiTokens...`): danh sách, tạo (`apiTokens/create`), xóa, bật/tắt. Bản thân token chỉ được hiển thị **một lần khi tạo** và không được lưu ở dạng đọc được: «Sao chép token này ngay bây giờ. Vì lý do bảo mật, nó không được lưu ở dạng đọc được và sẽ không được hiển thị lại.»

Chi tiết về 2FA, mật khẩu, đồng bộ hóa LDAP và định dạng subscription (JSON/Clash, fragmentation, noises, mux) được trình bày trong các phần riêng biệt tương ứng của hướng dẫn.

13.10. Thay đổi API trong 3.3.0 (quan trọng cho các tích hợp)

Trong phiên bản 3.3.0, cấu trúc đường dẫn API phía máy chủ đã thay đổi. Nếu bạn có các tích hợp bên ngoài (script, bot, bảng điều khiển trung tâm, tác vụ CI) truy cập bảng điều khiển qua HTTP, chúng **cần được sửa**, nếu không chúng sẽ ngừng hoạt động.

⚠ BREAKING CHANGE: các endpoint `/panel/setting/*` và `/panel/xray/*` đã chuyển sang `/panel/api`

Trước đây, quản lý cài đặt bảng điều khiển và cấu hình Xray nằm riêng biệt, dưới các đường dẫn `/panel/setting/*` và `/panel/xray/*`. Bây giờ cả hai tập hợp được đăng ký bên trong nhóm API chung `/panel/api`. Các đường dẫn cũ **đã bị xóa hoàn toàn** – yêu cầu đến chúng sẽ trả về 404.

Lý do thực hiện điều này: toàn bộ nhóm `/panel/api` đi qua kiểm tra quyền truy cập thống nhất, tức là các endpoint này bây giờ chấp nhận cùng header `Authorization: Bearer <token>` như phần còn lại của API. API token là quyền truy cập quản trị viên đầy đủ, và như vậy toàn bộ bề mặt API đã trở nên thống nhất.

Những gì KHÔNG thay đổi: các trang giao diện web (SPA routes) `/panel/settings` và `/panel/xray` vẫn còn nguyên vị trí – chỉ các endpoint API phía máy chủ mới bị ảnh hưởng.

Bảng tương ứng đường dẫn (cũ → mới)

Tiền tố cho tất cả các đường dẫn bên dưới – chỉ thêm `api/` sau `/panel/`.

Đường dẫn cũ (≤ 3.2.x)	Đường dẫn mới (3.3.0)	Phương thức
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST
<code>/panel/setting/apiTokens/delete/:id</code>	<code>/panel/api/setting/apiTokens/delete/:id</code>	POST
<code>/panel/setting/apiTokens/setEnabled/:id</code>	<code>/panel/api/setting/apiTokens/setEnabled/:id</code>	POST
<code>/panel/xray/</code>	<code>/panel/api/xray/</code>	POST
<code>/panel/xray/update</code>	<code>/panel/api/xray/update</code>	POST
<code>/panel/xray/getDefaultJsonConfig</code>	<code>/panel/api/xray/getDefaultJsonConfig</code>	GET

Đường dẫn cũ (≤ 3.2.x)	Đường dẫn mới (3.3.0)	Phương thức
/panel/xray/getXrayResult	/panel/api/xray/getXrayResult	GET
/panel/xray/getOutboundsTraffic	/panel/api/xray/getOutboundsTraffic	GET
/panel/xray/resetOutboundsTraffic	/panel/api/xray/resetOutboundsTraffic	POST
/panel/xray/testOutbound	/panel/api/xray/testOutbound	POST
/panel/xray/warp/:action	/panel/api/xray/warp/:action	POST
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (và /outbound-subs/*)	/panel/api/xray/outbound-subs (và /outbound-subs/*)	GET/POST/DELETE

Bản thân các tên đường dẫn con, phần thân yêu cầu và định dạng phản hồi không thay đổi – chỉ có **tiền tố** thay đổi.

Cách sửa các tích hợp hiện có

1. Tìm trong các script/câu hình của bạn tất cả các đường dẫn `/panel/setting/` và `/panel/xray/`.
2. Thay thế tiền tố: thêm `api/` ngay sau `/panel/` (ví dụ: `/panel/setting/all` → `/panel/api/setting/all`).
3. Không cần sửa phần thân yêu cầu, tham số và định dạng phản hồi – chỉ URL thay đổi.
4. Vì cài đặt và cấu hình Xray bây giờ nằm dưới `/panel/api`, chúng có thể (và nên) được truy cập bằng cùng API token `Authorization: Bearer <token>` như `/panel/api/inbounds/*` và các endpoint khác. Đừng quên về CSRF-middleware được bật cho toàn bộ nhóm `/panel/api`.

Ví dụ: đọc tất cả cài đặt qua API. Trước đây (≤ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Bây giờ (3.3.0) – đã thêm `api/` sau `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Tương tự cho khởi động lại bảng điều khiển: `POST /panel/api/setting/restartPanel`. Đường dẫn cũ `/panel/setting/restartPanel` bây giờ sẽ trả về 404.

API có kiểu: schema và tài liệu (Swagger / OpenAPI)

Trong 3.3.0, đặc tả OpenAPI đã trở nên hoàn toàn có kiểu. Trước đây, các phản hồi có kiểu được mô tả bằng đối tượng rỗng `{}`; bây giờ các component và schema (`components.schemas`) được tạo trực tiếp từ các mô hình dữ liệu. Nhờ đó:

- Swagger UI hiển thị các mô hình dữ liệu thực, chứ không phải các placeholder vô nghĩa.

- Các bộ tạo bên ngoài (`openapi-generator` và tương tự) có thể xây dựng client sẵn sàng theo ngôn ngữ cần thiết dựa trên đặc tả.
- Mỗi phản hồi có kiểu được đính kèm `$ref` đến mô hình cụ thể và kèm theo các ví dụ phản hồi.

Nơi xem tài liệu API:

- **Trang Swagger tích hợp.** Trong menu bảng điều khiển – mục «**Tài liệu API**» (SPA route `/panel/api-docs`). Đây liệt kê tương tác tất cả các endpoint với mô tả, phần thân yêu cầu và ví dụ phản hồi.
 - **Đặc tả OpenAPI 3.0 thô** được trả về tại địa chỉ `/panel/api/openapi.json` . URL này có thể đưa trực tiếp vào Postman, Insomnia hoặc `openapi-generator` . Đặc tả được tích hợp vào binary khi build; khi bảng điều khiển chạy dưới `webBasePath` không chuẩn, trường `servers` trong đặc tả tự động được ghi lại theo đường dẫn cơ sở hiện tại, để nút «Try it out» và các bộ tạo bên ngoài nhắm đến tiền tố đúng.
-

14. Telegram Bot

Bảng điều khiển 3X-UI có bot Telegram tích hợp sẵn, qua đó bạn có thể nhận thông báo về trạng thái máy chủ và các client, cũng như quản lý từng client trực tiếp từ ứng dụng nhắn tin. Bot hoạt động theo công nghệ long polling (thăm dò liên tục Telegram), do đó không cần tên miền bên ngoài hay cổng mở – chỉ cần kết nối ra ngoài đến các máy chủ Telegram.

Bot phân biệt hai loại người dùng:

- **Quản trị viên** – người dùng có Telegram User ID được chỉ định trong cài đặt bot (trường «User ID quản trị viên bot»). Có quyền truy cập vào tất cả các chức năng: thống kê máy chủ, sao lưu, quản lý client, khởi động lại Xray.
- **Client** – bất kỳ người dùng nào khác có Telegram User ID được liên kết với một client inbound cụ thể (trường `tgId` của client). Chỉ thấy thông tin về các gói đăng ký của chính mình.

Ví dụ: liên kết client với Telegram. Để người dùng nhận được thông kê về gói đăng ký của mình, Telegram User ID dạng số của họ được ghi vào trường `tgId` của client. Trong cài đặt JSON của client, điều này trông như sau:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

Sau đó, người dùng có User ID `123456789` có thể gửi cho bot lệnh `/usage ivan` và xem thống kê của mình. Quản trị viên có thể đặt cùng ID đó qua nút « Đặt người dùng Telegram» trong thẻ client – không cần chỉnh sửa JSON thủ công.

14.1. Bật và cấu hình bot

Tất cả các thông số của bot được đặt trong bảng điều khiển ở mục **Cài đặt** → **Telegram Bot**. Sau khi thay đổi cài đặt, chỉ cần lưu lại – bảng điều khiển áp dụng ngay, không cần khởi động lại. Nếu thay đổi cờ bật (`tgBotEnable`), token, User ID của quản trị viên hoặc địa chỉ API server, bảng điều khiển sẽ tự động dừng và khởi động lại bot với các thông số mới. Quy tắc cũ về việc phải khởi động lại bảng điều khiển sau khi đổi token không còn hiệu lực.

Trường (UI)	Khóa cài đặt	Giá trị mặc định	Mô tả
Bật Telegram Bot	<code>tgBotEnable</code>	<code>false</code>	Công tắc chính. Gợi ý: «Truy cập các tính năng của bảng điều khiển qua Telegram bot». Khi tắt, bot không khởi động và các tác vụ thông báo không được lên lịch.

Trường (UI)	Khóa cài đặt	Giá trị mặc định	Mô tả
Telegram Token	<code>tgBotToken</code>	(trống)	Token của bot. Gợi ý: «Cần lấy token từ trình quản lý bot Telegram <code>@botfather</code> ». Không có token hợp lệ, bot sẽ không khởi động.
SOCKS Proxy	<code>tgBotProxy</code>	(trống)	Proxy để kết nối đến Telegram. Gợi ý: «Nếu bạn cần proxy Socks5 để kết nối đến Telegram, hãy cấu hình theo hướng dẫn».
Telegram API Server	<code>tgBotAPIServer</code>	(trống)	API server Telegram thay thế. Gợi ý: «API server Telegram đang sử dụng. Để trống để dùng server mặc định».
User ID quản trị viên bot	<code>tgBotChatId</code>	(trống)	Một hoặc nhiều Telegram User ID của quản trị viên, phân cách bằng dấu phẩy. Gợi ý: «Để lấy User ID, sử dụng <code>@userinfobot</code> hoặc lệnh <code>/id</code> trong bot».
Tần suất thông báo cho quản trị viên từ bot	<code>tgRunTime</code>	<code>@daily</code>	Lịch gửi báo cáo định kỳ theo định dạng crontab. Gợi ý: «Nhập khoảng thời gian thông báo theo định dạng Crontab».
Sao lưu cơ sở dữ liệu	<code>tgBotBackup</code>	<code>false</code>	Gợi ý: «Gửi thông báo kèm file sao lưu cơ sở dữ liệu». Đính kèm bản sao lưu vào báo cáo định kỳ.
Thông báo đăng nhập	<code>tgBotLoginNotify</code>	<code>true</code>	Gợi ý: «Hiển thị tên người dùng, địa chỉ IP và thời gian khi có người cố gắng đăng nhập vào bảng điều khiển của bạn».
Ngưỡng tải CPU để thông báo	<code>tgCpu</code>	<code>80</code>	Ngưỡng mức sử dụng CPU tính theo phần trăm (kiểm tra giá trị 0–100). Gợi ý: «Thông báo cho quản trị viên trên Telegram nếu tải CPU vượt quá ngưỡng này (đơn vị: %)». Khi giá trị bằng 0, kiểm tra CPU bị tắt.
Ngôn ngữ Telegram Bot	—	—	Ngôn ngữ mà bot dùng để soạn thảo tất cả các tin nhắn.

Lấy token qua @BotFather

- Mở hội thoại với **@BotFather** trong Telegram.
- Gửi lệnh `/newbot` và làm theo hướng dẫn (tên bot và `username` duy nhất kết thúc bằng `bot`).
- BotFather sẽ cấp token có dạng `123456789:AA...`. Sao chép nó vào trường **Telegram Token**.

Lấy User ID quản trị viên

User ID là mã định danh số của tài khoản (không phải username). Có thể tìm nó theo hai cách:

- Nhắn tin cho bot **@userinfobot**.
- Khởi động bot đã cấu hình và gửi lệnh `/id` — bot sẽ trả về ID của bạn.

Nhập số này vào trường **User ID quản trị viên bot**. Để chỉ định nhiều quản trị viên, hãy liệt kê ID của họ cách nhau bằng dấu phẩy (ví dụ: `11111111,22222222`). Mỗi ID được kiểm tra là số nguyên; giá trị không hợp lệ sẽ gây ra lỗi khởi động bot.

Ví dụ: giá trị trường «User ID quản trị viên bot». Một quản trị viên — chỉ là một số:

123456789

Hai quản trị viên cách nhau bằng dấu phẩy (có thể không cần khoảng trắng):

123456789,987654321

Mỗi giá trị phải là số nguyên. Dạng như `@username` hoặc `123 456` (có khoảng trắng bên trong số) sẽ không được chấp nhận – bot sẽ không khởi động.

Proxy

Hỗ trợ các scheme `socks5://`, `http://` và `https://`. Nếu trường proxy để trống, bot sẽ cố dùng proxy chung của bảng điều khiển (nếu đã đặt và scheme được hỗ trợ). URL có scheme không được hỗ trợ hoặc cú pháp không hợp lệ sẽ bị bỏ qua – bot kết nối trực tiếp. Proxy hữu ích khi máy chủ bị chặn truy cập trực tiếp đến API Telegram.

Thông báo qua email (SMTP)

Ngoài Telegram, các sự kiện tương tự cũng có thể được nhận qua email. Kênh này được cấu hình trong mục **Cài đặt** → **Email** trên tab **SMTP Settings**:

Trường (UI)	Khóa cài đặt	Giá trị mặc định	Mô tả
Enable Email Notifications	<code>smtpEnable</code>	<code>false</code>	Công tắc chính cho thông báo email qua SMTP.
SMTP Host	<code>smtpHost</code>	(trống)	Host của SMTP server (ví dụ: <code>smtp.gmail.com</code>).
SMTP Port	<code>smtpPort</code>	<code>587</code>	Cổng của SMTP server.
SMTP Username	<code>smtpUsername</code>	(trống)	Tên người dùng để xác thực SMTP. Cũng được dùng làm địa chỉ người gửi (From).
SMTP Password	<code>smtpPassword</code>	(trống)	Mật khẩu để xác thực SMTP. Được lưu trữ ẩn; nếu mật khẩu đã được đặt, trường hiển thị dấu hiệu «đã cấu hình» và có thể để trống để giữ nguyên giá trị hiện tại.
Recipients	<code>smtpTo</code>	(trống)	Danh sách người nhận phân cách bằng dấu phẩy (ví dụ: <code>admin@example.com, ops@example.com</code>).
Encryption	<code>smtpEncryptionType</code>	<code>starttls</code>	Kiểu mã hóa kết nối: <code>none</code> (không mã hóa), <code>starttls</code> (STARTTLS) hoặc <code>tls</code> (TLS ngầm định).

Nút **Send Test Email** gửi email thử nghiệm và hiển thị kết quả theo từng bước: **Connection** (kết nối), **Authentication** (xác thực) và **Send** (gửi). Nếu có sự cố, kết quả chẩn đoán chỉ ra bước nào xảy ra lỗi (ví dụ: «Authentication failed – check username and password» hoặc «Server requires STARTTLS – change encryption type»), giúp dễ dàng chọn đúng thông số.

Trên tab thứ hai (**Notifications**) có thể chọn các sự kiện sẽ được gửi qua email – với cùng các thẻ nhóm như đối với Telegram (xem «Trung tâm sự kiện và lựa chọn thông báo» trong mục 14.5).

API Server Telegram

Theo mặc định, bot kết nối đến API Telegram chính thức. Trong trường hợp **Telegram API Server** có thể chỉ định địa chỉ Bot API server riêng (`telegram-bot-api`). URL được kiểm tra bảo mật; địa chỉ bị chặn hoặc không hợp lệ sẽ bị loại bỏ và sử dụng server mặc định.

14.2. Menu chính và các nút

Menu được gọi bằng lệnh `/start` . Các nút là bàn phím inline đính kèm tin nhắn; bộ nút phụ thuộc vào việc bạn là quản trị viên hay client.

Menu quản trị viên

Nút	Hành động
 Báo cáo sử dụng lưu lượng được sắp xếp	Liệt kê tất cả client được sắp xếp theo lưu lượng, kèm mức sử dụng của từng người; các email «thừa» không có dữ liệu được đánh dấu « Không có kết quả».
 Trạng thái máy chủ	Tổng quan về máy chủ (xem mục 14.5). Nút «  Làm mới» cập nhật dữ liệu.
 Đặt lại toàn bộ lưu lượng	Đặt lại bộ đếm lưu lượng của tất cả client. Yêu cầu xác nhận («Bạn có chắc không?  »), sau đó với mỗi client hiển thị «  Thành công» hoặc «  Thất bại», cuối cùng – «  Đã hoàn tất đặt lại lưu lượng cho tất cả client».
 Sao lưu DB	Gửi file cơ sở dữ liệu và <code>config.json</code> (xem mục 14.6).
 Nhật ký chặn	Gửi các file nhật ký các địa chỉ IP bị chặn do vượt giới hạn IP.
 Inbound	Tổng quan về tất cả inbound: Remark, cổng, lưu lượng, số lượng client, ngày hết hạn.
 Sắp hết hạn	Danh sách các inbound và client sắp hết lưu lượng hoặc hết hạn (xem mục 14.5).
 Lệnh	Hiển thị hướng dẫn về các lệnh quản trị viên.
 Trực tuyến	Số lượng và danh sách các client đang trực tuyến; nhấn vào email sẽ mở thẻ client. Nút «  Làm mới».
 Tất cả client	Mở lựa chọn inbound, sau đó danh sách các client của inbound đó – để xem/quản lý.
 Client mới	Khởi chạy trình hướng dẫn thêm client (chọn inbound → bản nháp → xác nhận).
 Cài đặt gói đăng ký / liên kết riêng / QR code	Chọn inbound và client để lấy liên kết gói đăng ký, liên kết riêng hoặc QR code.

Menu client

Client có bộ nút hạn chế:

Nút	Hành động
 Thông kê client	Hiển thị dữ liệu về tất cả gói đăng ký được liên kết với Telegram User ID của client.

Nút	Hành động
Lệnh	Hiển thị hướng dẫn về các lệnh client.
Cài đặt gói đăng ký	Chọn client của mình → liên kết gói đăng ký.
Liên kết riêng	Chọn client của mình → liên kết riêng.
QR code	Chọn client của mình → QR code.

Nếu người dùng không có client nào liên kết với Telegram User ID của họ, bot sẽ trả lời: « Không tìm thấy câu hình của bạn! Vui lòng yêu cầu quản trị viên sử dụng Telegram User ID của bạn trong câu hình. User ID của bạn: ...». Cần cung cấp ID này cho quản trị viên để họ nhập vào trường client.

14.3. Lệnh bot

Kể từ phiên bản 3.5.0, có **tám** lệnh được đăng ký trong menu «/» của Telegram:

Lệnh	Mô tả (từ menu)	Quyền truy cập	Chức năng
/start	Hiển thị menu chính	tất cả	Chào mừng; quản trị viên còn được hiển thị thêm «  Chào mừng đến với bot quản lý!» và menu chính.
/help	Hướng dẫn về bot	tất cả	Hiển thị lời chào chung và đề nghị chọn mục menu.
/status	Kiểm tra trạng thái bot	tất cả	Trả lời «  Bot đang hoạt động bình thường».
/id	Hiển thị Telegram ID của bạn	tất cả	Trả về «  User ID của bạn: ... ». Tiện lợi để lấy User ID của chính mình.
/usage	Hiển thị mức sử dụng của client: /usage email	tất cả (xem bên dưới)	Được thêm vào menu trong 3.5.0.
/inbound	Tìm kiếm inbound: /inbound remark (admin)	quản trị viên	Được thêm vào menu trong 3.5.0.
/restart	Khởi động lại lõi Xray (admin)	quản trị viên	Được thêm vào menu trong 3.5.0.
/clearall	Đặt lại lưu lượng của tất cả client (admin)	quản trị viên	Mới trong 3.5.0: yêu cầu xác nhận (các nút «Hủy» / «Xác nhận đặt lại lưu lượng») và đặt về 0 lưu lượng của tất cả client.

Chi tiết về các lệnh có tham số:

- **/usage [Email]** – tìm kiếm client theo email.
 - Đối với **quản trị viên** hiển thị thẻ client đầy đủ (với các nút quản lý).
 - Đối với **client** chỉ hiển thị gói đăng ký của chính họ với email đã chỉ định (theo liên kết Telegram User ID). Không có tham số, bot yêu cầu nhập email: « Vui lòng chỉ định email để tìm kiếm».
- **/inbound [tên kết nối]** – chỉ dành cho quản trị viên. Tìm kiếm inbound theo Remark và hiển thị các thông số cùng thống kê tất cả client. Không có tham số (hoặc đối với client) – « Lệnh không xác định».
- **/restart** – chỉ dành cho quản trị viên. Khởi động lại Xray Core. Các phản hồi có thể có: « Xray Core đã được khởi động lại thành công», « Xray Core chưa chạy» (nếu lõi không hoạt động), « Lỗi khi khởi

động lại Xray core. <Lỗi>». Bất kỳ tham số nào sau `/restart` sẽ dẫn đến thông báo lệnh không xác định kèm gợi ý `/restart`.

Trong các chat nhóm, lệnh dạng `/lệnh@botusername` chỉ được xử lý nếu username khớp với tên của bot hiện tại.

Ba thay đổi nữa trong 3.5.0: trong danh sách **client trực tuyến**, các nút được ghi nhãn dạng `email – remark inbound` (các tên trùng nhau ở các inbound khác nhau không còn bị lẫn lộn); các tin nhắn **sao lưu** và **nhật ký chặn** bắt đầu bằng dòng `Hostname==...` – tiện lợi khi một bot phục vụ nhiều panel; việc tìm thẻ client theo **Telegram ID** hoạt động bất kể định dạng JSON của cài đặt (trước đây các bản ghi «compact» đến từ các nút hoặc từ nhập liệu không được tìm thấy).

Hướng dẫn quản trị viên (nút «Lệnh»):

🔍 Để khởi động lại Xray Core: `/restart`
🌐 Để tìm kiếm client theo email: `/usage [Email]`
👤 Để tìm kiếm inbound (kèm thống kê client): `/inbound [tên kết nối]`
Telegram User ID của bạn: `/id`

Hướng dẫn client:

💰 Để xem thông tin về gói đăng ký của bạn: `/usage [Email]`
👤 Telegram User ID của bạn: `/id`

14.4. Quản lý client (chỉ quản trị viên)

Khi mở thẻ client (qua «Tất cả client», «Trực tuyến», «Sắp hết hạn» hoặc `/usage`), quản trị viên thấy thông tin về client (email, các inbound được liên kết, trạng thái «Đang hoạt động», trạng thái kết nối, ngày hết hạn, mức sử dụng lưu lượng) và các nút quản lý inline:

Nút	Mục đích
Làm mới	Tải lại thẻ client.
Đặt lại lưu lượng	Đặt lại bộ đếm lưu lượng của client. Yêu cầu xác nhận « Xác nhận đặt lại lưu lượng?».
Giới hạn lưu lượng	Đặt giới hạn lưu lượng. Các giá trị có sẵn: ☹ Không giới hạn (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB hoặc « Tùy chỉnh» – nhập số trên bàn phím số tích hợp (các nút 0–9, « » – đặt lại về 0, « » – xóa chữ số cuối, « Xác nhận: N»). Giá trị được đặt theo gigabyte.
Thay đổi ngày hết hạn	Các tùy chọn có sẵn: ☹ Không giới hạn, « Tùy chỉnh», thêm 7/10/14/20 ngày, 1/3/6/12 tháng. Số dương sẽ gia hạn (cộng số ngày vào ngày hết hạn hiện tại hoặc vào «bây giờ» nếu đã hết hạn); 0 xóa giới hạn thời hạn.
Nhật ký IP	Hiển thị các địa chỉ IP đã ghi nhận của client (kèm dấu thời gian nếu có). Từ nhật ký có thể « Làm mới» và « Xóa IP» (với xác nhận « Xác nhận xóa IP?»).
Giới hạn IP	Giới hạn IP đồng thời. Tùy chọn: ☹ Không giới hạn (0), 1–10 hoặc « Tùy chỉnh» (bàn phím số).

Nút	Mục đích
 Đặt người dùng Telegram	Hiển thị Telegram User ID đang được liên kết với client; cho phép xóa liên kết (« Xóa người dùng Telegram» với xác nhận). Việc liên kết người dùng mới được thực hiện qua tùy chọn liên hệ Telegram của hệ thống.
 Bật/Tắt	Bật hoặc tắt client. Yêu cầu xác nhận « Xác nhận bật/tắt người dùng? ».

Tất cả các thao tác thay đổi cấu hình (giới hạn lưu lượng/IP, ngày hết hạn, liên kết/hủy liên kết người dùng Telegram, bật/tắt) sẽ đánh dấu Xray để khởi động lại khi cần, để các thay đổi có hiệu lực. Sau khi thao tác thành công, bot hiển thị xác nhận dạng «  : ... » và hiển thị lại thẻ.

Mọi đầu vào số trong các trình hướng dẫn bị giới hạn ở giá trị < 999999.

14.5. Thông báo và báo cáo

Thông báo được gửi đến tất cả quản trị viên (tất cả User ID trong `tgBotChatId`).

Trung tâm sự kiện và lựa chọn thông báo

Thông báo được xây dựng trên một trung tâm sự kiện thống nhất, với hai kênh giao hàng — **Telegram** và **email (SMTP)**. Đối với mỗi kênh, bạn chọn riêng những sự kiện nào cần thông báo. Trong **Cài đặt** → **Telegram** việc này được thực hiện trên tab **Notifications**, trong **Cài đặt** → **Email** — trên tab cùng tên.

Các sự kiện được nhóm thành các thẻ; mỗi nhóm có công tắc chính kèm bộ đếm các sự kiện đã bật (n/tổng) và trạng thái trung gian khi chỉ một phần được chọn. Các nhóm có sẵn:

- **Outbound** — «Down» (`outbound.down`) và «Up» (`outbound.up`): outbound bị ngắt và phục hồi.
- **Xray Core** — «Crash» (`xray.crash`): lỗi Xray bị kết thúc bất thường.
- **Nodes** — «Down» (`node.down`) và «Up» (`node.up`): node trở nên không khả dụng hoặc phục hồi.
- **System** — «CPU high (%)» (`cpu.high`) và «Memory high (%)» (`memory.high`): tải CPU và RAM cao. Cả hai sự kiện đều có trường inline ngưỡng tính theo phần trăm ở bên cạnh.
- **Security** — «Login attempt» (`login.attempt`): cố gắng đăng nhập vào bảng điều khiển.

Bộ sự kiện đã bật được lưu riêng biệt: cho Telegram — trong `tgEnabledEvents` , cho Email — trong `smtpEnabledEvents` . Theo mặc định, cả hai kênh đều bật «Login attempt» và «CPU high» (giá trị `login.attempt,cpu.high`).

Thông báo đăng nhập bảng điều khiển

Được kiểm soát bằng tùy chọn **Thông báo đăng nhập** (`tgBotLoginNotify` , mặc định bật). Mỗi lần có người cố đăng nhập vào bảng điều khiển web, quản trị viên sẽ nhận được tin nhắn:

- Khi thành công: «  Đăng nhập vào bảng điều khiển thành công. » + host, tên người dùng, IP, thời gian.
- Khi thất bại: «  Lỗi đăng nhập vào bảng điều khiển. » + host, **lý do** (ví dụ: «Lỗi 2FA» khi nhập sai yếu tố thứ hai), tên người dùng, IP, thời gian.

Cảnh báo tải CPU và bộ nhớ cao

Mỗi phút bảng điều khiển kiểm tra mức sử dụng CPU và RAM. Nếu ngưỡng **tgCpu** > 0 và mức tải CPU trung bình trong một phút vượt quá ngưỡng đó, quản trị viên sẽ nhận được: «  Tải CPU là N%, vượt quá ngưỡng M%». Tương tự, tải RAM được kiểm tra so với ngưỡng **tgMemory** (mặc định 80%) – sự kiện «Memory high (%)».

Cả hai ngưỡng được đặt bằng các trường inline bên cạnh các sự kiện «CPU high (%)» và «Memory high (%)» trong nhóm **System** trên tab Notifications (xem «Trung tâm sự kiện và lựa chọn thông báo» ở trên). Đối với kênh Email có các khóa riêng **smtpCpu** và **smtpMemory**. Khi giá trị ngưỡng bằng 0, kiểm tra tương ứng bị tắt.

Báo cáo định kỳ (theo lịch)

Được lên lịch theo biểu thức cron từ trường **Tần suất thông báo** (**tgRunTime** , mặc định **@daily**). Nếu giá trị trống hoặc không hợp lệ, sử dụng **@daily** . Báo cáo bao gồm:

Trình tạo lịch

Trường **Tần suất thông báo cho quản trị viên từ bot** không được nhập thủ công mà thông qua trình tạo lịch. Đầu tiên chọn chế độ trong danh sách thả xuống:

- **@every** – **lặp lại theo khoảng thời gian** – xuất hiện trường số và chọn đơn vị (**Giây / Phút / Giờ**); kết quả được tạo thành biểu thức dạng **@every 6h** .
- **@hourly** – **mỗi giờ**, **@daily** – **mỗi ngày lúc 00:00**, **@weekly** – **mỗi tuần**, **@monthly** – **mỗi tháng** – các preset có sẵn được lưu dưới dạng macro tương ứng (**@hourly** , **@daily** , **@weekly** , **@monthly**).
- **Tùy chỉnh (crontab)** – trường nhập biểu thức crontab riêng. Bộ lên lịch của bảng điều khiển hoạt động với giây được bật, vì vậy biểu thức tùy chỉnh gồm **6 trường**: giây, phút, giờ, ngày trong tháng, tháng, ngày trong tuần (ví dụ: **0 30 8 * * *** – mỗi ngày lúc 08:30:00). Khi chuyển sang chế độ này, trường được điền sẵn bằng crontab tương đương của lựa chọn hiện tại, để có điểm xuất phát.

Ví dụ: giá trị trường «Tần suất thông báo» (**tgRunTime).** Hỗ trợ cả các từ viết tắt có sẵn lần định dạng crontab đầy đủ:

Giá trị	Thời điểm kích hoạt
@daily	một lần mỗi ngày vào nửa đêm (giá trị mặc định)
@hourly	mỗi giờ
@every 6h	mỗi 6 giờ
0 9 * * *	mỗi ngày lúc 09:00
0 9 * * 1	mỗi thứ Hai lúc 09:00
0 */12 * * *	mỗi 12 giờ (lúc 00:00 và 12:00)

Thứ tự các trường trong crontab: phút, giờ, ngày trong tháng, tháng, ngày trong tuần.

1. Dòng « Báo cáo theo lịch: <lich>» và ngày/giờ hiện tại.
2. **Trạng thái máy chủ** (xem bên dưới).
3. Khởi «Sắp hết hạn» theo inbound và client.
4. Thông báo cá nhân đến các client có liên kết Telegram User ID — mỗi client không phải quản trị viên nhận danh sách các gói đăng ký của mình sắp hết lưu lượng/thời hạn (kể cả các gói đã tắt).
5. Nêu bật **Sao lưu cơ sở dữ liệu** (`tgBotBackup`) — sao lưu DB gửi đến quản trị viên.

Trạng thái máy chủ bao gồm: host, phiên bản 3X-UI và Xray, IPv4/IPv6, thời gian hoạt động (tính theo ngày), tải trung bình (Load1/2/3), RAM (hiện tại/tổng), số lượng client trực tuyến, bộ đếm kết nối TCP/UDP, tổng lưu lượng mạng (↑/↓) và trạng thái Xray.

«Sắp hết hạn» hiển thị:

- theo inbound: số lượng đã tắt và số lượng «sắp hết», sau đó liệt kê các inbound đó (Remark, cổng, lưu lượng, ngày hết hạn);
- theo client: tương tự, cộng thêm thẻ client và các nút với email của họ (nhấn sẽ mở thẻ client).

Ngưỡng «sắp hết» được lấy từ cài đặt chung của bảng điều khiển: dự phòng lưu lượng (theo GB) và dự phòng thời hạn (theo ngày). Inbound/client được coi là «sắp hết» khi lưu lượng còn lại đến giới hạn nhỏ hơn ngưỡng HOẶC thời gian còn lại đến ngày hết hạn nhỏ hơn ngưỡng.

14.6. Sao lưu và nhật ký

- **Sao lưu DB** (nút « Sao lưu DB» hoặc tùy chọn trong báo cáo định kỳ): bot gửi thời gian sao lưu, file cơ sở dữ liệu (`x-ui.db` hoặc `x-ui.dump` cho PostgreSQL) và file cấu hình Xray `config.json`.

Tên file sao lưu mà bot gửi được tạo dựa trên địa chỉ máy chủ: sử dụng giá trị **webDomain**, và nếu không được đặt — IP công khai của máy chủ. Điều này giúp xác định file đến từ máy chủ nào khi thu thập sao lưu từ nhiều bảng điều khiển. Nếu không xác định được địa chỉ, sẽ dùng tên chung.

- **Nhật ký chặn** (nút « Nhật ký chặn»): gửi file nhật ký hiện tại và trước đó về các địa chỉ IP bị chặn do vượt giới hạn IP. Các file trống không được gửi.

14.7. Đặc điểm hoạt động

- **Tin nhắn dài** được chia thành các phần (ngưỡng ~2000 ký tự), bản phím inline được đính kèm vào phần cuối cùng.
- **Song song**: các lệnh và lần nhấn nút được xử lý đồng thời (pool tối đa 10 trình xử lý đồng thời).
- **Độ tin cậy gửi**: khi gặp lỗi kết nối, tin nhắn được gửi lại với độ trễ theo cấp số nhân (1s/2s/4s, tối đa 3 lần thử).
- **Bộ nhớ đệm**: dữ liệu «Trạng thái máy chủ» được lưu vào cache để các lần nhấn «Làm mới» thường xuyên không gây tải cho hệ thống.
- **Khởi động lại bot**: khi lưu cài đặt ảnh hưởng đến bot (cờ bật, token, User ID quản trị viên hoặc địa chỉ API server), bảng điều khiển tự dừng vòng lặp thăm dò trước đó và khởi động vòng mới với các thông số mới — không cần tải lại bảng điều khiển. Chỉ một phiên nhận cập nhật hoạt động cùng một lúc.

15. Cơ sở dữ liệu địa lý (geoip / geosite và tùy chỉnh)

Cơ sở dữ liệu địa lý là các tệp nhị phân `.dat` mà Xray-core sử dụng để định tuyến và lọc lưu lượng theo quốc gia (dải IP) hoặc theo danh mục tên miền. Panel có khả năng tải và cập nhật cả bộ tệp địa lý tiêu chuẩn lẫn các nguồn tùy chỉnh tùy ý do người dùng chỉ định qua URL. Tất cả tệp được lưu trong thư mục `bin` cạnh tệp nhị phân Xray (đường dẫn mặc định `bin`, có thể ghi đè bằng biến môi trường `XUI_BIN_FOLDER`).

15.1. geoip.dat và geosite.dat là gì

- **geoip.dat** – cơ sở dữ liệu ánh xạ «địa chỉ IP → mã quốc gia/khu vực». Được sử dụng trong các quy tắc định tuyến dưới dạng `geoip:<mã>`, ví dụ `geoip:ru`, `geoip:cn`, cũng như cho các nhân đặc biệt `geoip:private` (mạng riêng tư/cục bộ). Về bản chất đây là câu trả lời cho câu hỏi «IP này thuộc quốc gia nào».
- **geosite.dat** – cơ sở dữ liệu ánh xạ «tên miền → danh mục/danh sách». Được sử dụng dưới dạng `geosite:<danh mục>`, ví dụ `geosite:category-ads-all` (tên miền quảng cáo), `geosite:google`, `geosite:ru`. Về bản chất đây là các danh sách tên miền được nhóm lại.

Các tệp này cần thiết để xây dựng các quy tắc kiểu «toàn bộ lưu lượng đến IP/tên miền của Nga – kết nối trực tiếp, còn lại – qua outbound» và các quy tắc tương tự. Bản thân các quy tắc được thiết lập trong phần định tuyến của Xray; cơ sở dữ liệu địa lý chỉ cung cấp dữ liệu cho chúng. Nếu không có tệp địa lý cập nhật, các quy tắc tham chiếu đến `geoip:` / `geosite:` sẽ không hoạt động hoặc sẽ dựa vào các danh sách lỗi thời.

Ví dụ: quy tắc «tên miền và IP của Nga – kết nối trực tiếp». Quy tắc này trong phần định tuyến chuyển toàn bộ lưu lượng đến tài nguyên của Nga sang outbound có thể `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Tệp địa lý tiêu chuẩn và cách cập nhật

Panel chứa danh sách cho phép (allowlist) cố định gồm sáu tệp tiêu chuẩn với các nguồn tải đã được mã hóa cứng. Việc cập nhật được thực hiện qua `POST /panel/api/server/updateGeofile/:fileName` (hoặc không có tên tệp – để cập nhật tất cả cùng lúc).

Ví dụ: cập nhật một tệp và tất cả tệp qua API. Chỉ cập nhật `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Cập nhật tất cả sáu tệp tiêu chuẩn trong một yêu cầu (không chỉ định tên tệp):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Phản hồi thành công:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Tên tệp	Nguồn (kho phát hành)
geoup.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoup.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoup_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoup.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoup_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoup.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Đặc điểm cập nhật tệp tiêu chuẩn:

- **Nút cập nhật một tệp.** Trước khi tải xuống sẽ hiển thị xác nhận: «Bạn có thực sự muốn cập nhật tệp địa lý không?» với ghi chú «Thao tác này sẽ cập nhật tệp #filename#.» (tiếng Anh: *Do you really want to update the geofile? This will update the #filename# file.*). Khi thành công sẽ hiển thị thông báo «Tệp địa lý đã được cập nhật thành công» (tiếng Anh: *Geofile updated successfully*).
- **Nút «Update all»** (cập nhật tất cả) tải xuống tất cả sáu tệp. Xác nhận: «Thao tác này sẽ cập nhật tất cả tệp địa lý.» (tiếng Anh: *This will update all geofiles.*).
- **Tải xuống có điều kiện.** Nếu tệp cục bộ đã tồn tại, yêu cầu sẽ gửi kèm header `If-Modified-Since` với thời gian sửa đổi của tệp. Phản hồi `304 Not Modified` từ máy chủ có nghĩa là tệp không thay đổi – tệp sẽ không được tải lại, chỉ cập nhật mốc thời gian của tệp.
- **Bảo mật tên tệp.** Chỉ chấp nhận các tên có trong allowlist; tên được kiểm tra không chứa `..`, dấu phân cách đường dẫn `/` và `\`, đường dẫn tuyệt đối và phải khớp với mẫu `^[a-zA-Z0-9._-]+\..dat$`. Bất kỳ tên nào ngoài danh sách sẽ bị từ chối với lỗi «Invalid geofile name».
- **Khởi động lại Xray.** Sau khi tải tệp địa lý, Xray-core sẽ được khởi động lại để đọc lại các cơ sở dữ liệu đã cập nhật. Nếu không thể khởi động lại, thông báo lỗi sẽ chứa dòng tương ứng.

Cập nhật cơ sở dữ liệu địa lý từ dòng lệnh (x-ui)

Cơ sở dữ liệu địa lý cũng có thể được cập nhật mà không cần panel – thông qua menu tương tác `x-ui` (mục cập nhật tệp địa lý) hoặc bằng lệnh không tương tác `x-ui update-all-geofiles`. Đối với từng tệp trong bộ (geoup/geosite, bao gồm cả bộ IR và RU) sẽ hiển thị trạng thái riêng: «đã cập nhật», «đã là phiên bản mới nhất» hoặc «lỗi tải xuống». Khi tải xuống thất bại sẽ không in thông báo thành công giả. Việc khởi động lại Xray (và do đó ngắt các kết nối đang hoạt động) chỉ xảy ra nếu có ít nhất một tệp thực sự được cập nhật; nếu không có tệp nào thay đổi (tất cả đều trả về `304 Not Modified`), panel và Xray sẽ không được khởi động lại.

15.3. Tự động cập nhật dữ liệu địa lý bằng Xray (Geodata Auto-Update)

Các nguồn `.dat` bổ sung theo URL tùy ý được thêm không phải qua panel mà thông qua phần `geodata` gốc của Xray-core. Phần tương ứng được đặt trong cửa sổ modal cập nhật Xray (Dashboard → cập nhật Xray, `xrayUpdates`) – đây là tab «Geodata Auto-Update» (Tự động cập nhật Geodata). Panel ở đây chỉ chỉnh sửa khóa `geodata` trong mẫu cấu hình Xray; việc tải xuống, kiểm tra và tải lại nóng các tệp do bản thân lõi Xray thực hiện.

Ở phần trên của mục hiển thị gợi ý: «Xray tải xuống các tệp này theo lịch và tải lại nóng mà không cần khởi động lại. URL phải là HTTPS. Tệp phải đã tồn tại trong thư mục bin trước khi Xray có thể cập nhật nó.» (tiếng Anh: *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*).

Các trường của mục

- **Schedule (cron)** (Lịch cron) – chuỗi cron gồm 5 trường; giá trị mặc định `0 4 * * *` (hàng ngày lúc 04:00). Khi lưu sẽ kiểm tra rằng chuỗi chứa đúng 5 trường, nếu không sẽ hiển thị lỗi «Cron phải chứa 5 trường, ví dụ `0 4 * * *`».
- **Download through outbound (optional)** (Tải xuống qua outbound (tùy chọn)) – danh sách thả xuống với các thẻ outbound có sẵn (cộng outbound của subscription), qua đó Xray sẽ tải các tệp; các outbound có giao thức `blackhole` không xuất hiện trong danh sách. Trường có thể để trống – khi đó sẽ sử dụng kết nối trực tiếp. Lựa chọn này độc lập với outbound cho các yêu cầu của chính panel (xem §11): tự động cập nhật geodata có outbound tải xuống riêng của nó.
- **Danh sách tệp** – mỗi dòng xác định một cặp «URL + File name» (Tên tệp). URL phải bắt đầu bằng `https://` (nếu không sẽ hiển thị «Cần có HTTPS URL cho mỗi tệp.»). Tên tệp chỉ định đơn giản, không có đường dẫn và dấu phân cách – chỉ các ký tự `^[A-Za-z0-9._-]+$` (nếu không sẽ hiển thị «Tên tệp phải đơn giản, ví dụ `geosite_custom.dat` (không có đường dẫn.)). Khi nhập URL, panel sẽ cố gắng tự động điền tên tệp từ phần cuối của đường dẫn. Nút «Add file» (Thêm tệp) thêm một dòng, nút thùng rác xóa nó.

Nếu danh sách trống, hiển thị gợi ý: «Chưa có tệp nào được cấu hình. Tham chiếu các tệp trong quy tắc định tuyến dưới dạng `ext:geosite_custom.dat:category`» (tiếng Anh: *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category*).

Lưu

Nút «Save & Restart Xray» (Lưu và khởi động lại Xray) hiển thị xác nhận «Lưu cài đặt geodata?» với ghi chú «Mẫu cấu hình Xray sẽ được cập nhật và Xray sẽ được khởi động lại.» (tiếng Anh: *Save geodata settings? This updates the Xray config template and restarts Xray*). Sau khi lưu, khóa `geodata` được ghi vào mẫu cấu hình (`POST /panel/api/xray/update`) và Xray được khởi động lại (`POST /panel/api/server/restartXrayService`). Nếu danh sách tệp trống, khóa `geodata` sẽ bị xóa khỏi mẫu.

Các đặc điểm quan trọng:

- **Tệp phải đã tồn tại trong bin**. Xray chỉ cập nhật các tệp `.dat` đã có trong thư mục `bin` tại thời điểm khởi động. Do đó, tệp tùy chỉnh mới trước tiên phải được đặt vào `bin` theo cách thủ công (hoặc ít nhất tạo phiên bản trống/lỗi thời ở đó với tên cần thiết), và chỉ sau đó Xray mới bắt đầu duy trì nó theo lịch.

- **Tải lại nóng.** Sau khi tải xuống theo lịch, Xray sẽ đọc lại các cơ sở dữ liệu đã cập nhật mà không cần khởi động lại toàn bộ tiến trình.
- **Khả năng tương thích.** Các tệp địa lý đã tải xuống trước đó (cả tiêu chuẩn lẫn tùy chỉnh) tiếp tục hoạt động trong các quy tắc định tuyến với cú pháp `ext:` mà không có thay đổi.

Nếu danh sách trống, hiển thị gợi ý: «Chưa có nguồn địa lý tùy chỉnh nào – nhấn «Thêm» để tạo» (tiếng Anh: *No custom geo sources yet – click Add to create one*).

Các cột bảng và trường của nguồn

Trường (UI)	JSON	Giá trị mặc định	Mô tả
Type (Loại)	<code>type</code>	– (bắt buộc)	Loại tài nguyên: chỉ <code>geosite</code> hoặc <code>geoip</code> . Xác định tên tệp kết quả.
Alias (Bí danh)	<code>alias</code>	– (bắt buộc)	Định danh ngắn của nguồn. Tên tệp được tạo từ nó và loại.
URL (URL)	<code>url</code>	– (bắt buộc)	Liên kết trực tiếp đến tệp <code>.dat</code> (http/https).
Enabled (Đã bật)	–	–	Trạng thái hoạt động của nguồn trong danh sách.
Last updated (Cập nhật lần cuối)	<code>lastUpdatedAt</code>	<code>0</code>	Thời gian cập nhật thành công gần nhất (Unix time; <code>0</code> – chưa được cập nhật).
Routing (ext:...) (Định tuyến (ext:...))	–	–	Chuỗi sẵn sàng cho quy tắc định tuyến: <code>ext:<tệp.dat>:tag</code> .
Actions (Hành động)	–	–	Các nút «Chỉnh sửa», «Xóa», «Cập nhật ngay».

Ngoài ra, các trường dịch vụ được lưu trong cơ sở dữ liệu: `localPath` (đường dẫn thực tế đến tệp trong thư mục `bin`), `lastModified` (giá trị header `Last-Modified` từ máy chủ, được sử dụng cho tải xuống có điều kiện), `createdAt` và `updatedAt`.

Đặt tên tệp

Tên tệp kết quả được tạo tự động từ loại và bí danh:

- loại `geoip` → `geoip_<alias>.dat`;
- loại `geosite` → `geosite_<alias>.dat`.

Ví dụ, nguồn có loại `geosite` và bí danh `myads` sẽ tạo tệp `geosite_myads.dat`.

Ví dụ: thêm nguồn qua API. Thêm danh sách tên miền quảng cáo tùy chỉnh dưới dạng tài nguyên `geosite` với bí danh `myads`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

Panel sẽ tải tệp vào thư mục `bin` dưới tên `geosite_myads.dat`, lưu bản ghi và khởi động lại Xray.

Các nút và hành động

- **Add** (Thêm) – mở biểu mẫu «Add custom geo» (Thêm nguồn tùy chỉnh). Nút lưu – «Save» (Lưu). API: `POST /add`.
- **Edit** (Chỉnh sửa) – biểu mẫu «Edit custom geo» (Chỉnh sửa nguồn tùy chỉnh). API: `POST /update/:id`. Khi thay đổi loại hoặc bí danh, tệp cũ sẽ bị xóa và tệp mới sẽ được tải xuống lại.
- **Delete** (Xóa) – xác nhận «Xóa nguồn tùy chỉnh này không?» (tiếng Anh: *Delete this custom geo source?*). Xóa bản ghi khỏi cơ sở dữ liệu và bản thân tệp `.dat`. API: `POST /delete/:id`. Khi thành công: «Tệp địa lý tùy chỉnh «<tên>» đã được xóa».
- **Update now** (Cập nhật ngay) – tải lại nguồn cụ thể và cập nhật mốc thời gian. API: `POST /download/:id`. Khi thành công: «Geofile «<tên>» đã được cập nhật».
- **Cập nhật tất cả** – cập nhật tất cả nguồn tùy chỉnh cùng lúc. API: `POST /update-all`. Khi hoàn toàn thành công: «Tất cả nguồn địa lý tùy chỉnh đã được cập nhật» (tiếng Anh: *All custom geo sources updated*). Nếu ít nhất một nguồn không cập nhật được, thao tác được coi là thành công một phần với thông báo «Không thể cập nhật một hoặc nhiều nguồn địa lý tùy chỉnh» (tiếng Anh: *One or more custom geo sources failed to update*), và phản hồi liệt kê các nguồn thành công và thất bại.

Sau bất kỳ hành động nào trong số này (thêm, chỉnh sửa, xóa, cập nhật, cập nhật tất cả khi có thành công) Xray-core sẽ được khởi động lại.

Từng bước: thêm nguồn

1. Nhấn «Add» (Thêm).
2. Trong trường «Type» (Loại) chọn `geosite` hoặc `geoip`.
3. Trong trường «Alias» (Bí danh) nhập định danh (chỉ chữ thường Latin, chữ số, `-` và `_`; gợi ý placeholder: `a-z 0-9 _ -`).
4. Trong trường «URL» chỉ định liên kết trực tiếp đến tệp `.dat` (phải bắt đầu bằng `http://` hoặc `https://`).
5. Nhấn «Save» (Lưu). Panel sẽ ngay lập tức tải tệp vào thư mục `bin`, lưu bản ghi và khởi động lại Xray.

15.4. Xác thực và giới hạn

Khi tạo và chỉnh sửa nguồn sẽ thực hiện các kiểm tra nghiêm ngặt. Thông báo lỗi:

Điều kiện	Thông báo (RU)	Thông báo (EN)
Loại không phải geosite / geoip	Тип должен быть geosite или geoip	Type must be geosite or geoip
Bí danh trống	Укажите псевдоним	Alias is required
Ký tự không hợp lệ trong bí danh (không khớp <code>^[a-z0-9_-]+\$</code>)	Псевдоним содержит недопустимые символы	Alias must match allowed characters
Bí danh bị dành riêng	Этот псевдоним зарезервирован	This alias is reserved
URL trống	Укажите URL	URL is required
URL không phân tích được	Некорректный URL	URL is invalid
Scheme không phải http/https	URL должен использовать http или https	URL must use http or https
Host trống/không hợp lệ hoặc bị chặn bởi bảo vệ SSRF	Некорректный хост URL	URL host is invalid
Trùng lặp «loại + bí danh»	Такой псевдоним уже используется для этого типа	This alias is already used for this type
Không tìm thấy nguồn	Источник не найден	Custom geo source not found
Lỗi tải xuống	Ошибка загрузки	Download failed

Gợi ý trong biểu mẫu (xác thực phía client): «Bí danh: chỉ a-z, chữ số, - và _» (*Alias may only contain lowercase letters, digits, - and _*) và «URL phải bắt đầu bằng http:// hoặc https://» (*URL must start with http:// or https://*).

Các giới hạn kỹ thuật bổ sung:

- **Bí danh bị dành riêng.** Không thể sử dụng các bí danh xung đột với tệp tiêu chuẩn. Bị dành riêng (so sánh không phân biệt chữ hoa/thường, dấu gạch ngang được coi tương đương với dấu gạch dưới): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. Ví dụ, `geosite-ru` sẽ bị từ chối như `geosite_ru`.
- **Bảo vệ SSRF.** Host URL được phân giải thành IP, và nếu nó trở đến địa chỉ riêng tư/nội bộ, việc tải xuống sẽ bị chặn (người dùng thấy «Host URL không hợp lệ»). Điều này ngăn việc sử dụng panel để truy cập các dịch vụ nội bộ.
- **Bảo vệ path traversal.** Đường dẫn cuối cùng của tệp phải nằm trong thư mục `bin` (với việc giải quyết symlink); mọi nỗ lực thoát ra ngoài đều bị từ chối.
- **Kích thước tệp tối thiểu.** Tệp tải xuống chỉ được coi là hợp lệ nếu nó không nhỏ hơn 64 byte; tệp quá nhỏ sẽ bị từ chối với lỗi tải xuống.
- **Proxy và tải xuống có điều kiện.** Nếu cài đặt panel có cấu hình proxy, việc tải xuống sẽ đi qua đó; trong các trường hợp khác sử dụng kết nối trực tiếp với transport an toàn SSRF. Cũng như với tệp tiêu chuẩn, sử dụng `If-Modified-Since / 304 Not Modified` (tệp không thay đổi sẽ không được tải lại). Timeout tải xuống – 10 phút, kiểm tra khả năng truy cập URL (HEAD, nếu cần – GET một phần) – 12 giây.

15.5. Kiểm tra tự động khi khởi động panel

Khi khởi động, panel duyệt qua tất cả nguồn tùy chỉnh và kiểm tra sự tồn tại và tính toàn vẹn của tệp cục bộ cho từng nguồn (tệp không tồn tại, là thư mục hoặc nhỏ hơn 64 byte). Nếu tệp bị thiếu hoặc bị hỏng, sẽ thực hiện kiểm tra nguồn và thử tải xuống lại. Điều này đảm bảo rằng sau khi cài đặt lại hoặc mất thư mục `bin`, các tệp địa lý tùy chỉnh sẽ được khôi phục tự động.

15.6. Sử dụng cơ sở dữ liệu địa lý trong quy tắc định tuyến

Trong các quy tắc định tuyến Xray, cơ sở dữ liệu địa lý được sử dụng trong các trường như `domain / ip` qua các tiền tố:

- **geoip:** cho cơ sở dữ liệu IP — `geoip:<mã>`. Ví dụ: `geoip:ru`, `geoip:cn`, `geoip:private`. Lấy từ `geoip.dat` (hoặc `geoip_RU.dat` v.v., nếu quy tắc trỏ đến tệp cụ thể).
- **geosite:** cho cơ sở dữ liệu tên miền — `geosite:<danh mục>`. Ví dụ: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Lấy từ `geosite.dat`.

Ví dụ: chặn quảng cáo qua geosite. Quy tắc gửi tất cả tên miền quảng cáo vào «lỗ đen» (giả sử có outbound với thẻ `blocked` và giao thức `blackhole`):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Đối với các tệp **tùy chỉnh** sử dụng cú pháp tệp ngoài `ext:`. Gợi ý trong UI: «Trong quy tắc định tuyến hãy sử dụng giá trị như `ext:tệp.dat:tag` (thay thế tag).» (tiếng Anh: *In routing rules use the value column as ext:file.dat:tag (replace tag)*). Định dạng:

```
ext:<tên_tệp.dat>:<tag>
```

trong đó `<tên_tệp.dat>` — là `geoip_<alias>.dat` hoặc `geosite_<alias>.dat`, còn `<tag>` — danh sách/danh mục cụ thể bên trong tệp. Panel trong cột «Routing (ext:...）」 (Định tuyến (ext:...)) hiển thị mẫu sẵn sàng dạng `ext:geosite_myads.dat:tag` — chỉ cần thay `tag` bằng thẻ cần thiết. Tên tệp như vậy được chỉ định trong mục «Geodata Auto-Update» (xem §15.3) trong trường «File name» (Tên tệp) — ví dụ `geosite_custom.dat`; tham chiếu đến nó trong quy tắc dưới dạng `ext:geosite_custom.dat:category`.

Ví dụ: quy tắc dựa trên tệp tùy chỉnh. Nếu đã thêm nguồn loại `geosite` với bí danh `myads`, và danh sách bên trong tệp `.dat` được đánh dấu bằng thẻ `ads`, quy tắc định tuyến trông như sau:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

Đối với nguồn IP (loại `geoip` , bí danh `mycorp` , thẻ `office`) trường sẽ là `"ip":`
`["ext:geoip_mycorp.dat:office"]`.

16. Vận hành: sao lưu, nhật ký, cập nhật, CLI

Phần này đề cập đến việc bảo trì bảng điều khiển hàng ngày: tạo và khôi phục bản sao lưu cơ sở dữ liệu, xem nhật ký (log) của bảng điều khiển và Xray, khởi động lại và dừng dịch vụ, cập nhật Xray và bảng điều khiển, các tác vụ định kỳ (cron) và gỡ cài đặt bảng điều khiển. Một số thao tác được thực hiện từ giao diện web (các tab trên trang «Дашборд» và «Cài đặt bảng điều khiển»), một số – từ menu dòng lệnh `x-ui` trên máy chủ.

16.1. Sao lưu và khôi phục cơ sở dữ liệu

Toàn bộ dữ liệu của bảng điều khiển (inbound, khách hàng, nhóm, nút, cài đặt) được lưu trong một cơ sở dữ liệu duy nhất. Quản lý sao lưu có thể truy cập từ trang «Дашборд» ở tab «Sao lưu», tiêu đề khối – «Sao lưu và khôi phục».

Bảng điều khiển hỗ trợ hai công cụ cơ sở dữ liệu, và hành vi sao lưu phụ thuộc vào đó:

- **SQLite** (mặc định) – dữ liệu được lưu trong tệp `x-ui.db`.
- **PostgreSQL** – nếu bảng điều khiển được cấu hình trên PostgreSQL, khối hiển thị gợi ý:

«Bảng điều khiển này đang chạy trên PostgreSQL. «Sao lưu» tải xuống tệp lưu trữ `pg_dump` (.dump), còn «Khôi phục» tải nó lên thông qua `pg_restore`. Khôi phục cũng chấp nhận cơ sở dữ liệu SQLite (.db) hoặc dump di chuyển SQLite và nhập dữ liệu của chúng vào PostgreSQL. Máy chủ phải có cài đặt các công cụ client PostgreSQL (`pg_dump` và `pg_restore`).»

Xuất (tạo bản sao)

Nút «**Xuất cơ sở dữ liệu**» (tiếng Anh: `Back Up`) tải xuống tệp sao lưu về thiết bị của bạn.

Công cụ CSDL	Tên tệp	Quá trình diễn ra trên máy chủ
SQLite	<code>{host}_YYYY-MM-DD_HHMMSS.db</code>	Đầu tiên thực hiện checkpoint WAL để tệp chứa các bản ghi mới nhất, sau đó toàn bộ tệp được đọc và gửi để tải xuống
PostgreSQL	<code>{host}_YYYY-MM-DD_HHMMSS.dump</code>	Chạy <code>pg_dump</code> , tệp lưu trữ được gửi để tải xuống

Kể từ phiên bản 3.4.2, tên tệp sao lưu được tải xuống chứa địa chỉ mà bạn đã mở bảng điều khiển, và **ngày-giờ**: `{host}_YYYY-MM-DD_HHMMSS` với phần mở rộng `.db` (SQLite) hoặc `.dump` (PostgreSQL) – ví dụ `panel.example.com_2026-06-27_000000.db`. Nhờ đó các tệp được nhóm theo máy chủ và sắp xếp theo thời gian, còn nhiều bản sao trong cùng một ngày không ghi đè lên nhau. Cùng tên đó được dùng trong các bản sao lưu mà bot Telegram gửi (host được lấy từ miền/IP của bảng điều khiển, phương án dự phòng – `x-ui`).

Gợi ý trong giao diện:

- **SQLite**: «Nhấn để tải xuống tệp .db chứa bản sao lưu cơ sở dữ liệu hiện tại của bạn về thiết bị.»

- PostgreSQL: «Nhân để tải xuống dump PostgreSQL (.dump) của cơ sở dữ liệu hiện tại về thiết bị.»

Về mặt kỹ thuật, xuất là yêu cầu `GET /panel/api/server/getDb`. Tên tệp đính kèm được máy chủ tạo ra (Content-Disposition) tùy thuộc vào công cụ cơ sở dữ liệu.

Tên tệp sao lưu được tạo dựa trên địa chỉ máy chủ chứ không phải tên cố định `x-ui.db` / `x-ui.dump`. Khi tải xuống từ trình duyệt, nó được lấy từ địa chỉ bảng điều khiển trong thanh địa chỉ (host của yêu cầu), nếu không – từ miền web đã cấu hình, và nếu không có – từ IP công khai của máy chủ (trước tiên IPv4, sau đó IPv6), với phương án dự phòng là `x-ui`. Điều này giúp phân biệt dễ dàng các bản sao lưu từ các máy chủ khác nhau. Phần mở rộng vẫn là `.db` cho SQLite và `.dump` cho PostgreSQL; các bản sao lưu qua Telegram được đặt tên theo cùng miền/IP.

Ví dụ: tải xuống bản sao lưu qua API. Cùng một lần xuất có thể lấy bằng yêu cầu từ console – ví dụ: cho script sao lưu tự động. Cần phiên đã xác thực (cookie đăng nhập):

```
# 1) Đăng nhập và lưu cookie phiên
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Tải xuống tệp cơ sở dữ liệu (tên do máy chủ xác định: x-ui.db hoặc x-ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

Nếu bảng điều khiển được mở theo đường dẫn cơ sở (Web Base Path), cần thêm nó vào URL: `...:2053/<base_path>/panel/api/server/getDb`.

Nhập (khôi phục)

Nút «**Nhập cơ sở dữ liệu**» (tiếng Anh: `Restore`) mở hộp chọn tệp và tải nó lên máy chủ để khôi phục (POST `/panel/api/server/importDB`, trường biểu mẫu `db`).

Gợi ý trong giao diện:

- SQLite: «Nhân để chọn và tải lên tệp .db hoặc dump di chuyển (.dump) từ thiết bị của bạn để khôi phục cơ sở dữ liệu.»
- PostgreSQL: «Nhân để chọn và tải lên bản sao lưu PostgreSQL (.dump), cơ sở dữ liệu SQLite (.db) hoặc dump di chuyển SQLite để khôi phục cơ sở dữ liệu. Điều này sẽ thay thế tất cả dữ liệu hiện tại.»

Từ 3.5.0, loại tệp được xác định theo nội dung (PGDMP – tệp lưu trữ `pg_dump`; header «SQLite format 3» – cơ sở dữ liệu `.db`; các dòng `PRAGMA / BEGIN TRANSACTION` ở đầu – dump SQL dạng văn bản), chứ không theo phần mở rộng; hộp thoại chọn tệp chấp nhận `.dump`, `.db` trên cả hai DBMS. Các tệp SQLite tải lên panel PostgreSQL được nhập vào PostgreSQL trong một giao dịch duy nhất (xem 3.14); trước khi nhập, tệp được kiểm tra là CSDL panel thật, còn các lược đồ cũ tự động được di chuyển tiếp. Trước khi khôi phục tệp lưu trữ `pg_dump`, panel kiểm tra trước (trước khi dùng Xray) khả năng đọc của nó: nếu dump được tạo bởi PostgreSQL mới hơn, lỗi được hiển thị kèm lệnh chính xác – ví dụ «...run 'x-ui pgclient 17'...».

Quá trình nhập cho SQLite (quan trọng là nó có tính nguyên tử và có khôi phục):

1. Tập được tải lên được kiểm tra định dạng – đây phải là cơ sở dữ liệu SQLite hợp lệ; nếu không sẽ trả về lỗi «Invalid db file format».
2. Tập được lưu vào `x-ui.db.temp` tạm thời và trải qua kiểm tra tính toàn vẹn.
3. **Xray dừng lại** trước khi thay thế CSDL.
4. Cơ sở dữ liệu hiện tại được đổi tên thành `x-ui.db.backup` dự phòng (fallback).
5. Tập tạm được chuyển vào vị trí CSDL hoạt động, thực hiện khởi tạo và di chuyển lược đồ, sau đó di chuyển inbound.
6. **Nếu bất kỳ bước nào hoàn thành với lỗi** – thực hiện khôi phục: cơ sở dữ liệu cũ được phục hồi từ `x-ui.db.backup`, và Xray khởi động lại trên dữ liệu cũ.
7. Khi thành công, tập dự phòng được xóa, và **Xray tự động khởi động lại** trên dữ liệu đã khôi phục.

Thông báo giao diện theo kết quả:

Kết quả	Nội dung
Thành công	«Cơ sở dữ liệu đã được nhập thành công»
Lỗi nhập	«Đã xảy ra lỗi khi nhập cơ sở dữ liệu»
Lỗi đọc tệp	«Đã xảy ra lỗi khi đọc cơ sở dữ liệu»

Khôi phục sẽ thay thế hoàn toàn dữ liệu hiện tại. Vì Xray tạm thời dừng trong quá trình này, các kết nối hiện tại của khách hàng sẽ bị gián đoạn trong thời gian nhập.

Tập di chuyển giữa các công cụ (SQLite ⇌ PostgreSQL)

Ngoài bản sao lưu thông thường, có chức năng «**Tải xuống tập di chuyển**» (Download Migration, yêu cầu `GET /panel/api/server/getMigration`). Nó tạo ra một tệp có thể chuyển đổi để chuyển sang công cụ CSDL khác:

Từ 3.5.0, nút này chỉ hiển thị **trên các panel PostgreSQL** (trên SQLite, bản sao lưu `.db` thông thường nay được khôi phục thẳng vào panel PostgreSQL – không cần bản xuất riêng):

Công cụ hiện tại	Nội dung tải xuống	Tên tệp	Mục đích
PostgreSQL	Cơ sở dữ liệu SQLite được xây dựng từ dữ liệu PostgreSQL	<code>x-ui.db</code>	Chuyển bảng điều khiển trở lại SQLite

Gợi ý: «Nhấn để tải xuống cơ sở dữ liệu SQLite (.db), được xây dựng từ dữ liệu PostgreSQL của bạn và sẵn sàng để chạy bảng điều khiển trên SQLite.»

Việc chuyển đổi `.db` ⇌ `.dump` cho SQLite cũng có thể được thực hiện từ CLI bằng lệnh `x-ui migrateDB [file]` (xem mục 16.7). Ngoài ra, từ 3.5.0, việc di chuyển giữa các công cụ CSDL trở nên **có giao dịch và không mất mát** (bao gồm cả nhóm khách hàng và bộ đếm lưu lượng toàn cục cũng được chuyển; lần nhập thất bại không thay đổi CSDL đích), còn để cập nhật các công cụ client PostgreSQL đã xuất hiện lệnh `x-ui`

pgclient [phiên bản] và mục **10. Install/Upgrade client tools (pg_dump/pg_restore)** trong menu PostgreSQL (khi cần, kho lưu trữ chính thức của PostgreSQL sẽ được kết nối).

Sao lưu qua Telegram bot

Nếu Telegram bot được cấu hình (xem phần về thông báo), nó có thể gửi bản sao lưu trực tiếp vào chat của quản trị viên. Sao lưu qua Telegram bao gồm **hai tệp**: chính cơ sở dữ liệu (`x-ui.db` , hoặc `x-ui.dump` trên PostgreSQL) và cấu hình Xray `config.json` . Tin nhắn được đặt trước dòng « Thời gian sao lưu: ...».

Có hai cách nhận bản sao lưu trong Telegram:

1. **Theo yêu cầu.** Nút « Sao lưu CSDL» trong menu bot – bot ngay lập tức gửi các tệp vào chat hiện tại.
2. **Tự động cùng báo cáo.** Trong cài đặt bot có công tắc «**Sao lưu cơ sở dữ liệu**» (Database Backup) với mô tả «Gửi thông báo kèm tệp sao lưu cơ sở dữ liệu». Khi được bật, tại mỗi lần gửi báo cáo định kỳ, bot sau báo cáo sẽ gửi bản sao lưu tới tất cả quản trị viên. Chu kỳ gửi báo cáo được xác định bởi lịch cron của bot (xem mục 16.6). Giữa các tệp và giữa các quản trị viên, bot dừng lại để không vượt quá giới hạn Telegram.

Sao lưu qua bot chỉ được gửi nếu bot đang chạy; trên PostgreSQL nó cũng yêu cầu có `pg_dump` trên máy chủ.

16.2. Xem nhật ký

Bảng điều khiển có hai công cụ xem nhật ký độc lập, cả hai đều mở từ tab «**Nhật ký**» trên «Дашборд». Mỗi cửa sổ có thể làm mới (biểu tượng «làm mới» ở tiêu đề) và tải xuống nội dung hiển thị vào tệp `x-ui.log` (nút với biểu tượng tải xuống).

Nhật ký bảng điều khiển (ứng dụng / syslog)

Cửa sổ nhật ký bảng điều khiển (`POST /panel/api/server/logs/{count}`). Các điều khiển:

Thành phần	Giá trị mặc định	Mô tả
Số dòng	20	Danh sách thả xuống: 20 / 50 / 100 / 500 / 1000
Cấp độ	Info	Cấp độ tối thiểu: Debug / Info / Notice / Warning / Error
SysLog (hộp kiểm)	tắt	Lấy nhật ký từ đâu: từ bộ đệm ứng dụng hay từ nhật ký hệ thống
Tự động làm mới (hộp kiểm)	tắt	Đọc lại nhật ký mỗi 5 giây (xem bên dưới)

Hành vi phụ thuộc vào hộp kiểm **SysLog**:

- **Tắt (mặc định):** nhật ký được lấy từ bộ đệm vòng nội bộ của bảng điều khiển, được lọc theo cấp độ đã chọn. Các bản ghi hiển thị với cấp độ (DEBUG / INFO / NOTICE / WARNING / ERROR) và nguồn: `X-UI` : – thông báo của chính bảng điều khiển, `XRAY` : – thông báo chuyển tiếp từ Xray.

Các thông báo đơn giản không có dấu thời gian và cấp độ (ví dụ: thông báo hệ thống «Syslog is not supported» trên Windows) giờ hiển thị đầy đủ như nguyên bản. Định dạng YYYY/MM/DD LEVEL – nội dung được nhận dạng chặt chẽ; mọi thứ khác được xuất ra không phân tích, vì vậy các dòng như vậy không còn bị cắt bớt (trước đây ba từ đầu tiên bị nhầm xử lý như ngày/giờ/cấp độ).

- **Bật:** bảng điều khiển thực thi `journalctl -u x-ui --no-pager -n <count> -p <level>` trên máy chủ, tức là hiển thị nhật ký hệ thống của dịch vụ `x-ui`. Số dòng cho phép – từ 1 đến 10000; cấp độ nhận các giá trị syslog (`emerg/0` , `alert/1` , `crit/2` , `err/3` , `warning/4` , `notice/5` , `info/6` , `debug/7`). Trên Windows chế độ SysLog không được hỗ trợ – sẽ hiển thị cảnh báo rằng cần bỏ hộp kiểm và sử dụng nhật ký ứng dụng. Nếu `systemd` /dịch vụ không khả dụng, sẽ xuất hiện thông báo lỗi khi khởi động `journalctl`.

Ví dụ: cùng một nhật ký từ console máy chủ. Khi bảng điều khiển không khả dụng (ví dụ: không khởi động), nhật ký hệ thống có thể đọc trực tiếp – đây chính xác là lệnh mà bảng điều khiển thực thi ở chế độ SysLog:

```
# 100 dòng cuối cùng ở cấp warning và cao hơn
journalctl -u x-ui --no-pager -n 100 -p warning

# theo dõi nhật ký theo thời gian thực
journalctl -u x-ui -f
```

Cấp độ trong cửa sổ này lọc **đầu ra**. Cấp độ tối thiểu nào được ghi vào console/syslog được xác định bởi cấp độ ghi nhật ký của bảng điều khiển (biên môi trường, mặc định là `Info` ; vào tệp bảng điều khiển luôn ghi ở cấp `DEBUG`).

Nhật ký truy cập Xray (nhật ký access)

Cửa sổ riêng cho access-log của Xray (`POST /panel/api/server/xraylogs/{count}`). Nó phân tích các dòng nhật ký truy cập Xray và hiển thị chúng dưới dạng bảng: **Date, From, To, Inbound, Outbound, Email**.

Từ phiên bản 3.4.1, cửa sổ này và nút gọi nó trên thẻ trạng thái Xray được đặt tên là «**Nhật ký truy cập**» (`Access Logs`) – trước đây chúng chỉ được gọi là «Nhật ký». Việc đổi tên được thực hiện để không nhầm lẫn trình xem access-log của Xray với trình xem nhật ký của chính bảng điều khiển, trước đây có cùng tên.

Thành phần	Giá trị mặc định	Mô tả
Số dòng	20	20 / 50 / 100 / 500 / 1000
Bộ lọc	trống	Tìm kiếm văn bản theo chuỗi con (áp dụng khi nhấn Enter)
Tự động làm mới (hộp kiểm)	tắt	Đọc lại nhật ký mỗi 5 giây (xem bên dưới)
Direct (hộp kiểm)	bật	Hiển thị kết nối trực tiếp (lưu lượng qua freedom-outbound)
Blocked (hộp kiểm)	bật	Hiển thị kết nối bị chặn (lưu lượng vào blackhole-outbound)
Proxy (hộp kiểm)	bật	Hiển thị lưu lượng được proxy

Loại sự kiện được xác định tự động theo thẻ kết nối outbound trong dòng nhật ký: khớp với thẻ freedom → «DIRECT» (xanh lá), blackhole → «BLOCKED» (đỏ), tất cả còn lại → «PROXY» (xanh dương). Các dòng api → api và dòng trống bị bỏ qua.

Tự động làm mới. Trong cả hai cửa sổ nhật ký («Nhật ký» và «Nhật ký truy cập») đều có cờ «**Tự động làm mới**» (Auto Update). Nếu bật nó, nội dung nhật ký sẽ tự động đọc lại mỗi 5 giây với tất cả cài đặt hiện tại của cửa sổ được giữ nguyên – số dòng đã chọn, cấp độ/bộ lọc và các hộp kiểm Direct / Blocked / Proxy. Việc thăm dò dừng lại ngay khi cửa sổ được đóng hoặc cờ được bỏ.

Để cửa sổ này hiển thị các bản ghi, Xray phải có **nhật ký truy cập** được bật với đường dẫn đến tệp (không phải none) – xem bên dưới. Nếu access-log bị tắt hoặc tệp không khả dụng, cửa sổ sẽ trống («No Record...»).

16.3. Cấp độ và cấu hình ghi nhật ký Xray

Các tham số ghi nhật ký của chính Xray được thiết lập trên trang «**Cấu hình Xray**» trong khối «**Nhật ký**» (Log) với cảnh báo:

«Nhật ký có thể làm chậm máy chủ. Chỉ bật các loại nhật ký bạn cần khi cần thiết!»

Trường	Dịch	Giá trị mặc định	Mô tả
Cấp độ nhật ký (logLevel)	Log Level	warning	Mức độ chi tiết của nhật ký lỗi Xray. Các giá trị được phép: debug , info , notice , warning , error . Gợi ý: «Cấp độ nhật ký cho nhật ký lỗi, chỉ định thông tin cần ghi lại.»
Nhật ký truy cập (accessLog)	Access Log	none	Đường dẫn đến tệp nhật ký truy cập. Giá trị đặc biệt none tắt nhật ký truy cập. Gợi ý: «Đường dẫn đến tệp nhật ký truy cập. Giá trị đặc biệt «none» tắt nhật ký truy cập.»
Nhật ký lỗi (errorLog)	Error Log	trống (đường dẫn mặc định)	Đường dẫn đến tệp nhật ký lỗi; none tắt chúng. Gợi ý: «Đường dẫn đến tệp nhật ký lỗi. Giá trị đặc biệt «none» tắt nhật ký lỗi.»
Nhật ký DNS (dnsLog)	DNS Log	false (tắt)	Bật ghi nhật ký các yêu cầu DNS. Gợi ý: «Bật nhật ký các yêu cầu DNS.»
Che địa chỉ (maskAddress)	Mask Address	trống (tắt)	Khi được kích hoạt, địa chỉ IP thực sẽ tự động được thay thế bằng địa chỉ che trong nhật ký. Gợi ý: «Khi được kích hoạt, địa chỉ IP thực được thay thế bằng địa chỉ che trong nhật ký.»

Vì mặc định «**Nhật ký truy cập**» = none , cửa sổ «Nhật ký Xray» (mục 16.2) ban đầu trống. Để nó hoạt động, hãy chỉ định đường dẫn đến access-log ở đây và khởi động lại Xray.

Lưu ý: access-log trống chỉ ảnh hưởng đến cửa sổ này. Danh sách khách hàng trực tuyến trên «Dashbord» và giới hạn số lượng IP trong biểu mẫu khách hàng **không phụ thuộc** vào access-log – bảng điều khiển xác định khách hàng trực tuyến và đếm địa chỉ IP của họ thông qua online-stats API của

nhân Xray (thống kê kết nối). Trên các phiên bản nhân cũ không có API này, bảng điều khiển tự động quay lại phương pháp cũ (đọc access-log), và khi đó đường dẫn đến access-log ở đây vẫn cần thiết cho giới hạn IP.

Giới hạn số lượng IP và fail2ban. Bản thân giới hạn số lượng IP của khách hàng (trường «IP Limit» trong biểu mẫu khách hàng và khi thêm hàng loạt) chỉ được áp dụng trên máy chủ nếu **fail2ban** được cài đặt – chính nó mới chặn các địa chỉ vượt quá giới hạn. Bảng điều khiển kiểm tra sự hiện diện của fail2ban (GET /panel/api/server/fail2banStatus); nếu không có, trường «IP Limit» trở nên không khả dụng với gợi ý giải thích (trên Windows – thông báo riêng), và các giới hạn đã thiết lập trước đó trên các máy chủ như vậy tự động được đặt về 0, vì chúng không có tác dụng dù sao. Chặn fail2ban áp dụng cho cả TCP và UDP. Từ 3.5.0, kết nối «chết» (client biến mất mà không đóng TCP đúng cách) chỉ được ghi nhận **một lần**, chứ không phải ở mỗi lần quét 10 giây: dòng [LIMIT_IP] và chu trình ngắt kết nối chỉ lặp lại khi có kết nối lại thực sự, vì vậy bộ đếm fail2ban không bị tăng ảo và không cần nâng cao maxretry nữa (định dạng nhật ký và failregex không thay đổi). Trên các máy chủ thông thường, fail2ban giờ được cài đặt tự động khi cài đặt và cập nhật bảng điều khiển (xem mục 16.5).

Ví dụ: khối Log giúp cửa sổ «Nhật ký Xray» bắt đầu hiển thị bản ghi. Trong câu hình JSON của Xray nó trông như sau:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

Điều chỉnh – thay "access": "none" bằng đường dẫn đến tệp (ví dụ: "./access.log"). Sau khi lưu, hãy khởi động lại Xray, và bảng trong cửa sổ «Nhật ký Xray» sẽ được điền các dòng.

16.4. Quản lý Xray: dừng và khởi động lại

Trạng thái Xray được quản lý từ thẻ Xray trên «DashBoard». Trạng thái hiện tại hiển thị một trong các giá trị: **Đang chạy** (Running), **Đã dừng** (Stopped), **Không xác định** (Unknown), **Lỗi** (Error). Khi có lỗi, có thể xem gợi ý «Lỗi khi khởi động Xray».

Nút	Dịch	Endpoint	Hành động
Dừng	Stop	POST /panel/api/server/stopXrayService	Dừng tiến trình Xray. Khi thành công – thông báo cảnh báo «Xray service has been stopped».
Khởi động lại	Restart	POST /panel/api/server/restartXrayService	Khởi động lại (hoặc khởi động) Xray với cấu hình hiện tại. Khi thành công – thông báo «Xray service has been restarted successfully».

Sau bất kỳ thao tác nào, bảng điều khiển phát sóng trạng thái mới qua WebSocket, vì vậy trạng thái trên «Дашборд» được cập nhật mà không cần tải lại trang. Nếu thao tác hoàn thành với lỗi, trạng thái Xray trở thành «Lỗi», và văn bản lỗi xuất hiện trong thông báo.

Ngoài khởi động lại thủ công, bảng điều khiển tự kiểm tra xem có cần khởi động lại Xray không (tác vụ nền mỗi 30 giây) và tiến trình có bị sập không (kiểm tra mỗi giây) – xem mục 16.6.

Màn hình sức khỏe tunnel (tự động khởi động lại Xray)

Trong phiên bản 3.4.1 đã xuất hiện **màn hình sức khỏe tunnel** tùy chọn. Nếu được bật, bảng điều khiển định kỳ kiểm tra khả năng tiếp cận của URL được chỉ định và sau một số lần kiểm tra thất bại liên tiếp sẽ tự động khởi động lại nhân Xray – điều này giúp khôi phục tunnel đã ngừng truyền lưu lượng. Theo mặc định, màn hình **bị tắt** và được cấu hình **chỉ bằng biến môi trường của dịch vụ** (không có cài đặt của nó trong giao diện web – đó là thiết kế của tác giả).

Biến `XUI_TUNNEL_HEALTH_MONITOR=true` bật màn hình. Biến `XUI_TUNNEL_HEALTH_PROXY` nên được trỏ đến xray-inbound cục bộ (ví dụ `socks5://127.0.0.1:1080`) – khi đó thăm dò đi qua chính Xray và kiểm tra chính xác tunnel; không có nó, chỉ kiểm tra kết nối máy chủ, và khởi động lại sẽ không khắc phục vấn đề kết nối mạng của máy chủ. Các biến còn lại xác định các tham số kiểm tra:

Biến	Mục đích	Mặc định
<code>XUI_TUNNEL_HEALTH_MONITOR</code>	Bật màn hình (bật/tắt)	<code>false</code>
<code>XUI_TUNNEL_HEALTH_PROXY</code>	Proxy qua đó thăm dò đi (chỉ định xray-inbound cục bộ)	trống
<code>XUI_TUNNEL_HEALTH_URL</code>	URL được kiểm tra	<code>https://www.cloudflare.com/cdn-cgi/trace</code>
<code>XUI_TUNNEL_HEALTH_INTERVAL</code>	Khoảng thời gian giữa các lần kiểm tra	<code>30s</code>
<code>XUI_TUNNEL_HEALTH_TIMEOUT</code>	Thời gian chờ của một lần kiểm tra	<code>10s</code>
<code>XUI_TUNNEL_HEALTH_FAILURES</code>	Số lần thất bại liên tiếp trước khi khởi động lại	<code>3</code>
<code>XUI_TUNNEL_HEALTH_COOLDOWN</code>	Thời gian dừng tối thiểu giữa các lần khởi động lại	<code>5m</code>

Khởi động lại Xray ngắt kết nối của tất cả khách hàng đang kết nối, vì vậy có lý để giữ khoảng thời gian và ngưỡng số lần thất bại đủ lớn để sự cố ngẫu nhiên của một lần thăm dò không dẫn đến các lần khởi động lại không cần thiết.

16.5. Khởi động lại và cập nhật bảng điều khiển

Khởi động lại bảng điều khiển

Trên trang «**Cài đặt bảng điều khiển**» có hành động «**Khởi động lại bảng điều khiển**» (`Restart Panel` , `POST /panel/api/setting/restartPanel`). Khi xác nhận, bảng điều khiển khởi động lại **sau 3 giây**.

Thông báo:

- Xác nhận: «Bạn có chắc muốn khởi động lại bảng điều khiển không? Xác nhận, và việc khởi động lại sẽ xảy ra sau 3 giây. Nếu bảng điều khiển không khả dụng, hãy kiểm tra nhật ký máy chủ.»
- Thành công: «Bảng điều khiển đã được khởi động lại thành công».

Về mặt kỹ thuật trên Linux, việc khởi động lại được thực hiện bằng cách gửi tín hiệu `SIGHUP` đến tiến trình bảng điều khiển (hoặc thông qua hook đã đăng ký). Trên Windows, gửi `SIGHUP` không được hỗ trợ.

Tự cập nhật bảng điều khiển (Update Panel)

Trên «Дашборд» có chức năng «**Cập nhật bảng điều khiển**» (`Update Panel`) – cập nhật 3X-UI lên phiên bản mới nhất trực tiếp từ giao diện web.

Trước khi cập nhật, bảng điều khiển so sánh các phiên bản (`GET /panel/api/server/getPanelUpdateInfo`), yêu cầu phiên bản mới nhất của 3x-ui từ GitHub:

Trường	Dịch
Phiên bản bảng điều khiển hiện tại	Current panel version
Phiên bản bảng điều khiển mới nhất	Latest panel version
Bảng điều khiển đã cập nhật / «Đã cập nhật»	Panel is up to date / Up to date – hiển thị khi không có phiên bản mới

Khởi chạy cập nhật – `POST /panel/api/server/updatePanel` . Hộp thoại xác nhận:

- «Bạn có thực sự muốn cập nhật bảng điều khiển không?»
- «Điều này sẽ cập nhật 3X-UI lên phiên bản #version# và khởi động lại dịch vụ bảng điều khiển.»

Sau khi khởi chạy – thông báo bật lên «Đã bắt đầu cập nhật bảng điều khiển» (`Panel update started`); khi kiểm tra phiên bản thất bại – «Kiểm tra cập nhật bảng điều khiển thất bại» (`Panel update check failed`).

Những gì xảy ra trên máy chủ: tự cập nhật chỉ được hỗ trợ **trên Linux** (trên các HĐH khác sẽ trả về lỗi «panel web update is supported only on Linux installations»). Bảng điều khiển tải xuống script chính thức `update.sh` từ GitHub (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) và chạy nó trong một tiến trình riêng: ưu tiên thông qua `systemd-run` trong một unit riêng (`x-ui-web-update-<timestamp>`), và nếu không có `systemd` – như một tiến trình tách biệt độc lập. Sau khi hoàn thành, script cập nhật các thành phần và khởi động lại dịch vụ bảng điều khiển. Cần có `bash` để chạy.

Nếu trong quá trình cập nhật script tạo ra đường dẫn cơ sở web mới ngẫu nhiên (Web Base Path), dịch vụ `x-ui` khởi động lại tự động để đường dẫn mới hoạt động ngay lập tức. (Nếu không khởi động lại, máy chủ sẽ

tiếp tục phục vụ đường dẫn cũ, còn giao diện sẽ hiển thị đường dẫn mới, và địa chỉ mới sẽ không khả dụng cho đến khi khởi động lại thủ công.)

Kênh dev (rolling), từ 3.4.2. Nút phiên bản bảng điều khiển trên «Bảng điều khiển chính» nay luôn mở cửa sổ cập nhật (trước đây trên bản dựng ổn định mới nhất, nó dẫn đến trang phát hành GitHub), còn công tắc **«Kênh dev»** (`devChannelEnable`) luôn hiển thị – kể cả trên bản dựng ổn định. Khi bật nó, ở lần cập nhật tiếp theo có thể chuyển sang kênh dev «rolling» (các bản dựng theo mỗi commit vào `main`). Khi bật, một cảnh báo được hiển thị: «Các bản dựng dev theo dõi mỗi commit vào main và không phải là bản phát hành ổn định – không có rollback tự động». Nếu không có thao tác rõ ràng của người dùng, sẽ không có gì được cập nhật.

Kênh cập nhật Dev (rolling-bản dựng theo commit)

Ngoài cập nhật thông thường lên bản phát hành ổn định, có **«Kênh phát triển»** (`Dev`) tùy chọn. Công tắc xuất hiện trong cửa sổ cập nhật bảng điều khiển **chỉ trên bản dựng dev** (bản dựng CI, được tạo theo commit riêng); trên các bản phát hành ổn định nó không hiển thị. Khi bật, bảng điều khiển sẽ được cập nhật lên rolling-bản dựng `dev-latest`, theo dõi mỗi commit của nhánh `main` và không phải là bản phát hành ổn định – hiển thị cảnh báo rằng bản dựng dev không ổn định và không có khả năng rollback tự động. Ở chế độ dev, cửa sổ hiển thị «Commit hiện tại» / «Commit mới nhất» thay vì số phiên bản. Chức năng chỉ khả dụng trên Linux với `systemd`.

Trên bản dựng dev, bảng điều khiển hiển thị phiên bản của mình là `dev+<commit-ngắn>` thay vì số ổn định gây nhầm lẫn – trong badge thanh bên, trên thẻ «Дашборд», trong cửa sổ cập nhật, trong báo cáo trạng thái của Telegram bot và trong đầu ra lệnh `x-ui -v`. Trên các bản phát hành ổn định, cách hiển thị phiên bản không thay đổi.

Trên các nút (nodes), bảng điều khiển cùng 3x-ui này được cập nhật tập trung thông qua `POST /panel/api/nodes/updatePanel` – xem phần về các nút.

Cài đặt tự động fail2ban

Để giới hạn số lượng IP của khách hàng (mục 16.3) hoạt động ngay lập tức, khi cài đặt và cập nhật bảng điều khiển trên máy chủ thông thường, `fail2ban` giờ được cài đặt và cấu hình tự động (trước đây điều này chỉ xảy ra trong Docker image). Hành vi được kiểm soát bởi biến môi trường `XUI_ENABLE_FAIL2BAN`: cấu hình được thực hiện nếu biến không được đặt hoặc bằng `true`. Chạy thủ công có thể thực hiện bằng lệnh `x-ui setup-fail2ban`. Sự thất bại của cấu hình fail2ban không làm gián đoạn quá trình cài đặt hoặc cập nhật bảng điều khiển.

Cài đặt và cập nhật trên máy chủ chỉ có IPv6

Các script `install.sh` và `update.sh` giờ hoạt động chính xác trên các máy chủ chỉ có IPv6: tải xuống bản phát hành, script `x-ui.sh` và các tệp dịch vụ không còn ép buộc dùng IPv4 (`curl -4`), mà lấy giao thức khả dụng. Vì vậy, bảng điều khiển có thể được cài đặt và cập nhật cả trên máy chủ không có địa chỉ IPv4.

Các bản sửa lỗi script trong 3.5.0

- **Họ RHEL** (Rocky/Alma/RHEL/Oracle): cài đặt PostgreSQL cục bộ từ menu nay hoạt động (xác thực bằng mật khẩu thay vì ident), còn fail2ban cho giới hạn IP được cài từ **EPEL**.
- **Arch/Manjaro**: cài đặt và cập nhật không còn chạy nâng cấp toàn hệ thống `pacman -Syu` — chỉ cập nhật cơ sở dữ liệu gói.
- **ARM 32-bit**: nhị phân Xray được tải xuống được đặt tên `xray-linux-arm32` — đúng như cách panel khởi chạy nó (trước đây bản cập nhật đặt tệp dưới tên khác, và panel tiếp tục chạy lỗi cũ).
- **Chứng chỉ IP**: IPv4 công khai được tự phát hiện sẽ hiển thị để xác nhận (Enter — chấp nhận) trước khi phát hành; nếu từ chối — nhập thủ công có kiểm tra.
- **Chọn cổng ACME**: Enter (chấp nhận cổng 80 mặc định) không còn hiển thị thông báo sai «Your input is invalid».

Ghi đề cổng bảng điều khiển bằng biến `XUI_PORT`

Cổng lắng nghe của bảng điều khiển web có thể được ghi đề bằng biến môi trường `XUI_PORT` — nó chỉ có hiệu lực trong suốt thời gian hoạt động của tiến trình hiện tại và **không thay đổi** giá trị `webPort` đã lưu trong cơ sở dữ liệu. Các giá trị từ `1` đến `65535` được phép; giá trị trống, không đúng hoặc ngoài phạm vi bị bỏ qua (dùng `webPort`) với cảnh báo trong nhật ký. Điều này hữu ích khi triển khai, chủ yếu trong Docker: khi sử dụng bridge network, cổng container được công bố phải khớp với `XUI_PORT` — ví dụ: `XUI_PORT=8080` và `ports: "8080:8080"`.

Cập nhật và chuyển đổi phiên bản Xray-core

Ngay tại đây trên «Дашборд», bạn có thể quản lý phiên bản Xray-core độc lập với bảng điều khiển.

- **Cập nhật Xray** (`Xray Updates`) / **Chọn phiên bản** (`Version`) — danh sách thả xuống các phiên bản khả dụng. Gợi ý: «Chọn phiên bản bạn muốn» và cảnh báo «Quan trọng: các phiên bản cũ có thể không hỗ trợ các cài đặt hiện tại».
- Cài đặt/thay đổi phiên bản — `POST /panel/api/server/installXray/{version}`. Hộp thoại: «Chuyển đổi phiên bản Xray» / «Bạn có chắc muốn thay đổi phiên bản Xray không?». Khi thành công — «Xray đã được cập nhật thành công».

Ví dụ: thay đổi phiên bản Xray-core bằng yêu cầu API. Phiên bản được chỉ định bởi thẻ phát hành từ XTLS/Xray-core (với tiền tố `v`). Ví dụ: chuyển sang `v1.8.24`:

```
curl -s -b cookies.txt -X POST \
https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` — tệp với cookie từ ví dụ trong mục 16.1.) Sau khi cài đặt, Xray sẽ tự động khởi động lại với phiên bản đã chọn.

Trên máy chủ khi thay đổi phiên bản, Xray đầu tiên dừng lại, tải xuống tệp lưu trữ của phiên bản cần thiết từ GitHub (XTLS/Xray-core), nhị phân được giải nén và thay thế, sau đó Xray khởi động lại với việc kiểm tra kích thước kiểm tra của tệp lưu trữ/nhị phân.

16.6. Các tác vụ định kỳ (cron)

Bảng điều khiển đăng ký một số tác vụ nền khi khởi động. Lịch trình của chúng được cố định (không cấu hình được trong UI, ngoại trừ lịch trình báo cáo Telegram và đồng bộ hóa LDAP). Dưới đây – các tác vụ liên quan đến vận hành.

Tác vụ	Lịch trình	Mục đích
Kiểm tra hoạt động Xray	mỗi 1 giây	Kiểm soát tiến trình Xray đang chạy
Kiểm tra sự cần thiết khởi động lại Xray	mỗi 30 giây	Khởi động lại nếu cấu hình được đánh dấu là đã thay đổi
Thu thập lưu lượng Xray	mỗi 5 giây (bắt đầu sau 5 giây kể từ khi khởi động)	Thông kê lưu lượng của inbound/khách hàng
Kiểm tra IP khách hàng	mỗi 10 giây	Kiểm soát giới hạn IP theo nhật ký
Heartbeat và đồng bộ lưu lượng nút	mỗi 5 giây	Trao đổi với các nút
Xóa nhật ký	hàng ngày (<code>@daily</code>)	Xóa nhật ký giới hạn IP và persistent access-log, xoay nhật ký hiện tại thành <code>*.prev.log</code> . Từ 3.5.0, việc dọn dẹp hàng ngày còn làm sạch cả error-log của Xray (trước đây – chỉ access-log)
Giới hạn tăng trưởng nhật ký Xray	mỗi 10 phút (<code>@every 10m</code>)	Mới trong 3.5.0. Cắt bớt access-log và error-log của Xray khi bất kỳ tệp nào vượt quá 64 MiB ; các nhật ký đã tắt (<code>none</code> /trống) không bị động đến
Đặt lại lưu lượng theo chu kỳ	<code>@hourly</code> , <code>@daily</code> , <code>@weekly</code> , <code>@monthly</code>	Đặt lại bộ đếm lưu lượng của các inbound (và khách hàng của chúng) có chu kỳ đặt lại tự động tương ứng
Báo cáo Telegram	được đặt trong cài đặt bot (mặc định <code>@daily</code>)	Gửi báo cáo cho quản trị viên; khi bật tùy chọn – kèm bản sao lưu CSDL đính kèm (mục 16.1)
Đặt lại bộ nhớ hash của Telegram	mỗi 2 phút	Chỉ khi bot được bật
Kiểm soát tải CPU cho Telegram	mỗi 10 giây	Chỉ nếu ngưỡng CPU > 0 được đặt

Bổ sung:

- **Đặt lại lưu lượng định kỳ** chỉ kích hoạt cho các inbound có chọn chế độ đặt lại tự động tương ứng (hàng giờ/hàng ngày/hàng tuần/hàng tháng). Tác vụ đặt lại lưu lượng của chính inbound và tất cả khách hàng của nó.
- **Kiểm tra hết hạn và hết giới hạn.** Vô hiệu hóa khách hàng khi hết hạn và khi hết giới hạn lưu lượng được thực hiện trong phạm vi thông kê lưu lượng: khách hàng có `expiry_time` đã hết hạn hoặc dung lượng đã dùng hết được đánh dấu và vô hiệu hóa, khi cần thiết tính toán thời hạn tiếp theo (cho giới hạn tuần hoàn và chế độ «đếm từ lần sử dụng đầu tiên»). Trên «DashBoard» và trong các danh sách điều này được phản ánh bằng trạng thái «Đã hết hạn»/«Đã hết»/«Sắp hết».

- **Sao lưu tự động trong Telegram** – đây là tác dụng phụ của tác vụ báo cáo, không có lịch cron riêng chỉ cho sao lưu. Do đó tần suất sao lưu tự động bằng tần suất báo cáo của bot.

16.7. Menu console và CLI (`x-ui`)

Trên máy chủ, bảng điều khiển được quản lý bằng lệnh `x-ui` . Không có đối số, menu tương tác «3X-UI Panel Management Script» được mở; với đối số, lệnh con cụ thể được thực thi. Các mục menu liên quan đến vận hành:

Số trong menu	Mục	Hành động
1	Install	Cài đặt bảng điều khiển (tải xuống và chạy <code>install.sh</code>)
2	Update	Cập nhật tất cả thành phần x-ui lên phiên bản mới nhất mà không mất dữ liệu; sau đó – khởi động lại tự động
3	Update to Dev Channel (latest commit)	Cập nhật lên rolling-bản dựng <code>dev-latest</code> (commit mới nhất của nhánh <code>main</code>) với xác nhận (xem 16.5)
4	Update Menu	Chỉ cập nhật script menu <code>x-ui</code>
5	Legacy Version	Cài đặt phiên bản được chỉ định (cũ) của bảng điều khiển theo số đã nhập (ví dụ: <code>2.4.0</code>)
6	Uninstall	Xóa hoàn toàn bảng điều khiển và Xray (xem 16.8)
7	Reset Username & Password	Đặt lại đăng nhập/mật khẩu quản trị viên
8	Reset Web Base Path	Đặt lại đường dẫn cơ sở web của bảng điều khiển
9	Reset Settings	Đặt lại cài đặt về giá trị mặc định
10	Change Port	Thay đổi cổng bảng điều khiển
11	View Current Settings	Xem cài đặt hiện tại
12–14	Start / Stop / Restart	Khởi động, dừng, khởi động lại dịch vụ bảng điều khiển
15	Restart Xray	Chỉ khởi động lại Xray
16	Check Status	Trạng thái dịch vụ hiện tại
17	Logs Management	Xem và xóa nhật ký (xem bên dưới)
18–19	Enable / Disable Autostart	Bật/tắt khởi động tự động dịch vụ khi khởi động HĐH
27	Update Geo Files	Cập nhật tệp địa lý (GeoIP/GeoSite)
25	PostgreSQL Management	Quản lý PostgreSQL

Đánh số các mục menu đã thay đổi trong phiên bản 3.4.1: do thêm mục 3 «Update to Dev Channel», tất cả các mục tiếp theo dịch chuyển thêm một. Tổng số mục trở thành 28, lựa chọn được nhập trong phạm vi `[0–28]` .

Quản lý nhật ký trong CLI (mục 16)

Submenu «Logs Management» giờ mở bằng mục **17** (trước đây – 16):

- **Debug Log** – xem nhật ký dịch vụ theo luồng: `journalctl -u x-ui -e --no-pager -f -p debug` (trên Alpine – `grep` qua `/var/log/messages`).
- **Clear All logs** – xóa nhật ký hệ thống: `journalctl --rotate + journalctl --vacuum-time=1s` ,sau đó dịch vụ khởi động lại. (Không khả dụng trên Alpine.)

Các lệnh con trực tiếp `x-ui`

Tất cả các lệnh con khả dụng:

Lệnh	Mô tả
<code>x-ui</code>	Mở menu quản trị
<code>x-ui start</code>	Khởi động bảng điều khiển
<code>x-ui stop</code>	Dừng bảng điều khiển
<code>x-ui restart</code>	Khởi động lại bảng điều khiển
<code>x-ui restart-xray</code>	Khởi động lại Xray
<code>x-ui status</code>	Trạng thái hiện tại
<code>x-ui settings</code>	Hiển thị cài đặt hiện tại
<code>x-ui enable</code>	Bật khởi động tự động khi khởi động HĐH
<code>x-ui disable</code>	Tắt khởi động tự động
<code>x-ui log</code>	Xem nhật ký
<code>x-ui banlog</code>	Xem nhật ký chặn Fail2ban
<code>x-ui setup-fail2ban</code>	Cài đặt và cấu hình fail2ban cho giới hạn IP (xem 16.5)
<code>x-ui update</code>	Cập nhật bảng điều khiển

| `x-ui update-dev` | Cập nhật bảng điều khiển lên kênh phát triển (rolling-bản dựng `dev-latest`) || `x-ui update-all-geofiles` | Cập nhật tất cả tệp địa lý (với khởi động lại sau đó) || `x-ui migrateDB [file]` | Chuyển đổi cơ sở dữ liệu `.db` ⇌ `.dump` (SQLite) || `x-ui legacy` | Cài đặt phiên bản cũ || `x-ui install` | Cài đặt bảng điều khiển || `x-ui uninstall` | Xóa bảng điều khiển |

Lệnh `x-ui update` tải xuống và chạy `update.sh` chính thức (giống như cập nhật web từ mục 16.5), yêu cầu xác nhận: «This function will update all x-ui components to the latest version, and the data will not be lost.» Sau khi hoàn thành, bảng điều khiển tự động khởi động lại.

Cờ `-webCert` / `-webCertKey` trong lệnh con `setting` . Đường dẫn đến chứng chỉ và khóa riêng tư của bảng điều khiển web có thể được đặt trực tiếp trong lệnh con `x-ui setting -webCert <đường`

dẫn> `-webCertKey <đường dẫn>` – chỉ định bất kỳ cờ nào trong số này sẽ lưu đường dẫn tương ứng (như lệnh `con cert` riêng biệt), và bảng điều khiển ngay lập tức chuyển sang HTTPS.

Lấy API token qua CLI

Lệnh lấy API token qua CLI (mục menu/lệnh `x-ui`) không hiển thị token đã cấp trước đó. API token chỉ được lưu dưới dạng hash, vì vậy token hiện có không thể lấy ở dạng văn bản thuần. Nếu token đã được cấu hình, lệnh thông báo số lượng của chúng, khuyến nghị quản lý token trong bảng điều khiển (**Settings** → **API Tokens**, xem phần về API token) và ngay lập tức tạo **token dự phòng mới** có tên dạng `cli-fallback-<timestamp>` và hiển thị nó để CLI vẫn hữu ích mà không cần đăng nhập vào giao diện.

16.8. Xóa bảng điều khiển

Việc xóa được thực hiện từ CLI – mục menu **5 (Uninstall)** hoặc lệnh `x-ui uninstall`. Trước khi xóa, yêu cầu xác nhận (mặc định «không»): «Are you sure you want to uninstall the panel? xray will also be uninstalled!».

Khi xác nhận, script:

1. Dừng dịch vụ và tắt khởi động tự động (`systemctl stop/disable x-ui`, hoặc trên Alpine – `rc-service / rc-update`), xóa tệp unit dịch vụ và tải lại cấu hình systemd.
2. Xóa các thư mục dữ liệu và ứng dụng (`/etc/x-ui/`, thư mục cài đặt) và tệp env của dịch vụ (`/etc/default/x-ui`, `/etc/conf.d/x-ui` hoặc `/etc/sysconfig/x-ui` – tùy thuộc vào bản phân phối).
3. Xóa chính script `x-ui` và xuất thông báo «Uninstalled Successfully», cũng như lệnh để cài đặt lại.

Nếu bảng điều khiển sử dụng PostgreSQL (trong tệp env `XUI_DB_TYPE=postgres`), sau khi xóa tệp bảng điều khiển, script thêm hỏi xem có cần xóa cả máy chủ PostgreSQL cùng với tất cả cơ sở dữ liệu của nó không: «Also purge PostgreSQL and delete all of its data?». Yêu cầu xác nhận rõ ràng (mặc định – từ chối) và đi kèm với cảnh báo: việc xóa sẽ ảnh hưởng đến **TẤT CẢ** cơ sở dữ liệu PostgreSQL trên máy, kể cả thuộc về các ứng dụng khác, và không thể phục hồi. Khi từ chối, PostgreSQL và dữ liệu của nó vẫn nguyên vẹn.

Việc xóa không thể phục hồi: cùng với bảng điều khiển, Xray và tất cả dữ liệu (kể cả cơ sở dữ liệu) bị xóa. Nếu dữ liệu có thể cần thiết, hãy xuất cơ sở dữ liệu trước (mục 16.1).

16.9. Lệnh `x-ui migrateDB`

Bắt đầu từ phiên bản 3.3.0, script quản lý `x-ui.sh` đã nhận lệnh `con migrateDB` – wrapper xung quanh nhị phân tích hợp `x-ui` (`x-ui migrate-db`) để chuyển đổi cơ sở dữ liệu bảng điều khiển SQLite giữa hai định dạng: nhị phân `.db` và dump văn bản có thể chuyển đổi `.dump` (văn bản SQL thông thường).

Lệnh làm gì

Lệnh hoạt động theo hai hướng, và hướng được xác định **tự động** theo tệp đầu vào:

Hướng	Tên gọi	Quá trình diễn ra
<code>.db → .dump</code>	dump (xuất)	cơ sở dữ liệu nhị phân SQLite được xuất thành tệp SQL văn bản
<code>.dump → .db</code>	restore (khôi phục)	từ tệp SQL văn bản, cơ sở dữ liệu nhị phân SQLite được tái tạo

Bên dưới, script gọi nhị phân bảng điều khiển:

- xuất: `x-ui migrate-db --src <đầu vào> --dump <đầu ra>`
- khôi phục: `x-ui migrate-db --restore <đầu vào> --out <đầu ra>`

Cú pháp gọi

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** – tệp đầu vào (đối số đầu tiên). Nếu không chỉ định, cơ sở dữ liệu bảng điều khiển được cài đặt mặc định sẽ được lấy: `/etc/x-ui/x-ui.db`.
- **[output]** – đường dẫn đến tệp đầu ra (đối số thứ hai). Tùy chọn: khi vắng mặt, tên được chọn tự động bên cạnh tệp đầu vào (xem bên dưới).

Ví dụ:

```
x-ui migrateDB                               # xuất /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db      # tạo .db từ dump
```

Cách xác định hướng

Script xem phần mở rộng của tệp đầu vào:

- `*.db`, `*.sqlite`, `*.sqlite3` → chế độ **dump** (xuất thành văn bản);
- `*.dump`, `*.sql` → chế độ **restore** (tạo cơ sở dữ liệu).

Nếu phần mở rộng không được nhận dạng, script đọc 16 byte đầu tiên của tệp: chữ ký `SQLite format 3` có nghĩa là cơ sở dữ liệu nhị phân (chế độ dump), ngược lại tệp được coi là dump (chế độ restore).

Tên tệp đầu ra, nếu đối số thứ hai không được đặt:

- khi xuất – cùng tên với đầu vào, với phần mở rộng `.dump`;
- khi khôi phục – cùng tên với phần mở rộng `.db`.

Kiểm tra bảo vệ và hành vi

- **Sự hiện diện của nhị phân.** Nếu nhị phân `x-ui` không tìm thấy hoặc không thể thực thi – xuất lỗi «x-ui binary not found ... Is the panel installed?».

- **Hỗ trợ chức năng trong bản dựng.** Script kiểm tra rằng nhị phân hỗ trợ `migrate-db --dump/--restore` (thông qua `x-ui migrate-db -h`). Nếu không – đề nghị cập nhật bảng điều khiển bằng lệnh `x-ui update`.
- **Sự tồn tại của tệp đầu vào.** Khi không có tệp đầu vào, in lỗi và dòng với cú pháp gọi.
- **Ghi đè đầu ra.** Nếu tệp đầu ra đã tồn tại, yêu cầu xác nhận (mặc định «không»); không có xác nhận thì thao tác bị hủy. Khi khôi phục, tệp đầu ra cũ được xóa trước.
- **Bảo vệ cơ sở dữ liệu «đang sống».** Khi khôi phục vào cơ sở dữ liệu mặc định `/etc/x-ui/x-ui.db`, khi bảng điều khiển đang chạy, thao tác bị từ chối với yêu cầu trước tiên dừng bảng điều khiển (`x-ui stop`) hoặc chọn đường dẫn đầu ra khác. Điều này ngăn việc ghi đè cơ sở dữ liệu hoạt động của dịch vụ đang chạy.
- Khi tạo cơ sở dữ liệu thất bại, tệp đầu ra chưa hoàn chỉnh sẽ bị xóa.

Tại sao điều này cần thiết

- **Sao lưu.** Văn bản `.dump` – có thể đọc được bởi con người, thuận tiện để lưu trữ trong hệ thống kiểm soát phiên bản và để xem nội dung cơ sở dữ liệu theo sự khác biệt.
- **Chuyển đổi.** Dump có thể chuyển đổi giữa các máy và ổn định với sự khác biệt về phiên bản định dạng tệp SQLite – trên máy chủ mới, `.db` hoạt động được tạo từ nó.
- **Chẩn đoán.** Từ `.dump` bạn có thể nhìn thấy bằng mắt cấu trúc và dữ liệu của bảng điều khiển mà không cần có công cụ SQLite trong tay.

Chế độ tương tác

Ngoài gọi trực tiếp, việc chuyển đổi có thể truy cập từ menu tương tác. Trong submenu PostgreSQL (`x-ui` → phần làm việc với PostgreSQL) có mục **9. Convert SQLite .db <-> .dump**: nó hỏi đường dẫn đến tệp đầu vào (mặc định `/etc/x-ui/x-ui.db`) và đến tệp đầu ra (có thể để trống để đặt tên tự động), và hướng, như trong chế độ CLI, được xác định tự động.

Tài liệu được chuẩn bị dựa trên mã nguồn 3X-UI. Nếu một số mục trong giao diện của phiên bản bạn khác – ưu tiên cho hành vi của bảng điều khiển và gợi ý trong chính UI.